ỦY BAN NHÂN DÂN TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

LÊ VÕ HOÀNG OANH - 3122410288

BIỆN THỊ MỸ PHỤNG - 3122410326

HEALTHASSISTANT: NỀN TẢNG TƯ VẤN SỨC KHỎE THÔNG MINH VÀ ĐẶT LỊCH KHÁM TRỰC TUYẾN TÍCH HỢP AI

KHÓA LUẬN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

TRÌNH ĐỘ ĐÀO TẠO: ĐẠI HỌC

TP. HỒ CHÍ MINH, THÁNG 05 NĂM 2026

ỦY BAN NHÂN DÂN TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

LÊ VÕ HOÀNG OANH - 3122410288

BIỆN THỊ MỸ PHỤNG - 3122410326

HEALTHASSISTANT: NỀN TẢNG TƯ VẤN SỨC KHỎE THÔNG MINH VÀ ĐẶT LỊCH KHÁM TRỰC TUYẾN TÍCH HỢP AI

KHÓA LUẬN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

TRÌNH ĐỘ ĐÀO TẠO: ĐẠI HỌC

NGƯỜI HƯỚNG DẪN: TS. ĐỖ NHƯ TÀI

TP. HỒ CHÍ MINH, THÁNG 05 NĂM 2026

LỜI CAM ĐOAN

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi, các số liệu và kết quả nghiên cứu nêu trong khóa luận là trung thực, được các đồng tác giả cho phép sử dụng và chưa từng được công bố trong bất kì một công trình nào khác.

Tác giả khóa luận

Lê Võ Hoàng Oanh

Biện Thị Mỹ Phụng

LỜI CẢM ƠN

Để hoàn thành khóa luận tốt nghiệp này, chúng em đã nhận được rất nhiều sự đồng hành, hỗ trợ cả về kiến thức lẫn tinh thần từ nhiều cá nhân và tập thể.

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến Trường Đại học Sài Gòn cùng các thầy cô Khoa Công nghệ thông tin. Bốn năm học tập tại trường không chỉ trang bị cho chúng em những kiến thức chuyên môn nền tảng, mà còn là bệ phóng quan trọng giúp chúng em tự tin thực hiện đề tài nghiên cứu này.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc nhất đến Thầy Đỗ Như Tài. Thầy đã luôn dành thời gian định hướng, chỉ dẫn và đưa ra những lời khuyên chuyên môn vô cùng kịp thời để chúng em đi đúng hướng. Sự kiên nhẫn và những góp ý thẳng thắn, sát sao của Thầy là yếu tố then chốt góp phần giúp đề tài hoàn thành một cách trọn vẹn.

Cuối cùng, chúng em xin cảm ơn gia đình và bạn bè - những người đã luôn bên cạnh động viên, chia sẻ áp lực và là chỗ dựa tinh thần vững chắc để chúng em yên tâm hoàn thành chặng đường này.

Mặc dù đã dành nhiều thời gian và tâm huyết cho đề tài, nhưng chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong sẽ nhận được những ý kiến đóng góp, nhận xét từ quý Thầy Cô để đề tài được hoàn thiện và thực tế hơn.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI CAM ĐOAN i

LỜI CẢM ƠN ii

MỤC LỤC 1

DANH MỤC CÁC CỤM TỪ VIẾT TẮT 3

DANH MỤC CÁC BẢNG 4

DANH MỤC HÌNH 5

TÓM TẮT KHÓA LUẬN 8

CHƯƠNG 1. MỞ ĐẦU 10

1.1. Tính cấp thiết của đề tài 10

1.2. Tổng quan tình hình nghiên cứu 11

1.3. Mục tiêu nghiên cứu 13

1.4. Nhiệm vụ nghiên cứu 13

1.5. Phạm vi và đối tượng nghiên cứu 14

1.6. Phương pháp nghiên cứu 16

1.7. Ý nghĩa khoa học và thực tiễn 18

1.8. Cấu trúc của khóa luận 19

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 21

2.1. Tổng quan về hệ thống hỗ trợ y tế thông minh 21

2.2. Trí tuệ nhân tạo và AI Agents 22

2.3. Công nghệ tự động hóa luồng công việc với n8n 24

2.4. Kiến trúc truy xuất thông tin tăng cường (RAG) 26

2.5. Phát triển ứng dụng web với kiến trúc hiện đại 27

2.6. Công nghệ đóng gói và triển khai hệ thống 32

2.7. Kiểm thử phần mềm và các công cụ kiểm thử 36

2.8. Bảo mật và an toàn dữ liệu y khoa 48
2.9. Tổng quan các nghiên cứu và ứng dụng liên quan 50
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG 55
3.1. Phân tích yêu cầu hệ thống 55
3.2. Kiến trúc hệ thống tổng thể 67
3.3. Quy trình xử lý của AI Agent và Luồng tự động hóa (n8n) 88
3.4. Thiết kế cơ sở dữ liệu 95
3.5. Thiết kế workflow AI và cơ chế xử lý 119
3.6. Thiết kế giao diện và Trải nghiệm người dùng (UI/UX) 122
CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ 130
4.1. Môi trường triển khai 130
4.2. Triển khai hệ thống Microservices và workflow AI 133
4.3. Kết quả triển khai giao diện 137
4.4. Tổ chức thực nghiệm và kịch bản kiểm thử 144
4.5. Đánh giá và phân tích kết quả 156
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 159
5.1. Tóm tắt kết quả nghiên cứu 159
5.2. Đóng góp của đề tài 160
5.3. Hạn chế của nghiên cứu 160
5.4. Hướng phát triển trong tương lai 161
5.5. Kết luận 163
TÀI LIỆU THAM KHẢO 165

DANH MỤC CÁC CỤM TỪ VIẾT TẮT

STT	Từ viết tắt	Từ tiếng Anh/Việt
1	AI	Artificial Intelligence
2	AI Agent	Artificial Intelligence Agent
3	API	Application Programming Interface
4	AWS	Amazon Web Services
5	BLEU	Bilingual Evaluation Understudy
6	CI/CD	Continuous Integration / Continuous Deployment
7	HTTPS	HyperText Transfer Protocol Secure
8	JWT	JSON Web Token
9	KB	Knowledge Base
10	LLM	Large Language Model
11	RAG	Retrieval-Augmented Generation
12	ROUGE-1	Recall-Oriented Understudy for Gisting Evaluation - 1-gram
13	ROUGH-L	Recall-Oriented Understudy for Gisting Evaluation - Longest Common Subsequence
14	UI/UX	User Interface / User Experience
15	VPS	Virtual Private Server

DANH MỤC CÁC BẢNG

Bảng 3.1. Tổng quan thiết kế API cho user-service 76
Bảng 3.2. Tổng quan thiết kế API cho doctor-service 77
Bảng 3.3. Tổng quan thiết kế API cho chat-service 77
Bảng 3.4. Tổng quan thiết kế API cho appointment-service 78
Bảng 3.5. Mô tả public.account 100
Bảng 3.6. Mô tả public.health_profile 102
Bảng 3.7. Mô tả public.doctor 103
Bảng 3.8. Mô tả public.doctor_schedule 104
Bảng 3.9. Mô tả public.doctor_request 105
Bảng 3.10. Mô tả public.salary 106
Bảng 3.11. Mô tả public.doctor_assistant_profile 107
Bảng 3.12. Mô tả public.doctor_assistant_knowledge 108
Bảng 3.13. Mô tả public.appointment 110
Bảng 3.14. Mô tả public.appointment_result 111
Bảng 3.15. Mô tả public.payment 112
Bảng 3.16. Mô tả public.refund 113
Bảng 3.17. Mô tả public.chat 114
Bảng 3.18. Mô tả public.chat_member 116
Bảng 3.19. Mô tả public.message 117
Bảng 4.1. Thông số kỹ thuật của VPS 130
Bảng 4.2. Công nghệ sử dụng trong hệ thống 131
Bảng 4.3. Bảng test case module Đăng nhập 145
Bảng 4.4. Bảng test case module Quản lý bảng lương 147
Bảng 4.5. Bảng test case module Hồ sơ sức khỏe 148
Bảng 4.6. Kết quả kiểm thử giao diện người dùng 150
Bảng 4.7. Bảng test case module Tìm kiếm và lọc bác sĩ 151
Bảng 4.8. Kết quả đánh giá workflow AI 154
Bảng 4.9. Kết quả kiểm thử API bằng Postman Collection Runner 155

DANH MỤC HÌNH

Hình 2.1. Minh họa hoạt động của n8n 24
Hình 2.2. Minh họa về kiến trúc truy xuất thông tin tăng cường 26
Hình 2.3. Minh họa về Kiến trúc Microservices qua hệ thống mua sắm trực tuyến 28
Hình 2.4. Minh họa về cách thay đổi thành phần của ReactJS 29
Hình 2.5. Minh họa về vòng đời của giao tiếp thời gian thực 30
Hình 2.6. So sánh đồng bộ và bất đồng bộ 31
Hình 2.7. Minh họa đóng gói ứng dụng được quản lý bởi Docker. 33
Hình 2.8. Các lợi ích nổi bật khi sử dụng nền tảng Cloud 35
Hình 2.9. Kiến trúc ứng dụng/công nghệ JUnit 5.....38
Hình 2.10. Tổng quan về mocking trong Unit Test.....40
Hình 2.11. Cách thức Diffblue Cover hoạt động.....41
Hình 2.12. Kiến trúc hoạt động của JaCoCo 43
Hình 2.13. Kiến trúc hoạt động của Playwright 44
Hình 2.14. Minh họa kết quả kiểm thử API bằng Postman Collection Runner ..47
Hình 3.1. Sơ đồ Use case tổng quát của hệ thống HealthAssistant.....62
Hình 3.2. Sơ đồ mô tả quy trình đặt lịch và khám bệnh.....64
Hình 3.3. Sơ đồ mô tả quy trình đăng kí trở thành bác sĩ.....65
Hình 3.4. Sơ đồ mô tả quy trình nhắn tin với Chatbot AI của hệ thống.....66
Hình 3.5. Sơ đồ mô tả quy trình nhắn tin với Chatbot AI trợ lý của bác sĩ.....66
Hình 3.6. Sơ đồ mô tả quy trình nhắn tin bác sĩ 67
Hình 3.7. Sơ đồ C4 Context của hệ thống HealthAssistant.....70
Hình 3.8. Sơ đồ C4 Container của hệ thống HealthAssistant.....71
Hình 3.9. Mô tả kiến trúc tổng thể của hệ thống HealthAssistant.....72
Hình 3.10. Một vài vai trò của API Gateway trong các hệ thống Microservices74
Hình 3.11. Luồng xử lý xác thực của các yêu cầu từ gửi dùng đến hệ thống 84
Hình 3.12. Minh họa đơn giản về xác thực bằng JWT.....85
Hình 3.13. Sơ đồ minh họa cơ chế phân quyền dựa trên vai trò 86
Hình 3.14. Workflow tổng quan của hệ thống AI 89
Hình 3.15. Quy trình xử lý của General AI Agent 90
Hình 3.16. Quy trình xử lý của Personal AI Agent 91

Hình 3.17. Quy trình tóm tắt triệu chứng 93
Hình 3.18. Quy trình xử lý hình ảnh.....93
Hình 3.19. Quy trình phân loại vai trò.....94
Hình 3.20. Sơ đồ cấu trúc cơ sở dữ liệu mức ý niệm của hệ thống.....95
Hình 3.21. Mối quan hệ của các bảng trong user_serivce_db.....103
Hình 3.22. Mối quan hệ của các bảng trong doctor_serivce_db 109
Hình 3.23. Mối quan hệ của các bảng trong appointment_serivce_db 114
Hình 3.24. Mối quan hệ của các bảng trong chat_serivce_db.....118
Hình 3.25. Bố cục tổng quan của hệ thống.....125
Hình 3.26. Bố cục tổng quan của hệ thống khi thu nhỏ Sidebar 125
Hình 3.27. Minh họa về Modal: Đánh giá bác sĩ.....126
Hình 3.28. Bố cục chức năng nhắn tin 127
Hình 3.29. Bố cục chức năng đặt lịch khám.....128
Hình 4.1. Quy trình triển khai thủ công được áp dụng.....133
Hình 4.2. Dockerfile của api-gateway 134
Hình 4.3. Trích đoạn docker-compose.yml dùng trong triển khai. 134
Hình 4.4. Kết quả build image và khởi tạo các container ở local.....135
Hình 4.5. Trạng thái hoạt động của các container sau khi triển khai trên VPS.135
Hình 4.6. Giao diện trang Projects trên Netlify.....138
Hình 4.7. Giao diện thêm dự án lên Netlify 138
Hình 4.8. Kết quả sau khi thêm dự án lên Netlity 138
Hình 4.9. Cấu hình xuất bản ứng dụng.....139
Hình 4.10. Thông tin về phiên bản được xuất bản gần nhất.....139
Hình 4.11. Giao diện chức năng tìm kiếm bác sĩ 140
Hình 4.12. Giao diện chức năng quản lý lịch hẹn 140
Hình 4.13. Giao diện chức năng nhắn tin với AI ChatBot 141
Hình 4.14. Giao diện chức năng thống kê 141
Hình 4.15. Giao diện chức năng quản lý lương.....141
Hình 4.16. Giao diện chức năng quản lý lịch hẹn 142
Hình 4.17. Giao diện chức năng quản lý lịch làm việc 142
Hình 4.18. Giao diện quản lý trợ lý 142
Hình 4.19. Giao diện chức năng thống kê 143

Hình 4.20. Giao diện chức năng quản lý lịch hẹn 143
Hình 4.21. Giao diện chức năng quản lý bảng lương.....143
Hình 4.22. Giao diện chức năng kiểm duyệt bác sĩ.....144
Hình 4.23. Màn hình report của JaCoCo.....153
Hình 4.24. Workflow kiểm thử chất lượng phản hồi của workflow AI.....154
Hình 4.25. Kết quả kiểm thử API bằng Postman Collection Runner.....155

TÓM TẮT KHÓA LUẬN

Trong bối cảnh chuyển đổi số trong lĩnh vực y tế và nhu cầu hỗ trợ chăm sóc sức khỏe từ xa ngày càng gia tăng, các hệ thống tư vấn sức khỏe truyền thống vẫn còn nhiều hạn chế về khả năng hỗ trợ liên tục, cá nhân hóa thông tin và giảm tải cho đội ngũ y tế. Đồng thời, sự phát triển mạnh mẽ của trí tuệ nhân tạo tạo điều kiện thuận lợi cho việc xây dựng các hệ thống hỗ trợ tư vấn sức khỏe thông minh có khả năng phản hồi tự nhiên và hỗ trợ người dùng mọi lúc, mọi nơi. Khóa luận này tập trung nghiên cứu, thiết kế và triển khai một hệ thống AI Agent hỗ trợ tư vấn sức khỏe thông minh, kết hợp mô hình ngôn ngữ lớn, kỹ thuật tăng cường truy xuất kết hợp sinh phản hồi và quy trình xử lý đa tác vụ (Workflow) nhằm nâng cao chất lượng phản hồi và hỗ trợ quy trình chăm sóc sức khỏe ban đầu.

Về mặt phương pháp, khóa luận xây dựng một workflow AI đa bước cho bài toán tư vấn sức khỏe, bao gồm các chức năng phân tích triệu chứng, truy xuất tri thức y khoa, tư vấn sức khỏe cá nhân hóa, hỗ trợ quy trình đặt lịch khám và chuyển tiếp cho bác sĩ trong các trường hợp cần tư vấn chuyên sâu. Hệ thống sử dụng cơ chế RAG nhằm kết hợp khả năng sinh ngôn ngữ tự nhiên của mô hình AI với dữ liệu tri thức y khoa được lưu trữ dưới dạng cơ sở dữ liệu vector, giúp tăng độ chính xác, giảm hiện tượng sinh thông tin sai lệch và đảm bảo phản hồi phù hợp với ngữ cảnh người dùng. Đồng thời, kỹ thuật thiết kế prompt và cơ chế phân vai AI Agent được áp dụng nhằm tối ưu khả năng xử lý hội thoại và phân tách nhiệm vụ trong hệ thống.

Về mặt hệ thống, khóa luận triển khai kiến trúc Microservices hiện đại nhằm đảm bảo khả năng mở rộng, khả năng bảo trì và tính linh hoạt trong quá trình triển khai. Hệ thống được xây dựng theo mô hình các service độc lập bao gồm user-service, doctor-service, chat-service, appointment-service, notification-service và api-gateway. Trong đó, api-gateway đóng vai trò điều phối và định tuyến request giữa frontend và các backend service thông qua các giao diện lập trình ứng dụng (Application Programming Interface - API), còn chat-service đảm nhiệm việc giao tiếp với workflow AI và lưu trữ lịch sử hội thoại nhằm duy trì ngữ cảnh trao đổi với người dùng. Notification-service được sử dụng để gửi email cho bác sĩ trong các trường hợp AI không thể xử lý đầy đủ hoặc phát hiện yêu cầu tư vấn chuyên sâu cũng như các thông báo về lịch hẹn khác. Các công nghệ như Spring Boot, ReactJS,

Docker, PostgreSQL, Redis Cloud và RESTful API được sử dụng nhằm tối ưu hiệu suất xử lý và khả năng tích hợp giữa các thành phần trong hệ thống.

Để đánh giá hiệu quả hệ thống, khóa luận tiến hành thực nghiệm trên nhiều kịch bản hội thoại sức khỏe khác nhau, kết hợp đánh giá chất lượng phản hồi AI và kiểm thử chức năng hệ thống. Chất lượng phản hồi được đánh giá thông qua các chỉ số BLEU, ROUGE-1 và ROUGE-L nhằm đo lường mức độ tương đồng giữa phản hồi của AI và dữ liệu tham chiếu. Đồng thời, hệ thống được kiểm thử thông qua Unit Test, Postman Collection Runner và Playwright nhằm đánh giá tính chính xác, độ ổn định và khả năng hoạt động của các chức năng chính cũng như giao diện người dùng. Kết quả thực nghiệm cho thấy hệ thống có khả năng phản hồi nhanh, duy trì độ ổn định tốt và cung cấp nội dung tư vấn phù hợp với ngữ cảnh hội thoại trong phần lớn trường hợp thử nghiệm.

Tổng thể, khóa luận đã chứng minh tính khả thi của việc ứng dụng AI Agent kết hợp RAG và kiến trúc Microservices trong bài toán hỗ trợ tư vấn sức khỏe thông minh. Các kết quả đạt được không chỉ có ý nghĩa học thuật trong lĩnh vực ứng dụng trí tuệ nhân tạo vào y tế mà còn mở ra tiềm năng ứng dụng thực tiễn trong hỗ trợ chăm sóc sức khỏe từ xa, giảm tải cho hệ thống y tế và nâng cao khả năng tiếp cận dịch vụ tư vấn sức khỏe cho người dùng. Đồng thời, khóa luận cũng đặt nền tảng cho các nghiên cứu mở rộng trong tương lai như tích hợp dữ liệu y khoa thời gian thực, nâng cao khả năng suy luận của AI Agent và mở rộng hệ thống sang các lĩnh vực hỗ trợ y tế chuyên sâu hơn.

CHƯƠNG 1. MỞ ĐẦU

1.1. Tính cấp thiết của đề tài

Chăm sóc sức khỏe luôn là mối quan tâm hàng đầu của cộng đồng. Trong thời đại công nghệ hiện nay, khi nhịp sống ngày càng bận rộn, việc tiếp cận bác sĩ cũng như nhận được hỗ trợ y tế đúng thời điểm trở thành một khó khăn đối với nhiều người. Nhu cầu được tư vấn sức khỏe từ xa và đặt lịch khám trực tuyến ngày càng gia tăng, đặc biệt là ngoài giờ hành chính hoặc trong các tình huống không thể trực tiếp đến cơ sở y tế.

Bên cạnh đó, quy trình tư vấn và đặt lịch khám tại nhiều phòng khám hiện nay vẫn còn mang tính thủ công, gây mất thời gian cho cả bác sĩ lẫn bệnh nhân. Đặc biệt, đối với mô hình phòng khám chỉ có một bác sĩ vận hành, một mình bác sĩ sẽ phải kiêm nhiệm toàn bộ các khâu từ tiếp nhận lịch hẹn, tư vấn chuyên môn và trả lời tin nhắn/cuộc gọi khi bệnh nhân cần đến. Sự quá tải này khiến việc vận hành bị đình trệ, dễ xảy ra sai sót, trùng lịch và mang lại trải nghiệm không tốt cho bệnh nhân khi để họ phải chờ đợi phản hồi quá lâu.

Trong khi đó, xu hướng chuyển đổi số và ứng dụng trí tuệ nhân tạo trong lĩnh vực y tế đang phát triển mạnh mẽ. Tuy nhiên, phần lớn các giải pháp hiện nay thường hướng đến bệnh viện hoặc hệ thống y tế lớn, đòi hỏi chi phí triển khai cao và khó phù hợp với các phòng khám tư nhân nhỏ lẻ. Vì vậy, nhu cầu về một nền tảng hỗ trợ tư vấn sức khỏe và quản lý lịch khám thông minh, chi phí thấp, dễ triển khai và phù hợp với mô hình phòng khám nhỏ là rất cần thiết.

Xuất phát từ thực tế đó, đề tài được nghiên cứu và phát triển nhằm hỗ trợ kết nối giữa bác sĩ và bệnh nhân trên nền tảng web. Hệ thống cho phép bác sĩ tạo hồ sơ cá nhân, mở rộng khả năng tiếp cận bệnh nhân. Điểm cốt lõi của nền tảng là việc triển khai hệ sinh thái gồm ba vai trò trợ lý AI (Artificial Intelligence - AI) chuyên biệt: tiếp tân, y tá và trợ lý bác sĩ (tư vấn chuyên môn) dựa trên nguồn tri thức (Knowledge Base - KB) do chính bác sĩ cung cấp. Thay vì hoạt động dưới dạng nhiều chatbot độc lập, các vai trò AI được tích hợp và phối hợp linh hoạt trong cùng một khung trò chuyện, hoạt động như một trợ lý ảo hỗ trợ tư vấn sức khỏe 24/7, giúp giải đáp thắc mắc y tế, tối ưu hóa quy trình đặt lịch và giảm tải hoàn toàn áp lực vận hành cho bác sĩ.

Đề tài không chỉ góp phần giảm tải một phần áp lực vận hành cho các phòng khám nhỏ mà còn nâng cao trải nghiệm người dùng, mở rộng khả năng tiếp cận bệnh nhân cho bác sĩ và phù hợp với xu hướng ứng dụng trí tuệ nhân tạo trong chuyển đổi số y tế hiện nay.

1.2. Tổng quan tình hình nghiên cứu

Trong những năm gần đây, cùng với sự phát triển của công nghệ thông tin và trí tuệ nhân tạo, lĩnh vực y tế số đã nhận được nhiều sự quan tâm nghiên cứu trên toàn thế giới cũng như tại Việt Nam. Nhiều hệ thống hỗ trợ tư vấn sức khỏe trực tuyến, chatbot y tế và nền tảng đặt lịch khám đã được phát triển nhằm nâng cao khả năng tiếp cận dịch vụ chăm sóc sức khỏe cho người dùng.

Tại Việt Nam, quá trình chuyển đổi số trong lĩnh vực y tế đang diễn ra mạnh mẽ với sự xuất hiện của nhiều nền tảng như BookingCare, YouMed cùng các ứng dụng đặt lịch riêng của các bệnh viện lớn. Các hệ thống này góp phần hỗ trợ bệnh nhân tìm kiếm bác sĩ, đặt lịch khám và số hóa quy trình tiếp nhận khám chữa bệnh.

Bên cạnh đó, một số phòng khám và bệnh viện cũng đã bắt đầu triển khai chatbot AI trên website hoặc fanpage nhằm hỗ trợ trả lời tự động các câu hỏi thường gặp. Tuy nhiên, phần lớn các chatbot hiện nay vẫn hoạt động theo cơ chế kịch bản cố định, chủ yếu hỗ trợ các câu hỏi cơ bản như thời gian làm việc, giá dịch vụ hoặc thông tin liên hệ. Khi người dùng đặt các câu hỏi chuyên môn hoặc mô tả triệu chứng phức tạp, hệ thống thường chưa thể xử lý linh hoạt theo ngữ cảnh.

Ngoài ra, đa số các giải pháp hiện tại chủ yếu phù hợp với bệnh viện hoặc các phòng khám có quy mô lớn. Đối với các phòng khám tư nhân nhỏ, đặc biệt là mô hình chỉ có một bác sĩ vận hành, vẫn còn thiếu một nền tảng tích hợp vừa hỗ trợ tư vấn thông minh vừa hỗ trợ quy trình đặt lịch với chi phí triển khai phù hợp.

Trên thế giới, theo nghiên cứu “Language Models are Few-Shot Learners” của Tom B. Brown và cộng sự [2], sự phát triển của các mô hình LLM đã thúc đẩy mạnh mẽ xu hướng ứng dụng AI trong nhiều lĩnh vực, trong đó có y tế số. Nhiều hệ thống như Ada Health, Babylon Health hay mô hình Med-PaLM của Google DeepMind đã cho thấy khả năng hỗ trợ phân tích triệu chứng, tư vấn sức khỏe sơ bộ và cá nhân hóa phản hồi cho người dùng [1].

Song song đó, xu hướng xây dựng hệ thống đa tác nhân AI (Multi-Agent) cũng đang phát triển mạnh với các framework như AutoGen, CrewAI và LangGraph [4]-[6]. Các hệ thống này cho phép nhiều AI Agent phối hợp với nhau để xử lý các nhiệm vụ chuyên biệt thay vì chỉ sử dụng một chatbot đơn lẻ. Tuy nhiên, phần lớn các giải pháp quốc tế hiện nay vẫn hướng đến quy mô lớn, yêu cầu hạ tầng công nghệ phức tạp và chi phí triển khai cao.

Qua quá trình khảo sát các nghiên cứu và hệ thống hiện có trong và ngoài nước, có thể nhận thấy vẫn chưa có nhiều giải pháp hướng trực tiếp đến nhóm phòng khám tư nhân quy mô nhỏ, đặc biệt là mô hình chỉ có một bác sĩ vận hành. Các hệ thống hiện nay thường chỉ tập trung vào một chức năng riêng lẻ như quản lý lịch khám hoặc chatbot hỗ trợ cơ bản, chưa kết hợp được khả năng tư vấn thông minh, điều phối vai trò AI và hỗ trợ quản lý vận hành phòng khám trong cùng một nền tảng thống nhất. Nhiều giải pháp vẫn thiếu khả năng quản lý bác sĩ, quản lý tri thức chuyên môn, quản lý lịch làm việc và hỗ trợ phân quyền quản trị phù hợp với mô hình phòng khám tư nhân quy mô nhỏ.

Xuất phát từ khoảng trống đó, đề tài được đề xuất nhằm xây dựng một nền tảng web hỗ trợ kết nối giữa bác sĩ và bệnh nhân với chi phí thấp, dễ triển khai và phù hợp với các phòng khám nhỏ. Hệ thống ứng dụng workflow đa tác nhân AI trên nền tảng n8n với hai nhánh chức năng chính. Nhánh thứ nhất là General AI Agent hoạt động độc lập như một trợ lý sức khỏe tổng quát, hỗ trợ người dùng giải đáp các câu hỏi y tế phổ biến. Nhánh thứ hai là Personal AI Agent dành riêng cho từng bác sĩ, bao gồm ba vai trò AI chuyên biệt: tiếp tân, y tá, trợ lý bác sĩ. Các vai trò này phối hợp linh hoạt trong cùng một khung trò chuyện và hoạt động dựa trên nguồn tri thức chuyên môn do chính bác sĩ cung cấp. Kiến trúc này giúp hệ thống vừa đảm bảo khả năng hỗ trợ sức khỏe cơ bản cho người dùng phổ thông, vừa cho phép cá nhân hóa trải nghiệm tư vấn và đặt lịch theo từng phòng khám hoặc bác sĩ cụ thể.

Bên cạnh đó, theo nghiên cứu “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” của Patrick Lewis và cộng sự [3], kỹ thuật Retrieval-Augmented Generation (RAG) cho phép kết hợp khả năng truy xuất tri thức với mô hình ngôn ngữ lớn nhằm nâng cao độ chính xác và tính cập nhật của phản hồi AI. Trong đề tài này, kỹ thuật RAG được kết hợp với cơ sở dữ liệu vector để giúp AI

phản hồi dựa trên nguồn tri thức chuyên môn do chính bác sĩ cung cấp, từ đó hạn chế hiện tượng tạo thông tin sai lệch của mô hình AI.

1.3. Mục tiêu nghiên cứu

Đề tài hướng đến nghiên cứu và xây dựng hệ thống HealthAssistant nhằm hỗ trợ kết nối giữa bác sĩ và bệnh nhân trên nền tảng web, đồng thời góp phần tối ưu hóa quy trình tư vấn, quản lý và đặt lịch khám cho các phòng khám tư nhân quy mô nhỏ.

Cụ thể, đề tài tập trung thực hiện các mục tiêu sau:

- Phân tích nhu cầu thực tế của các phòng khám tư nhân quy mô nhỏ và nhu cầu tư vấn sức khỏe trực tuyến của người dùng.
- Xây dựng nền tảng web hỗ trợ kết nối giữa bác sĩ và bệnh nhân với các chức năng như quản lý thông tin bác sĩ, quản lý lịch làm việc, quản lý lịch hẹn, quản lý tri thức và hỗ trợ tư vấn trực tuyến.
- Xây dựng cơ chế phân quyền và quản trị hệ thống nhằm hỗ trợ quản lý bác sĩ, kiểm duyệt tài khoản và vận hành nền tảng.
- Nghiên cứu và ứng dụng nền tảng n8n trong việc xây dựng workflow tự động hóa cho hệ thống AI Agent (Artificial Intelligence Agent - AI Agent).
- Thiết kế workflow AI đa tác nhân trên nền tảng n8n với hai nhánh chức năng gồm General AI Agent hỗ trợ tư vấn sức khỏe tổng quát và Personal AI Agent dành riêng cho từng bác sĩ với các vai trò tiếp tân, y tá và trợ lý bác sĩ.
- Ứng dụng kỹ thuật RAG kết hợp cơ sở dữ liệu vector nhằm giúp AI truy xuất và phản hồi dựa trên nguồn tri thức chuyên môn do bác sĩ cung cấp.
- Đánh giá hiệu suất và khả năng hỗ trợ tư vấn, hỗ trợ đặt lịch khám và tính phù hợp của hệ thống trong môi trường phòng khám tư nhân quy mô nhỏ.

1.4. Nhiệm vụ nghiên cứu

Để đạt được các mục tiêu đã đề ra, đề tài tập trung thực hiện các nhiệm vụ nghiên cứu sau:

- Tìm hiểu các công nghệ liên quan đến chatbot y tế, hệ thống đa tác nhân AI, kỹ thuật RAG và cơ sở dữ liệu vector.
- Khảo sát nhu cầu thực tế của các phòng khám tư nhân quy mô nhỏ và quy trình tương tác giữa bác sĩ với bệnh nhân trong hoạt động tư vấn và đặt lịch khám.

- Phân tích và thiết kế kiến trúc nền tảng web hỗ trợ kết nối giữa bác sĩ và bệnh nhân.
- Nghiên cứu và ứng dụng nền tảng n8n trong việc xây dựng workflow tự động hóa cho hệ thống AI.
- Thiết kế workflow AI đa tác nhân với hai nhánh chức năng gồm General AI Agent hỗ trợ giải đáp các câu hỏi sức khỏe phổ biến và Personal AI Agent dành riêng cho từng bác sĩ.
- Xây dựng cơ chế phối hợp giữa các vai trò AI trong Personal AI Agent gồm tiếp tân, y tá và trợ lý bác sĩ nhằm hỗ trợ quy trình tiếp nhận, tư vấn ban đầu và hỗ trợ quy trình đặt lịch khám trong cùng một khung trò chuyện.
- Xây dựng cơ chế quản lý KB cho phép bác sĩ cung cấp và cập nhật dữ liệu chuyên môn phục vụ quá trình tư vấn của AI.
- Ứng dụng kỹ thuật RAG kết hợp cơ sở dữ liệu vector nhằm nâng cao khả năng truy xuất thông tin và độ chính xác trong phản hồi của AI.
- Xây dựng các chức năng chính của hệ thống như quản lý tài khoản, quản lý hồ sơ bác sĩ, quản lý tri thức, trò chuyện trực tuyến, đặt lịch khám và các chức năng quản trị hệ thống.
- Triển khai thử nghiệm và đánh giá hiệu quả hoạt động của hệ thống đối với mô hình phòng khám tư nhân quy mô nhỏ.

1.5. Phạm vi và đối tượng nghiên cứu

1.5.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các công nghệ, mô hình và phương pháp phục vụ xây dựng hệ thống tư vấn sức khỏe và đặt lịch khám trực tuyến tích hợp AI, bao gồm:

- Nền tảng web hỗ trợ kết nối giữa bác sĩ và bệnh nhân.
- Hệ thống chatbot y tế và các mô hình LLM.
- Kiến trúc đa tác nhân AI trong xử lý hội thoại.
- Workflow AI Agent trên nền tảng n8n.
- Kỹ thuật RAG kết hợp cơ sở dữ liệu vector phục vụ truy xuất tri thức.
- Các cơ chế quản lý tri thức, quản lý lịch làm việc, quản trị hệ thống và hỗ trợ quy trình đặt lịch khám.

- Quy trình tư vấn sức khỏe trực tuyến và quản lý tương tác giữa bác sĩ với bệnh nhân tại các phòng khám tư nhân quy mô nhỏ.

1.5.2. Phạm vi nghiên cứu

Phạm vi về nội dung và dịch vụ y tế

Đề tài tập trung nghiên cứu và xây dựng hệ thống hỗ trợ tư vấn sức khỏe sơ bộ và đặt lịch khám trực tuyến cho phòng khám quy mô nhỏ. Hệ thống hỗ trợ các nghiệp vụ như:

- Tiếp nhận thông tin ban đầu từ bệnh nhân.
- Giải đáp các câu hỏi sức khỏe cơ bản.
- Hỗ trợ tìm bác sĩ phù hợp.
- Quản lý lịch làm việc và lịch hẹn khám.
- Hỗ trợ quy trình đặt lịch khám trực tuyến.
- Quản lý KB cho các vai trò trợ lý AI.
- Hỗ trợ các chức năng quản trị hệ thống như quản lý tài khoản và kiểm duyệt hồ sơ bác sĩ.

Đề tài không tập trung vào các chức năng chẩn đoán y khoa chuyên sâu, kê đơn thuốc hoặc thay thế vai trò chuyên môn của bác sĩ trong quá trình điều trị.

Phạm vi về công nghệ

- Nền tảng workflow AI: Sử dụng n8n để xây dựng và vận hành workflow AI.
- Kiến trúc hệ thống: Kết hợp ứng dụng web responsive với workflow automation trên nền tảng n8n nhằm phù hợp với các phòng khám tư nhân quy mô nhỏ, đồng thời không yêu cầu hạ tầng triển khai phức tạp.
- Mô hình triển khai: Hệ thống được triển khai theo kiến trúc Microservices nhằm tăng khả năng mở rộng, tách biệt các dịch vụ chức năng và hỗ trợ bảo trì hệ thống.
- Tích hợp hệ thống: Kết nối giữa giao diện web, backend và workflow AI thông qua API và Webhook.
- Workflow AI: Hệ thống hỗ trợ hai AI Agent chính gồm General AI Agent và Personal AI Agent. Trong đó, Personal AI Agent hỗ trợ ba vai trò trợ lý AI với KB tùy chỉnh theo từng bác sĩ.

- Cơ chế truy xuất tri thức: Sử dụng kỹ thuật RAG kết hợp cơ sở dữ liệu vector nhằm hỗ trợ AI truy xuất dữ liệu tri thức chuyên môn trong quá trình tư vấn.

Phạm vi về khách thể

Đối tượng hướng đến của đề tài là các phòng khám tư nhân quy mô nhỏ, đặc biệt là mô hình chỉ có một bác sĩ vận hành và có nhu cầu hỗ trợ tư vấn sức khỏe trực tuyến, quản lý tương tác với bệnh nhân và hỗ trợ quy trình đặt lịch khám. Ngoài ra, hệ thống cũng hướng đến nhóm người dùng có nhu cầu tìm kiếm thông tin sức khỏe cơ bản và đặt lịch trực tuyến thông qua nền tảng web.

Phạm vi về thời gian thực nghiệm

Đề tài được nghiên cứu, thiết kế và triển khai trong phạm vi thời gian thực hiện khóa luận tốt nghiệp. Quá trình thực nghiệm tập trung vào việc kiểm thử các chức năng của hệ thống, đánh giá khả năng xử lý hội thoại và chất lượng phản hồi của hệ thống AI trong môi trường phát triển cục bộ.

1.6. Phương pháp nghiên cứu

Để xây dựng và triển khai hệ thống HealthAssistant, đề tài được thực hiện dựa trên phương pháp nghiên cứu ứng dụng, kết hợp giữa phương pháp nghiên cứu lý thuyết, phương pháp thiết kế và phát triển hệ thống, cùng với phương pháp thực nghiệm và đánh giá hệ thống. Việc kết hợp nhiều phương pháp nghiên cứu giúp đảm bảo hệ thống không chỉ có cơ sở khoa học và kỹ thuật rõ ràng mà còn đáp ứng được tính thực tiễn, khả năng triển khai và mở rộng trong môi trường thực tế.

1.6.1. Phương pháp nghiên cứu ứng dụng

Phương pháp nghiên cứu ứng dụng được sử dụng nhằm giải quyết bài toán thực tế trong lĩnh vực chăm sóc sức khỏe trực tuyến, đặc biệt là nhu cầu tư vấn sức khỏe sơ bộ và hỗ trợ đặt lịch khám nhanh chóng trên cùng một nền tảng. Đề tài tập trung xây dựng một hệ thống có khả năng hỗ trợ người dùng mô tả triệu chứng sức khỏe thông qua hội thoại tự nhiên với AI, từ đó đưa ra các phản hồi tư vấn ban đầu và hỗ trợ gợi ý hành động phù hợp.

Bên cạnh chức năng tư vấn, hệ thống còn tích hợp chức năng đặt lịch khám trực tuyến với bác sĩ, giúp người dùng có thể chuyển tiếp từ quá trình tư vấn sang đặt lịch trong cùng một quy trình liên tục.

1.6.2. Phương pháp nghiên cứu lý thuyết

Đề tài tiến hành nghiên cứu và tổng hợp các cơ sở lý thuyết liên quan đến hệ thống tư vấn sức khỏe trực tuyến, kiến trúc phần mềm hiện đại và tìm hiểu mô hình AI. Trong đó, nhóm tập trung nghiên cứu mô hình AI Agent kết hợp với cơ chế RAG nhằm nâng cao khả năng phản hồi theo ngữ cảnh và hỗ trợ truy xuất thông tin phù hợp trong quá trình hội thoại với người dùng.

Các kiến thức về PostgreSQL, WebSocket, RabbitMQ, Docker và kiến trúc Microservices cũng được nghiên cứu nhằm xây dựng hệ thống có khả năng giao tiếp realtime, xử lý bất đồng bộ, dễ dàng triển khai và dễ dàng mở rộng.

Ngoài các công nghệ backend và AI, đề tài cũng nghiên cứu các công nghệ phát triển frontend và workflow tự động hóa. ReactJS được sử dụng để xây dựng giao diện người dùng, Zustand hỗ trợ quản lý trạng thái phía client, Tailwind CSS và Ant Design hỗ trợ thiết kế giao diện trực quan và đồng nhất. Bên cạnh đó, nền tảng n8n được sử dụng để xây dựng workflow AI cho hệ thống.

1.6.3. Phương pháp thiết kế và phát triển hệ thống

Đề tài áp dụng các nguyên lý và quy trình kỹ thuật phần mềm để xây dựng sản phẩm.

Kiến trúc phần mềm: Thiết kế theo kiến trúc Microservices nhằm đảm bảo khả năng mở rộng và dễ dàng bảo trì. Các chức năng chính của hệ thống được tách thành nhiều service độc lập bao gồm: user-service, doctor-service, chat-service, appointment-service và notification-service. Việc tách biệt các service giúp hệ thống có thể phát triển, triển khai và nâng cấp từng thành phần một cách độc lập mà không ảnh hưởng đến toàn bộ hệ thống.

Quy trình phát triển: Thực hiện theo hướng Agile - Iterative, tập trung phát triển theo từng chức năng nhỏ, sau đó tiến hành kiểm tra, sửa lỗi và tiếp tục mở rộng chức năng mới. Phương pháp này giúp hệ thống được cải tiến liên tục trong suốt quá trình phát triển, đồng thời hỗ trợ phát hiện và xử lý lỗi sớm trước khi triển khai các chức năng tiếp theo.

1.6.4. Phương pháp thực nghiệm và đánh giá hệ thống

Sau quá trình phát triển, hệ thống được triển khai thử nghiệm nhằm đánh giá hiệu năng, tính ổn định và khả năng đáp ứng của các chức năng chính. Việc đánh giá tập

trung vào hai nhóm nội dung chính gồm khả năng phản hồi của workflow AI và hiệu năng xử lý của hệ thống backend.

Đối với workflow AI, đề tài tiến hành đánh giá chất lượng phản hồi của hệ thống chatbot thông qua các chỉ số BLEU, ROUGE-1 và ROUGE-L bằng cách so sánh phản hồi sinh ra với dữ liệu tham chiếu trong tập kiểm thử. Việc đánh giá nhằm xác định mức độ liên quan và khả năng truy xuất thông tin của hệ thống AI trong quá trình tư vấn sức khỏe.

Đối với hiệu năng hệ thống backend, đề tài sử dụng Postman Collection Runner để mô phỏng khoảng 100 request đồng thời đối với chức năng trọng tâm nhằm đánh giá khả năng xử lý request và thời gian phản hồi của hệ thống trong điều kiện tải thử nghiệm.

Bên cạnh đó, đề tài còn tiến hành kiểm thử giao diện và luồng chức năng người dùng bằng Playwright nhằm đảm bảo tính chính xác của các thao tác trên giao diện web như đăng nhập, quản lý bảng lương và quản lý hồ sơ sức khỏe người dùng. Đồng thời, kiểm thử đơn vị được thực hiện bằng JUnit kết hợp công cụ Diffblue Cover nhằm tự động sinh và thực thi các ca kiểm thử, từ đó kiểm tra tính chính xác của một số hàm xử lý nghiệp vụ tại tầng Service của hệ thống backend.

1.7. Ý nghĩa khoa học và thực tiễn

1.7.1. Ý nghĩa khoa học

Ứng dụng AI trong chăm sóc sức khỏe

Đề tài góp phần làm rõ khả năng ứng dụng AI trong lĩnh vực chăm sóc sức khỏe trực tuyến thông qua việc xây dựng hệ thống HealthAssistant tích hợp AI Agent và cơ chế RAG. Hệ thống sử dụng vector database thông qua nền tảng Pinecone để lưu trữ các dữ liệu như thông tin chuyên khoa, KB của trợ lý AI của mỗi bác sĩ, từ đó hỗ trợ trợ lýAI của bác sĩ phản hồi dựa trên nguồn dữ liệu đã được kiểm soát thay vì trả lời tự do như các chatbot thông thường. Điều này góp phần hạn chế các phản hồi sai lệch và nâng cao tính an toàn trong quá trình tư vấn sức khỏe trực tuyến.

Tích hợp AI vào quy trình nghiệp vụ

Khác với các chatbot truyền thống chỉ dừng ở mức hội thoại, hệ thống tích hợp AI trực tiếp vào quy trình nghiệp vụ chăm sóc sức khỏe. AI hỗ trợ phân tích hình ảnh sức khỏe, phân tích triệu chứng, gợi ý bác sĩ và hỗ trợ quy trình đặt lịch khám trực

tuyến ngay trong quá trình tương tác với người dùng. Điều này cho thấy khả năng kết hợp giữa AI và hệ thống nghiệp vụ trong các nền tảng chăm sóc sức khỏe hiện đại.

1.7.2. Ý nghĩa thực tiễn

Đối với người dùng: Hệ thống giúp người dùng tiếp cận dịch vụ tư vấn sức khỏe và đặt lịch khám trực tuyến trên cùng một nền tảng thống nhất. Người dùng có thể trao đổi triệu chứng với AI để nhận phản hồi tư vấn ban đầu trước khi thực hiện đặt lịch khám với bác sĩ phù hợp.

Đối với bác sĩ: Nền tảng hỗ trợ số hóa quy trình đặt lịch và tương tác với người dùng. Chức năng giao tiếp realtime và gửi thông báo email giúp nâng cao khả năng kết nối giữa bác sĩ và người dùng. Ngoài ra, hệ thống còn tích hợp các trợ lý AI theo từng vai trò như tiếp tân, y tá và trợ lý bác sĩ nhằm hỗ trợ phản hồi các câu hỏi ban đầu, cung cấp thông tin và hỗ trợ quy trình đặt lịch khám, từ đó giúp giảm tải khối lượng tương tác thủ công và hạn chế yêu cầu bác sĩ phải trực tuyến liên tục để phản hồi người dùng.

1.8. Cấu trúc của khóa luận

Nội dung cốt lõi của khóa luận được triển khai qua 5 chương chính, cụ thể như sau:

Chương 1: Mở đầu - Trình bày tính cấp thiết của đề tài xuất phát từ nhu cầu thực tế của các phòng khám tư nhân quy mô nhỏ trong việc tối ưu hóa quy trình vận hành và nâng cao khả năng tiếp cận bệnh nhân. Chương này tập trung làm rõ tổng quan tình hình nghiên cứu trong và ngoài nước liên quan đến y tế số, chatbot y tế và hệ thống đa tác nhân AI. Đồng thời, chương này cũng xác định rõ mục tiêu, nhiệm vụ, phạm vi, đối tượng, phương pháp nghiên cứu và ý nghĩa khoa học cũng như thực tiễn của đề tài.

Chương 2: Cơ sở lý thuyết - Giới thiệu tổng quan về hệ thống hỗ trợ y tế thông minh và các khái niệm nền tảng về Trí tuệ nhân tạo, mô hình LLM và xu hướng phát triển của AI Agents. Chương này đi sâu trình bày lý thuyết về công nghệ tự động hóa luồng công việc với n8n, kiến trúc RAG kết hợp với cơ sở dữ liệu vector nhằm tối ưu hóa tri thức chuyên môn cho AI. Bên cạnh đó, các công nghệ phát triển ứng dụng Website hiện đại, kiến trúc Microservices, công nghệ đóng gói, triển khai hệ thống

cùng các tiêu chuẩn bảo mật, an toàn dữ liệu y khoa và tổng quan các nghiên cứu liên quan cũng được hệ thống hóa trong chương này.

Chương 3: Thiết kế hệ thống - Tập trung vào khâu phân tích yêu cầu hệ thống và đặc tả kiến trúc hệ thống tổng thể theo mô hình Microservices. Nội dung cốt lõi của chương này là việc thiết kế quy trình xử lý của AI Agent và luồng tự động hóa dựa trên nền tảng n8n, cụ thể là cấu trúc hai nhánh chức năng: General AI Agent (tư vấn tổng quát) và Personal AI Agent (ba vai trò riêng biệt: tiếp tân, y tá, trợ lý bác sĩ) phối hợp linh hoạt dựa trên KB do bác sĩ cung cấp. Đồng thời, chương này cũng trình bày thiết kế cơ sở dữ liệu, thiết kế workflow AI và cơ chế xử lý, thiết kế giao diện, trải nghiệm người dùng (UI/UX) cho nền tảng web.

Chương 4: Kiểm thử và đánh giá - Mô tả chi tiết môi trường và thao tác thực tế để triển khai các dịch vụ Microservices, Frontend. Đồng thời, chương này cũng trình bày nội dung về kiểm tra hệ thống bao gồm: tổ chức thực nghiệm và xây dựng các kịch bản kiểm thử nhằm giả lập quy trình tương tác thực tế giữa bệnh nhân với hệ thống. Từ đó, tiến hành đánh giá, phân tích kết quả thu được về khả năng khả năng hỗ trợ tư vấn sức khỏe và chất lượng phản hồi của workflow AI.

Chương 5: Kết luận và hướng phát triển - Tổng hợp lại toàn bộ các kết quả nghiên cứu đã đạt được của đề tài, những đóng góp thiết thực về mặt khoa học và thực tiễn đối với mô hình phòng khám tư nhân quy mô nhỏ. Đồng thời, chương này cũng thẳng thắn chỉ ra những hạn chế còn tồn tại của hệ thống trong phạm vi nghiên cứu và hiện thực đề tài. Qua đó đề xuất các hướng cải tiến, phát triển tiềm năng trong tương lai và đưa ra kết luận chung cho toàn bộ khóa luận.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về hệ thống hỗ trợ y tế thông minh

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ trong lĩnh vực y tế, các hệ thống hỗ trợ y tế thông minh ngày càng đóng vai trò quan trọng trong việc nâng cao khả năng tiếp cận dịch vụ chăm sóc sức khỏe, tối ưu quy trình vận hành và hỗ trợ tương tác giữa bệnh nhân với đội ngũ y tế.

Về bản chất, hệ thống hỗ trợ y tế thông minh là tập hợp các thành phần phần mềm và công nghệ có khả năng tiếp nhận dữ liệu sức khỏe, phân tích thông tin, hỗ trợ tư vấn và đưa ra phản hồi phù hợp cho người dùng. Các hệ thống này thường được triển khai trên nền tảng web, ứng dụng di động hoặc tích hợp với các thiết bị theo dõi sức khỏe, cho phép người dùng trao đổi thông tin sức khỏe từ xa mà không phụ thuộc hoàn toàn vào việc đến trực tiếp cơ sở y tế. Trong những năm gần đây, sự phát triển của AI - đặc biệt là các mô hình LLM - đã tạo điều kiện cho các hệ thống y tế thông minh có khả năng giao tiếp tự nhiên hơn với người dùng, hỗ trợ phân loại triệu chứng ban đầu (triage), giải đáp thắc mắc sức khỏe và hỗ trợ tự động hóa các tác vụ hành chính như đặt lịch và tạo báo cáo.

Bên cạnh chức năng tư vấn, nhiều hệ thống hiện đại còn tích hợp các chức năng như quản lý hồ sơ sức khỏe điện tử, đặt lịch khám trực tuyến, giao tiếp thời gian thực với bác sĩ và theo dõi lịch sử khám bệnh. Việc tích hợp nhiều chức năng trong cùng một nền tảng giúp tạo ra quy trình chăm sóc sức khỏe liền mạch, giảm bớt các thao tác thủ công và nâng cao trải nghiệm người dùng. Điều này đặc biệt có ý nghĩa trong mô hình phòng khám tư nhân quy mô nhỏ, nơi bác sĩ thường phải tự đảm nhiệm nhiều công việc khác nhau - từ tư vấn, tiếp nhận lịch hẹn đến hỗ trợ khách hàng - khi chưa có đủ nguồn lực để duy trì đội ngũ vận hành riêng biệt.

Song song với sự phát triển của AI, xu hướng xây dựng hệ thống y tế thông minh theo kiến trúc Microservices cũng ngày càng phổ biến. Thay vì xây dựng toàn bộ hệ thống dưới dạng một khối duy nhất (monolithic), kiến trúc Microservices cho phép chia hệ thống thành nhiều dịch vụ độc lập như quản lý người dùng, quản lý bác sĩ, trò chuyện trực tuyến, đặt lịch khám và gửi thông báo. Mỗi dịch vụ có thể được phát triển, triển khai và mở rộng riêng biệt, giúp tăng tính linh hoạt, khả năng bảo trì và

khả năng chịu lỗi của hệ thống. Điều này đặc biệt phù hợp với các nền tảng tích hợp AI do khối lượng xử lý và nhu cầu mở rộng có thể thay đổi đáng kể theo thời gian.

Một trong những vấn đề quan trọng trong hệ thống hỗ trợ y tế thông minh là khả năng truy xuất và quản lý tri thức chuyên môn một cách chính xác. Các mô hình AI truyền thống thường phản hồi dựa trên dữ liệu huấn luyện tĩnh và có thể phát sinh hiện tượng "ảo giác" (hallucination) - tức tạo ra thông tin không có căn cứ thực tế. Nghiên cứu của Nishisako và cộng sự trong bài "Reducing Hallucinations and Trade-Offs in Responses in Generative AI Chatbots for Cancer Information" (2025) [12] xác nhận rằng chatbot tích hợp RAG với nguồn thông tin y tế đáng tin cậy tạo ra phản hồi chính xác hơn và ít ảo giác hơn so với mô hình LLM độc lập trong lĩnh vực tư vấn thông tin ung thư.

Để khắc phục hạn chế này, nhiều hệ thống hiện nay áp dụng kỹ thuật RAG, cho phép AI truy xuất thông tin từ nguồn tri thức được kiểm soát trước khi sinh phản hồi. Việc kết hợp RAG với cơ sở dữ liệu vector giúp nâng cao khả năng tìm kiếm thông tin theo ngữ nghĩa và tăng độ chính xác trong quá trình tư vấn sức khỏe.

Nhìn chung, các hệ thống hỗ trợ y tế thông minh đang trở thành xu hướng quan trọng trong lĩnh vực y tế số nhờ khả năng kết hợp giữa AI, dữ liệu và tự động hóa nhằm nâng cao hiệu quả chăm sóc sức khỏe. Việc nghiên cứu và xây dựng hệ thống HealthAssistant không chỉ phù hợp với xu hướng ứng dụng AI trong y tế hiện nay mà còn hướng đến việc đề xuất một mô hình tư vấn sức khỏe khả thi - có thể triển khai thực tế tại các phòng khám tư nhân quy mô nhỏ tại Việt Nam thông qua việc tận dụng các nền tảng đám mây và dịch vụ AI sẵn có.

2.2. Trí tuệ nhân tạo và AI Agents

2.2.1. Tổng quan về trí tuệ nhân tạo

Theo Stuart Russell và Peter Norvig trong tài liệu "Artificial Intelligence: A Modern Approach" [8], trí tuệ nhân tạo là lĩnh vực thuộc khoa học máy tính, tập trung vào việc xây dựng các hệ thống có khả năng mô phỏng những hành vi thông minh của con người như học tập, suy luận, nhận thức, xử lý ngôn ngữ tự nhiên và ra quyết định. AI hướng đến việc phát triển các tác nhân (agents) có khả năng cảm nhận môi trường, phân tích dữ liệu và thực hiện hành động nhằm đạt được mục tiêu cụ thể.

Sự phát triển của AI đã trải qua nhiều giai đoạn, từ các hệ chuyên gia dựa trên luật (rule-based systems) đến các phương pháp học máy và học sâu. Theo Ian Goodfellow, Yoshua Bengio và Aaron Courville trong cuốn "Deep Learning" [9], học máy cho phép hệ thống tự động học từ dữ liệu thay vì phụ thuộc hoàn toàn vào các quy tắc được lập trình thủ công, trong khi học sâu sử dụng các mạng nơ-ron nhiều lớp để giải quyết các bài toán phức tạp như nhận dạng hình ảnh, giọng nói và xử lý ngôn ngữ tự nhiên.

Hiện nay, một trong những hướng phát triển nổi bật của AI là các mô hình LLM. Theo nghiên cứu "Language Models are Few-Shot Learners" của Tom B. Brown và cộng sự [2], các mô hình này được huấn luyện trên lượng dữ liệu văn bản khổng lồ bằng kỹ thuật học sâu, cho phép máy tính hiểu ngữ cảnh, sinh văn bản và thực hiện nhiều tác vụ chỉ với một số lượng nhỏ ví dụ đầu vào. Các mô hình tiêu biểu hiện nay bao gồm OpenAI GPT series, Anthropic Claude và Google Gemini. Nhờ khả năng xử lý ngôn ngữ linh hoạt, các mô hình LLM đang được ứng dụng rộng rãi trong giáo dục, tài chính, chăm sóc khách hàng và đặc biệt là y tế.

Trong lĩnh vực y tế, theo Eric Topol trong cuốn "Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again" [10], AI có khả năng hỗ trợ phân tích dữ liệu bệnh án, nhận diện triệu chứng, hỗ trợ chẩn đoán và cung cấp tư vấn sơ bộ cho người dùng. Các mô hình LLM có thể tiếp nhận mô tả triệu chứng dưới dạng ngôn ngữ tự nhiên, phân tích ngữ cảnh và đưa ra phản hồi phù hợp, góp phần nâng cao trải nghiệm tương tác giữa người dùng và hệ thống, đồng thời hỗ trợ giảm tải cho nhân viên y tế trong các tác vụ sàng lọc và cung cấp thông tin ban đầu.

2.2.2. Khái niệm AI Agents

Theo Stuart Russell và Peter Norvig trong tài liệu "Artificial Intelligence: A Modern Approach" [8], AI Agent là một hệ thống trí tuệ nhân tạo có khả năng cảm nhận môi trường, phân tích dữ liệu, đưa ra quyết định và thực hiện hành động nhằm đạt được mục tiêu xác định. Khác với các hệ thống AI truyền thống chỉ phản hồi thụ động dựa trên dữ liệu đầu vào, AI Agent có mức độ tự chủ cao hơn nhờ khả năng lập kế hoạch, ghi nhớ ngữ cảnh và phối hợp nhiều bước xử lý để hoàn thành nhiệm vụ.

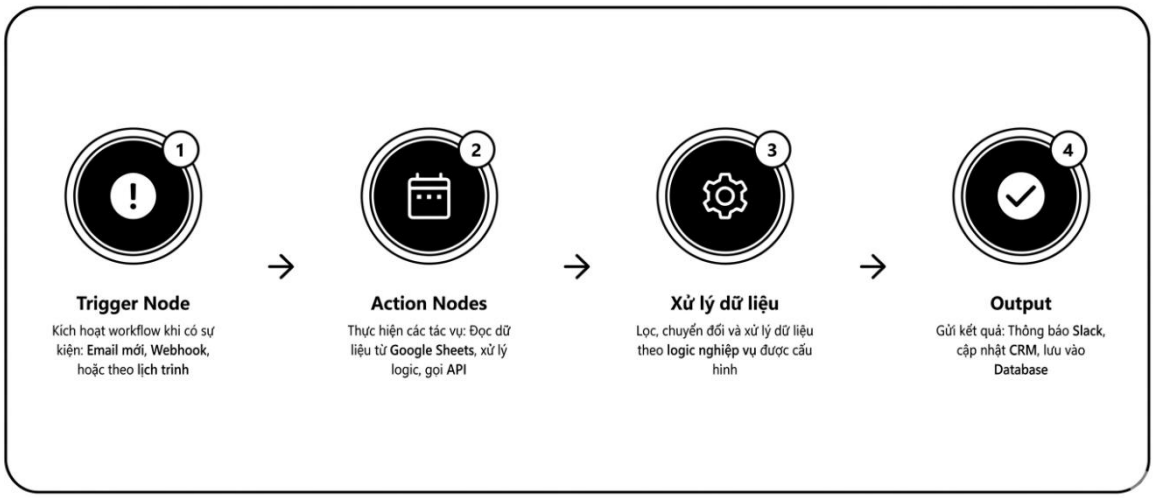
Sự phát triển của các mô hình LLM đã mở rộng đáng kể khả năng của AI Agent. Theo bài viết "LLM Powered Autonomous Agents" của Lilian Weng [11], các agent hiện đại không chỉ có khả năng sinh phản hồi bằng ngôn ngữ tự nhiên mà còn có thể

sử dụng các công cụ bên ngoài, truy xuất KB, gọi API và tương tác với các hệ thống khác để xử lý các tác vụ phức tạp. Điều này giúp AI Agent hoạt động linh hoạt hơn so với chatbot truyền thống, đồng thời nâng cao khả năng hỗ trợ người dùng trong nhiều lĩnh vực chuyên môn như y tế, giáo dục và tài chính.

Trong lĩnh vực y tế, AI Agent có thể hỗ trợ tiếp nhận thông tin triệu chứng từ người dùng, truy xuất dữ liệu y khoa, hỗ trợ sàng lọc ban đầu và cung cấp thông tin tham khảo phù hợp với ngữ cảnh bệnh nhân. Ngoài ra, AI Agent còn có khả năng phối hợp với các hệ thống khác như cơ sở dữ liệu bệnh án, hệ thống đặt lịch khám hoặc nền tảng quản lý phòng khám nhằm nâng cao hiệu quả hỗ trợ chăm sóc sức khỏe. Nhờ khả năng tự động hóa quy trình và hỗ trợ ra quyết định, AI Agent góp phần giảm tải cho nhân viên y tế trong các tác vụ hỗ trợ thông tin và nâng cao trải nghiệm tương tác của người dùng với hệ thống chăm sóc sức khỏe thông minh.

2.3. Công nghệ tự động hóa luồng công việc với n8n

2.3.1. Giới thiệu về n8n



Hình 2.1. Minh họa hoạt động của n8n

n8n là một nền tảng tự động hóa quy trình công việc (workflow automation) mã nguồn mở, được thiết kế theo mô hình dựa trên nút (node-based workflow). Nền tảng này cho phép người dùng xây dựng và quản lý các quy trình xử lý thông qua giao diện kéo - thả trực quan, hỗ trợ kết nối giữa nhiều hệ thống, dịch vụ, cơ sở dữ liệu và API mà không yêu cầu lập trình phức tạp.

Khác với nhiều nền tảng tự động hóa thương mại, n8n hỗ trợ triển khai theo mô hình self-hosted, cho phép hệ thống được cài đặt và vận hành trên máy chủ riêng của

tổ chức. Điều này giúp tăng khả năng kiểm soát dữ liệu và hạ tầng vận hành, đặc biệt phù hợp với các hệ thống xử lý dữ liệu nhạy cảm như thông tin y tế và hồ sơ bệnh nhân.

Ngoài khả năng tích hợp đa dạng dịch vụ, n8n còn hỗ trợ xây dựng các luồng xử lý logic phức tạp thông qua hàng trăm node có sẵn, bao gồm xử lý dữ liệu, điều kiện rẽ nhánh, gọi API, truy vấn cơ sở dữ liệu và tích hợp các mô hình AI hiện đại. Nhờ đó, n8n có thể đóng vai trò trung tâm điều phối trong các hệ thống ứng dụng thông minh, hỗ trợ tự động hóa nhiều tác vụ nghiệp vụ khác nhau.

Trong lĩnh vực y tế, n8n có thể được sử dụng để xây dựng các workflow tự động như tiếp nhận thông tin bệnh nhân, xử lý dữ liệu tư vấn sức khỏe, kết nối hệ thống đặt lịch khám và phối hợp với các mô hình AI nhằm hỗ trợ cung cấp thông tin y khoa. Việc sử dụng nền tảng workflow automation giúp tăng khả năng mở rộng hệ thống, giảm khối lượng thao tác thủ công và nâng cao hiệu quả vận hành trong các cơ sở y tế quy mô nhỏ.

2.3.2. Ứng dụng n8n làm “bộ não” điều phối

Trong các hệ thống ứng dụng AI hiện đại, việc điều phối luồng xử lý giữa nhiều thành phần như LLM, cơ sở dữ liệu, API và KB đóng vai trò quan trọng nhằm đảm bảo khả năng xử lý linh hoạt và tự động hóa quy trình nghiệp vụ. n8n cung cấp cơ chế xây dựng workflow trực quan dựa trên node, hỗ trợ kết nối và điều phối các thành phần này trong cùng một hệ thống.

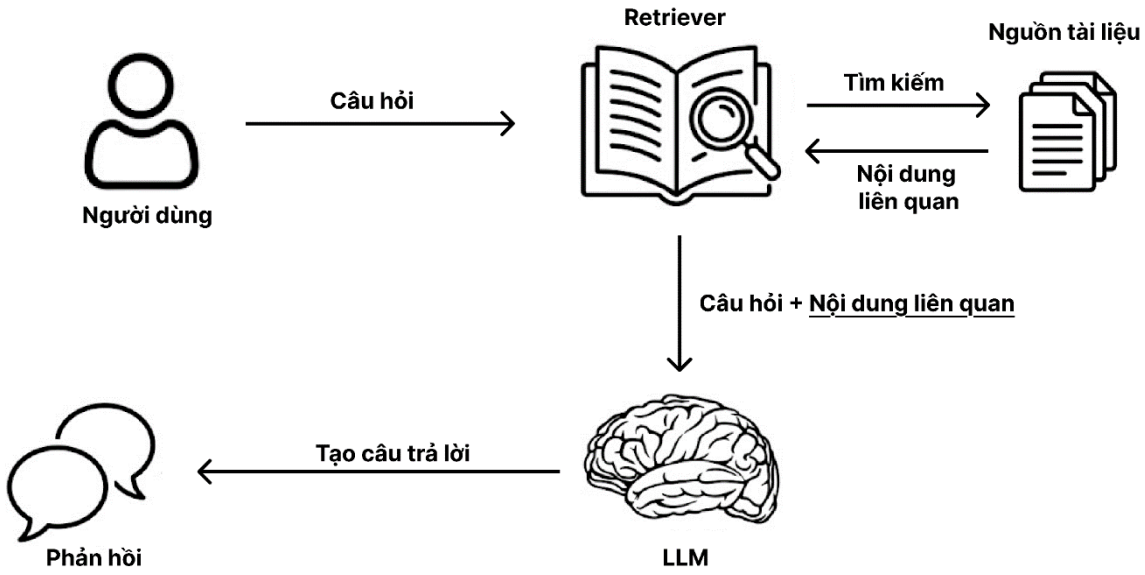
Thông qua các workflow xử lý tự động, n8n cho phép hệ thống tiếp nhận dữ liệu đầu vào, phân luồng xử lý theo điều kiện, gọi API, truy xuất dữ liệu và trả kết quả về cho hệ thống chính. Ngoài ra, nền tảng này còn hỗ trợ tích hợp với các mô hình AI, cơ sở tri thức và các hệ thống lưu trữ ngữ cảnh hội thoại nhằm nâng cao khả năng xử lý thông tin theo ngữ cảnh.

Trong lĩnh vực ứng dụng AI cho y tế, n8n có thể được sử dụng như một lớp điều phối trung gian giữa các thành phần của hệ thống như AI Agent, cơ sở dữ liệu bệnh nhân, hệ thống đặt lịch khám và cơ sở tri thức y khoa. Nền tảng này hỗ trợ xây dựng các workflow phục vụ tiếp nhận triệu chứng, truy xuất thông tin y khoa, hỗ trợ tìm bác sĩ phù hợp và phối hợp xử lý giữa nhiều AI Agent khác nhau tùy theo loại yêu cầu của người dùng.

Việc sử dụng n8n giúp hệ thống dễ dàng mở rộng, thuận tiện trong quá trình phát triển và bảo trì, đồng thời nâng cao khả năng tích hợp giữa các thành phần AI và các dịch vụ nghiệp vụ trong hệ thống chăm sóc sức khỏe thông minh.

2.4. Kiến trúc truy xuất thông tin tăng cường (RAG)

2.4.1. Khái niệm RAG



Hình 2.2. Minh họa về kiến trúc truy xuất thông tin tăng cường

Theo nghiên cứu "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" của Patrick Lewis và cộng sự [3], Retrieval-Augmented Generation (RAG) là kiến trúc kết hợp giữa khả năng sinh văn bản của mô hình ngôn ngữ lớn và cơ chế truy xuất thông tin từ các nguồn dữ liệu bên ngoài. Thay vì chỉ dựa trên tri thức đã được huấn luyện sẵn trong mô hình, RAG cho phép hệ thống truy xuất các tài liệu liên quan từ cơ sở tri thức trước khi tạo phản hồi.

Cơ chế này giúp cải thiện độ chính xác và tính cập nhật của thông tin, đồng thời giảm hiện tượng "ảo giác" - tình trạng mô hình sinh ra thông tin không chính xác hoặc không có căn cứ. Nhờ khả năng bổ sung ngữ cảnh từ nguồn dữ liệu đáng tin cậy, RAG được ứng dụng rộng rãi trong các hệ thống hỏi đáp, trợ lý AI và các bài toán yêu cầu độ tin cậy cao về thông tin.

Trong lĩnh vực y tế, RAG đóng vai trò quan trọng vì các phản hồi của hệ thống cần dựa trên tài liệu chuyên môn, hướng dẫn y khoa và tri thức được xác thực. Việc kết hợp cơ chế truy xuất tri thức với mô hình LLM giúp nâng cao chất lượng tư vấn và giảm nguy cơ cung cấp thông tin sai lệch cho người dùng.

2.4.2. Quy trình hoạt động của RAG

Quy trình hoạt động của hệ thống RAG thường bao gồm ba giai đoạn chính:

- **Indexing (Đánh chỉ mục):** Các tài liệu, hướng dẫn hoặc dữ liệu tri thức được chuyển đổi thành vector thông qua mô hình nhúng. Các vector này sau đó được lưu trữ trong cơ sở dữ liệu vector nhằm hỗ trợ tìm kiếm theo ngữ nghĩa.
- **Retrieval (Truy xuất):** Khi người dùng gửi câu hỏi, hệ thống sẽ chuyển nội dung câu hỏi thành vector và thực hiện tìm kiếm trong cơ sở dữ liệu vector để lấy ra các tài liệu hoặc đoạn văn bản có mức độ liên quan cao nhất.
- **Generation (Sinh phản hồi):** Các nội dung được truy xuất sẽ được đưa vào làm ngữ cảnh cho mô hình LLM. Dựa trên thông tin này, mô hình sẽ tạo ra câu trả lời phù hợp với truy vấn của người dùng.

Trong lĩnh vực y tế, quy trình RAG giúp hệ thống AI có thể truy xuất thông tin từ các tài liệu chuyên môn, hướng dẫn điều trị hoặc cơ sở tri thức y khoa trước khi tạo phản hồi. Điều này góp phần nâng cao độ chính xác của nội dung tư vấn, hỗ trợ cung cấp thông tin phù hợp với ngữ cảnh và hạn chế nguy cơ sinh ra thông tin sai lệch.

2.5. Phát triển ứng dụng web với kiến trúc hiện đại

2.5.1. Tổng quan

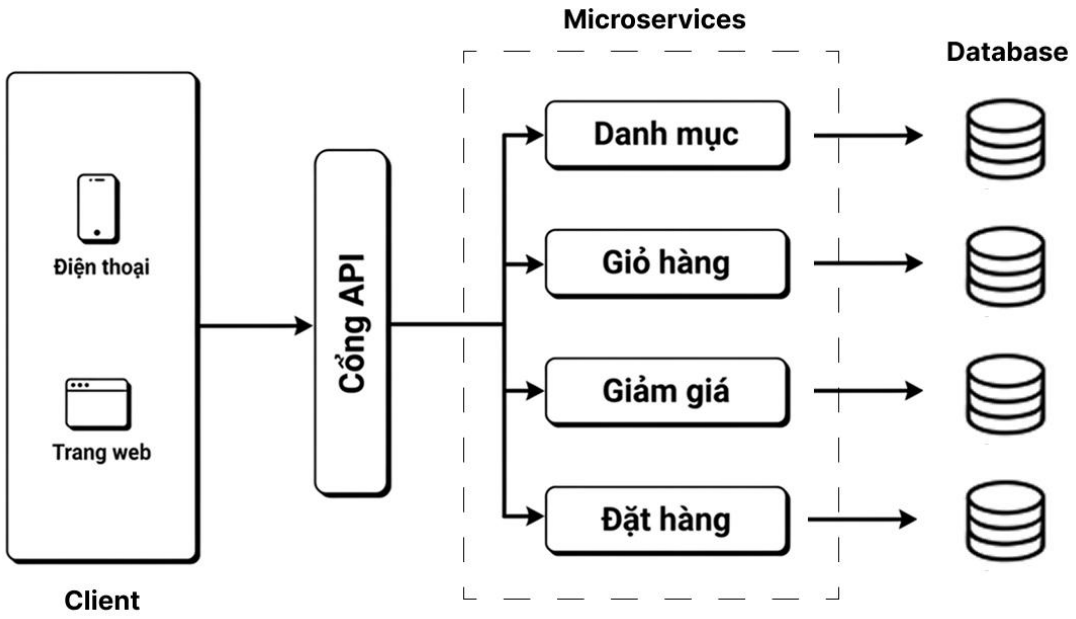
Trong bối cảnh chuyển đổi số và sự phát triển mạnh mẽ của các hệ thống trực tuyến, ứng dụng web hiện đại ngày càng yêu cầu khả năng xử lý linh hoạt, hiệu năng cao và dễ dàng mở rộng. Đặc biệt đối với các hệ thống tích hợp AI, xử lý dữ liệu thời gian thực hoặc phục vụ số lượng lớn người dùng đồng thời, mô hình phát triển web truyền thống dần bộc lộ nhiều hạn chế về khả năng mở rộng và bảo trì.

Trước đây, nhiều hệ thống web được xây dựng theo kiến trúc Monolithic, trong đó toàn bộ chức năng được triển khai trong một khối thống nhất. Mặc dù mô hình này phù hợp với các hệ thống nhỏ, nhưng khi quy mô hệ thống tăng lên, việc cập nhật hoặc mở rộng chức năng thường gặp khó khăn do các thành phần phụ thuộc chặt chẽ vào nhau.

Để khắc phục các hạn chế đó, các hệ thống hiện đại có xu hướng chuyển sang sử dụng kiến trúc phân tán và hướng dịch vụ nhằm tăng tính linh hoạt trong phát triển và triển khai hệ thống. Trong đó, kiến trúc Microservices, giao tiếp realtime và xử lý

bất đồng bộ là những hướng tiếp cận được sử dụng phổ biến trong các nền tảng web hiện đại.

2.5.2. Kiến trúc Microservices



Hình 2.3. Minh họa về Kiến trúc Microservices qua hệ thống mua sắm trực tuyến

Kiến trúc Microservices là mô hình thiết kế phần mềm trong đó hệ thống được chia thành nhiều dịch vụ nhỏ độc lập. Mỗi dịch vụ đảm nhiệm một nhóm chức năng riêng biệt và có thể được phát triển, triển khai hoặc mở rộng độc lập với các dịch vụ khác.

Khác với mô hình Monolithic, Microservices hướng đến việc giảm sự phụ thuộc giữa các thành phần trong hệ thống. Các service thường giao tiếp với nhau thông qua REST API hoặc Message Broker, giúp tăng tính linh hoạt trong quá trình vận hành và bảo trì.

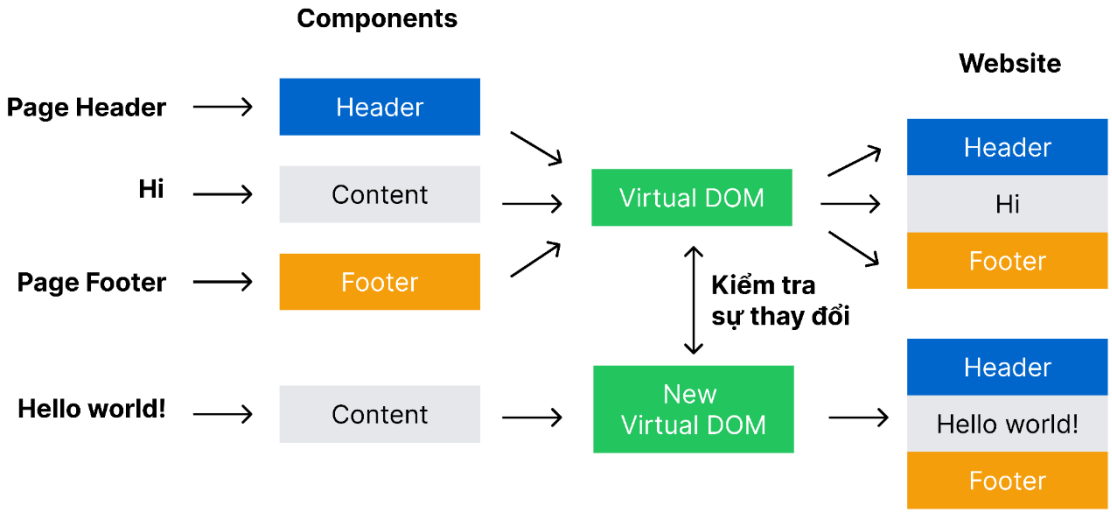
Một số ưu điểm nổi bật của kiến trúc Microservices gồm:

- Hỗ trợ mở rộng hệ thống linh hoạt theo từng chức năng
- Giảm ảnh hưởng dây chuyền khi xảy ra lỗi
- Dễ bảo trì và nâng cấp hệ thống
- Tăng khả năng triển khai độc lập
- Phù hợp với mô hình phát triển Agile

Bên cạnh ưu điểm, kiến trúc này cũng đặt ra một số thách thức như:

- Quản lý giao tiếp giữa các service
- Đồng bộ dữ liệu phân tán
- Quản lý bảo mật giữa các thành phần
- Giám sát và triển khai hệ thống phức tạp hơn

2.5.3. Phát triển giao diện người dùng với ReactJS



Hình 2.4. Minh họa về cách thay đổi thành phần của ReactJS

Trong các ứng dụng web hiện đại, giao diện người dùng đóng vai trò quan trọng trong việc nâng cao trải nghiệm tương tác và khả năng sử dụng hệ thống. Các nền tảng web hiện nay thường yêu cầu giao diện có khả năng cập nhật dữ liệu nhanh, phản hồi linh hoạt và hỗ trợ tương tác liên tục với người dùng.

ReactJS là thư viện JavaScript phổ biến được sử dụng rộng rãi trong phát triển frontend nhờ mô hình component-based. Theo mô hình này, giao diện được chia thành nhiều thành phần nhỏ có thể tái sử dụng, giúp việc phát triển và quản lý mã nguồn trở nên thuận lợi hơn.

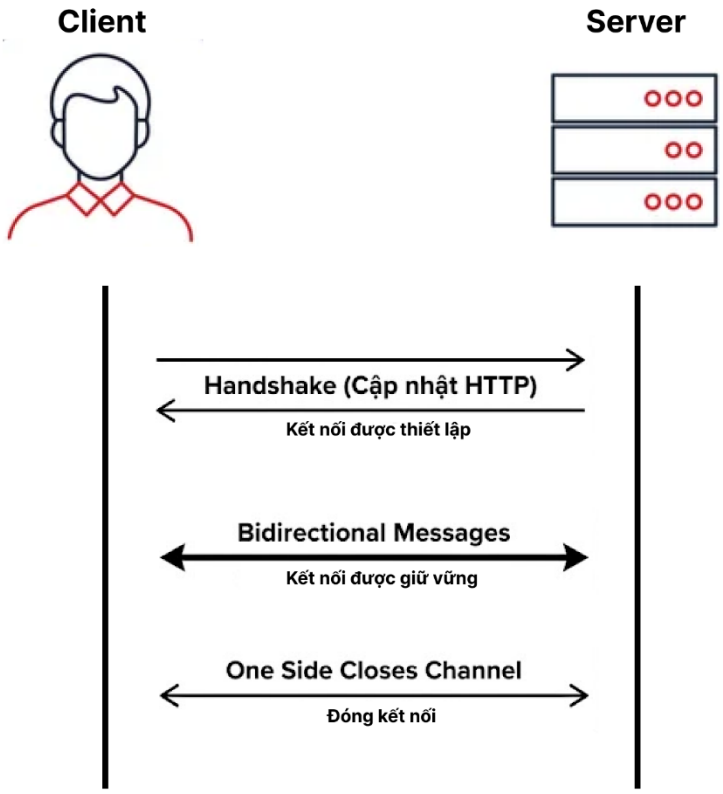
Ngoài khả năng tái sử dụng component, ReactJS còn sử dụng cơ chế Virtual DOM nhằm tối ưu hiệu năng hiển thị. Khi dữ liệu thay đổi, React chỉ cập nhật các thành phần cần thiết thay vì tải lại toàn bộ trang, từ đó giảm thời gian phản hồi và cải thiện trải nghiệm người dùng [17].

Một số ưu điểm của ReactJS [17]:

- Hỗ trợ xây dựng giao diện tương tác cao
- Tăng khả năng tái sử dụng mã nguồn

- Tối ưu hiệu năng hiển thị
- Dễ tích hợp với REST API hoặc WebSocket
- Có hệ sinh thái lớn và cộng đồng phát triển mạnh

2.5.4. Giao tiếp thời gian thực với WebSocket



Hình 2.5. Minh họa về vòng đời của giao tiếp thời gian thực

Trong các hệ thống hỗ trợ trao đổi trực tuyến, cơ chế giao tiếp thời gian thực đóng vai trò quan trọng nhằm đảm bảo dữ liệu được truyền tải liên tục và giảm độ trễ trong quá trình tương tác.

HTTP truyền thống hoạt động theo mô hình request-response, trong đó client phải liên tục gửi request để cập nhật dữ liệu mới từ server. Cách tiếp cận này có thể làm tăng số lượng request và gây ảnh hưởng đến hiệu năng hệ thống trong các ứng dụng realtime.

Để giải quyết vấn đề này, WebSocket được phát triển nhằm hỗ trợ kết nối hai chiều liên tục giữa client và server. Sau khi kết nối được thiết lập, dữ liệu có thể được gửi và nhận ngay lập tức mà không cần tạo kết nối mới cho mỗi lần truyền tải.

Một số ưu điểm của WebSocket:

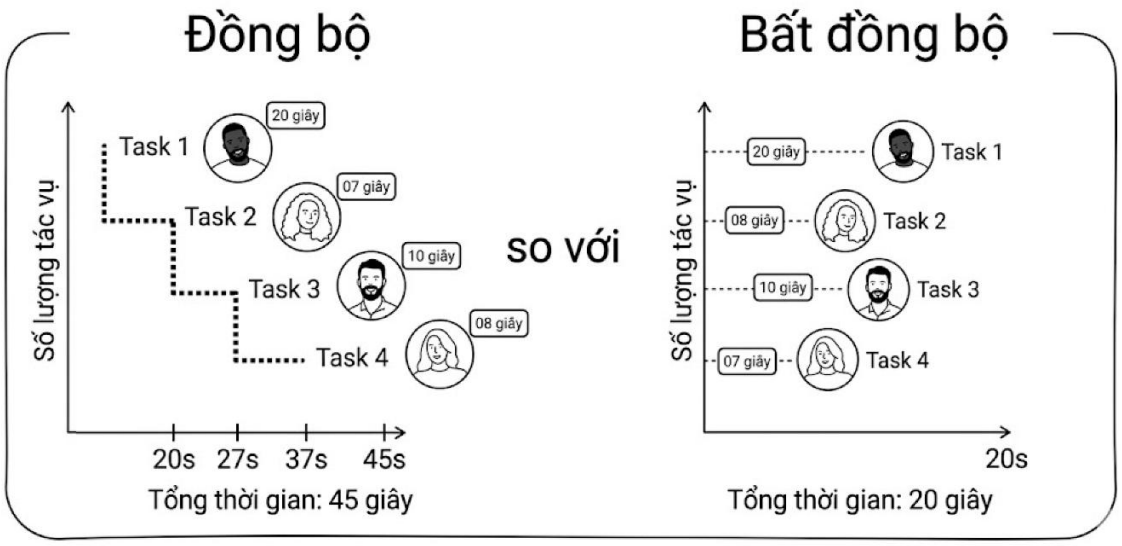
- Giảm độ trễ khi truyền dữ liệu
- Hỗ trợ giao tiếp hai chiều liên tục
- Giảm số lượng request HTTP
- Tăng hiệu quả cho các hệ thống realtime

Hiện nay, WebSocket được sử dụng phổ biến trong:

- Hệ thống chat trực tuyến
- Ứng dụng livestream
- Theo dõi dữ liệu realtime
- Hệ thống thông báo trực tiếp

Trong các nền tảng tư vấn sức khỏe trực tuyến, việc áp dụng WebSocket giúp nâng cao trải nghiệm hội thoại giữa người dùng với hệ thống hoặc đội ngũ hỗ trợ chuyên môn.

2.5.5. Xử lý bất đồng bộ



Hình 2.6. So sánh đồng bộ và bất đồng bộ

Trong các hệ thống web hiện đại, việc xử lý toàn bộ tác vụ theo cơ chế đồng bộ có thể làm tăng thời gian phản hồi và giảm hiệu năng hệ thống. Đối với các tác vụ không yêu cầu phản hồi ngay lập tức như gửi email, thông báo hoặc xử lý nền, cơ chế bất đồng bộ thường được sử dụng nhằm giảm tải cho backend.

Message Queue là mô hình trung gian cho phép các service giao tiếp với nhau thông qua hàng đợi thông điệp. Thay vì xử lý trực tiếp tác vụ, service sẽ gửi message vào queue và một service khác sẽ tiếp nhận và xử lý sau đó.

Trong các hệ thống tích hợp nhiều dịch vụ độc lập, cơ chế Message Queue đóng vai trò quan trọng trong việc đảm bảo khả năng giao tiếp linh hoạt giữa các thành phần.

2.5.6. Redis và cơ chế lưu trữ tạm thời

Redis là hệ quản trị dữ liệu dạng in-memory hoạt động với tốc độ truy xuất cao. Khác với cơ sở dữ liệu quan hệ truyền thống, Redis tập trung vào khả năng xử lý nhanh các dữ liệu tạm thời hoặc dữ liệu cần truy cập thường xuyên [21].

Redis thường được sử dụng trong:

- Cache dữ liệu
- Quản lý session
- Lưu trạng thái realtime
- Hỗ trợ xử lý dữ liệu tạm thời

Nhờ cơ chế lưu trữ trên bộ nhớ RAM, Redis có khả năng phản hồi nhanh hơn đáng kể so với các hệ quản trị dữ liệu lưu trữ trên ổ đĩa. Điều này giúp cải thiện hiệu năng cho các hệ thống có yêu cầu realtime hoặc số lượng request lớn [21].

2.6. Công nghệ đóng gói và triển khai hệ thống

2.6.1. Tổng quan

Trong các hệ thống phần mềm hiện đại, đặc biệt là các hệ thống phân tán và kiến trúc Microservices, việc đóng gói và triển khai ứng dụng đóng vai trò quan trọng nhằm đảm bảo tính nhất quán giữa các môi trường phát triển, kiểm thử và vận hành thực tế.

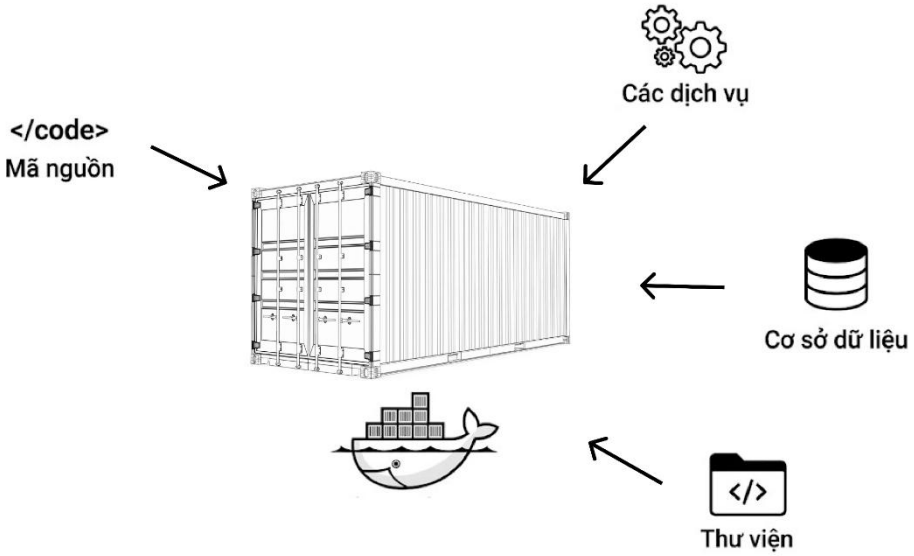
Trước đây, quá trình triển khai phần mềm thường phụ thuộc nhiều vào cấu hình thủ công của máy chủ, dẫn đến các vấn đề như:

- Khác biệt môi trường giữa các máy
- Khó tái sử dụng cấu hình triển khai
- Phát sinh lỗi phụ thuộc thư viện
- Khó mở rộng và bảo trì hệ thống

Để giải quyết các hạn chế này, xu hướng hiện nay là sử dụng công nghệ containerization nhằm đóng gói toàn bộ ứng dụng cùng môi trường chạy vào một thực thể độc lập. Cách tiếp cận này giúp hệ thống có khả năng triển khai linh hoạt và đảm bảo tính ổn định trên nhiều môi trường khác nhau.

Bên cạnh đó, các hệ thống hiện đại còn áp dụng mô hình triển khai tự động và hạ tầng phân tán nhằm nâng cao khả năng mở rộng, tính sẵn sàng và hiệu quả vận hành.

2.6.2. Công nghệ Docker trong triển khai ứng dụng



Hình 2.7. Minh họa đóng gói ứng dụng được quản lý bởi Docker.

Docker là nền tảng containerization phổ biến được sử dụng rộng rãi trong phát triển và triển khai phần mềm hiện nay. Docker cho phép đóng gói ứng dụng cùng toàn bộ thư viện, dependency và môi trường thực thi vào một container độc lập [16].

Khác với máy ảo truyền thống, container hoạt động nhẹ hơn do chia sẻ kernel của hệ điều hành host thay vì khởi tạo toàn bộ hệ điều hành riêng biệt. Điều này giúp:

- Giảm tài nguyên tiêu thụ
- Tăng tốc độ khởi động ứng dụng
- Dễ dàng triển khai trên nhiều môi trường
- Đảm bảo tính nhất quán khi vận hành

Mỗi container thường chỉ đảm nhiệm một chức năng hoặc service riêng biệt, phù hợp với kiến trúc Microservices. Các service có thể được triển khai, cập nhật hoặc mở rộng độc lập mà không ảnh hưởng đến toàn bộ hệ thống.

Một số ưu điểm nổi bật của Docker [16]:

- Tính di động cao giữa các môi trường
- Hỗ trợ triển khai nhanh chóng
- Dễ quản lý dependency
- Giảm lỗi khác biệt môi trường
- Phù hợp với CI/CD (Continuous Integration/Continuous Deployment - CI/CD)

Hiện nay Docker được sử dụng phổ biến trong:

- Hệ thống Microservices
- Cloud-native application
- DevOps workflow
- AI và Machine Learning platform

Nhờ khả năng đóng gói linh hoạt và tính ổn định cao, Docker trở thành một trong những công nghệ quan trọng trong quá trình triển khai các hệ thống hiện đại.

2.6.3. Docker Compose và quản lý đa container

Trong các hệ thống phân tán, ứng dụng thường bao gồm nhiều service hoạt động đồng thời như backend, frontend, database, cache hoặc message broker. Việc quản lý và khởi chạy từng container riêng lẻ có thể gây khó khăn trong quá trình phát triển và triển khai.

Docker Compose được xây dựng nhằm hỗ trợ quản lý nhiều container thông qua một tệp cấu hình tập trung. Thông qua file docker-compose.yml, lập trình viên có thể định nghĩa [16]:

- Danh sách service
- Network giữa các container
- Volume lưu trữ dữ liệu
- Environment variables
- Cổng kết nối hệ thống

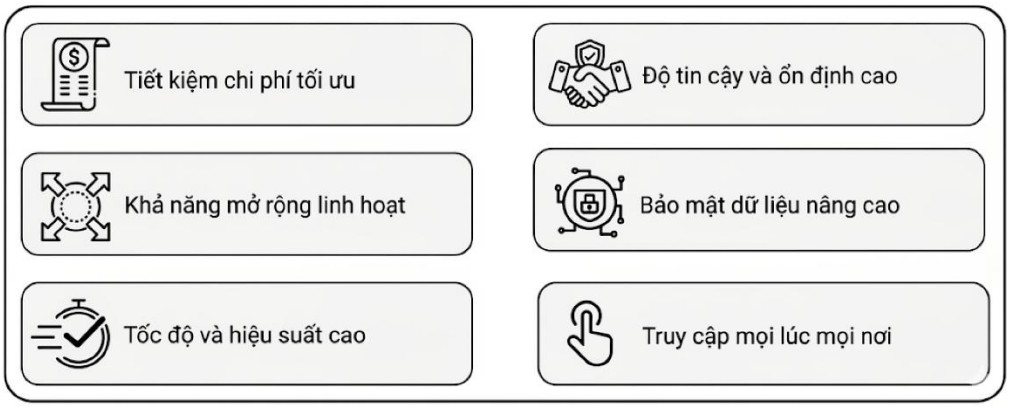
Việc sử dụng Docker Compose mang lại nhiều lợi ích:

- Đơn giản hóa quá trình triển khai
- Tự động hóa cấu hình hệ thống
- Dễ tái sử dụng môi trường

- Thuận lợi trong phát triển Microservices

Ngoài ra, Docker Compose còn hỗ trợ việc mở rộng và kiểm thử hệ thống trong môi trường local trước khi triển khai thực tế.

2.6.4. Triển khai hệ thống trên nền tảng Cloud



Hình 2.8. Các lợi ích nổi bật khi sử dụng nền tảng Cloud

Sự phát triển của điện toán đám mây đã tạo điều kiện cho các hệ thống hiện đại có khả năng triển khai linh hoạt và mở rộng theo nhu cầu thực tế.

Khác với mô hình máy chủ vật lý truyền thống, Cloud Computing cho phép:

- Triển khai hệ thống nhanh chóng
- Mở rộng tài nguyên linh hoạt
- Tăng tính sẵn sàng của hệ thống
- Giảm chi phí đầu tư hạ tầng ban đầu
- Bớt thao tác cấu hình bảo mật thủ công do các nền tảng Cloud sẽ hỗ trợ điều này

Hiện nay, nhiều nền tảng cloud phổ biến hỗ trợ triển khai ứng dụng container như:

- Amazon Web Services (AWS)
- Google Cloud Platform
- Microsoft Azure
- DigitalOcean

Đối với các hệ thống Microservices, môi trường Cloud giúp việc triển khai và quản lý các service trở nên linh hoạt hơn nhờ khả năng:

- Auto scaling

- Load balancing
- Monitoring hệ thống
- Quản lý tài nguyên động

2.6.5. CI/CD và tự động hóa triển khai

Trong quy trình phát triển phần mềm hiện đại, CI và CD đóng vai trò quan trọng trong việc tự động hóa kiểm thử và triển khai ứng dụng. Quy trình CI/CD mang lại nhiều giá trị như:

- Tự động build và kiểm thử mã nguồn
- Giảm lỗi triển khai thủ công
- Tăng tốc độ phát hành phiên bản mới
- Hỗ trợ cập nhật hệ thống liên tục

Quy trình CI/CD thường bao gồm:

- Sau khi Commit/Push lên mã nguồn
- Tự động build ứng dụng
- Chạy kiểm thử đã viết sẵn (nếu có)
- Đóng gói Docker Image và đẩy lên kho chứa tập trung (Docker Hub, GitHub Packages).
- Tự động kết nối với máy chủ, kéo Docker Image, dừng phiên bản hiện tại và chạy phiên bản mới.

Việc kết hợp Docker với CI/CD giúp đảm bảo tính nhất quán giữa môi trường phát triển và môi trường production, đồng thời tăng khả năng tự động hóa trong quá trình vận hành hệ thống.

2.7. Kiểm thử phần mềm và các công cụ kiểm thử

2.7.1. Tổng quan về kiểm thử phần mềm

Kiểm thử phần mềm là quá trình đánh giá và xác minh hệ thống nhằm đảm bảo phần mềm hoạt động đúng với yêu cầu đặt ra, phát hiện lỗi và nâng cao chất lượng hệ thống trước khi triển khai thực tế. Trong quá trình phát triển phần mềm hiện đại, kiểm thử đóng vai trò quan trọng trong việc đảm bảo tính ổn định, khả năng mở rộng và độ tin cậy của ứng dụng.

Kiểm thử phần mềm có thể được thực hiện ở nhiều mức khác nhau như kiểm thử đơn vị, kiểm thử tích hợp, kiểm thử hệ thống, kiểm thử API và kiểm thử giao diện người dùng. Việc kết hợp nhiều phương pháp kiểm thử giúp đánh giá hệ thống một cách toàn diện từ tầng xử lý nghiệp vụ đến giao diện tương tác với người dùng.

Ngoài ra, kiểm thử phần mềm còn hỗ trợ giảm chi phí bảo trì, phát hiện lỗi sớm trong quá trình phát triển và nâng cao trải nghiệm người dùng. Trong các mô hình phát triển hiện đại như Agile và DevOps, kiểm thử thường được tích hợp xuyên suốt vòng đời phát triển phần mềm nhằm đảm bảo chất lượng sản phẩm liên tục.

Vai trò của kiểm thử phần mềm

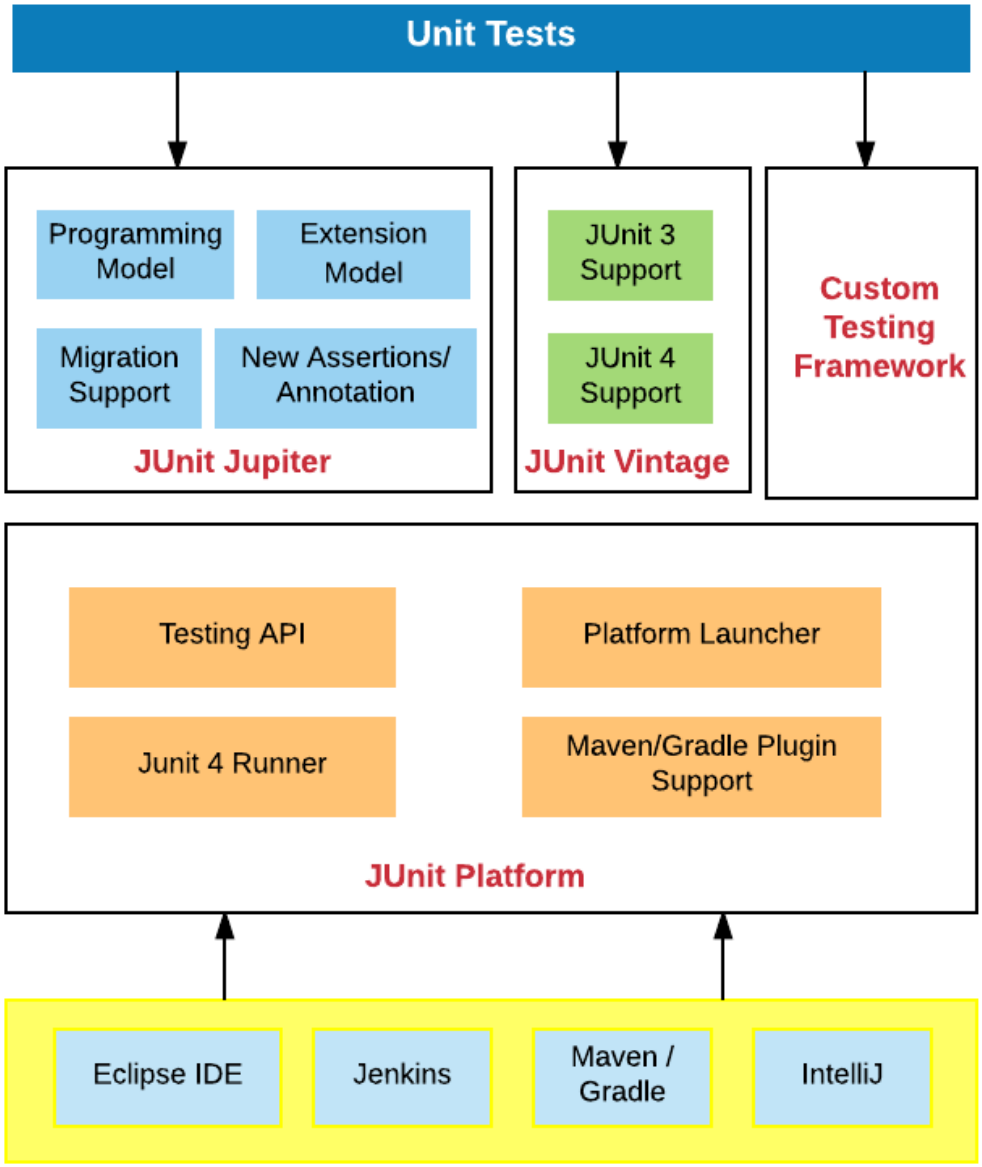
- Phát hiện lỗi trước khi triển khai hệ thống.
- Đảm bảo phần mềm đáp ứng đúng yêu cầu chức năng.
- Nâng cao độ ổn định và độ tin cậy của ứng dụng.
- Hỗ trợ giảm chi phí sửa lỗi và bảo trì hệ thống.
- Tăng chất lượng trải nghiệm người dùng.

2.7.2. JUnit 5

JUnit 5 là framework kiểm thử đơn vị dành cho ngôn ngữ Java, được sử dụng rộng rãi trong phát triển ứng dụng backend. Đây là phiên bản cải tiến của JUnit với kiến trúc linh hoạt hơn, hỗ trợ mở rộng và tích hợp tốt với các công cụ phát triển hiện đại như Maven, Gradle và Spring Boot. [25], [26]

JUnit 5 được xây dựng dựa trên ba thành phần chính:

- JUnit Platform: nền tảng hỗ trợ chạy framework kiểm thử.
- JUnit Jupiter: cung cấp API và annotation để viết test.
- JUnit Vintage: hỗ trợ chạy các test của phiên bản JUnit cũ.



Hình 2.9. Kiến trúc ứng dụng/công nghệ JUnit 5

Nguồn: Ajitesh Kumar Shukla (2017) [40].

JUnit 5 cung cấp nhiều annotation hỗ trợ xây dựng test case như @Test, @BeforeEach, @AfterEach, @DisplayName, @ParameterizedTest, giúp tổ chức và quản lý quá trình kiểm thử hiệu quả hơn. Framework này cho phép kiểm tra tính đúng đắn của các hàm xử lý nghiệp vụ thông qua việc so sánh kết quả thực tế với kết quả mong đợi. Ngoài ra, JUnit 5 còn hỗ trợ kiểm thử ngoại lệ thông qua assertThrows(), kiểm thử timeout và kiểm thử tham số hóa, giúp mở rộng khả năng xây dựng bộ kiểm thử linh hoạt và dễ bảo trì. [25]

Ưu điểm của JUnit 5 [25], [26]

- Dễ tích hợp với hệ sinh thái Java và Spring Boot.
- Hỗ trợ annotation linh hoạt và dễ mở rộng.
- Hỗ trợ kiểm thử tham số hóa (Parameterized Testing).
- Tích hợp tốt với các công cụ CI/CD.
- Hỗ trợ tổ chức test rõ ràng và dễ bảo trì.
- Có cộng đồng sử dụng rộng rãi và tài liệu phong phú.

Hạn chế của JUnit 5 [27]

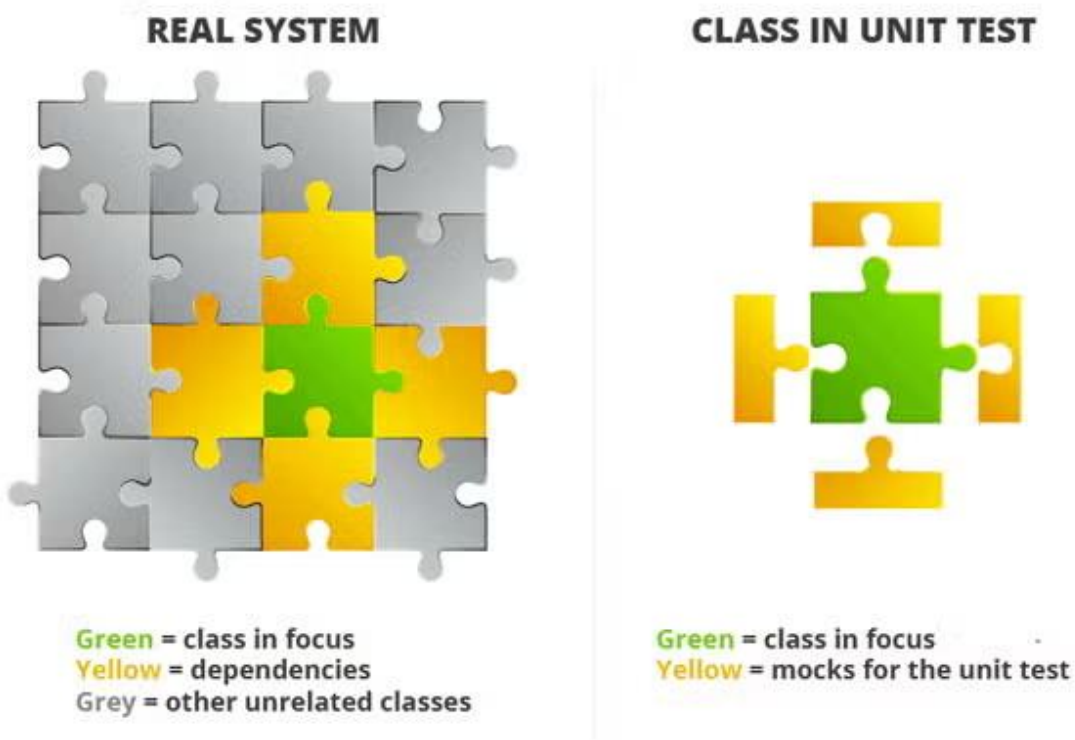
- Chủ yếu hỗ trợ Unit Test, không thay thế hoàn toàn Integration Test.
- Một số tính năng nâng cao cần kết hợp với thư viện khác như Mockito.
- Người mới cần thời gian làm quen với kiến trúc mới của JUnit 5.

2.7.3. Mockito

Mockito là thư viện hỗ trợ tạo đối tượng giả lập trong kiểm thử đơn vị Java [28]. Công cụ này cho phép mô phỏng hành vi của các dependency như repository, external API hoặc service khác mà không cần kết nối trực tiếp tới cơ sở dữ liệu hay hệ thống bên ngoài.

Trong kiểm thử backend, việc cô lập logic nghiệp vụ là rất quan trọng nhằm đảm bảo kết quả kiểm thử không bị ảnh hưởng bởi các thành phần phụ thuộc. Mockito hỗ trợ thực hiện điều này thông qua các annotation như @Mock, @InjectMocks và @Spy. Ngoài ra, Mockito còn hỗ trợ:

- Stubbing: mô phỏng dữ liệu trả về của hàm.
- Verify Interaction: xác minh số lần và cách thức gọi hàm.
- Exception Testing: mô phỏng ngoại lệ trong quá trình xử lý.
- Argument Matcher: hỗ trợ kiểm tra tham số truyền vào bằng các matcher như any(), eq(), contains().
- Behavior Verification: kiểm tra luồng xử lý và hành vi tương tác giữa các thành phần trong hệ thống.



Hình 2.10. Tổng quan về mocking trong Unit Test

Nguồn: Jrebel [41]

Mockito hoạt động dựa trên cơ chế tạo đối tượng giả lập tại runtime, giúp kiểm thử tập trung vào logic nghiệp vụ của lớp cần kiểm tra mà không phụ thuộc vào môi trường triển khai thực tế. Điều này đặc biệt hữu ích trong các ứng dụng backend sử dụng kiến trúc nhiều lớp như Controller - Service - Repository. [28]

Trong hệ sinh thái Spring Boot, Mockito thường được sử dụng cùng với JUnit 5 để kiểm thử các lớp Service hoặc Controller. Thư viện này hỗ trợ tích hợp tốt với Spring Test thông qua annotation @MockBean, cho phép thay thế bean thật bằng mock object trong quá trình kiểm thử. [28]

Ưu điểm của Mockito [28]

- Giảm phụ thuộc giữa các thành phần khi kiểm thử.
- Tăng tốc độ thực thi Unit Test.
- Hỗ trợ mô phỏng nhiều tình huống xử lý khác nhau.
- Dễ tích hợp với JUnit và Spring Boot.
- Giúp kiểm thử độc lập, dễ xác định lỗi.
- Hỗ trợ kiểm thử theo hướng TDD (Test Driven Development).

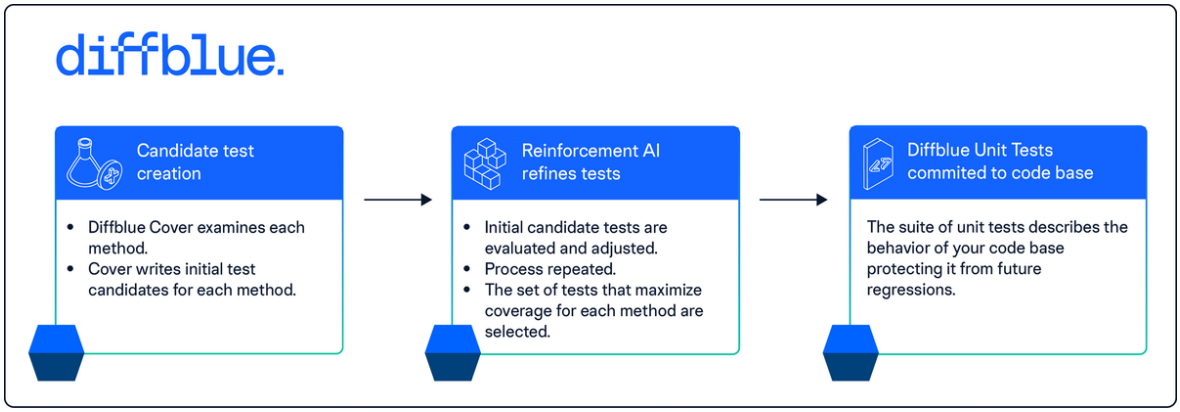
- Cú pháp đơn giản, dễ đọc và dễ bảo trì.

Hạn chế của Mockito [28]

- Không thay thế được Integration Test.
- Việc lạm dụng mock có thể khiến test thiếu tính thực tế.
- Một số trường hợp như static method hoặc final class cần cấu hình bổ sung.

2.7.4. Diffblue Cover

Diffblue Cover là công cụ sử dụng trí tuệ nhân tạo để tự động sinh Unit Test cho ứng dụng Java [29], [30]. Công cụ này sử dụng kỹ thuật phân tích mã nguồn kết hợp với AI nhằm xác định các luồng xử lý trong chương trình, từ đó tạo ra các test case phù hợp cho từng phương thức.



Hình 2.11. Cách thức Diffblue Cover hoạt động

Nguồn: Diffblue Cover Documentation [42]

Khác với việc viết test thủ công, Diffblue Cover có khả năng tự động:

- Sinh dữ liệu kiểm thử.
- Kiểm tra kết quả đầu ra.
- Tạo assertion tương ứng.
- Bao phủ nhiều nhánh xử lý trong mã nguồn.
- Phát hiện các trường hợp ngoại lệ và tình huống biên (Edge Case).

Diffblue Cover hoạt động bằng cách phân tích hành vi của mã nguồn và tự động xây dựng các bài kiểm thử có thể biên dịch và thực thi trực tiếp. Các test được sinh ra thường sử dụng JUnit và có thể kết hợp với Mockito để mô phỏng dependency trong quá trình kiểm thử. Công cụ này hỗ trợ tích hợp với các môi trường phát triển

phổ biến như IntelliJ IDEA, Maven và Gradle, giúp tự động hóa quy trình xây dựng bộ kiểm thử trong phát triển phần mềm hiện đại [29], [30].

Việc sử dụng Diffblue Cover giúp giảm thời gian xây dựng bộ kiểm thử và nâng cao độ bao phủ mã nguồn. Ngoài ra, công cụ còn hỗ trợ phát hiện các trường hợp xử lý đặc biệt mà lập trình viên có thể bỏ sót khi viết test thủ công. [30]

Ưu điểm của Diffblue Cover [29], [30]

- Tự động sinh test case bằng AI.
- Giảm thời gian viết kiểm thử thủ công.
- Hỗ trợ tăng độ bao phủ mã nguồn.
- Tích hợp với JUnit 5, Maven và Gradle.
- Hỗ trợ phát hiện các nhánh xử lý chưa được kiểm thử.
- Giúp cải thiện chất lượng và tính ổn định của mã nguồn.

Hạn chế của Diffblue Cover [29], [30]

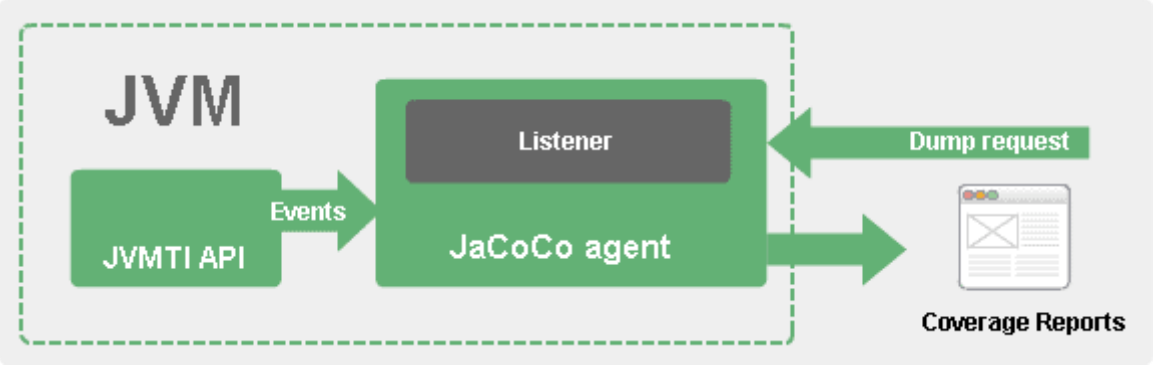
- Chủ yếu hỗ trợ ngôn ngữ Java.
- Một số test sinh tự động có thể khó đọc hoặc khó bảo trì.
- Hiệu quả phụ thuộc vào cấu trúc và chất lượng mã nguồn.
- Một số tính năng nâng cao yêu cầu phiên bản thương mại.

2.7.5. JaCoCo

JaCoCo (Java Code Coverage) là công cụ đo độ bao phủ mã nguồn dành cho ứng dụng Java. Công cụ này cho phép thống kê tỷ lệ mã nguồn đã được thực thi trong quá trình kiểm thử thông qua các chỉ số như:

- Line Coverage: tỷ lệ số dòng mã đã được thực thi.
- Branch Coverage: tỷ lệ các nhánh điều kiện đã được kiểm thử.
- Method Coverage: tỷ lệ phương thức đã được gọi trong quá trình kiểm thử.
- Class Coverage: tỷ lệ lớp đã được kiểm thử.

JaCoCo hoạt động bằng cách chèn mã theo dõi vào chương trình trong quá trình biên dịch hoặc thực thi nhằm ghi nhận các phần mã nguồn đã được chạy khi thực hiện test. [31]



Hình 2.12. Kiến trúc hoạt động của JaCoCo

Nguồn: Baeldung [43]

Công cụ này thường được tích hợp với Maven hoặc Gradle để tự động tạo báo cáo kiểm thử dưới dạng HTML, XML hoặc CSV [32]. Thông qua các báo cáo này, lập trình viên có thể đánh giá mức độ đầy đủ của bộ kiểm thử và xác định các phần mã nguồn chưa được kiểm thử [31].

JaCoCo hỗ trợ tốt trong quy trình kiểm thử tự động và Continuous Integration (CI), giúp theo dõi chất lượng kiểm thử trong suốt vòng đời phát triển phần mềm. Ngoài ra, công cụ còn có thể tích hợp với các nền tảng như Jenkins, SonarQube và GitHub Actions để phân tích chất lượng mã nguồn. [33]

Ưu điểm của JaCoCo [31]-[33]

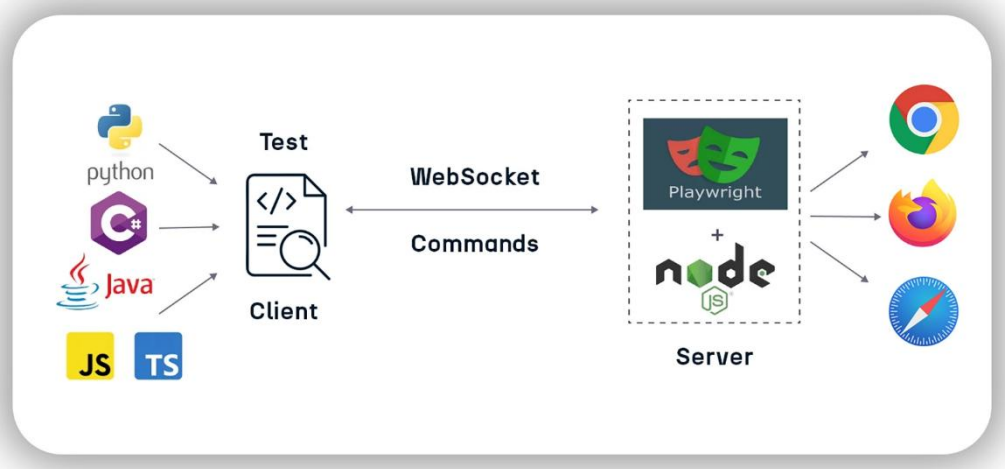
- Hỗ trợ đo coverage chi tiết.
- Tạo báo cáo trực quan dưới dạng HTML.
- Dễ tích hợp với Maven, Gradle và Spring Boot.
- Hỗ trợ đánh giá chất lượng kiểm thử.
- Theo dõi được mức độ bao phủ của từng lớp và phương thức.
- Hỗ trợ tích hợp với các công cụ CI/CD.

Hạn chế của JaCoCo [31]

- Độ bao phủ cao không đồng nghĩa với chất lượng test cao.
- Một số đoạn mã sinh tự động có thể làm sai lệch kết quả coverage.
- Không đánh giá được tính đúng đắn của logic kiểm thử.
- Có thể ảnh hưởng nhẹ đến hiệu năng khi instrument mã nguồn.

2.7.6. Playwright

Playwright là framework kiểm thử giao diện người dùng mã nguồn mở được phát triển bởi Microsoft. Công cụ này hỗ trợ kiểm thử tự động trên nhiều trình duyệt như Chromium, Firefox và WebKit, đồng thời hỗ trợ nhiều ngôn ngữ lập trình như JavaScript, TypeScript, Java và Python [34], [35].



Hình 2.13. Kiến trúc hoạt động của Playwright

Nguồn: MM Software [44]

Playwright cho phép mô phỏng các thao tác thực tế của người dùng trên trình duyệt như:

- Đăng nhập hệ thống.
- Nhập dữ liệu biểu mẫu.
- Nhấn nút chức năng.
- Điều hướng giữa các trang.
- Kiểm tra nội dung hiển thị.

Một trong những đặc điểm nổi bật của Playwright là cơ chế Auto-Waiting, cho phép hệ thống tự động chờ phần tử xuất hiện trước khi thao tác, từ đó giảm lỗi trong quá trình kiểm thử tự động [34]. Ngoài ra, Playwright còn hỗ trợ:

- Chạy kiểm thử ở chế độ headless.

- Chụp ảnh màn hình khi xảy ra lỗi.
- Quay video quá trình kiểm thử.
- Thực hiện kiểm thử End-to-End (E2E Testing).
- Giả lập nhiều thiết bị và kích thước màn hình.
- Hỗ trợ kiểm thử song song nhằm tăng tốc độ thực thi.

Playwright hoạt động dựa trên cơ chế điều khiển trực tiếp trình duyệt thông qua API hiện đại, giúp tăng độ ổn định và giảm lỗi so với nhiều framework kiểm thử giao diện truyền thống. Công cụ này cũng hỗ trợ theo dõi mạng, kiểm tra request/response và xử lý authentication trong quá trình kiểm thử. Nhờ khả năng hỗ trợ đa trình duyệt và tự động hóa mạnh mẽ, Playwright được sử dụng phổ biến trong kiểm thử giao diện web hiện đại nhằm đảm bảo tính ổn định và trải nghiệm người dùng trên nhiều nền tảng khác nhau [36].

Ưu điểm của Playwright [34]-[36]

- Hỗ trợ đa trình duyệt.
- Tốc độ thực thi nhanh.
- Hỗ trợ kiểm thử E2E hiệu quả.
- Tích hợp screenshot và video recording.
- Cơ chế Auto-Waiting giúp giảm lỗi khi kiểm thử.
- Hỗ trợ chạy kiểm thử song song.
- Dễ tích hợp với CI/CD và các hệ thống kiểm thử hiện đại.

Hạn chế của Playwright [34]

- Cần tài nguyên hệ thống tương đối cao khi chạy nhiều trình duyệt.
- Một số tính năng nâng cao yêu cầu hiểu biết về automation testing.
- Việc bảo trì test có thể phức tạp khi giao diện thay đổi thường xuyên.

2.7.7. Postman và Postman Collection Runner

Postman là công cụ hỗ trợ kiểm thử API phổ biến, cho phép gửi request HTTP và kiểm tra dữ liệu phản hồi từ hệ thống backend [37]. Công cụ hỗ trợ nhiều phương thức như GET, POST, PUT và DELETE, đồng thời cho phép xây dựng bộ sưu tập API phục vụ kiểm thử hệ thống.

Postman hỗ trợ viết script kiểm thử bằng JavaScript nhằm kiểm tra [38]:

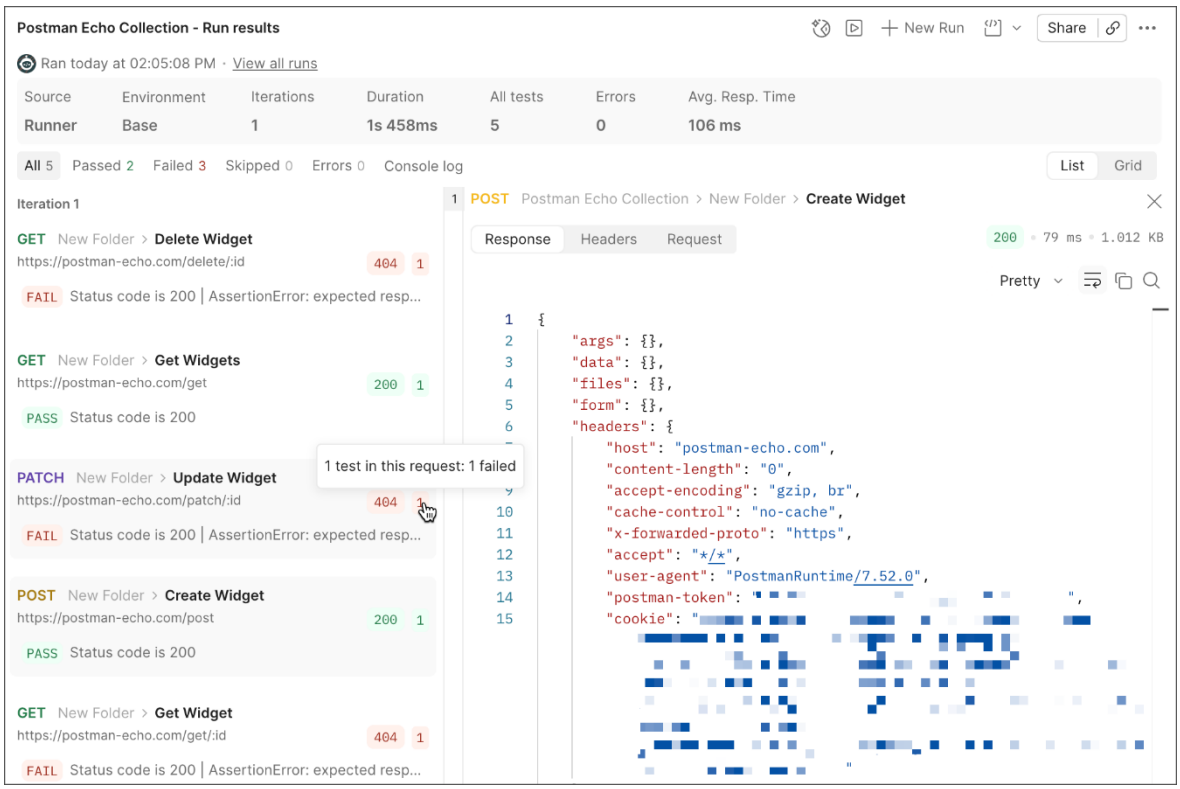
- Mã trạng thái HTTP.
- Cấu trúc dữ liệu JSON.
- Thời gian phản hồi của API.
- Tính chính xác của dữ liệu trả về.
- Header và thông tin xác thực của request/response.

Ngoài ra, Postman Collection Runner cho phép thực thi tự động nhiều request trong một collection nhằm hỗ trợ kiểm thử luồng hoạt động của hệ thống backend. Công cụ này có thể chạy lặp nhiều lần với dữ liệu đầu vào khác nhau thông qua file CSV hoặc JSON, giúp hỗ trợ kiểm thử dữ liệu hàng loạt [39].

Postman còn hỗ trợ quản lý Environment và Variables, cho phép tái sử dụng cấu hình như URL, token hoặc thông tin xác thực giữa nhiều request khác nhau [38]. Điều này giúp quá trình kiểm thử API trở nên linh hoạt và dễ quản lý hơn.

Bên cạnh đó, Postman hỗ trợ tích hợp với Newman - công cụ chạy Collection bằng command line - nhằm phục vụ quy trình kiểm thử tự động và CI/CD [37]. Các kết quả kiểm thử có thể được xuất ra dưới dạng báo cáo để theo dõi và đánh giá chất lượng API.

Nhờ giao diện trực quan và khả năng tự động hóa cao, Postman được sử dụng rộng rãi trong quá trình phát triển và kiểm thử RESTful API.



Hình 2.14. Minh họa kết quả kiểm thử API bằng Postman Collection Runner

Nguồn: Postman Learning Center [45]

Ưu điểm của Postman [37]-[39]

- Giao diện trực quan và dễ sử dụng.
- Hỗ trợ kiểm thử API nhanh chóng.
- Hỗ trợ script kiểm thử tự động.
- Dễ quản lý collection và môi trường kiểm thử.
- Hỗ trợ chia sẻ API và cộng tác nhóm.
- Tích hợp tốt với CI/CD thông qua Newman.
- Hỗ trợ kiểm thử dữ liệu hàng loạt.

Hạn chế của Postman [37], [38]

- Khó quản lý khi số lượng request quá lớn.
- Script kiểm thử phức tạp có thể khó bảo trì.
- Một số tính năng nâng cao yêu cầu tài khoản trả phí.
- Không phù hợp để thay thế hoàn toàn kiểm thử hiệu năng chuyên sâu.

2.8. Bảo mật và an toàn dữ liệu y khoa

2.8.1. Tổng quan

Trong lĩnh vực y tế số, dữ liệu sức khỏe được xem là loại dữ liệu đặc biệt nhạy cảm do chứa nhiều thông tin cá nhân như hồ sơ bệnh án, lịch sử điều trị và thông tin định danh người dùng. Nếu dữ liệu bị rò rỉ hoặc sử dụng sai mục đích, hậu quả có thể ảnh hưởng trực tiếp đến quyền riêng tư và an toàn của người bệnh. Theo WHO trong "Ethics and Governance of Artificial Intelligence for Health" (2021) [13], các hệ thống y tế số và ứng dụng AI trong y tế cần đảm bảo quyền riêng tư dữ liệu, tính minh bạch, an toàn và trách nhiệm giải trình trong quá trình xử lý thông tin sức khỏe. Vì vậy, bảo mật dữ liệu y khoa không chỉ là yêu cầu kỹ thuật mà còn là yếu tố then chốt để đảm bảo quyền riêng tư và duy trì độ tin cậy của các nền tảng chăm sóc sức khỏe trực tuyến.

2.8.2. Các nguyên tắc bảo mật

Trong lĩnh vực an toàn thông tin, các yêu cầu bảo vệ dữ liệu thường được xây dựng dựa trên mô hình nền tảng là CIA Triad - bộ ba nguyên tắc cốt lõi gồm: Tính bảo mật, Tính toàn vẹn và Tính sẵn sàng. Mô hình này được áp dụng rộng rãi trong y tế, bao gồm cả khuôn khổ bảo mật HIPAA (tiêu chuẩn bảo mật dữ liệu y tế của Hoa Kỳ) và các tiêu chuẩn quản lý bảo mật thông tin quốc tế như ISO/IEC 27001.

Tính bảo mật (Confidentiality): Đảm bảo dữ liệu chỉ được truy cập bởi những đối tượng được cấp quyền. Trong hệ thống y tế, điều này đặc biệt quan trọng do dữ liệu bệnh án và thông tin sức khỏe cá nhân cần được kiểm soát nghiêm ngặt để tránh rò rỉ ra bên ngoài.

Tính toàn vẹn (Integrity): Đảm bảo dữ liệu không bị thay đổi trái phép trong quá trình lưu trữ hoặc truyền tải. Điều này đặc biệt quan trọng đối với hồ sơ bệnh án và dữ liệu điều trị, vì sai lệch thông tin có thể ảnh hưởng trực tiếp đến quyết định y khoa.

Tính sẵn sàng (Availability): Đảm bảo hệ thống và dữ liệu luôn có thể truy cập khi cần thiết. Trong lĩnh vực y tế, việc gián đoạn hệ thống - có thể ảnh hưởng nghiêm trọng đến quá trình hỗ trợ và chăm sóc người bệnh.

2.8.3. Mã hóa và bảo vệ dữ liệu

Mã hóa dữ liệu là một trong những kỹ thuật quan trọng nhất trong bảo mật hệ thống y tế số, giúp giảm nguy cơ rò rỉ thông tin khi dữ liệu bị truy cập trái phép hoặc

bị đánh cắp. Theo các tiêu chuẩn bảo mật quốc tế (NIST, ISO/IEC 27001), dữ liệu y khoa thường được bảo vệ theo hai hình thức chính:

- Mã hóa dữ liệu khi truyền tải (Data in Transit): Đảm bảo an toàn cho dữ liệu trong quá trình luân chuyển giữa Client và Server hoặc giữa các dịch vụ nội bộ qua môi trường mạng, thường thực hiện thông qua các giao thức TLS/SSL.
- Mã hóa dữ liệu khi lưu trữ (Data at Rest): Đảm bảo an toàn cho dữ liệu khi được ghi nhận và lưu trữ cố định trong các hệ quản trị cơ sở dữ liệu hoặc hệ thống tệp tin của máy chủ, thường áp dụng các thuật toán như AES-256.

2.8.4. Đạo đức và quản trị AI trong y tế

Theo WHO trong "Ethics and Governance of Artificial Intelligence for Health" (2021) [13], các hệ thống AI trong lĩnh vực y tế cần tuân thủ sáu nguyên tắc đạo đức cốt lõi nhằm đảm bảo tính an toàn, minh bạch và công bằng trong quá trình vận hành:

- **Bảo vệ quyền tự chủ của con người**: Con người phải luôn kiểm soát các hệ thống y tế và quyết định lâm sàng. AI chỉ đóng vai trò hỗ trợ; quyền quyết định điều trị cuối cùng thuộc về bác sĩ và bệnh nhân. Dữ liệu cá nhân phải được bảo vệ bằng sự đồng ý có thông tin đầy đủ theo các khung pháp lý hiện hành.
- **Thúc đẩy sức khỏe, an toàn con người và lợi ích công cộng**: Các nhà phát triển AI y tế phải đáp ứng các yêu cầu pháp lý về độ an toàn, độ chính xác và hiệu quả lâm sàng cho từng ứng dụng cụ thể, đồng thời xây dựng cơ chế kiểm soát và cải tiến chất lượng liên tục.
- **Đảm bảo tính minh bạch, khả năng giải thích và dễ hiểu**: Công nghệ AI không được là một "hộp đen". Quá trình thiết kế, phát triển, nguồn gốc dữ liệu huấn luyện và cơ chế logic dẫn đến kết quả của AI phải được công khai đầy đủ để các chuyên gia y tế có thể hiểu, đánh giá và giải trình.
- **Thúc đẩy trách nhiệm và nghĩa vụ giải trình**: Khi AI đưa ra khuyến nghị dẫn đến quyết định sai sót, hệ thống quản trị phải quy định rõ ràng trách nhiệm thuộc về ai (nhà phát triển, cơ sở y tế hay bác sĩ điều trị) và phải có cơ chế để người dùng khiếu nại và được bồi thường.
- **Đảm bảo tính toàn diện và công bằng**: AI y tế phải được thiết kế và huấn luyện dựa trên tập dữ liệu đa dạng, không bị định kiến (bias) về tuổi tác, giới

tính, thu nhập, chủng tộc hay các đặc điểm khác được bảo vệ theo các quy định nhân quyền, để mọi người đều có thể tiếp cận công bằng.

- **Thúc đẩy AI có khả năng đáp ứng và bền vững**: Các nhà thiết kế, phát triển và người dùng phải liên tục và minh bạch đánh giá các ứng dụng AI trong thực tế để xác định mức độ đáp ứng yêu cầu. Đồng thời, AI phải phù hợp với định hướng phát triển bền vững của hệ thống y tế và môi trường.

2.9. Tổng quan các nghiên cứu và ứng dụng liên quan

2.9.1. Xu hướng ứng dụng AI

Trong những năm gần đây, trí tuệ nhân tạo đang được ứng dụng ngày càng rộng rãi trong lĩnh vực y tế nhằm hỗ trợ tư vấn sức khỏe, phân tích dữ liệu bệnh án, hỗ trợ chẩn đoán và tự động hóa quy trình chăm sóc người bệnh. Theo Tổ chức Y tế Thế giới (WHO), trong "Ethics and Governance of Artificial Intelligence for Health" (2021) [13], AI có tiềm năng cải thiện khả năng tiếp cận dịch vụ y tế, tối ưu hiệu quả vận hành và hỗ trợ ra quyết định lâm sàng.

Sự phát triển của các mô hình LLM như GPT đã thúc đẩy mạnh mẽ xu hướng xây dựng các hệ thống AI hội thoại trong y tế. Các hệ thống này có khả năng giao tiếp tự nhiên với người dùng, hỗ trợ giải đáp thông tin sức khỏe, phân tích triệu chứng ban đầu và hỗ trợ phân loại bệnh nhân, cũng như hỗ trợ quản lý quy trình chăm sóc. Ở phía cơ sở y tế, các mô hình LLM còn được ứng dụng vào ghi chép hồ sơ, hỗ trợ ra quyết định lâm sàng và đào tạo nhân lực y tế.

Tuy nhiên, một thách thức lớn của các mô hình LLM trong y tế là hiện tượng "ảo giác" - tức là việc mô hình tạo ra thông tin sai lệch hoặc không có căn cứ thực tế, điều có thể gây hại nghiêm trọng trong bối cảnh lâm sàng. Để khắc phục hạn chế này, nhiều nghiên cứu hiện nay tập trung vào kỹ thuật RAG - cơ chế kết hợp mô hình LLM với truy xuất tri thức từ nguồn dữ liệu bên ngoài đáng tin cậy - nhằm nâng cao độ chính xác và tính có căn cứ của phản hồi.

2.9.2. Các nghiên cứu về AI hội thoại trong y tế

AI hội thoại (Conversational AI), đặc biệt là các hệ thống Chatbot ứng dụng mô hình LLM, đang trở thành một trong những hướng nghiên cứu và ứng dụng mang tính đột phá trong lĩnh vực y tế số. Các nghiên cứu thực nghiệm và tổng quan quy mô

lớn gần đây đã định hình rõ ràng cả về tiềm năng, thách thức lẫn xu hướng giải pháp công nghệ mới cho kiến trúc này.

Tiềm năng và các kịch bản ứng dụng cốt lõi

Theo bài nghiên cứu tổng quan hệ thống "Transforming healthcare with chatbots: Uses and applications - coping review" đăng tải trên PMC [14], các nghiên cứu trên thế giới đã chứng minh chatbot y tế hiện đại không còn dừng lại ở việc phản hồi theo kịch bản cứng nhắc (rule-based) mà đã dịch chuyển sang khả năng duy trì ngữ cảnh và tương tác thời gian thực. Chatbot y tế được ghi nhận đạt hiệu quả cao trong 4 nhóm tác vụ chính:

- Tư vấn sức khỏe ban đầu (Health Triage & Screening): Hỗ trợ người dùng sàng lọc triệu chứng, phân loại mức độ khẩn cấp của bệnh trạng trước khi quyết định đến cơ sở y tế.
- Theo dõi người bệnh từ xa (Remote Patient Monitoring): Đóng vai trò là trợ lý ảo đồng hành cùng bệnh nhân mắc bệnh mãn tính, nhắc nhở lịch uống thuốc, theo dõi các chỉ số sinh tồn và chăm sóc hậu phẫu.
- Hỗ trợ chăm sóc tâm lý (Mental Health Support): Cung cấp các liệu pháp trò chuyện hành vi nhận thức (CBT), giải tỏa căng thẳng cho người dùng trong bối cảnh khan hiếm chuyên gia tâm lý học lâm sang.
- Giảm tải cho nhân viên y tế (Administrative Workflow Optimization): Tự động hóa việc trả lời các câu hỏi thường gặp (FAQs), đặt lịch hẹn khám, giúp tối ưu hóa thời gian cho đội ngũ y bác sĩ.

Rào cản kỹ thuật và thách thức trong y khoa

Mặc dù sở hữu tiềm năng lớn, các nghiên cứu học thuật cũng chỉ ra những lỗ hổng nghiêm trọng của AI hội thoại khi đối mặt với dữ liệu y tế nhạy cảm:

- Hiện tượng "ảo tưởng" thông tin: Các mô hình ngôn ngữ lớn có xu hướng tạo ra các phản hồi nghe có vẻ hợp lý nhưng thực chất lại sai lệch về mặt y khoa, gây nguy hiểm trực tiếp đến tính mạng người bệnh.
- Thiếu tính cá nhân hóa sâu (Lack of Personalization): Khó khăn trong việc tích hợp toàn vẹn dữ liệu bệnh án riêng biệt của từng cá nhân để đưa ra lời khuyên chính xác theo thời gian thực.

- Khó kiểm chứng nguồn tri thức (Black-box Nature): Câu trả lời sinh ra từ các tham số trọng số của mô hình thay vì truy xuất từ một nguồn phác đồ điều trị cố định, gây cản trở cho việc kiểm chứng của bác sĩ.
- Hạn chế xử lý tình huống phức tạp: AI gặp thách thức lớn khi người bệnh khai báo các triệu chứng chồng chéo hoặc các trường hợp cấp cứu y khoa cần can thiệp lâm sàng tức thì.

Xu hướng giải pháp công nghệ mới (Mô hình kết hợp RAG)

Để khắc phục các hạn chế trên, xu hướng nghiên cứu hiện nay đang chuyển từ các chatbot AI truyền thống sang kiến trúc RAG, cho phép AI truy xuất thông tin từ nguồn tri thức bên ngoài trước khi sinh phản hồi.

Theo nghiên cứu thực nghiệm "Development and Evaluation of a Retrieval-Augmented Generation-Based Electronic Medical Record Chatbot System" [15], việc kết hợp giữa sức mạnh hiểu ngữ cảnh của mô hình LLM và một kho cơ sở tri thức chuyên môn cố định (như Bệnh án điện tử - EMR, hoặc Phác đồ điều trị chuẩn) là chìa khóa cốt lõi.

Trong kiến trúc này, khi người dùng đặt câu hỏi, hệ thống sẽ thực hiện tìm kiếm ngữ nghĩa trong cơ sở dữ liệu y khoa đáng tin cậy trước để rút trích các tài liệu liên quan. Sau đó, các tài liệu chuẩn này mới được chuyển vào làm đầu vào để ràng buộc mô hình LLM sinh câu trả lời. Kết quả thực nghiệm cho thấy mô hình kết hợp này giúp:

- Triệt tiêu tối đa hiện tượng ảo tưởng thông tin nhờ việc ép AI chỉ được nói dựa trên dữ liệu có sẵn.
- Tăng độ tin cậy và minh bạch khi hệ thống có thể dẫn nguồn trực tiếp từ hồ sơ bệnh án hoặc tài liệu y khoa chính thống.
- Đảm bảo tính cá nhân hóa vượt trội nhờ khả năng đọc và hiểu sâu sắc lịch sử khám bệnh cá nhân được lưu trữ bảo mật trong hệ thống.

2.9.3. Các nền tảng và ứng dụng y tế thông minh hiện nay

Tại thị trường Việt Nam, xu thế chuyển đổi số y tế đã thúc đẩy sự ra đời của nhiều nền tảng và ứng dụng y tế thông minh. Các nền tảng này đóng vai trò là cầu nối kỹ thuật số giữa cơ sở y tế, đội ngũ bác sĩ và bệnh nhân, giúp tối ưu hóa quy trình khám

chữa bệnh truyền thống. Dưới đây là ba nền tảng y tế thông minh tiêu biểu đang được triển khai rộng rãi:

Nền tảng BookingCare

BookingCare là một trong những nền tảng tiên phong tại Việt Nam tập trung chuyên sâu vào giải pháp đặt lịch khám bệnh trực tuyến.

Mô hình hoạt động: Hoạt động như một nền tảng công nghệ chuyên biệt về dịch vụ y tế, kết nối bệnh nhân với mạng lưới các bệnh viện công, tư và phòng khám chuyên khoa uy tín tại Hà Nội và TP.HCM.

Tính năng cốt lõi: Cho phép người dùng chủ động lựa chọn bác sĩ, cơ sở y tế và khung giờ khám dựa trên hệ thống thông tin được xác thực, bao gồm giá dịch vụ, kinh nghiệm bác sĩ và đánh giá được công khai minh bạch.

Ưu điểm nổi bật: Giúp người bệnh xác định đúng chuyên khoa và bác sĩ phù hợp với tình trạng của mình, từ đó giảm thiểu tình trạng quá tải cục bộ và rút ngắn thời gian chờ đợi tại các bệnh viện lớn.

Hệ sinh thái y tế YouMed

YouMed (thành lập năm 2017) phát triển theo hướng một hệ sinh thái y tế số tích hợp đa dịch vụ, phục vụ cả người bệnh lẫn các cơ sở y tế đối tác.

Mô hình hoạt động: Cung cấp ứng dụng cho người bệnh đặt lịch khám, đồng thời triển khai giải pháp chuyển đổi số cho các phòng khám và bệnh viện đối tác, bao gồm tư vấn sức khỏe trực tuyến và sàn thương mại điện tử y tế.

Tính năng cốt lõi: Bên cạnh đặt lịch, YouMed tích hợp lưu trữ hồ sơ và lịch sử khám cá nhân, nhắc nhở lịch tái khám, hỗ trợ thanh toán phí khám trực tuyến, và cung cấp kho nội dung kiến thức y khoa được kiểm duyệt bởi đội ngũ bác sĩ và dược sĩ chuyên môn.

Ưu điểm nổi bật: Tạo ra một vòng khép kín từ khâu tìm kiếm thông tin, đặt lịch hẹn, lưu trữ hồ sơ khám cá nhân cho đến giai đoạn chăm sóc sức khỏe chủ động sau khám.

Ứng dụng MedPro

MedPro là nền tảng đặt lịch khám bệnh trực tuyến tập trung tối ưu hóa quy trình tiếp nhận bệnh nhân, hiện đã triển khai tại hơn 60 bệnh viện và 300 cơ sở y tế trên

toàn quốc, trong đó có nhiều bệnh viện công lập lớn như Bệnh viện Đại học Y Dược TP.HCM, Bệnh viện Chợ Rẫy, Bệnh viện Nhi Đồng 1...

Mô hình hoạt động: Kết nối trực tiếp với hệ thống quản lý bệnh viện, cho phép người bệnh hoàn tất đăng ký và lấy số thứ tự ưu tiên ngay trên điện thoại thông minh trước khi đến cơ sở y tế.

Tính năng cốt lõi: Người bệnh có thể thực hiện toàn bộ quy trình bao gồm đăng ký khám bệnh theo ngày/giờ, thanh toán chi phí trực tuyến (qua ví điện tử, thẻ ngân hàng), nhận phiếu khám điện tử, và tra cứu kết quả cận lâm sàng ngay trên ứng dụng.

Ưu điểm nổi bật: Giải quyết hiệu quả tình trạng ùn tắc tại khu vực tiếp nhận của các bệnh viện lớn bằng cách giảm thiểu các bước xếp hàng chờ lấy số, chờ đóng tiền và chờ nhận kết quả bản giấy.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

Hệ thống HealthAssistant được xây dựng nhằm hỗ trợ kết nối giữa bác sĩ và bệnh nhân trên nền tảng web, đồng thời giải quyết bài toán tư vấn sức khỏe và tối ưu hóa quy trình đặt lịch khám tại các phòng khám tư nhân quy mô nhỏ. Điểm nổi bật của hệ thống là việc tích hợp nhiều vai trò trợ lý AI hoạt động trong cùng một khung trò chuyện, giúp hỗ trợ bệnh nhân xuyên suốt quá trình tư vấn và đặt lịch khám. Ngoài chức năng hỗ trợ tư vấn sức khỏe, hệ thống còn hỗ trợ quản lý thông tin bác sĩ, lịch làm việc, lịch hẹn khám, dữ liệu tri thức AI và các chức năng quản trị người dùng trên cùng một nền tảng. Hệ thống hướng đến các mục tiêu chính:

- Hỗ trợ tư vấn sức khỏe sơ bộ cho bệnh nhân thông qua AI chatbot hoạt động 24/7.
- Hỗ trợ bác sĩ mở rộng khả năng tiếp cận và tương tác với bệnh nhân trên nền tảng trực tuyến.
- Tự động hóa các tác vụ hành chính như hỗ trợ tìm bác sĩ, kiểm tra lịch làm việc và tạo dữ liệu phục vụ cho quy trình đặt lịch khám.
- Cho phép cá nhân hóa tri thức tư vấn dựa trên dữ liệu chuyên môn do chính bác sĩ cung cấp.
- Hỗ trợ quản lý thông tin bác sĩ, lịch làm việc, lịch hẹn và dữ liệu vận hành của hệ thống.

3.1.1. Yêu cầu chức năng

Bệnh nhân có thể tương tác với hệ thống thông qua giao diện web để mô tả triệu chứng, đặt câu hỏi y tế và nhận phản hồi tư vấn sơ bộ. Hệ thống hỗ trợ ba vai trò trợ lý AI chuyên biệt hoạt động trong cùng một khung trò chuyện. Mỗi vai trò được định hướng theo chức năng nghiệp vụ riêng và sử dụng KB tương ứng nhằm hỗ trợ tạo phản hồi phù hợp với chuyên môn và nhu cầu của từng phòng khám:

- Tiếp tân: Hỗ trợ cung cấp thông tin phòng khám, giải đáp các câu hỏi liên quan đến quy trình khám bệnh, hỗ trợ đặt lịch hẹn, sắp xếp lịch khám và xử lý các yêu cầu mang tính hành chính.

- Y tá: Hỗ trợ hướng dẫn khai báo thông tin ban đầu, cung cấp hướng dẫn chuẩn bị trước khi khám hoặc xét nghiệm, hỗ trợ chăm sóc sau khám và hướng dẫn sử dụng thuốc đã được bác sĩ kê đơn.
- Trợ lý bác sĩ: Hỗ trợ trao đổi về triệu chứng, giải thích các thông tin sức khỏe, hỗ trợ tư vấn liên quan đến tình trạng sức khỏe và cung cấp hỗ trợ y tế không mang tính chẩn đoán.

Yêu cầu chức năng tổng quát:

- Bác sĩ có thể tạo hồ sơ cá nhân và quản lý dữ liệu tri thức cho từng vai trò AI thông qua giao diện quản trị mà không cần kiến thức lập trình.
- Hệ thống hỗ trợ tìm kiếm bác sĩ, kiểm tra lịch làm việc và gợi ý các khung giờ khám phù hợp cho bệnh nhân.
- Cho phép bệnh nhân thực hiện đặt lịch khám trực tuyến và nhận xác nhận lịch hẹn từ hệ thống.
- Hệ thống hỗ trợ lưu trữ lịch sử tư vấn và lịch hẹn nhằm hỗ trợ tra cứu và quản lý thông tin bệnh nhân.
- Hệ thống hỗ trợ gửi thông báo nhắc lịch khám cho bệnh nhân thông qua email.
- Hỗ trợ quản lý thông tin bác sĩ, lịch làm việc, lịch hẹn và dữ liệu vận hành của hệ thống.
- Hệ thống hỗ trợ cơ chế phân quyền người dùng theo từng vai trò nhằm đảm bảo kiểm soát truy cập và quản lý chức năng phù hợp với từng nhóm người dùng.
- Quản trị viên có thể quản lý tài khoản người dùng, kiểm duyệt bác sĩ và kiểm soát dữ liệu hệ thống.

3.1.2. Yêu cầu phi chức năng

1. **Tính dễ sử dụng:** Giao diện web cần thân thiện, hỗ trợ responsive trên nhiều thiết bị và cung cấp khả năng tương tác bằng ngôn ngữ tự nhiên.
2. **Bảo mật dữ liệu:** Hệ thống cần đảm bảo an toàn cho dữ liệu người dùng thông qua cơ chế xác thực, phân quyền truy cập và bảo vệ dữ liệu quan trọng.
3. **Hiệu suất:** Hệ thống cần đảm bảo khả năng phản hồi nhanh và hoạt động ổn định trong quá trình sử dụng.

4. **Khả năng mở rộng:** Kiến trúc hệ thống cần hỗ trợ mở rộng thêm workflow AI, vai trò trợ lý AI và cơ sở tri thức trong tương lai.
5. **Chi phí triển khai:** Ưu tiên sử dụng các công nghệ mã nguồn mở và mô hình self-hosted nhằm giảm chi phí vận hành cho các phòng khám tư nhân nhỏ.
6. **Tính an toàn trong hỗ trợ y tế:** Hệ thống chỉ đóng vai trò hỗ trợ tư vấn sơ bộ và không thay thế chẩn đoán chuyên môn của bác sĩ. Trong các trường hợp có dấu hiệu nghiêm trọng, hệ thống cần khuyến nghị người dùng đến cơ sở y tế để được thăm khám trực tiếp.

3.1.3. Yêu cầu người dùng

1. **Bệnh nhân:** Có thể sử dụng hệ thống trực tiếp trên trình duyệt web để tư vấn sức khỏe và đặt lịch khám mà không cần cài đặt ứng dụng riêng.
2. **Bác sĩ:** Có quyền quản lý hồ sơ cá nhân, lịch làm việc, lịch hẹn khám bệnh, dữ liệu tri thức cho trợ lý AI và theo dõi các thống kê liên quan trong hệ thống.
3. **Quản trị viên hệ thống:** Có quyền quản lý tài khoản, thống kê hệ thống, quản lý thông tin bác sĩ, kiểm duyệt bác sĩ, quản lý lịch hẹn và quản lý dữ liệu vận hành của nền tảng.
4. **Quản trị viên quản lý bác sĩ :** Có quyền quản lý thông tin bác sĩ, kiểm duyệt hồ sơ bác sĩ, theo dõi lịch hẹn và quản lý các dữ liệu liên quan đến vận hành bác sĩ trên hệ thống.
5. **Quản trị viên kiểm duyệt:** Có quyền kiểm duyệt hồ sơ bác sĩ, quản lý thông tin bác sĩ và theo dõi các thống kê liên quan đến hoạt động kiểm duyệt.
6. **Quản trị viên hỗ trợ:** Có quyền tham gia cuộc hội thoại và trả lời tin nhắn.

3.1.4. User Stories

User Stories được sử dụng nhằm mô tả các nhu cầu và mong muốn thực tế của người dùng khi tương tác với hệ thống HealthAssistant. Thông qua việc xây dựng User Stories, lúc phát triển có thể xác định rõ các chức năng cần thiết, luồng xử lý nghiệp vụ và trải nghiệm người dùng mong muốn trong quá trình sử dụng hệ thống. Các User Stories trong hệ thống được xây dựng dựa trên các nhóm người dùng chính gồm: người dùng (bệnh nhân), bác sĩ và nhóm quản trị viên. Mỗi User Story phản ánh một mục tiêu cụ thể mà người dùng muốn đạt được khi sử dụng nền tảng.

Người lạ

- Là người lạ, tôi muốn đăng ký, đăng nhập và khôi phục mật khẩu để truy cập hệ thống khi cần.
- Là người lạ, tôi muốn xem các trang giới thiệu hệ thống, bác sĩ, hỗ trợ, phản hồi, điều khoản và chính sách để tìm hiểu thông tin trước khi sử dụng dịch vụ.

Bệnh nhân

- Là bệnh nhân, tôi muốn trò chuyện với AI để nhận tư vấn sức khỏe, gợi ý bác sĩ phù hợp và tạo lịch hẹn trong cuộc trò chuyện.
- Là bệnh nhân, tôi muốn tìm kiếm, lọc và xem thông tin bác sĩ để lựa chọn bác sĩ phù hợp.
- Là bệnh nhân, tôi muốn trò chuyện và đặt lịch hẹn với bác sĩ để được hỗ trợ khám bệnh.
- Là bệnh nhân, tôi muốn quản lý cuộc trò chuyện bao gồm xem danh sách, tìm kiếm, xem các ảnh liên quan, đổi tên và xóa hội thoại.
- Là bệnh nhân, tôi muốn quản lý lịch hẹn bao gồm xem danh sách, xem chi tiết, lọc, cập nhật lý do khám bệnh và hủy lịch hẹn.
- Là bệnh nhân, tôi muốn đánh giá bác sĩ sau khi buổi khám kết thúc.
- Là bệnh nhân, tôi muốn quản lý thông tin sức khỏe cá nhân như cân nặng, chiều cao và các chỉ số y tế.
- Là bệnh nhân, tôi muốn quản lý thông tin cá nhân và thay đổi mật khẩu khi cần.
- Là bệnh nhân, tôi muốn gửi góp ý và trò chuyện với bộ phận hỗ trợ để giải quyết vấn đề phát sinh.

Bác sĩ

- Là bác sĩ, tôi muốn xem thống kê theo hôm nay, tháng này hoặc khoảng thời gian để theo dõi hiệu quả làm việc.
- Là bác sĩ, tôi muốn quản lý trợ lý AI bao gồm xem danh sách, tạo, chỉnh sửa và xóa trợ lý hỗ trợ tư vấn người dùng.
- Là bác sĩ, tôi muốn quản lý lịch làm việc và ngày nghỉ đặc biệt để phù hợp với kế hoạch cá nhân.

- Là bác sĩ, tôi muốn quản lý hồ sơ bác sĩ bao gồm xem hồ sơ, gửi yêu cầu cập nhật, theo dõi trạng thái và hủy yêu cầu khi cần.
- Là bác sĩ, tôi muốn quản lý lịch hẹn bao gồm xem danh sách, xem chi tiết, lọc lịch hẹn, cập nhật trạng thái và thêm kết quả khám bệnh.
- Là bác sĩ, tôi muốn xem, tìm kiếm và xuất bảng lương để theo dõi thu nhập của mình.
- Là bác sĩ, tôi muốn quản lý thông tin cá nhân và thay đổi mật khẩu khi cần.
- Là bác sĩ, tôi muốn gửi góp ý và trao đổi với bộ phận hỗ trợ để giải quyết vấn đề phát sinh.

Quản trị viên toàn quyền

- Là quản trị viên toàn quyền, tôi muốn xem thống kê theo hôm nay, tháng này và khoảng thời gian để theo dõi hoạt động hệ thống.
- Là quản trị viên toàn quyền, tôi muốn quản lý tài khoản bao gồm xem danh sách, xem chi tiết, tìm kiếm, tạo tài khoản admin mới và khóa hoặc mở khóa tài khoản.
- Là quản trị viên toàn quyền, tôi muốn xem và lọc thông tin bác sĩ để quản lý dữ liệu bác sĩ trên hệ thống.
- Là quản trị viên toàn quyền, tôi muốn quản lý yêu cầu đăng ký hoặc cập nhật thông tin bác sĩ bao gồm xem danh sách, xem chi tiết, chỉnh sửa, duyệt hoặc từ chối yêu cầu.
- Là quản trị viên toàn quyền, tôi muốn quản lý lịch hẹn bao gồm xem danh sách, xem chi tiết, tìm kiếm, hủy lịch hẹn và hoàn tiền cho lịch hẹn.
- Là quản trị viên toàn quyền, tôi muốn quản lý bảng lương bao gồm xem danh sách, tạo, lọc, hủy bảng lương và xuất bảng lương.
- Là quản trị viên toàn quyền, tôi muốn quản lý thông tin cá nhân và thay đổi mật khẩu khi cần.

Quản trị viên quản lý bác sĩ

- Là quản trị viên quản lý bác sĩ, tôi muốn xem thống kê theo hôm nay, tháng này và khoảng thời gian để theo dõi hoạt động quản lý bác sĩ.
- Là quản trị viên quản lý bác sĩ, tôi muốn xem và lọc thông tin bác sĩ để quản lý dữ liệu bác sĩ trên hệ thống.

- Là quản trị viên quản lý bác sĩ, tôi muốn quản lý yêu cầu đăng ký hoặc cập nhật thông tin bác sĩ bao gồm xem danh sách, xem chi tiết, chỉnh sửa, duyệt hoặc từ chối yêu cầu.
- Là quản trị viên quản lý bác sĩ, tôi muốn quản lý lịch hẹn bao gồm xem danh sách, xem chi tiết, tìm kiếm, hủy lịch hẹn và hoàn tiền cho lịch hẹn.
- Là quản trị viên quản lý bác sĩ, tôi muốn quản lý bảng lương bao gồm xem danh sách, tạo, lọc, hủy bảng lương và xuất bảng lương.
- Là quản trị viên quản lý bác sĩ, tôi muốn quản lý thông tin cá nhân và thay đổi mật khẩu khi cần.

Quản trị viên kiểm duyệt

- Là quản trị viên kiểm duyệt, tôi muốn xem thống kê theo hôm nay, tháng này và khoảng thời gian để theo dõi hoạt động kiểm duyệt.
- Là quản trị viên kiểm duyệt, tôi muốn xem và lọc thông tin bác sĩ để kiểm tra dữ liệu bác sĩ trên hệ thống.
- Là quản trị viên kiểm duyệt, tôi muốn quản lý yêu cầu đăng ký hoặc cập nhật thông tin bác sĩ bao gồm xem danh sách, xem chi tiết, tạo yêu cầu đăng ký bác sĩ, chỉnh sửa, duyệt hoặc từ chối yêu cầu.
- Là quản trị viên kiểm duyệt, tôi muốn quản lý thông tin cá nhân và thay đổi mật khẩu khi cần.

Quản trị viên hỗ trợ

- Là quản trị viên hỗ trợ, tôi muốn xem danh sách cuộc trò chuyện và hỗ trợ người dùng hoặc bác sĩ qua tin nhắn trực tuyến.
- Là quản trị viên hỗ trợ, tôi muốn quản lý thông tin cá nhân và thay đổi mật khẩu khi cần.

3.1.5. Use Case

Dựa trên các User Stories đã xây dựng cho từng nhóm người dùng, nhóm thực hiện tiến hành phân tích và gom nhóm các nhu cầu có tính chất tương đồng nhằm hình thành các nhóm chức năng chính của hệ thống. Các nhóm chức năng này được sử dụng làm cơ sở để xây dựng biểu đồ Use Case tổng quát cho hệ thống HealthAssistant. Quá trình phân tích được thực hiện theo hướng:

- Xác định vai trò khi tham gia hệ thống.
- Xác định mục tiêu mà từng vai trò sẽ đạt được.
- Gom các chức năng có liên quan thành một Use Case tổng quát để giảm độ phức tạp của sơ đồ.
- Loại bỏ các chức năng quá chi tiết khỏi sơ đồ tổng quát nhằm đảm bảo tính trực quan và dễ theo dõi.

Từ quá trình phân tích User Stories, hệ thống được xác định gồm các nhóm actor chính:

- Người lạ
- Bệnh nhân
- Bác sĩ
- Quản trị viên toàn quyền
- Quản trị viên quản lý bác sĩ
- Quản trị viên kiểm duyệt
- Quản trị viên hỗ trợ

Các chức năng chi tiết trong User Stories sau đó được gom nhóm thành các Use Case tổng quát như sau:



Hình 3.1. Sơ đồ Use case tổng quát của hệ thống HealthAssistant

3.1.6. Activity Diagram

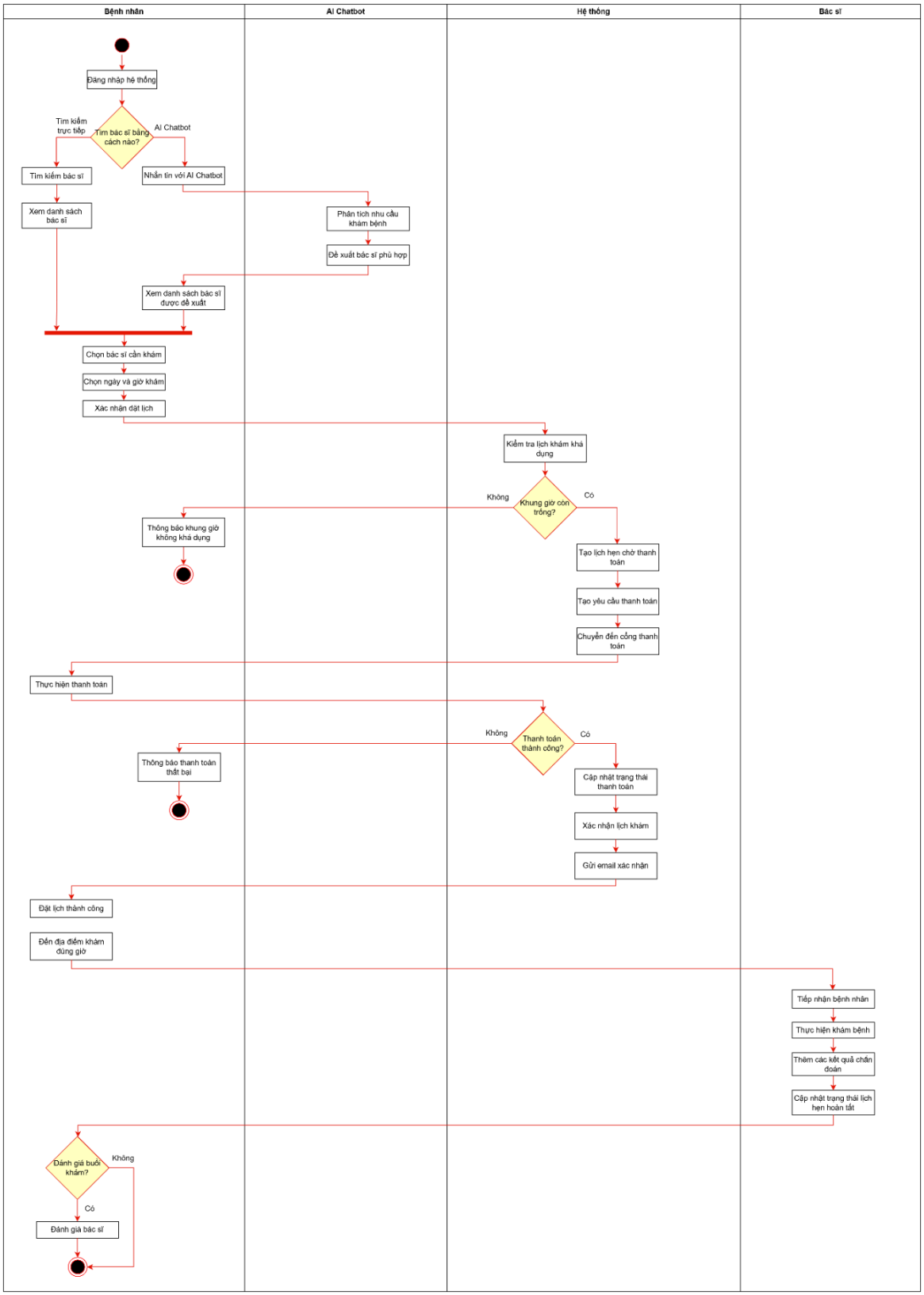
Activity Diagram được sử dụng nhằm mô tả luồng xử lý nghiệp vụ của hệ thống theo trình tự thực hiện giữa các actor và các thành phần liên quan. Thông qua biểu đồ này, có thể quan sát được quá trình xử lý của hệ thống từ lúc người dùng gửi yêu cầu cho đến khi nghiệp vụ hoàn tất. Trong hệ thống HealthAssistant, Activity Diagram được sử dụng để mô tả các luồng quan trọng sau:

- Đặt lịch và khám bệnh
- Đăng kí trở thành bác sĩ
- Nhắn tin với Chatbot AI của hệ thống
- Nhắn tin với Chatbot AI trợ lý của bác sĩ
- Nhắn tin bác sĩ

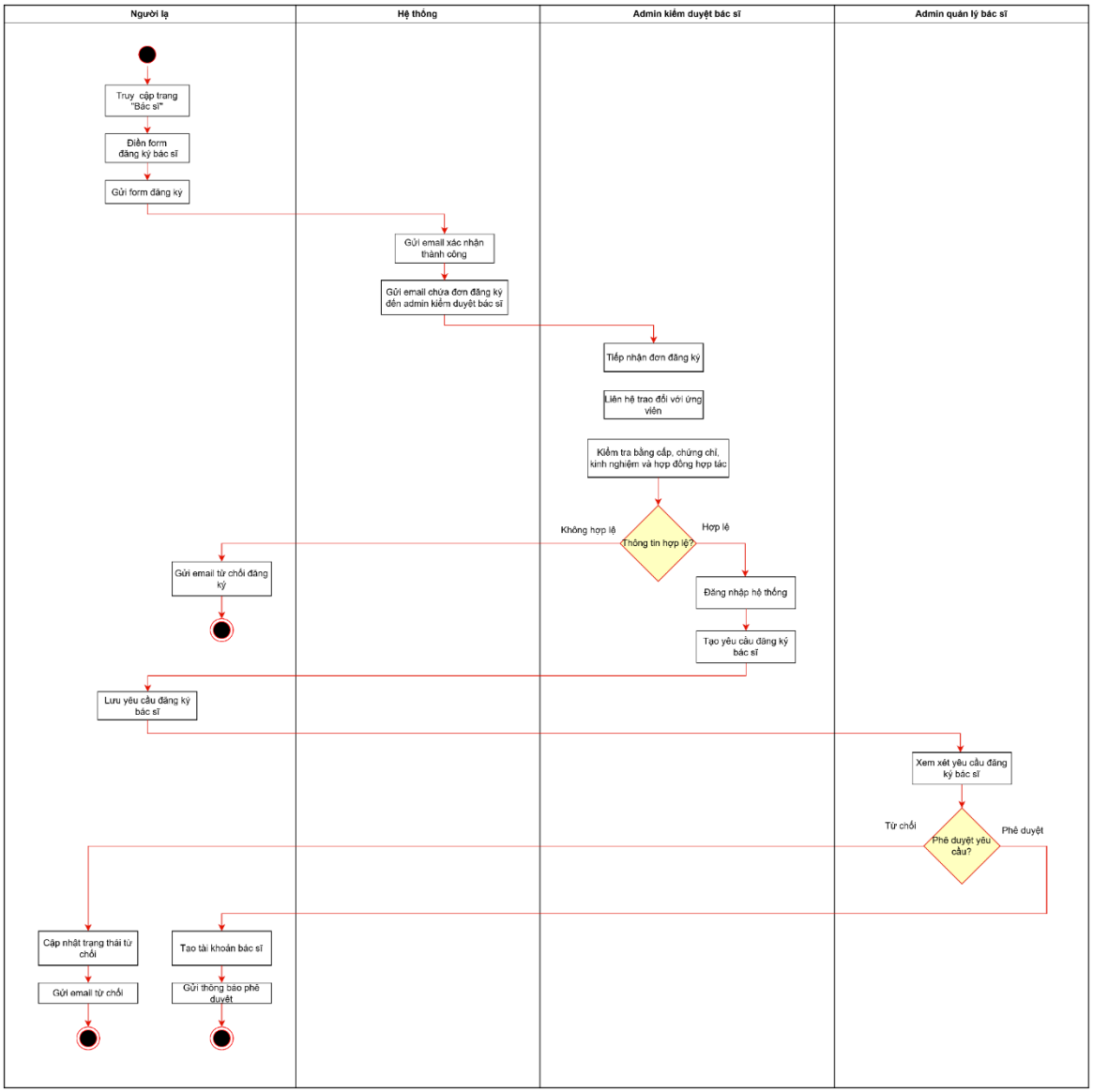
Các biểu đồ được xây dựng theo mô hình swimlane nhằm thể hiện rõ trách nhiệm xử lý của từng actor hoặc service trong hệ thống như bệnh nhân, bác sĩ, AI Chatbot, quản trị viên,.....Việc sử dụng Activity Diagram giúp nhóm phát triển:

- Hiểu rõ luồng nghiệp vụ của từng chức năng.
- Xác định các bước xử lý và điều kiện rẽ nhánh trong hệ thống.
- Phân tích trách nhiệm giữa các thành phần trong kiến trúc Microservices.

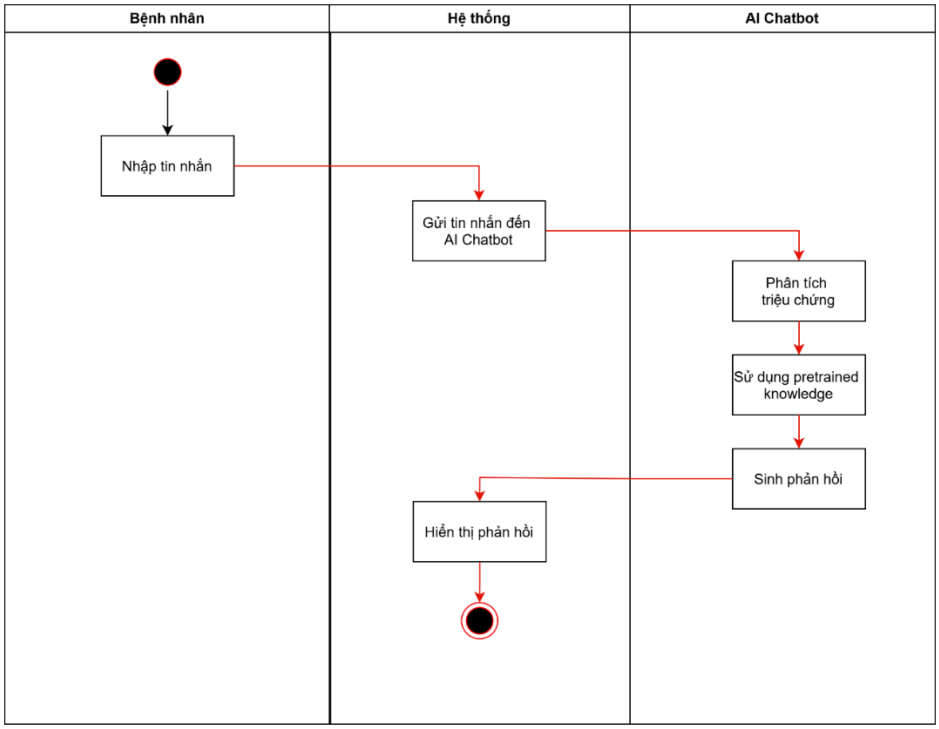
Sau đây là các biểu đồ hoạt động cho những chức năng chính của hệ thống HealthAssistant:



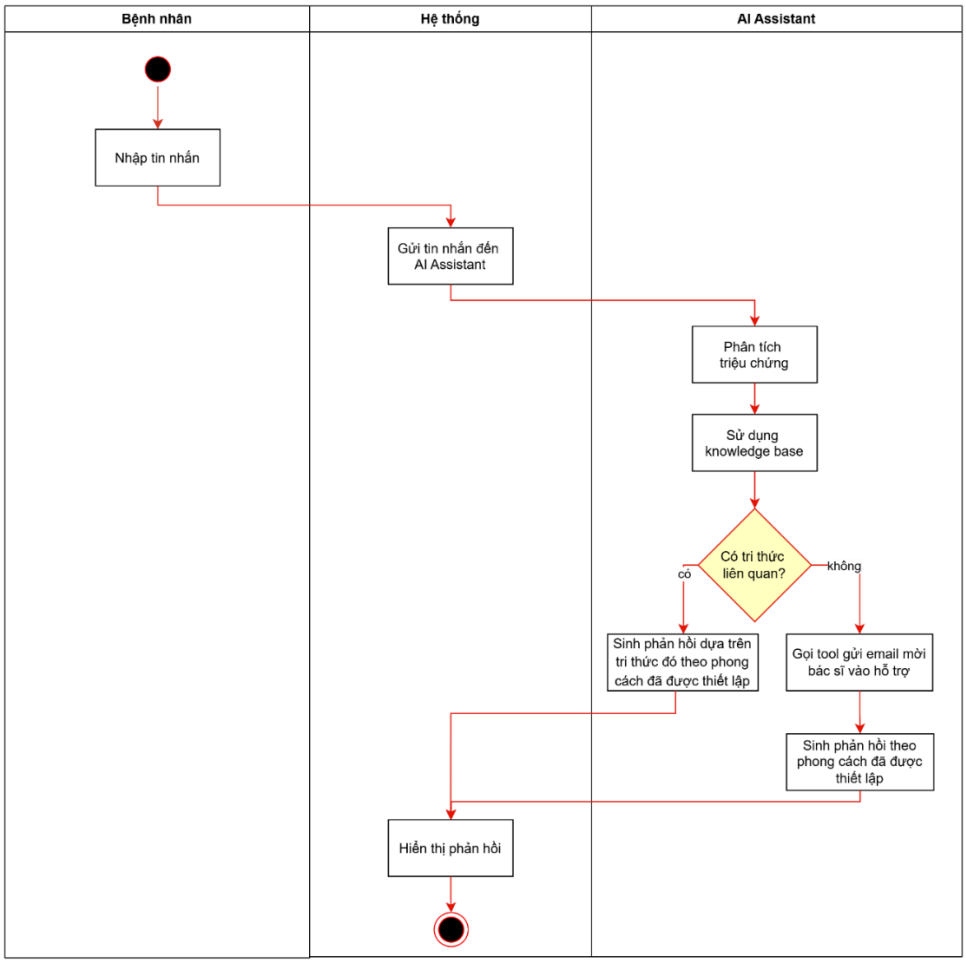
Hình 3.2. Sơ đồ mô tả quy trình đặt lịch và khám bệnh



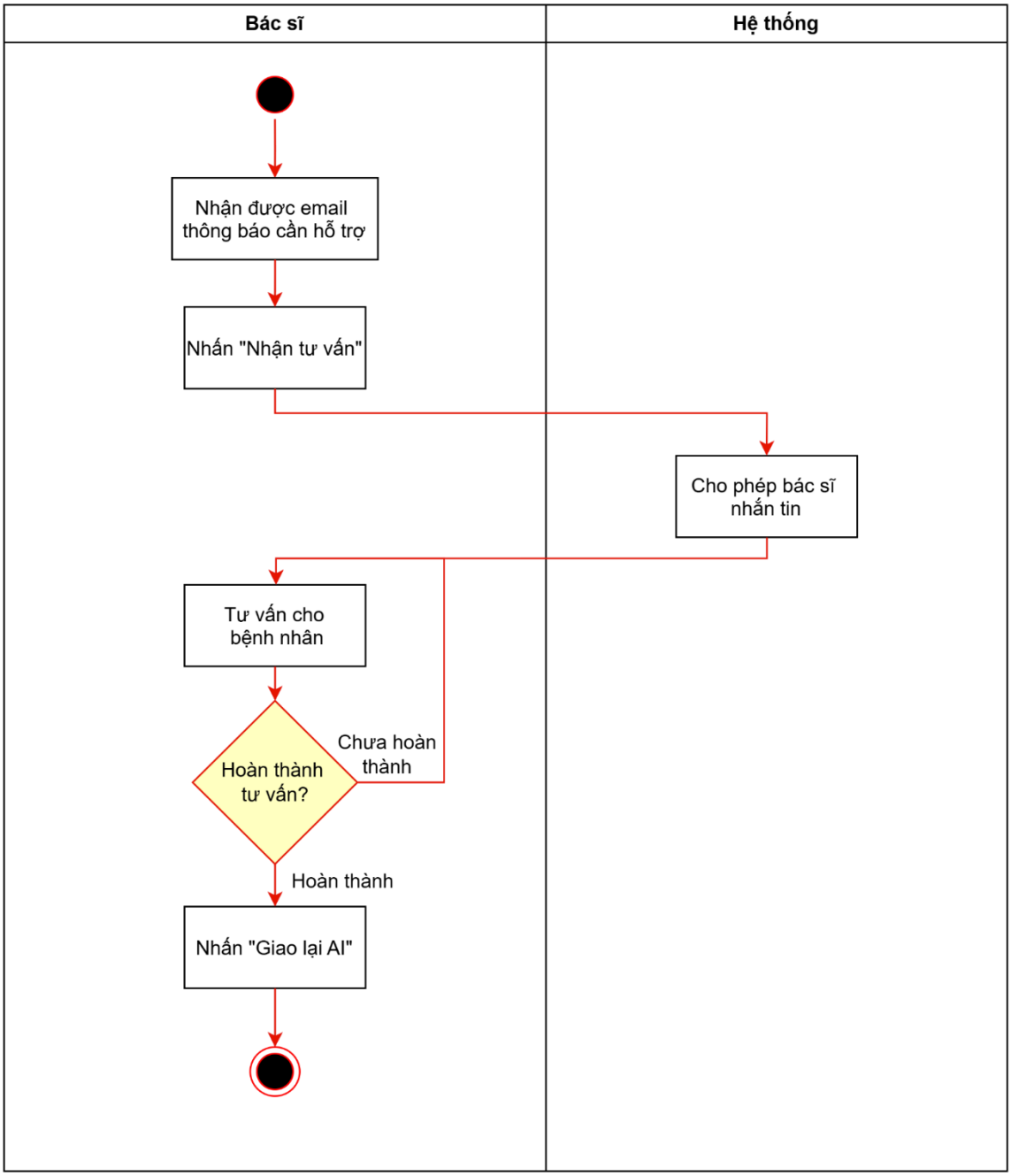
Hình 3.3. Sơ đồ mô tả quy trình đăng kí trở thành bác sĩ



Hình 3.4. Sơ đồ mô tả quy trình nhắn tin với Chatbot AI của hệ thống



Hình 3.5. Sơ đồ mô tả quy trình nhắn tin với Chatbot AI trợ lý của bác sĩ



Hình 3.6. Sơ đồ mô tả quy trình nhắn tin bác sĩ

3.2. Kiến trúc hệ thống tổng thể

3.2.1. Định hướng kiến trúc hệ thống

Hệ thống HealthAssistant được định hướng phát triển như một nền tảng hỗ trợ chăm sóc sức khỏe có khả năng mở rộng lâu dài, phục vụ đồng thời nhiều chức năng độc lập như quản lý người dùng, quản lý bác sĩ, đặt lịch khám, trò chuyện thời gian thực, gửi thông báo đặt lịch... Trong tương lai, hệ thống có thể tiếp tục tích hợp thêm các thành phần thông minh như hẹn lịch nhắc nhở, AI theo dõi sức khỏe hằng ngày,…

Vì vậy, kiến trúc hệ thống cần đáp ứng tốt các yêu cầu về khả năng mở rộng, tính linh hoạt và dễ bảo trì.

Sau quá trình phân tích đặc thù bài toán, nhóm quyết định lựa chọn kiến trúc Microservices thay cho mô hình Monolithic truyền thống dựa trên các định hướng chính sau:

- Phân tách nghiệp vụ độc lập: Hệ thống được chia thành nhiều dịch vụ riêng biệt như user-service, doctor-service, appointment-service, chat-service và notification-service. Mỗi service đảm nhiệm một miền nghiệp vụ cụ thể, giúp giảm phụ thuộc chéo và dễ dàng mở rộng chức năng trong tương lai.
- Tăng khả năng mở rộng hệ thống: Một số chức năng như chat thời gian thực hoặc tìm kiếm bác sĩ có tần suất truy cập cao hơn các module khác. Kiến trúc Microservices cho phép mở rộng độc lập từng dịch vụ theo đặc điểm tải riêng mà không cần scale toàn bộ hệ thống.
- Cải thiện khả năng bảo trì và triển khai: So với kiến trúc Monolithic, Microservices giúp giảm tình trạng phụ thuộc giữa các module, hạn chế việc phải build và deploy lại toàn bộ hệ thống khi chỉ thay đổi một chức năng nhỏ. Điều này giúp quá trình phát triển và bảo trì trở nên linh hoạt hơn.
- Hỗ trợ xử lý thời gian thực và bất đồng bộ: Hệ thống đồng thời tồn tại các tác vụ yêu cầu phản hồi tức thời (real-time chat, online status) và các tác vụ xử lý nền (gửi email hàng loạt, workflow automation). Việc tách service giúp tối ưu riêng cho từng loại workload và nâng cao hiệu năng tổng thể.
- Đảm bảo khả năng cô lập lỗi: Trong hệ thống y tế, các chức năng cốt lõi như đặt lịch khám, quản lý hồ sơ hoặc tìm kiếm bác sĩ cần duy trì hoạt động ổn định. Với kiến trúc Microservices, nếu chat-service hoặc notification-service gặp sự cố thì các chức năng nghiệp vụ quan trọng khác vẫn có thể hoạt động bình thường.
- Định hướng mở rộng trong tương lai: Kiến trúc Microservices tạo nền tảng thuận lợi để tích hợp thêm các module mới như AI Assistant, hệ thống gợi ý bác sĩ hoặc các dịch vụ phân tích dữ liệu mà không làm ảnh hưởng lớn đến các thành phần hiện có.
- Tăng tính độc lập dữ liệu giữa các dịch vụ: Mỗi Microservice được thiết kế theo hướng quản lý không gian dữ liệu riêng nhằm hạn chế phụ thuộc trực

tiếp giữa các service, từ đó nâng cao tính độc lập và khả năng phát triển lâu dài của hệ thống.

Từ những định hướng trên, kiến trúc Microservices được đánh giá là phù hợp với đặc thù của hệ thống HealthAssistant, đồng thời đáp ứng tốt yêu cầu mở rộng, ổn định và linh hoạt trong quá trình phát triển lâu dài.

3.2.2. Mô hình kiến trúc C4

Trong quá trình thiết kế kiến trúc hệ thống, việc mô tả rõ cấu trúc và mối quan hệ giữa các thành phần đóng vai trò quan trọng nhằm giúp người phát triển cũng như người đọc dễ dàng hình dung cách hệ thống hoạt động. Đối với các hệ thống có nhiều thành phần như HealthAssistant, việc sử dụng các sơ đồ kiến trúc truyền thống gây khó khăn trong việc theo dõi tổng thể hệ thống và luồng giao tiếp giữa các thành phần.

Để giải quyết vấn đề này, nhóm thực hiện lựa chọn sử dụng mô hình C4 nhằm trực quan hóa kiến trúc hệ thống theo nhiều mức độ khác nhau. Mô hình C4 được đề xuất bởi Simon Brown và hiện được sử dụng rộng rãi trong thiết kế kiến trúc phần mềm hiện đại, đặc biệt đối với các hệ thống phân tán và Microservices. Tên gọi "C4" được hình thành dựa trên bốn cấp độ mô tả kiến trúc gồm:

- Context (Ngữ cảnh hệ thống)
- Container (Các khối thực thi bên trong hệ thống)
- Component (Các thành phần bên trong container)
- Code (Mức mã nguồn)

Mỗi cấp độ sẽ tập trung mô tả hệ thống dưới một góc nhìn khác nhau, từ tổng quan đến chi tiết. Nhờ đó, người đọc có thể từng bước hiểu được:

- Hệ thống phục vụ ai.
- Hệ thống giao tiếp với các dịch vụ nào.
- Bên trong hệ thống gồm những thành phần gì.
- Các thành phần tương tác với nhau như thế nào.

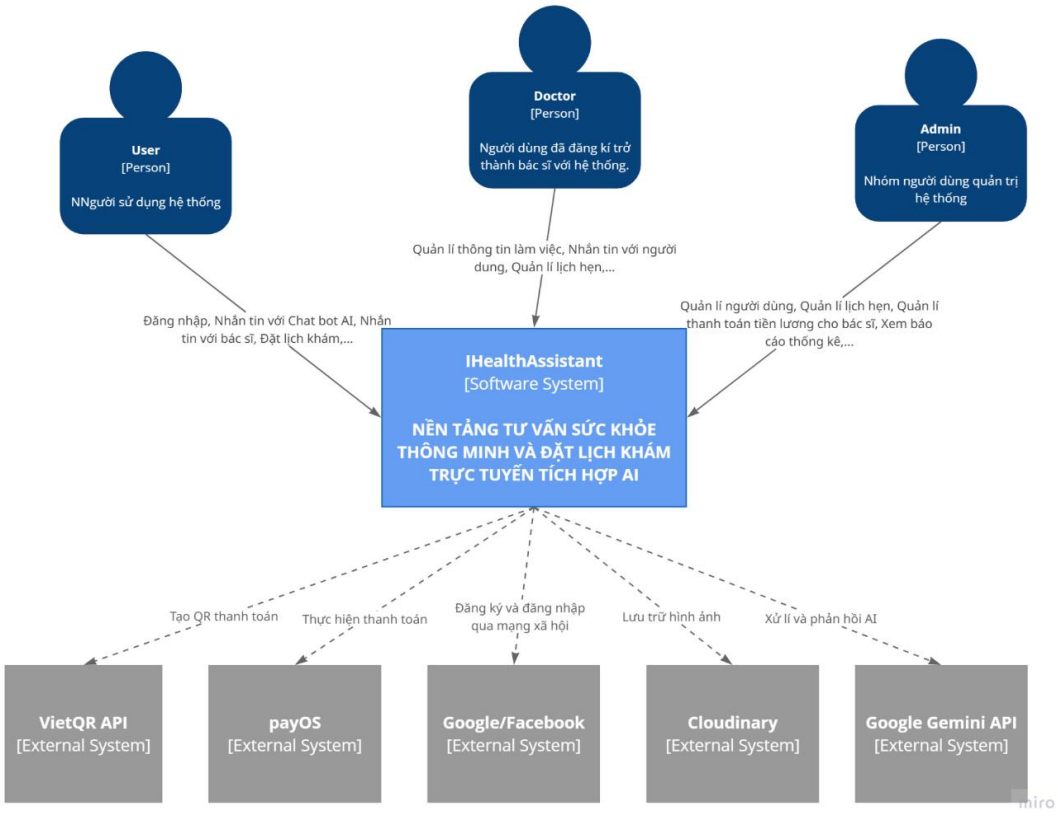
Khác với các sơ đồ UML truyền thống thường tập trung vào chi tiết kỹ thuật hoặc nghiệp vụ riêng lẻ, mô hình C4 hướng đến việc mô tả kiến trúc theo cách trực quan, dễ đọc và dễ tiếp cận hơn, kể cả đối với những người chưa có nhiều kinh nghiệm về

kiến trúc phần mềm. Việc áp dụng mô hình C4 giúp quá trình thiết kế và trình bày kiến trúc hệ thống trở nên trực quan hơn. Trong phạm vi đề tài HealthAssistant, nhóm thực hiện sử dụng hai mức mô tả chính của mô hình C4 gồm:

- C4 Context Diagram nhằm mô tả mối quan hệ tổng quan giữa hệ thống với người dùng và các hệ thống bên ngoài.
- C4 Container Diagram nhằm mô tả các thành phần chính bên trong hệ thống như frontend, API Gateway, các microservices, cơ sở dữ liệu và các dịch vụ hỗ trợ.

C4 Context

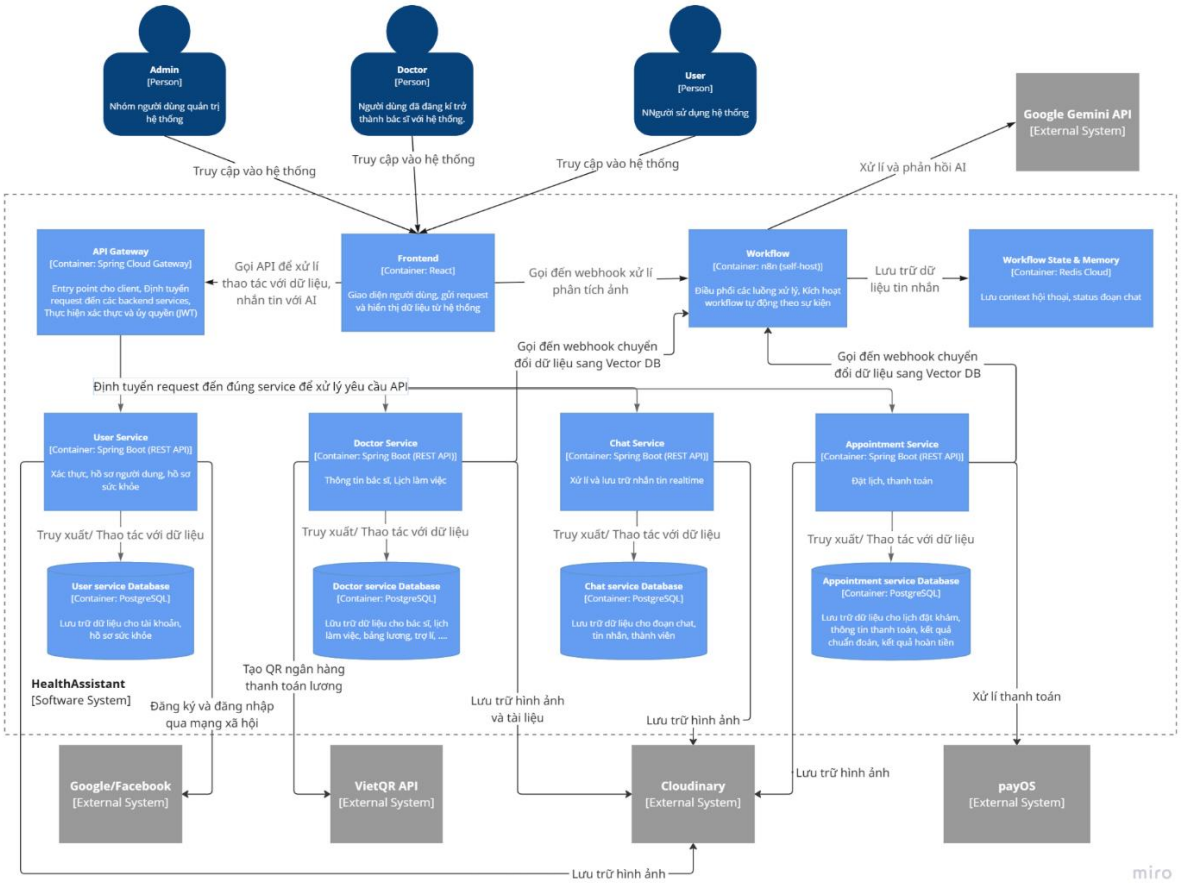
Biểu đồ Context mô tả mối quan hệ tổng quan giữa hệ thống HealthAssistant với các nhóm người dùng và các dịch vụ bên ngoài như hệ thống thanh toán, xác thực mạng xã hội, AI provider và dịch vụ lưu trữ đa phương tiện.



Hình 3.7. Sơ đồ C4 Context của hệ thống HealthAssistant

C4 Container

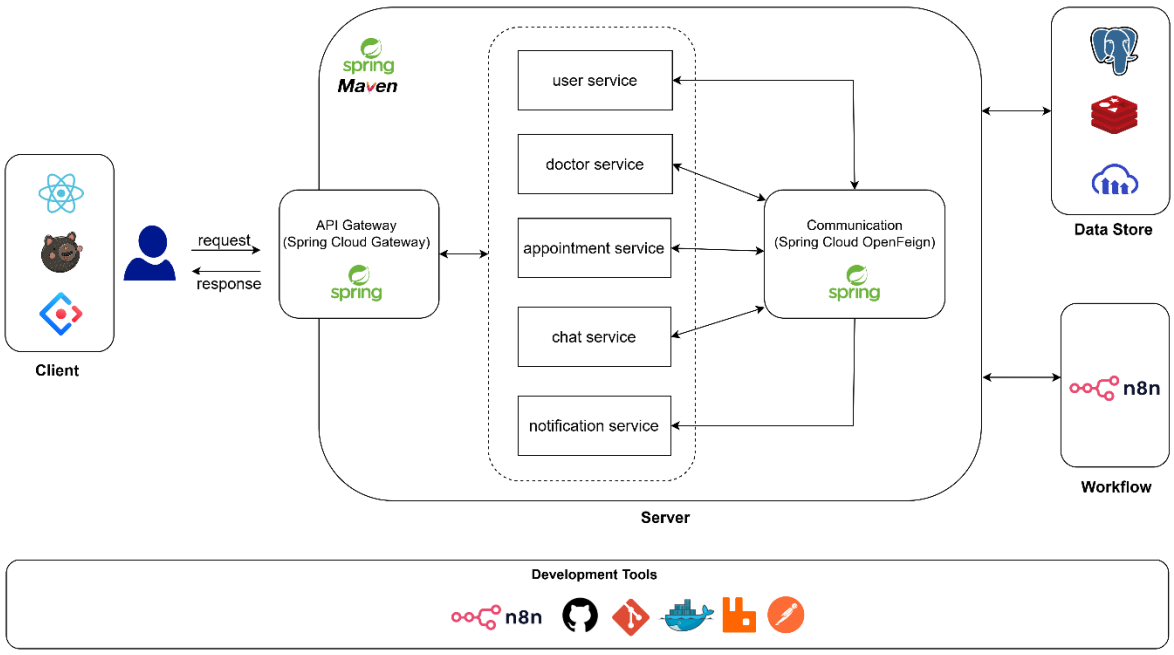
Biểu đồ Container mô tả các thành phần chính bên trong hệ thống HealthAssistant bao gồm frontend, API Gateway, các microservices, cơ sở dữ liệu và các thành phần hạ tầng hỗ trợ.



Hình 3.8. Sơ đồ C4 Container của hệ thống HealthAssistant

3.2.3. Sơ đồ kiến trúc tổng quan

Kiến trúc này bao gồm các thành phần từ phía giao diện người dùng (Client), lớp điều phối trung gian (API Gateway), các dịch vụ nghiệp vụ độc lập (Microservices), cơ chế giao tiếp nội bộ, cho đến tầng lưu trữ dữ liệu và xử lý tác vụ nền như trong **Error! Reference source not found.** sau.



Hình 3.9. Mô tả kiến trúc tổng thể của hệ thống HealthAssistant

Về mặt tổng quan, luồng hoạt động của hệ thống được diễn ra theo các bước sau:

Bước 1. Người dùng gửi yêu cầu từ phía Client:

Người dùng tương tác với hệ thống thông qua giao diện web được xây dựng bằng ReactJS [17]. Các thao tác như đăng nhập, tìm kiếm bác sĩ, đặt lịch khám hoặc nhắn tin đều được gửi đến hệ thống dưới dạng HTTP Request.

Bước 2. API Gateway tiếp nhận và điều phối request:

Toàn bộ request từ Client đều đi qua API Gateway đóng vai trò là cổng truy cập tập trung [24]. Thành phần này chịu trách nhiệm:

- Xác thực JWT Token.
- Kiểm tra quyền truy cập.
- Định tuyến request đến đúng Microservice tương ứng.
- Ẩn cấu trúc nội bộ của hệ thống phía sau Gateway.

Bước 3. Microservices xử lý nghiệp vụ độc lập:

Sau khi được định tuyến, request sẽ được chuyển đến các dịch vụ chuyên biệt:

- user-service: Xử lý các nghiệp vụ tài khoản và hồ sơ người dùng.
- doctor-service: Xử lý các nghiệp vụ bác sĩ.
- appointment-service: Xử lý các nghiệp vụ đặt lịch khám.
- chat-service: Xử lý nhắn tin thời gian thực.

- notification-service: Gửi thông báo email tự động.

Bước 4. Giao tiếp nội bộ giữa các dịch vụ:

Trong trường hợp một service cần sử dụng dữ liệu hoặc chức năng từ service khác, hệ thống sử dụng Spring Cloud OpenFeign [20] để thực hiện cơ chế gọi API nội bộ:

- appointment-service cần lấy thông tin bác sĩ từ doctor-service.
- chat-service cần xác thực thông tin người dùng từ user-service.

Bước 5. Tương tác với tầng dữ liệu và dịch vụ hỗ trợ:

Các Microservices tiếp tục tương tác với các thành phần phía dưới:

- PostgreSQL lưu trữ dữ liệu nghiệp vụ chính.
- Redis Cloud hỗ trợ quản lý trạng thái online.
- Cloudinary lưu trữ tài nguyên đa phương tiện.
- n8n hỗ trợ xử lý workflow tự động.
- RabbitMQ hỗ trợ xử lý gửi email bất đồng bộ.

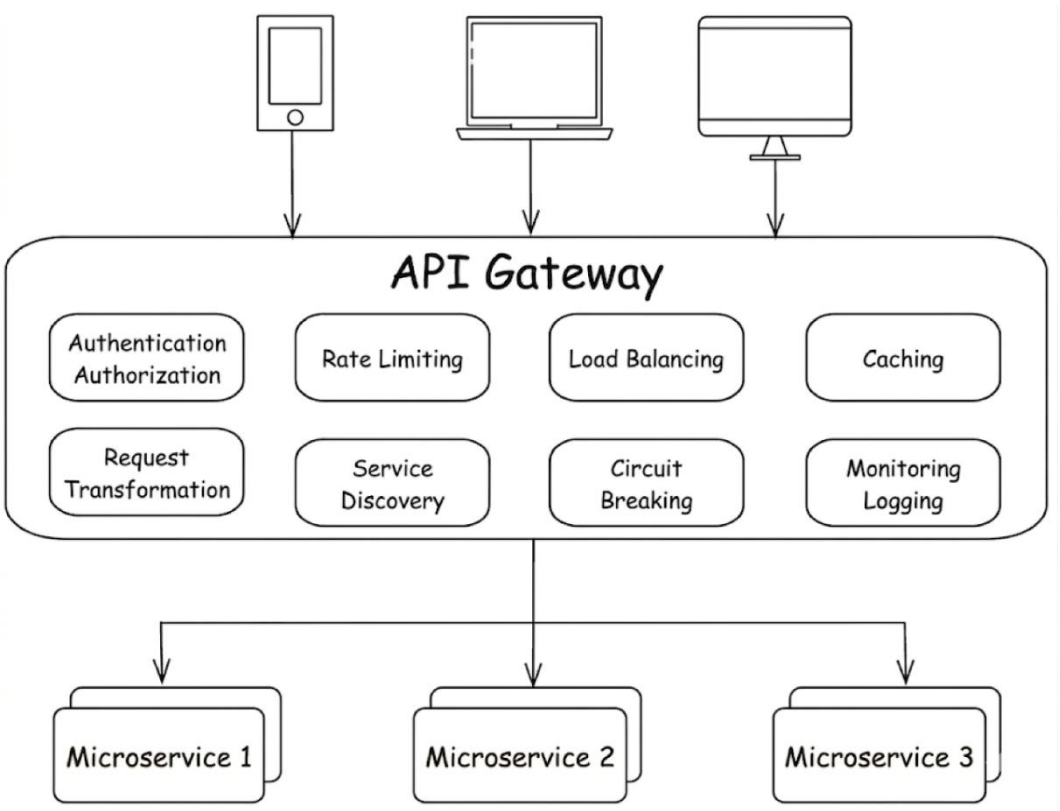
Bước 6. Xử lý bất đồng bộ và realtime:

Đối với các tác vụ tiêu tốn thời gian như gửi email hoặc thông báo hàng loạt, hệ thống sử dụng RabbitMQ để đưa message vào hàng đợi thay vì xử lý trực tiếp trên luồng request-response nhằm giảm thời gian phản hồi cho người dùng. Bên cạnh đó, chat-service sử dụng WebSocket để thiết lập kết nối hai chiều liên tục giữa client và server, cho phép truyền tải tin nhắn và trạng thái online theo thời gian thực với độ trễ thấp.

Bước 7. Trả kết quả về Client:

Sau khi xử lý hoàn tất, kết quả sẽ được phản hồi ngược lại từ Microservice → API Gateway → Client để hiển thị cho người dùng.

3.2.4. API Gateway



Hình 3.10. Một vài vai trò của API Gateway trong các hệ thống Microservices

Trong hệ thống HealthAssistant, toàn bộ request từ phía client đều phải đi qua API Gateway trước khi được chuyển tiếp đến các Microservices phía sau. API Gateway đóng vai trò là lớp giao tiếp và xác thực tập trung của hệ thống, giúp quản lý thống nhất quá trình xử lý request giữa frontend và backend service. Thay vì để client giao tiếp trực tiếp với từng service riêng lẻ, API Gateway hoạt động như một "điểm vào duy nhất", chịu trách nhiệm tiếp nhận request, kiểm tra tính hợp lệ và định tuyến đến service tương ứng. Trong hệ thống HealthAssistant, API Gateway đảm nhiệm các chức năng chính gồm:

- Kiểm tra JWT Token từ request gửi lên.
- Xác thực tính hợp lệ của token.
- Thiết lập CORS nhằm kiểm soát quyền truy cập từ phía frontend.
- Quản lý whitelist endpoint cho các API công khai không yêu cầu xác thực.
- Trích xuất thông tin người dùng từ token nha accountId và role.
- Chuyển tiếp request đến đúng Microservice tương ứng.

Sau khi xác thực thành công, Gateway sẽ chuyển tiếp các thông tin định danh người dùng như accountId và role đến các service phía sau nhằm phục vụ cho quá trình xử lý nghiệp vụ nội bộ và phân quyền chức năng theo từng vai trò người dùng. Việc tập trung cơ chế xác thực tại API Gateway giúp giảm sự lặp lại logic bảo mật giữa các service, đồng thời tăng tính nhất quán trong toàn hệ thống. Ngoài ra, mô hình này còn giúp hệ thống dễ mở rộng, dễ bảo trì và tăng khả năng kiểm soát bảo mật trong kiến trúc Microservices.

3.2.5. Giao tiếp giữa các thành phần

3.2.5.2. Thiết kế API

Hệ thống HealthAssistant được xây dựng theo kiến trúc Microservices, trong đó mỗi service đảm nhiệm một nhóm chức năng riêng biệt như quản lý người dùng, bác sĩ, lịch hẹn và trò chuyện trực tuyến. Để các thành phần trong hệ thống có thể giao tiếp hiệu quả, đề tài sử dụng REST API làm chuẩn giao tiếp chính kết hợp với WebSocket cho các chức năng thời gian thực. Việc thiết kế API theo hướng tách biệt giúp hệ thống dễ mở rộng, thuận tiện trong bảo trì và hỗ trợ phát triển độc lập giữa các service.

Thiết kế REST API

REST API là mô hình thiết kế API phổ biến sử dụng giao thức HTTP để trao đổi dữ liệu giữa client và server. REST API hoạt động theo nguyên tắc stateless, nghĩa là mỗi request đều chứa đầy đủ thông tin cần thiết để server xử lý mà không cần lưu trạng thái trước đó. Trong REST API, các tài nguyên được biểu diễn thông qua URL và thao tác xử lý dữ liệu được thực hiện thông qua các phương thức HTTP như:

- GET: Lấy dữ liệu
- POST: Tạo mới dữ liệu
- PUT/PATCH: Cập nhật dữ liệu
- DELETE: Xóa dữ liệu

Hệ thống HealthAssistant sử dụng REST API làm cơ chế giao tiếp chính giữa frontend ReactJS và các backend services. Các endpoint được thiết kế theo mô hình resource-based, đại diện cho từng tài nguyên cụ thể trong hệ thống như người dùng, bác sĩ, lịch hẹn hoặc tin nhắn. Dữ liệu trao đổi giữa client và server sử dụng định dạng JSON nhằm đảm bảo tính phổ biến và dễ tích hợp giữa các nền tảng khác nhau.Việc

áp dụng REST API giúp hệ thống dễ kiểm thử, dễ mở rộng và phù hợp với mô hình frontend/backend tách biệt của kiến trúc hiện đại.

Thiết kế API theo từng service

Dựa trên kiến trúc Microservices, hệ thống được chia thành nhiều service độc lập. Mỗi service cung cấp một nhóm API riêng phục vụ cho từng chức năng cụ thể của hệ thống.

Bảng 3.1. Tổng quan thiết kế API cho user-service

user-service		
Mô tả	user-service chịu trách nhiệm quản lý thông tin người dùng, xác thực tài khoản và hồ sơ sức khỏe cá nhân. Các nhóm API chính bao gồm: - /auth/*: Xác thực và quản lý phiên đăng nhập - /account/*: Quản lý thông tin tài khoản - /health-profile/*: Quản lý hồ sơ sức khỏe Một số endpoint trong service:	
Method	**Endpoint**	**Chức năng**
POST	/auth/sign-in	Đăng nhập hệ thống
POST	/auth/sign-up	Đăng ký tài khoản
GET	/account/{id}	Lấy thông tin tài khoản theo accoutnId
PUT	/account/{id}	Cập nhật thông tin tài khoản theo accoutnId
GET	/health-profile/account/{accountId}	Lấy hồ sơ sức khỏe theo accountId
POST	/health-profile/	Tạo hồ sơ sức khỏe mới

Bảng 3.2. Tổng quan thiết kế API cho doctor-service

doctor-service		
Mô tả	doctor-service quản lý thông tin bác sĩ, lịch làm việc, yêu cầu bác sĩ. bảng lương và trợ lý Chatbot AI. Các nhóm API chính bao gồm: - /doctor/*: Quản lý thông tin bác sĩ - /doctor-schedule/*: Quản lý lịch làm việc - /salary/*: Quản lý chi trả lương cho bác sĩ - /doctor-request/*: Quản lý yêu cầu bác sĩ - /assistants/*: Quản lý trợ lý ChatBot AI của bác sĩ - /assistant-profiles/*: Quản lý tri thức cho ChatBot AI Một số endpoint trong service:	
Method	**Endpoint**	**Chức năng**
GET	/doctor	Lấy danh sách bác sĩ
GET	/doctor/{id}	Lấy chi tiết bác sĩ
GET	/doctor-schedule	Lấy lịch làm việc
GET	/doctor-request/doctor/{doctorId}	Lấy các yêu cầu theo doctorId
GET	/salary/{id}	Lấy bảng lương theo mã
POST	/assistant-profiles /doctor/{doctorId}	Lấy danh sách trợ lý theo doctorId

Bảng 3.3. Tổng quan thiết kế API cho chat-service

chat-service		
Mô tả	chat-service phục vụ chức năng trò chuyện trực tuyến giữa người dùng và bác sĩ. Service này kết hợp giữa REST API và WebSocket nhằm hỗ trợ realtime communication. Các nhóm API chính bao gồm: - /chat/*: Quản lý cuộc trò chuyện - /message/*: Quản lý tin nhắn Một số endpoint trong service:	
Method	**Endpoint**	**Chức năng**

GET	/chat	Lấy danh sách cuộc trò chuyện
GET	/chat/conversations/{accountId}	Lấy danh sách cuộc trò chuyện theo accountId
GET	/message/chat/{chatId}	Lấy lịch sử tin nhắn theo chatId

Bảng 3.4. Tổng quan thiết kế API cho appointment-service

appointment-service		
Mô tả	appointment -service chịu trách nhiệm xử lý các chức năng liên quan đến đặt lịch khám, thanh toán và kết quả khám bệnh.. Các nhóm API chính bao gồm: - /appointment/*: Quản lý lịch hẹn khám bệnh - /appointment/result/*: Quản lý kết quả khám bệnh Một số endpoint trong service:	
Method	**Endpoint**	**Chức năng**
GET	/appointment/account/{accountId}	Lấy lịch hẹn theo người dùng
PUT	/appointment/refund	Hoàn tiền lịch hẹn
POST	/appointment/result/{appointmentId}	Thêm kết quả khám theo mã lịch hẹn
GET	/appointment/result/{resultId}	Lấy kết quả khám theo mã kết quả

Thiết kế giao tiếp thời gian thực qua Websocket

Đối với các chức năng yêu cầu cập nhật dữ liệu theo thời gian thực như trò chuyện trực tuyến giữa người dùng và bác sĩ, hệ thống sử dụng WebSocket để duy trì kết nối hai chiều liên tục giữa client và server.

Khác với REST API hoạt động theo cơ chế request-response truyền thống, WebSocket cho phép server chủ động gửi dữ liệu đến client ngay khi có sự kiện mới

phát sinh mà không cần client gửi request liên tục. Trong hệ thống HealthAssistant, WebSocket được sử dụng cho các chức năng:

- Gửi và nhận tin nhắn realtime
- Đồng bộ trạng thái cuộc trò chuyện
- Hiển thị tin nhắn tức thời giữa người dùng và bác sĩ
- Giảm độ trễ trong quá trình trao đổi dữ liệu

Việc kết hợp REST API và WebSocket giúp hệ thống vừa đảm bảo khả năng quản lý dữ liệu ổn định, vừa đáp ứng tốt các chức năng tương tác thời gian thực trong quá trình tư vấn trực tuyến.

3.2.5.1. Giao tiếp giữa các services

Trong kiến trúc Microservices, các dịch vụ nghiệp vụ không hoạt động hoàn toàn tách biệt mà thường xuyên cần trao đổi dữ liệu và phối hợp xử lý để hoàn thành một tác vụ tổng thể. Vì vậy, hệ thống được thiết kế theo mô hình kết hợp giữa giao tiếp đồng bộ và giao tiếp bất đồng bộ nhằm cân bằng giữa tốc độ phản hồi, hiệu năng xử lý và khả năng mở rộng hệ thống.

Giao tiếp đồng bộ với OpenFeign

Đối với các tác vụ yêu cầu phản hồi tức thời hoặc cần dữ liệu trực tiếp từ service khác trong quá trình xử lý request, hệ thống sử dụng Spring Cloud OpenFeign để thực hiện cơ chế giao tiếp đồng bộ giữa các Microservices thông qua REST API [20]. Một ví dụ về cơ chế này được áp dụng trong trường hợp: appointment-service cần gọi doctor-service để kiểm tra thông tin bác sĩ và lịch khám.

Việc sử dụng OpenFeign giúp đơn giản hóa quá trình gọi API nội bộ, giảm lặp lại và tăng tính nhất quán trong giao tiếp giữa các dịch vụ. Ngoài ra, các service không cần quan tâm trực tiếp đến chi tiết triển khai của nhau mà chỉ cần giao tiếp thông qua các endpoint đã được chuẩn hóa.

Giao tiếp bất đồng bộ với RabbitMQ

Đối với tác vụ tiêu tốn thời gian xử lý hoặc không yêu cầu phản hồi ngay lập tức, hệ thống sử dụng RabbitMQ làm Message Broker nhằm triển khai cơ chế giao tiếp bất đồng bộ giữa các dịch vụ [23]. Cơ chế này chủ yếu được áp dụng cho các tác vụ: Gửi loại email nào đó sau khi thực hiện một hành động nào đó.

Khi một sự kiện xảy ra, service nguồn sẽ gửi message vào RabbitMQ thay vì xử lý trực tiếp tác vụ đó. Các service phía sau sẽ đóng vai trò consumer để tiếp nhận và xử lý message độc lập.

Ví dụ: Sau khi appointment-service tạo lịch khám thành công, hệ thống sẽ đẩy một message vào queue. Sau đó, notification-service sẽ consume message này để thực hiện gửi email xác nhận lịch hẹn cho người dùng.

Cách tiếp cận này giúp giảm tải cho luồng xử lý chính, tăng tốc độ phản hồi API và hạn chế tình trạng blocking request khi số lượng người dùng tăng cao.

Chiến lược kết hợp Sync và Async Communication

Hệ thống không sử dụng duy nhất một cơ chế giao tiếp mà kết hợp đồng thời cả synchronous và asynchronous communication nhằm tối ưu cho từng loại nghiệp vụ cụ thể: Các nghiệp vụ cần phản hồi trực tiếp và dữ liệu realtime sử dụng OpenFeign. Các tác vụ nền hoặc xử lý kéo dài sử dụng RabbitMQ để xử lý bất đồng bộ. Cách tiếp cận này giúp hệ thống đạt được sự cân bằng giữa:

- Tốc độ phản hồi
- Khả năng mở rộng
- Hiệu năng xử lý
- Tính ổn định trong môi trường Microservices

3.2.6. Kiến trúc dữ liệu

Kiến trúc dữ liệu của hệ thống HealthAssistant được thiết kế theo định hướng phân tách dữ liệu theo từng miền nghiệp vụ nhằm đảm bảo tính độc lập giữa các Microservices, tăng khả năng mở rộng và giảm phụ thuộc chéo trong quá trình phát triển hệ thống.

Thay vì sử dụng một cơ sở dữ liệu dùng chung cho toàn bộ hệ thống, mỗi service sẽ quản lý một database riêng biệt bên trong cùng hệ quản trị PostgreSQL. Cách tiếp cận này giúp mỗi service sở hữu và kiểm soát hoàn toàn dữ liệu thuộc phạm vi nghiệp vụ của mình, đồng thời hạn chế việc truy cập trực tiếp dữ liệu giữa các service. Kiến trúc dữ liệu của hệ thống được xây dựng theo hướng kết hợp nhiều mô hình lưu trữ khác nhau nhằm tối ưu cho từng loại dữ liệu và từng loại công việc cụ thể.

- PostgreSQL phù hợp với dữ liệu nghiệp vụ cần tính toàn vẹn cao.
- Redis Cloud được sử dụng để lưu trữ dữ liệu tạm thời như bộ nhớ hội thoại của workflow AI và trạng thái kết nối realtime của người dùng.
- RabbitMQ hỗ trợ xử lý message bất đồng bộ.
- Cloudinary phục vụ lưu trữ tài nguyên đa phương tiện.

PostgreSQL - Cơ sở dữ liệu nghiệp vụ chinh

Hệ thống sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu quan hệ trung tâm cho các Microservices:

- user-service
- doctor-service
- appointment-service
- chat-service

PostgreSQL được lựa chọn nhờ khả năng đảm bảo tính toàn vẹn dữ liệu, hỗ trợ transaction mạnh mẽ và phù hợp với các nghiệp vụ có tính ràng buộc cao trong lĩnh vực y tế.

Redis Cloud - Trạng thái hoạt động và lịch sử hội thoại

Bên cạnh PostgreSQL, hệ thống sử dụng Redis Cloud nhằm hỗ trợ các tác vụ yêu cầu tốc độ truy xuất cao và xử lý thời gian thực. Redis Cloud được sử dụng chủ yếu cho:

- Lưu trạng thái online của người dùng.
- Lưu lịch sử hội thoại phục vụ bộ nhớ ngữ cảnh cho các AI Agent.

Trong chức năng AI HealthAssistant, việc lưu lịch sử hội thoại trên Redis Cloud giúp AI có thể duy trì ngữ cảnh trò chuyện liên tục, từ đó tạo cảm giác giao tiếp tự nhiên và thông minh hơn thay vì xử lý từng tin nhắn độc lập. Lịch sử tin nhắn dài hạn vẫn được lưu xuống PostgreSQL nhằm đảm bảo khả năng đồng bộ dữ liệu và hiển thị lại trên giao diện người dùng. Do Redis hoạt động trên bộ nhớ RAM nên tốc độ đọc/ghi rất nhanh, phù hợp với các dữ liệu có tính realtime hoặc dữ liệu tạm thời cần truy xuất thường xuyên.

RabbitMQ - Kiến trúc Message Queue bất đồng bộ

Hệ thống sử dụng RabbitMQ làm Message Broker nhằm xử lý tác vụ bất đồng bộ liên quan đến gửi email thông báo. Khi xảy ra các sự kiện nghiệp vụ như đặt lịch khám thành công hoặc hủy lịch khám, các dịch vụ nguồn (ví dụ appointment-service) sẽ gửi yêu cầu HTTP đến notification-service để khởi tạo quá trình gửi email thông báo. Tuy nhiên, thay vì thực hiện gửi email trực tiếp trên luồng xử lý request-response, notification-service sẽ chuyển đổi dữ liệu thành message và publish vào RabbitMQ thông qua cơ chế Message Queue bất đồng bộ. Sau đó, các thành phần consumer được triển khai bằng cơ chế @RabbitListener sẽ tiếp nhận các message từ hàng đợi RabbitMQ và thực hiện quy trình gửi email độc lập ở phía sau. Cách tiếp cận này giúp giảm thời gian phản hồi cho request chính, hạn chế blocking trong quá trình xử lý và tăng khả năng chịu tải của hệ thống khi số lượng email cần gửi tăng cao. Cách tiếp cận này giúp:

- Giảm thời gian phản hồi cho người dùng.
- Hạn chế blocking request.
- Tăng khả năng xử lý tải đồng thời.
- Nâng cao tính ổn định của hệ thống.

Cloudinary - Quản lý tài nguyên đa phương tiện

Đối với các dữ liệu đa phương tiện, hệ thống sử dụng Cloudinary làm nền tảng lưu trữ đám mây nhằm giảm tải dung lượng cho server backend và tối ưu băng thông truyền tải [22]. Các tài nguyên được lưu trữ trên Cloudinary bao gồm:

- Ảnh: Đại diện người dùng, kết quả khám bệnh, hình trong tin nhắn.
- Tài liệu: tài nguyên tri thức (KB) do bác sĩ cung cấp nhằm phục vụ các vai trò trợ lý AI tương ứng.

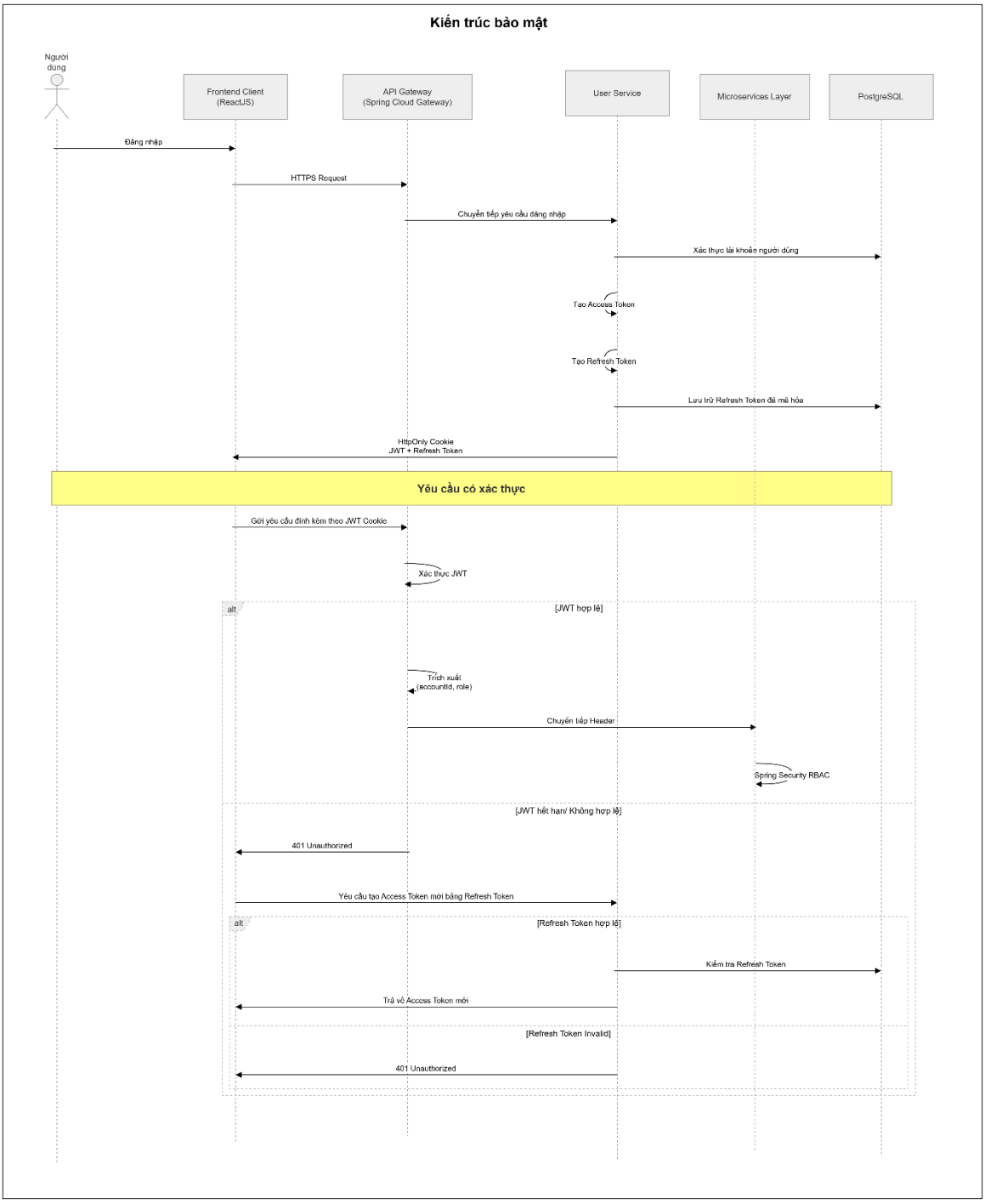
Việc tách riêng nơi lưu trữ tài nguyên khỏi hệ thống backend giúp cải thiện hiệu năng, tăng khả năng mở rộng và đơn giản hóa quá trình quản lý tài nguyên tĩnh. Một vài vai trò chính của Cloudinary [22]:

- Lưu trữ dữ liệu đa phương tiện.
- Giảm tải cho server backend.
- Tối ưu tốc độ truy cập tài nguyên.
- Hỗ trợ mở rộng lưu trữ linh hoạt.

3.2.7. Kiến trúc bảo mật

Do hệ thống HealthAssistant xử lý nhiều dữ liệu nhạy cảm liên quan đến tài khoản người dùng, lịch sử khám bệnh và thông tin trao đổi giữa bệnh nhân với bác sĩ, kiến trúc bảo mật được thiết kế theo hướng tập trung vào xác thực tập trung, phân quyền phân tán và cơ chế xác thực phi trạng thái nhằm tăng tính bảo mật và khả năng mở rộng trong môi trường Microservices:

- Xác thực tập trung: Là cơ chế quy gom toàn bộ quy trình kiểm tra danh tính người dùng về một đầu mối duy nhất (tại API Gateway). Thay vì mỗi dịch vụ phải tự thực hiện việc kiểm tra tên đăng nhập và mật khẩu, hành động xác thực chỉ diễn ra một lần duy nhất tại cửa ngõ hệ thống. Cơ chế này giúp giảm thiểu bề mặt tấn công, đồng bộ hóa chính sách bảo mật và đơn giản hóa việc quản lý vòng đời phiên làm việc của người dùng.
- Phân quyền phân tán: Khác với xác thực, việc quyết định một người dùng có quyền thực hiện một hành động cụ thể hay không (ví dụ: Bác sĩ được xem bệnh án, Bệnh nhân không được sửa lịch khám) sẽ được giao cho từng Microservice tự quyết định dựa trên ngữ cảnh nghiệp vụ riêng biệt. Sau khi API Gateway xác thực thành công, thông tin về vai trò và quyền hạn của người dùng sẽ được đính kèm vào mã token để chuyển tiếp xuống các dịch vụ phía sau, giúp các dịch vụ này tự chủ trong việc kiểm soát truy mà không cần gọi ngược lại máy chủ xác thực.
- Cơ chế xác thực phi trạng thái: Là giải pháp xác thực không lưu trữ trạng thái phiên làm việc trên bộ nhớ của máy chủ backend. Thay vào đó, mọi thông tin nhận diện và quyền hạn của người dùng đều được đóng gói trọn gói vào một mã token tự chứa (Self-contained token, điển hình là JWT) lưu ở phía Client. Mỗi khi Client gửi yêu cầu, máy chủ chỉ cần giải mã và kiểm tra tính toàn vẹn của chữ ký số trên token đó để chấp nhận hoặc từ chối. Cơ chế phi trạng thái này giúp triệt tiêu sự phụ thuộc vào bộ nhớ máy chủ, cho phép hệ thống mở rộng quy mô các Microservices một cách linh hoạt.



Hình 3.11. Luồng xử lý xác thực của các yêu cầu từ gửi dùng đến hệ thống

Cơ chế xác thực bằng JWT



Hình 3.12. Minh họa đơn giản về xác thực bằng JWT

Hệ thống sử dụng JSON Web Token (JWT) làm cơ chế xác thực chính giữa Client và backend. Sau khi người dùng đăng nhập thành công, user-service sẽ tạo Access Token và Refresh Token để phục vụ quá trình xác thực và duy trì phiên đăng nhập.

- Access Token được sử dụng cho các request thông thường.
- Refresh Token được sử dụng để cấp lại Access Token khi token cũ hết hạn.

Refresh Token chỉ được tạo khi người dùng lựa chọn cơ chế "Remember Me", giúp cân bằng giữa trải nghiệm người dùng và khả năng kiểm soát phiên đăng nhập.

Để tăng mức độ an toàn, Refresh Token không được lưu trực tiếp dưới dạng raw token trong cơ sở dữ liệu mà được mã hóa băm (hash) trước khi lưu trữ nhằm hạn chế rủi ro rò rỉ thông tin xác thực. Khi Access Token hết hạn: hệ thống sẽ thử tạo lại Access Token mới thông qua Refresh Token, nếu quá trình này thất bại, người dùng sẽ bị đăng xuất và yêu cầu xác thực lại.

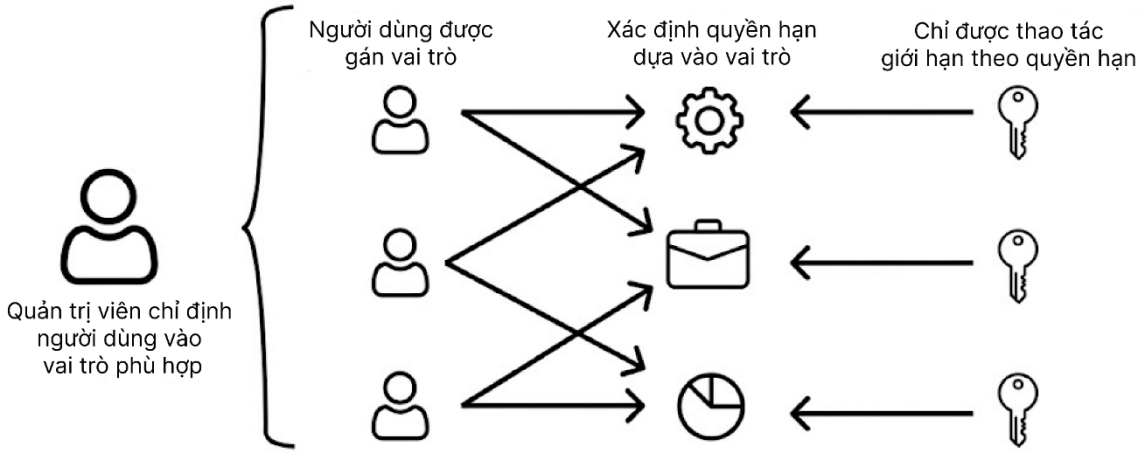
Bảo mật Cookie và lưu trữ Token

JWT được lưu trữ phía client thông qua HttpOnly Cookie thay vì localStorage nhằm hạn chế nguy cơ đánh cắp token thông qua JavaScript phía trình duyệt. Hệ thống cấu hình:

- HttpOnly để ngăn truy cập token từ client-side script.
- SameSite=None nhằm hỗ trợ giao tiếp cross-origin giữa frontend và backend trong môi trường tách biệt domain.
- Secure=false trong môi trường phát triển nội bộ; trong môi trường production có thể kích hoạt Secure=true để đảm bảo cookie chỉ được truyền tải thông qua HTTPS.

Cách tiếp cận này giúp tăng tính an toàn trong quá trình quản lý phiên đăng nhập đồng thời phù hợp với kiến trúc frontend-backend tách biệt của hệ thống.

Phân quyền theo vai trò (RBAC)



Hình 3.13. Sơ đồ minh họa cơ chế phân quyền dựa trên vai trò

Hệ thống áp dụng mô hình Role-Based Access Control (RBAC) nhằm kiểm soát quyền truy cập theo từng nhóm người dùng như:

- USER
- DOCTOR
- ADMIN
- ADMIN_MANAGE_DOCTOR
- ADMIN_REVIEWER_DOCTOR
- ADMIN_SUPPORT

Mặc dù Gateway chịu trách nhiệm xác thực JWT, việc kiểm tra quyền truy cập cụ thể vẫn được thực hiện riêng tại từng Microservice thông qua Spring Security [20]. Điều này giúp các service chủ động kiểm soát nghiệp vụ bảo mật của riêng mình thay vì phụ thuộc hoàn toàn vào Gateway. Qua đó giúp tăng tính linh hoạt, giảm phụ thuộc giữa các thành phần và phù hợp với đặc thù của kiến trúc Microservices.

Bảo mật dữ liệu người dùng và mật khẩu

Hệ thống có sử dụng thư viện mã hóa để mã hóa mật khẩu người dùng, mã hóa token, mã hóa số ngân hang của bác sĩ trước khi lưu trữ vào cơ sở dữ liệu. Điều này giúp hạn chế rủi ro lộ thông tin ngay cả khi dữ liệu bị truy cập trái phép.

Ngoài ra, các dữ liệu đa phương tiện như: ảnh đại diện, tài liệu khám bệnh, hình ảnh trao đổi trong chat được lưu trữ tách biệt trên Cloudinary nhằm giảm tải cho hệ thống backend và hỗ trợ quản lý tài nguyên linh hoạt hơn.

3.2.8. Triển khai hệ thống

Hệ thống HealthAssistant được triển khai theo mô hình frontend và backend tách biệt nhằm tăng khả năng mở rộng, dễ bảo trì và phù hợp với kiến trúc Microservices. Các thành phần trong hệ thống được triển khai trên nhiều nền tảng khác nhau tùy theo đặc thù sử dụng và yêu cầu vận hành. Trong quá trình phát triển, frontend được triển khai độc lập trên nền tảng cloud hosting, trong khi các backend service và hạ tầng hỗ trợ được container hóa và triển khai đồng thời thông qua Docker Compose.

Triển khai Frontend

Phần frontend của hệ thống được xây dựng bằng ReactJS và triển khai trên nền tảng Netlify nhằm hỗ trợ quá trình build và publish ứng dụng web nhanh chóng. Netlify là nền tảng cloud hosting chuyên dùng để triển khai các ứng dụng web tĩnh và frontend hiện đại như ReactJS, VueJS, Angular, đồng thời hỗ trợ các ứng dụng có Server-Side Rendering thông qua serverless functions. Nền tảng này hỗ trợ triển khai tự động thông qua Git repository và cung cấp các tính năng:

- CI/CD tự động
- HTTPS miễn phí cho mọi dự án
- CDN toàn cầu giúp tăng tốc độ truy cập (thế nhưng hạ tầng chủ yếu đặt tại Mỹ, độ trễ với người dùng Việt Nam có thể cao hơn so với các nền tảng tại Đông Nam Á)
- Hỗ trợ routing cho Single Page Application
- Dễ dàng rollback phiên bản khi có lỗi nhờ mỗi lần deploy được lưu trữ độc lập

Frontend đóng vai trò giao diện tương tác chính giữa người dùng và hệ thống HealthAssistant, đồng thời thực hiện giao tiếp với backend thông qua API Gateway.

Triển khai Backend và các dịch vụ hỗ trợ

Backend và các dịch vụ liên quan của hệ thống HealthAssistant được triển khai trên môi trường Linux VPS nhằm đảm bảo khả năng vận hành ổn định và hỗ trợ triển khai đồng thời nhiều service trong kiến trúc Microservices. Toàn bộ các backend

service và hạ tầng hỗ trợ được container hóa bằng Docker và quản lý tập trung thông qua Docker Compose. Việc sử dụng VPS giúp hệ thống có môi trường runtime độc lập, hỗ trợ triển khai các service backend, database và các thành phần hỗ trợ khác trong cùng một hạ tầng máy chủ. Trong mô hình hiện tại, các service được khởi chạy dưới dạng container bao gồm:

- api-gateway
- user-service
- doctor-service
- appointment-service
- chat-service
- notification-service

Bên cạnh đó, hệ thống còn triển khai các dịch vụ hỗ trợ:

- PostgreSQL
- RabbitMQ
- n8n

Docker Compose được sử dụng nhằm:

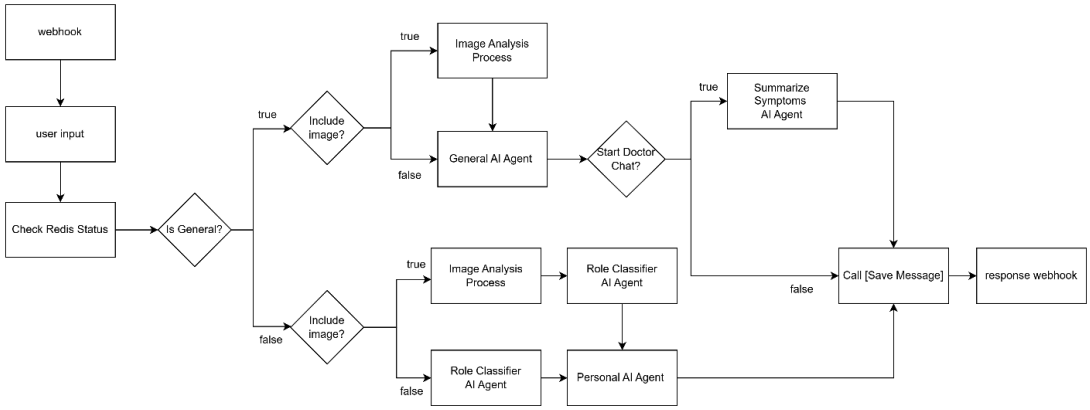
- Khởi chạy đồng thời toàn bộ service.
- Thiết lập network nội bộ giữa các container.
- Đồng bộ môi trường triển khai giữa quá trình phát triển và vận hành thực tế.

Thông qua mô hình triển khai này, hệ thống có thể dễ dàng mở rộng các service độc lập, đồng thời hỗ trợ bảo trì và cập nhật từng thành phần mà không ảnh hưởng đến toàn bộ hệ thống.

3.3. Quy trình xử lý của AI Agent và Luồng tự động hóa (n8n)

Trong hệ thống HealthAssistant, n8n được sử dụng như nền tảng điều phối trung tâm cho các quy trình xử lý AI. n8n đóng vai trò kết nối giữa hệ thống backend, các AI Agent, mô hình LLM bộ nhớ hội thoại và KB. Do hệ thống bao gồm nhiều bước xử lý và nhiều nhánh chức năng khác nhau, workflow được thiết kế theo mô hình node-based nhằm hỗ trợ quản lý trực quan, dễ mở rộng và thuận tiện trong quá trình bảo trì hệ thống.

3.3.1. Workflow tổng quan của hệ thống AI

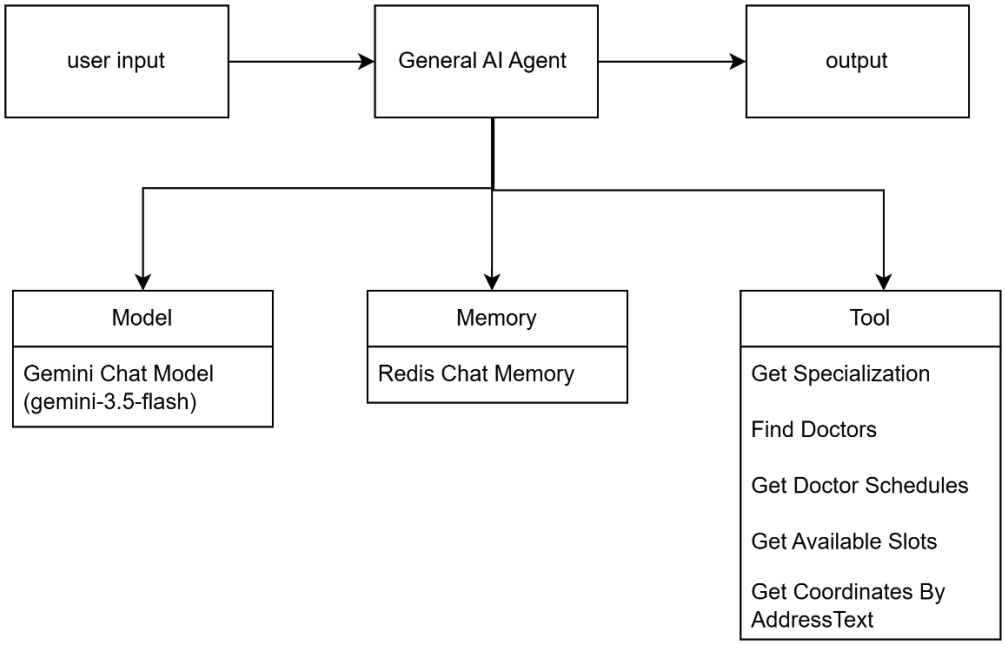


Hình 3.14. Workflow tổng quan của hệ thống AI

Khi người dùng gửi yêu cầu từ giao diện web, dữ liệu sẽ được chuyển đến hệ thống thông qua Webhook. Sau khi tiếp nhận dữ liệu đầu vào và kiểm tra trạng thái bộ nhớ hội thoại trong Redis, hệ thống sẽ phân loại yêu cầu để lựa chọn nhánh phù hợp. Nếu đầu vào là cuộc tư vấn tổng quát, hệ thống sẽ chuyển đến General AI Agent. Nếu đầu vào là cuộc tư vấn riêng với bác sĩ, hệ thống sẽ phân loại vai trò bằng Role Classifier AI Agent trước khi vào Personal AI Agent.

Trong cả hai nhánh, hệ thống đều hỗ trợ xử lý hình ảnh nếu người dùng gửi kèm ảnh liên quan đến chỉ số sức khỏe. Kết quả phân tích hình ảnh sẽ được bổ sung vào ngữ cảnh hội thoại trước khi gửi đến AI Agent xử lý. Sau khi AI hoàn tất phản hồi, hệ thống có thể kích hoạt quy trình tóm tắt triệu chứng phục vụ cho quá trình đặt lịch khám. Cuối cùng, phản hồi sẽ được lưu vào hệ thống và gửi trả về backend thông qua Webhook.

3.3.2. Quy trình xử lý của General AI Agent



Hình 3.15. Quy trình xử lý của General AI Agent

General AI Agent được sử dụng cho các yêu cầu tư vấn sức khỏe tổng quát và các chức năng hỗ trợ chung của hệ thống.

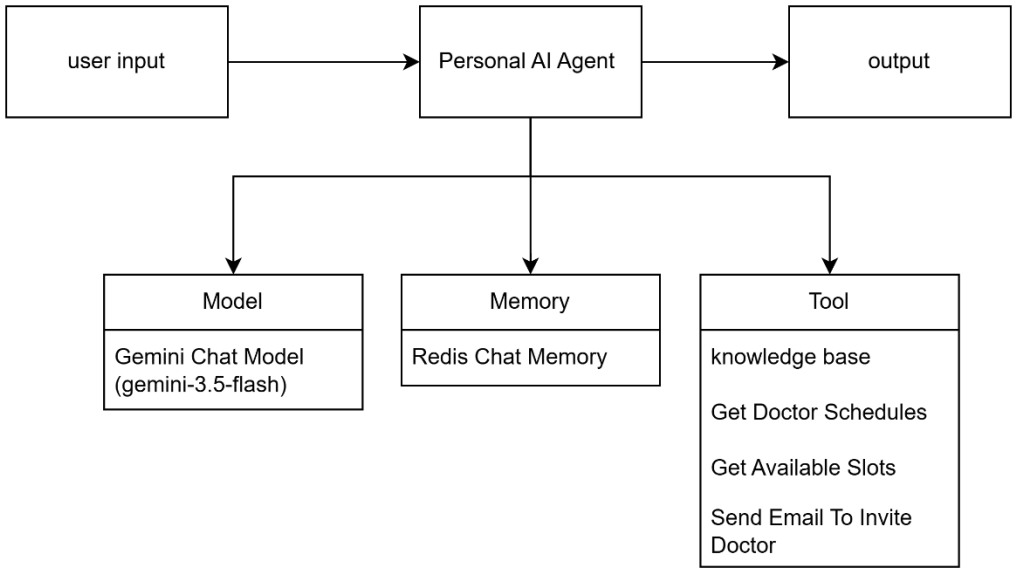
Quy trình xử lý bao gồm các thành phần chính:

- User Input: Tiếp nhận nội dung người dùng gửi từ giao diện web.
- General AI Agent: Đóng vai trò xử lý trung tâm của workflow. Thành phần này tiếp nhận dữ liệu đầu vào, phân tích yêu cầu và tạo phản hồi phù hợp.
- Memory: Sử dụng Redis Chat Memory nhằm lưu trữ lịch sử hội thoại và duy trì ngữ cảnh trong suốt quá trình tương tác.
- Model: Mô hình Gemini 3.5 Flash được sử dụng để sinh phản hồi dựa trên dữ liệu đầu vào và ngữ cảnh hội thoại.
- Tool: Tập hợp các công cụ hỗ trợ gọi API backend nhằm thực hiện các chức năng như:
    - Tìm bác sĩ.
    - Kiểm tra lịch làm việc.
    - Kiểm tra khung giờ khám còn trống.
    - Lấy tọa độ dựa trên địa chỉ văn bản.
    - Xác định chuyên khoa phù hợp.

- Output: Phản hồi được sinh ra.

General AI Agent chủ yếu sử dụng tri thức tổng quát của mô hình kết hợp với các API nghiệp vụ để hỗ trợ người dùng. Đối với chức năng đặt lịch khám, hệ thống AI không thực hiện tạo lịch hẹn trực tiếp. Sau khi người dùng lựa chọn bác sĩ và khung giờ phù hợp, mô hình AI sẽ sinh phản hồi theo cấu trúc metadata được định nghĩa trước trong prompt. Dữ liệu metadata này được gửi về backend để xử lý và hiển thị giao diện xác nhận lịch hẹn cũng như thanh toán trên frontend. Lịch hẹn chỉ được tạo chính thức sau khi người dùng hoàn tất bước xác nhận.

3.3.3. Quy trình xử lý của Personal AI Agent



Hình 3.16. Quy trình xử lý của Personal AI Agent

Personal AI Agent được sử dụng cho các vai trò trợ lý AI chuyên biệt hoạt động dựa trên tri thức do bác sĩ cung cấp. Mỗi vai trò được định hướng theo phạm vi nghiệp vụ riêng, tuy nhiên nội dung phản hồi thực tế phụ thuộc vào dữ liệu tri thức được cung cấp cho từng vai trò thông qua cơ chế RAG. Hệ thống hiện hỗ trợ ba vai trò AI gồm:

- Tiếp tân: Hỗ trợ cung cấp thông tin phòng khám, giải đáp các câu hỏi liên quan đến quy trình khám bệnh, hỗ trợ đặt lịch hẹn, sắp xếp lịch khám và xử lý các yêu cầu mang tính hành chính.

- Y tá: Hỗ trợ hướng dẫn khai báo thông tin ban đầu, cung cấp hướng dẫn chuẩn bị trước khi khám hoặc xét nghiệm, hỗ trợ chăm sóc sau khám và hướng dẫn sử dụng thuốc đã được bác sĩ kê đơn.
- Trợ lý bác sĩ: Hỗ trợ trao đổi về triệu chứng, giải thích các thông tin sức khỏe, hỗ trợ tư vấn liên quan đến tình trạng sức khỏe và cung cấp hỗ trợ y tế không mang tính chẩn đoán.

Mỗi vai trò được cấu hình với:

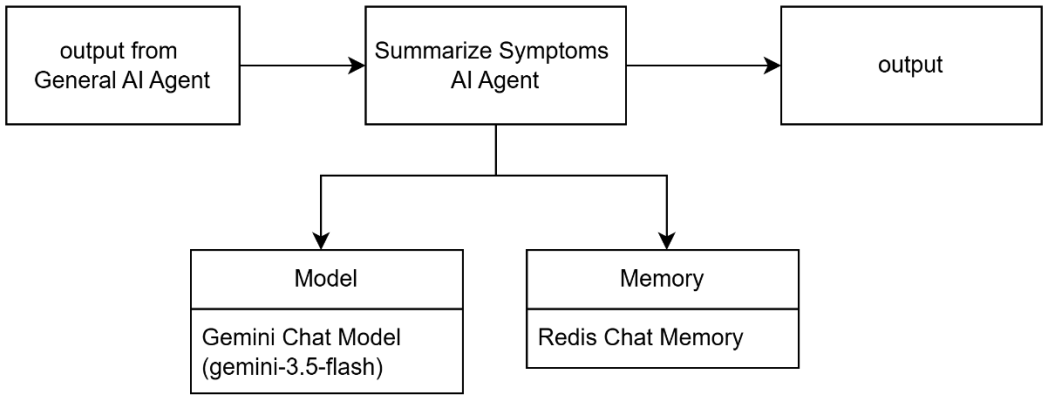
- System prompt (phong cách trả lời) riêng.
- Cơ sở tri thức riêng.
- Dữ liệu embedding riêng phục vụ cơ chế RAG.

Quy trình xử lý của Personal AI Agent tương tự General AI Agent nhưng được bổ sung khả năng truy xuất tri thức chuyên biệt trước khi gửi ngữ cảnh đến mô hình Gemini 3.5 Flash. Các thành phần chính gồm:

- User Input: Tiếp nhận yêu cầu từ người dùng.
- Personal AI Agent: Xử lý phản hồi dựa trên vai trò AI và nguồn tri thức tương ứng.
- Memory: Sử dụng Redis Chat Memory nhằm lưu trữ lịch sử hội thoại và duy trì ngữ cảnh trong suốt quá trình tương tác.
- Model: Mô hình Gemini 3.5 Flash được sử dụng để sinh phản hồi.
- Tool: Bao gồm các công cụ hỗ trợ truy xuất knowledge base và gọi API backend phục vụ các chức năng nghiệp vụ của hệ thống như kiểm tra lịch làm việc, truy xuất khung giờ khám và gửi email mời bác sĩ tư vấn trực tiếp khi gặp vấn đề ngoài tri thức hoặc vượt quá khả năng xử lý của AI Agent.
- Output: Phản hồi được sinh ra.

Quy trình này giúp cá nhân hóa phản hồi AI dựa trên chuyên môn và dữ liệu do bác sĩ cung cấp. Tương tự General AI Agent, Personal AI Agent chỉ đóng vai trò hỗ trợ sinh metadata hỗ trợ cho quy trình đặt lịch. Việc xác nhận lịch hẹn và thanh toán được thực hiện trên giao diện hệ thống sau khi người dùng xác nhận thông tin.

3.3.4. Quy trình tóm tắt triệu chứng



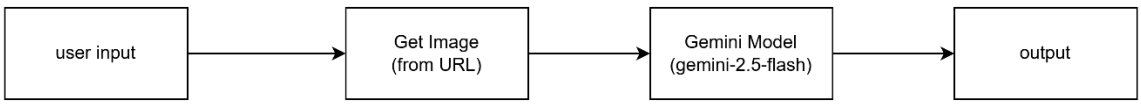
Hình 3.17. Quy trình tóm tắt triệu chứng

Sau khi AI hoàn tất quá trình tư vấn, hệ thống có thể kích hoạt quy trình tóm tắt triệu chứng nhằm tổng hợp nội dung hội thoại thành bản mô tả ngắn gọn. Quy trình này bao gồm:

- Output from General AI Agent: Tiếp nhận nội dung phản hồi từ General AI Agent.
- Summarize Symptoms AI Agent: Xử lý tóm tắt thông tin triệu chứng và nội dung tư vấn.
- Memory: Truy xuất lịch sử hội thoại từ Redis Chat Memory nhằm cung cấp ngữ cảnh cho quá trình tóm tắt.
- Model: Mô hình Gemini 3.5 Flash phục vụ sinh nội dung tóm tắt.
- Output: Kết quả sau khi tóm tắt.

Kết quả tóm tắt có thể được sử dụng để hỗ trợ bác sĩ trong quá trình tư vấn hoặc hỗ trợ quy trình đặt lịch khám bệnh.

3.3.5. Quy trình xử lý hình ảnh



Hình 3.18. Quy trình xử lý hình ảnh

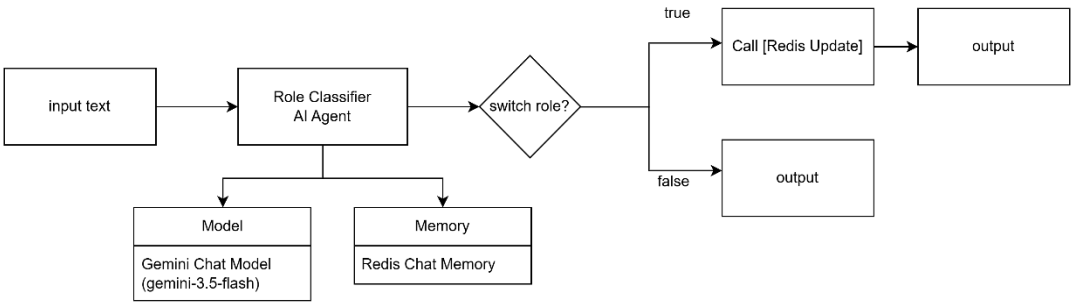
Hệ thống hỗ trợ người dùng gửi kèm hình ảnh chứa các thông tin y tế. Khi phát hiện có hình ảnh đầu vào, quy trình xử lý hình ảnh sẽ được kích hoạt để:

- Tiếp nhận file hình ảnh.

- Phân tích nội dung hình ảnh bằng mô hình AI.
- Sinh mô tả hoặc thông tin liên quan đến triệu chứng trong ảnh.
- Bổ sung kết quả phân tích vào đầu vào ban đầu.

Kết quả phân tích hình ảnh được sử dụng như đầu vào mới cho AI Agent.

3.3.6. Quy trình phân loại vai trò



Hình 3.19. Quy trình phân loại vai trò

Quy trình phân loại vai trò được sử dụng nhằm xác định vai trò AI phù hợp trước khi dữ liệu được gửi đến Personal AI Agent. Cơ chế này giúp hệ thống lựa chọn đúng ngữ cảnh chuyên môn và nguồn tri thức tương ứng với nhu cầu của người dùng. Quy trình xử lý bao gồm các thành phần chính như sau:

- Input Text: Tiếp nhận nội dung văn bản do người dùng gửi lên hệ thống.
- Role Classifier AI Agent: Đóng vai trò phân tích nội dung đầu vào và xác định vai trò AI phù hợp với ngữ cảnh hội thoại. Hệ thống hiện hỗ trợ ba vai trò gồm: Tiếp tân, Y tá, Trợ lý bác sĩ.
- Model: Mô hình Gemini 3.5 Flash được sử dụng để thực hiện tác vụ phân loại vai trò dựa trên nội dung hội thoại.
- Memory: Sử dụng Redis Chat Memory để lưu trữ lịch sử hội thoại nhằm phục vụ duy trì ngữ cảnh và hỗ trợ các AI Agent trong các bước xử lý tiếp theo.

Sau khi xác định vai trò, hệ thống sẽ kiểm tra xem vai trò mới có khác với vai trò hiện tại hay không:

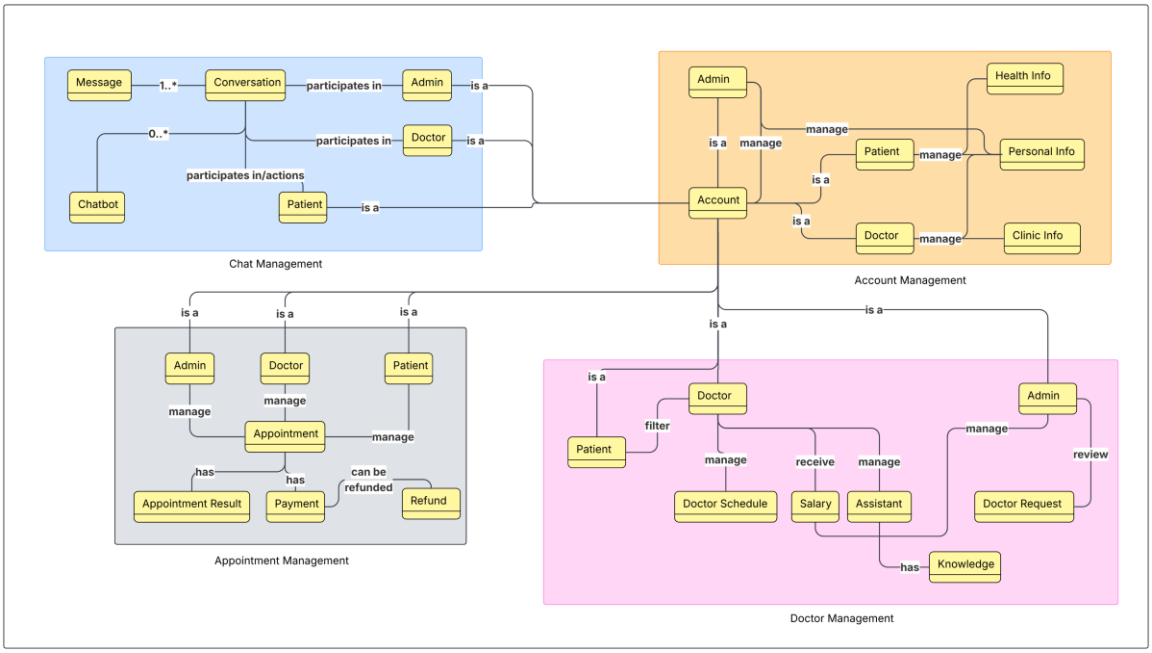
- Nếu vai trò thay đổi, hệ thống sẽ gọi cơ chế cập nhật trạng thái vai trò thông qua Redis Update trước khi tiếp tục xử lý.
- Nếu vai trò không thay đổi, dữ liệu sẽ được chuyển trực tiếp đến bước xử lý tiếp theo.

Quy trình này giúp hệ thống duy trì đúng ngữ cảnh chuyên môn cho từng cuộc hội thoại, đồng thời hỗ trợ cá nhân hóa phản hồi AI theo từng vai trò tương ứng.

3.4. Thiết kế cơ sở dữ liệu

3.4.1. Thiết kế cơ sở dữ liệu mức ý niệm

Mục tiêu của giai đoạn thiết kế cơ sở dữ liệu mức ý niệm là xây dựng một mô hình dữ liệu tổng quan, phản ánh chính xác các thực thể cốt lõi và mối quan hệ logic giữa chúng dựa trên yêu cầu nghiệp vụ của hệ thống. Mô hình này hoàn toàn độc lập với các hệ quản trị cơ sở dữ liệu cụ thể cũng như cấu trúc lưu trữ vật lý sau này. Nhằm tối ưu hóa kiến trúc dữ liệu, hệ thống sử dụng ký pháp lược đồ lớp để phân rã cơ sở dữ liệu thành 4 vùng nghiệp vụ được liên kết chặt chẽ với nhau thông qua thực thể tài khoản (Account) như sau:



Hình 3.20. Sơ đồ cấu trúc cơ sở dữ liệu mức ý niệm của hệ thống

Lưu ý quan trọng: Thực thể **Admin** (Quản trị viên) không đại diện cho một cá nhân hay một chủ thể duy nhất, mà được thiết kế như một lớp khái niệm đại diện (Abstract Role) cho một nhóm các chủ thể có cùng quyền hạn quản trị đặc thù. Việc trừu tượng hóa này nhằm đơn giản hóa mô hình dữ liệu ở tầng ý niệm, giúp bao quát toàn bộ các nhóm vai trò quản trị trong hệ thống thực tế.

Quản lý tài khoản (Account Management)

Đây là vùng nghiệp vụ hạt nhân, đóng vai trò định danh và thiết lập quyền hạn cho toàn bộ các đối tượng tương tác trong hệ thống:

- Thực thể trung tâm: **Account** (Tài khoản) là thực thể cha mang tính tổng quát hóa. Các thực thể **Admin** (đại diện cho nhóm Quản trị viên), **Patient** (Bệnh nhân), và **Doctor** (Bác sĩ) kế thừa toàn bộ thuộc tính cơ bản của **Account** thông qua mối quan hệ chuyên biệt hóa (is a).
- Mối quan hệ phụ thuộc:
    - **Admin** nắm quyền điều phối cao nhất, cho phép thực hiện hành động quản lý (manage) đối với thực thể tổng **Account**, cũng có thể quản lý dữ liệu cá nhân là **Personal Info** (Thông tin cá nhân).
    - **Patient** có quyền quản lý (manage) hai nhóm dữ liệu cá nhân chuyên sâu là **Personal Info** (Thông tin cá nhân) và **Health Info** (Hồ sơ sức khỏe).
    - **Doctor** chịu trách nhiệm quản lý (manage) dữ liệu **Personal Info** (Thông tin cá nhân) và **Clinic Info** (Thông tin hành nghề).

Quản lý lịch hẹn (Appointment Management)

Vùng nghiệp vụ này chịu trách nhiệm lưu trữ và quản lý toàn bộ vòng đời của lịch hẹn khám bệnh:

- Thực thể trung tâm: **Appointment** (Lịch hẹn) - Điểm chạm kết nối cả ba vai trò: **Admin**, **Doctor**, **Patient**. Ba vai trò này đều có mối quan hệ tương tác và quản lý (manage) trực tiếp đối với lịch hẹn này tùy theo nghiệp vụ (Bệnh nhân đặt và quản lý lịch khám, Bác sĩ tiếp nhận và quản lý lịch khám, Admin điều phối chung).
- Mối quan hệ phụ thuộc:
    - Mỗi một Appointment sẽ gắn liền (has) với một Appointment Result (Kết quả khám bệnh lâm sàng/đơn thuốc) và một Payment (Hóa đơn thanh toán phí khám).
    - Thực thể Payment có mối quan hệ đến Refund (Hoàn tiền), cho phép xử lý nghiệp vụ hoàn trả chi phí (can be refunded) trong trường hợp lịch hẹn bị hủy bỏ theo quy định.

Nhắn tin (Chat Management)

Vùng nghiệp vụ này cấu trúc luồng thông tin liên lạc và nhắn tin hỗ trợ thời gian thực trong hệ thống:

- Thực thể trung tâm: **Conversation** (Cuộc hội thoại) - Một cuộc hội thoại được duy trì nhờ sự tham gia (participates in) của 2 tác nhân khác nhau trong số các tác nhân: **Admin**, **Doctor**, **Patient**, **ChatBot**.
- Mối quan hệ phụ thuộc: **Conversation** chứa một hoặc nhiều (1..*) thực thể **Message** (Tin nhắn cụ thể).

Quản lý bác sĩ (Doctor Management)

Vùng nghiệp vụ này tập trung lưu trữ các dữ liệu phục vụ cho công tác vận hành phòng khám và quản lý nội bộ của bác sĩ:

- Thực thể trung tâm: **Doctor** trực tiếp tạo ra và làm chủ luồng thông tin cho vùng nghiệp vụ này.
- Mối quan hệ phụ thuộc:
    - **Patient** tìm kiếm (filter) **Doctor**.
    - **Doctor** nắm quyền điều phối và quản lý (manage) trực tiếp đối với **Doctor Schedule** (Lịch trình làm việc) và **Assistant** (Người trợ lý). Trong đó, thực thể **Assistant** phụ thuộc và gắn liền (has) với một kho thông tin chuyên môn chuyên biệt mang tên **Knowledge** (Khối kiến thức tri thức hỗ trợ).
    - **Salary** (Bảng lương) chịu sự phụ thuộc ràng buộc từ hai phía: **Admin** quản lý Salary và **Doctor** nhận/xem Salary.
    - Khi có nhu cầu thay đổi thông tin bác sĩ, thực thể **Doctor** sẽ khởi tạo và quản lý (manage) thực thể **Doctor Request** (Yêu cầu của bác sĩ) và đợi tiến trình xem xét, phê duyệt (review) từ phía **Admin**.

3.4.2. Thiết kế cơ sở dữ liệu mức luận lý

Giai đoạn thiết kế cơ sở dữ liệu mức luận lý có nhiệm vụ chuyển hóa mô hình ý niệm thành cấu trúc lưu trữ dữ liệu cụ thể, định nghĩa các bảng, kiểu dữ liệu, các ràng buộc và mối quan hệ giữa các thực thể. Ở mức độ này, kiến trúc hệ thống đã bắt đầu gắn liền với một hệ quản trị cơ sở dữ liệu mục tiêu.

Hệ quản trị cơ sơ dữ liệu

Như đã đề cập ở 3.2. Kiến trúc hệ thống tổng thể, hệ thống sử dụng **PostgreSQL** làm hệ quản trị cơ sở dữ liệu cho toàn bộ hệ thống. Quyết định này dựa trên ba luận điểm kỹ thuật chiến lược:

- PostgreSQL hỗ trợ hiệu quả kiến trúc Microservices thông qua khả năng triển khai nhiều cơ sở dữ liệu độc lập tương ứng với từng dịch vụ (user_service_db, doctor_service_db, appointment_service_db, chat_service_db). Mô hình phân tách này giúp cô lập dữ liệu giữa các service, giảm thiểu phụ thuộc, hạn chế điểm nghẽn tập trung (Single Point of Failure) và tăng khả năng mở rộng hệ thống theo chiều ngang.
- Bên cạnh đó, PostgreSQL cung cấp khả năng xử lý dữ liệu hỗn hợp thông qua kiểu dữ liệu jsonb, cho phép lưu trữ linh hoạt các loại dữ liệu như nội dung hội thoại AI, metadata hệ thống, hoặc dữ liệu sức khỏe mà không làm mất đi hiệu năng truy vấn nhờ hệ thống indexing tối ưu.
- Hỗ trợ mạnh mẽ các phần mở rộng không gian địa lý như PostGIS, cho phép lưu trữ và xử lý dữ liệu vị trí (geography(point, 4326)). Đây là nền tảng quan trọng để triển khai các tính năng tìm kiếm bác sĩ gần người dùng theo bán kính, tối ưu hóa trải nghiệm định vị trong hệ thống.

Nguyên tắc chuyển đổi từ mức ý niệm sang mức luận lý

Quá trình chuyển đổi từ sơ đồ thực thể ý niệm sang hệ thống bảng tuân thủ các nguyên tắc tối ưu hóa thiết kế sau:

- Cơ chế liên kết liên dịch vụ (Cross-Service Communication): Nhằm đảm bảo tính đóng gói độc lập tối đa của từng Microservice, hệ thống không sử dụng cơ chế đồng bộ dữ liệu dư thừa. Khi một dịch vụ (ví dụ: Dịch vụ Lịch hẹn hoặc Dịch vụ Chat) cần thông tin chi tiết của một tài khoản, tầng ứng dụng của dịch vụ đó sẽ thực hiện một cuộc gọi API trực tiếp (Synchronous API Call) sang user-service dựa trên mã định danh account_id duy nhất để truy xuất dữ liệu theo thời gian thực.
- Chuyển đổi thực thể kế thừa (is a):
    - Đối với hai vai trò Admin và Patient, hệ thống áp dụng giải pháp Single Table Inheritance (Kế thừa một bảng) bằng cách gộp chung vào bảng

account và phân biệt quyền hạn tường minh bằng trường dữ liệu chuyên biệt role.

- Riêng đối với vai trò Doctor, do đặc thù nghiệp vụ y tế yêu cầu lưu trữ các thông tin bác sĩ (booking_fee, specialization, experience_years...), hệ thống áp dụng giải pháp Class Table Inheritance (Bảng riêng biệt). Thực thể Doctor được tách thành một bảng doctor nằm tại doctor_service_db và ánh xạ ngược lại với bảng tài khoản thông qua khóa liên kết account_id.

- Ràng buộc toàn vẹn tại Tầng Ứng: Vì các cơ sở dữ liệu nằm trên các phân vùng vật lý khác nhau, hệ thống không thể sử dụng các ràng buộc khóa ngoại cứng kiểm tra chéo giữa các database của PostgreSQL. Do đó, tất cả các ràng buộc toàn vẹn xuyên dịch vụ đều được kiểm soát chủ động bởi Tầng ứng dụng. Ví dụ: Trước khi một Appointment (Lịch hẹn) được ghi nhận xuống appointment_service_db, mã nguồn backend sẽ tự động gọi dịch vụ quản lý bác sĩ (doctor - service) để xác thực tính tồn tại hợp lệ của doctor_id được chỉ định.
- Cải tiến tối ưu hóa luồng dữ liệu: So với mức ý niệm, mức luận lý đã có sự cải tiến sâu sắc: Thực thể Conversation được chuyển hóa thành cấu trúc gồm bảng chat (quản lý thông tin phòng chat) kết hợp với bảng chat_member (quản lý danh sách thành viên tham gia). Thiết kế này mở rộng khả năng xử lý linh hoạt cho các phòng chat đa nhiệm (giữa Patient - Doctor, Patient - Chatbot, hoặc có sự tham gia trò chuyện của Quản trị viên với Patient hoặc Doctor).

Đặc tả bảng luận lý theo từng dịch vụ

- **Dịch vụ người dùng (user_service_db)**

Bảng: public.account (Quản lý tài khoản và định danh)

Bảng này chứa thông tin tài khoản chung của toàn hệ thống, sử dụng trường role để phân định chi tiết các vai trò Admin đặc thù, Patient, Doctor. Mỗi vai trò người dùng được định nghĩa dưới dạng giá trị enum cố định. Cụ thể, hệ thống bao gồm sáu vai trò đã được đề cập trước đó như sau:

- Bệnh nhân: USER
- Bác sĩ: DOCTOR

- Quản trị viên hệ thống: ADMIN
- Quản trị viên quản lý bác sĩ: ADMIN_MANAGE_DOCTOR
- Quản trị viên kiểm duyệt: ADMIN_REVIEWER_DOCTOR
- Quản trị viên hỗ trợ: ADMIN_SUPPORT

Bảng 3.5. Mô tả public.account

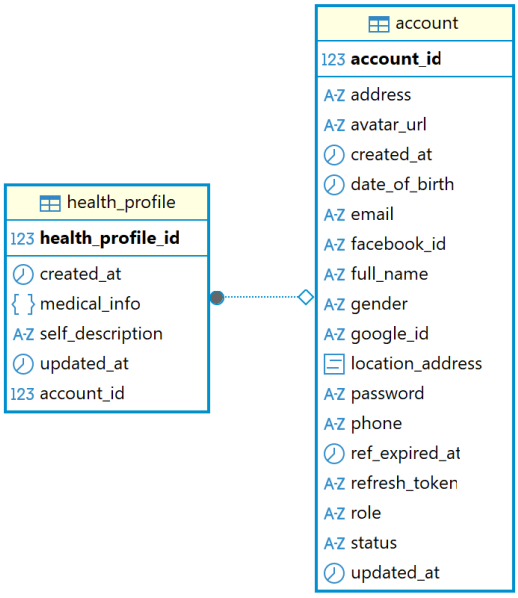
Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa / Mô tả
account_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của tài khoản người dùng trên hệ thống.
full_name	varchar(255)	NOT NULL	Họ và tên đầy đủ của chủ tài khoản.
email	varchar(255)	UNIQUE, NULL	Địa chỉ email của người dùng, dùng để đăng nhập hoặc liên hệ.
phone	varchar(10)	UNIQUE, NULL	Số điện thoại liên lạc (chuẩn hóa 10 ký tự), dùng để xác thực.
password	varchar(255)	NULL	Chuỗi mật khẩu đã được mã hóa (băm) an toàn (để trống nếu đăng nhập qua mạng xã hội).
date_of_birth	date	NULL	Ngày, tháng, năm sinh của người dùng.
gender	varchar(255)	NULL, CHECK	Giới tính, giới hạn trong các giá trị: MALE, FEMALE.
avatar_url	text	NULL	Đường dẫn ảnh đại diện của người dùng.
address	varchar(255)	NULL	Địa chỉ cư trú dạng văn bản thô.
location_address	geography (point, 4326)	NULL	Dữ liệu không gian (Spatial Data). Lưu trữ tọa độ Kinh độ (Lng) và Vĩ độ (Lat).
google_id	varchar(255)	UNIQUE, NULL	Mã định danh duy nhất liên kết với tài khoản Google (OAuth2).

facebook_id	varchar(255)	UNIQUE, NULL	Mã định danh duy nhất liên kết với tài khoản Facebook (OAuth2).
role	varchar(255)	NOT NULL, CHECK	Vai trò hệ thống: ADMIN USER DOCTOR ADMIN_MANAGE_DOCTOR ADMIN_REVIEWER_DOCTOR ADMIN_SUPPORT
status	varchar(255)	NULL, CHECK	Trạng thái tài khoản: ACTIVE (Hoạt động), INACTIVE (Tạm khóa).
refresh_token	varchar(255)	NULL	Chuỗi Token dùng để cấp lại Access Token mới khi hết hạn.
ref_expired_at	timestamp(6)	NULL	Thời điểm hết hạn chính xác của Refresh Token (có kèm múi giờ).
created_at	timestamp(6)	NULL	Thời điểm tài khoản được khởi tạo.
updated_at	timestamp(6)	NULL	Thời điểm cập nhật thông tin tài khoản gần nhất.

Bảng: public.health_profile (Hồ sơ sức khỏe cá nhân hóa của Bệnh nhân)

Bảng 3.6. Mô tả public.health_profile

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa / Mô tả
health_profile_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bản ghi hồ sơ sức khỏe.
account_id	int8	UNIQUE, FOREIGN KEY	Mã liên kết trực tiếp đến public.account(account_id). Đảm bảo ràng buộc một tài khoản chỉ có duy nhất một hồ sơ sức khỏe.
medical_info	jsonb	NULL	Dữ liệu bán cấu trúc dạng JSONB. Lưu trữ linh hoạt các thông tin lâm sàng động như: nhóm máu, tiền sử dị ứng, danh sách bệnh lý mãn tính, các loại thuốc đang sử dụng.
self_description	text	NULL	Ghi chú tự mô tả hoặc bổ sung thêm từ phía bệnh nhân về tình trạng sức khỏe hiện tại.
created_at / updated_at	timestamp(6)	NULL	Thời điểm khởi tạo và cập nhật lần cuối hồ sơ sức khỏe hệ thống.



Hình 3.21. Mối quan hệ của các bảng trong user_serivce_db

- **Dịch vụ bác sĩ (doctor_service_db)**

Bảng: public.doctor (Thông tin bác sĩ)

Bảng 3.7. Mô tả public.doctor

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa / Mô tả
doctor_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bác sĩ.
account_id	int8	NOT NULL	Mã tài khoản liên kết (user-service).
specialization	varchar(255)	NOT NULL	Chuyên khoa y tế đảm nhiệm.
description	text	NOT NULL	Mô tả bác sĩ
booking_fee/ booking_duration	int4	NOT NULL	Chi phí và thời gian cho một lượt khám bệnh.
experience_years	float8	NOT NULL	Số năm kinh nghiệm làm việc thực tế.

rating_avg/ rating_count	numeric(2,1)/ int4	NULL	Điểm đánh giá trung bình và số lượt đánh giá từ người dùng.
bank_id/ bank_name/ bank_number	varchar(255)	NULL	Thông tin tài khoản ngân hàng phục vụ thanh toán lương.
created_at/ updated_at	timestamp(6)	NULL	Thời điểm khởi tạo và cập nhật lần cuối thông tin bác sĩ.

Bảng: public.doctor_schedule (Lịch trình làm việc của Bác sĩ)

Bảng 3.8. Mô tả public.doctor_schedule

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa / Mô tả**
schedule_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bản ghi lịch trình làm việc.
doctor_id	int8	FOREIGN KEY	Khóa ngoại liên kết trực tiếp đến public.doctor (doctor_id) nhằm xác định lịch trình này thuộc về bác sĩ nào.
weekday	int4	NULL	Thứ trong tuần (Ví dụ quy ước: 2 cho Thứ Hai … 8 cho Chủ Nhật).
start_time	time(6)	NULL	Thời gian bắt đầu ca nhận lịch hẹn khám.
end_time	time(6)	NULL	Thời gian kết thúc ca nhận lịch hẹn khám.
day_off	date	NULL	Ngày nghỉ cụ thể được đăng ký.

status	varchar(255)	CHECK	Trạng thái của khung lịch trình, giới hạn nghiêm ngặt trong tập giá trị: ACTIVE, INACTIVE, DELETED.
created_at/ updated_at	timestamp(6)	NULL	Thời điểm khởi tạo và cập nhật lần cuối bản ghi lịch trình hệ thống.

Bảng: public.doctor_request (Quản lý các yêu cầu đăng ký và thay đổi thông tin của Bác sĩ)

Bảng 3.9. Mô tả public.doctor_request

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa / Mô tả**
request_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bản ghi yêu cầu/đề xuất.
doctor_id	int8	FOREIGN KEY, NULL	Khóa ngoại tham chiếu đến public.doctor(doctor_id). Giá trị là NULL nếu là yêu cầu đăng ký mới làm bác sĩ.
type	varchar(255)	CHECK	Phân loại yêu cầu, giới hạn trong 2 nhóm: BECOME_DOCTOR_REQUEST EDIT_PROFILE_REQUEST
full_name	varchar(255)	NULL	Họ và tên của bác sĩ/ứng viên gửi yêu cầu.
email	varchar(255)	NULL	Địa chỉ email liên hệ.
phone	varchar(10)	NULL	Số điện thoại liên lạc.
specialization	varchar(255)	NULL	Chuyên khoa y tế đăng ký hoặc muốn thay đổi.

experience_years	float8	CHECK	Số năm kinh nghiệm làm việc thực tế của bác sĩ.
booking_fee	int4	CHECK	Mức phí khám bệnh đăng ký.
booking_duration	int4	CHECK	Thời lượng cho một ca khám bệnh.
description	text	NULL	Thông tin giới thiệu bác sĩ.
status	varchar(255)	CHECK	Trạng thái kiểm duyệt phân cấp của yêu cầu: PENDING_REVIEWER REJECTED_REVIEWER PENDING_MANAGE_REVIEW REJECTED_MANAGE APPROVED, DELETED
created_at/ updated_at	timestamp(6)	NULL	Thời điểm khởi tạo và cập nhật lần cuối yêu cầu.

Bảng: public.salary (Quản lý bảng lương)

Bảng 3.10. Mô tả public.salary

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa / Mô tả**
salary_id	int8	PRIMARY KEY, IDENTITY	Mã định danh bảng lương tháng.
doctor_id	int8	NOT NULL, FOREIGN KEY	Khóa ngoại liên kết trực tiếp bảng doctor.
month / year	int4 / int4	NOT NULL	Kỳ tính lương (Tháng / Năm).
total_revenue	numeric(12,2)	NULL	Tổng doanh thu tạo ra từ các ca khám trong tháng.
commission_percentage	numeric(5,2)	NULL	Tỷ lệ phần trăm hoa hồng bác sĩ được nhận.

salary_amount	numeric(12,2)	NULL	Thực nhận của bác sĩ sau khi tính toán theo tỷ lệ sẽ nhận.
appointment_ids	varchar(255)	NOT NULL	Danh sách chuỗi mã lịch hẹn phục vụ tạo bảng lương.
status	varchar(255)	CHECK	Trạng thái bảng lương: ACTIVE, INACTIVE
created_at / updated_at	timestamp(6)	NULL	Thời điểm khởi tạo và cập nhật lần cuối bảng lương.

Bảng: public.doctor_assistant_profile (Thông tin Người trợ lý)

Bảng 3.11. Mô tả public.doctor_assistant_profile

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa / Mô tả**
assistant_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bản ghi thông tin trợ lý.
doctor_id	int8	NOT NULL, FOREIGN KEY	Khóa ngoại tham chiếu trực tiếp đến public.doctor(doctor_id). Đảm bảo mỗi trợ lý bắt buộc phải thuộc quyền quản lý của một bác sĩ cụ thể.
name	varchar(255)	NULL	Họ và tên hoặc tên định danh của trợ lý ảo.
role	varchar(255)	NOT NULL, CHECK	Vai trò tác nghiệp chuyên biệt, giới hạn nghiêm ngặt trong tập giá trị: RECEPTIONIST (Tiếp tân), NURSE (Y tá), MEDICAL_ASSISTANT (Trợ lý bác sĩ).
system_prompt	text	NULL	Kỹ thuật Prompt Engineering. Lưu trữ câu lệnh cấu hình ngữ cảnh, hành vi và quy tắc phản hồi

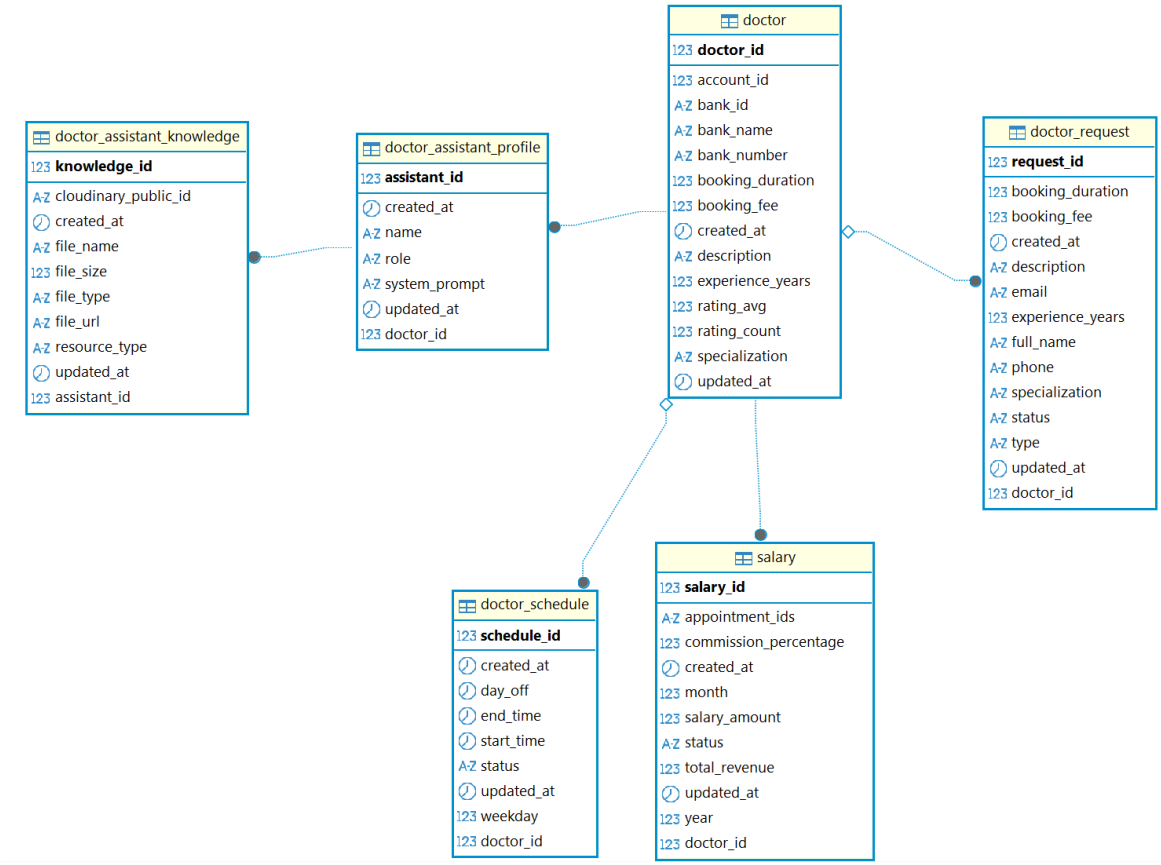
			dành riêng cho Trợ lý ảo AI Agent.
created_at / updated_at	timestamp(6)	NULL	Thời điểm khởi tạo và cập nhật lần cuối hồ sơ trợ lý trên hệ thống.

Bảng: public.doctor_assistant_knowledge (Tri thức hỗ trợ Trợ lý)

Bảng 3.12. Mô tả public.doctor_assistant_knowledge

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa / Mô tả**
knowledge_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bản ghi tài liệu tri thức.
assistant_id	int8	NOT NULL, FOREIGN KEY	Khóa ngoại tham chiếu trực tiếp đến public.doctor_assistant_profile (assistant_id). Xác định tài liệu này thuộc quyền sở hữu/sử dụng của trợ lý nào.
file_name	varchar(255)	NULL	Tên gốc của tệp tài liệu tri thức y tế được tải lên.
file_size	int8	NULL	Dung lượng của tệp tin (tính bằng đơn vị byte).
file_type	varchar(255)	NULL	Định dạng tệp tin (Ví dụ: application/pdf, text/plain, application/msword).
file_url	text	NULL	Đường dẫn đầy đủ để truy cập và tải tệp tài liệu trực tuyến.
cloudinary_public_id	varchar(255)	NULL	Mã định danh quản lý tệp tin duy nhất trên máy chủ lưu trữ đám mây Cloudinary, phục vụ tác vụ truy xuất hoặc xóa file.

resource_type	varchar(255)	NULL	Phân loại tài nguyên lưu trữ trên Cloudinary (Ví dụ: raw, image, video).
created_at / updated_at	timestamp(6)	NULL	Thời điểm tệp tài liệu tri thức được tải lên và cập nhật lần cuối trên hệ thống.



Hình 3.22. Mối quan hệ của các bảng trong doctor_serivce_db

- **Dịch vụ lịch hẹn (appointment_service_db)**

Bảng: public.appointment (Quản lý lịch hẹn khám bệnh)

Bảng 3.13. Mô tả public.appointment

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa / Mô tả
appointment_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của ca hẹn khám.
account_id	int8	NOT NULL	Mã tài khoản Bệnh nhân đặt lịch (user-service).
doctor_id	int8	NOT NULL	Mã bác sĩ tiếp nhận ca khám (doctor-service).
appointment_date	date	NOT NULL	Ngày diễn ra cuộc hẹn.
start_time / end_time	time(6)	NOT NULL	Khung giờ cụ thể của ca khám bệnh.
fee	int4	NOT NULL	Chi phí thực tế thu tại thời điểm đặt lịch.
reason	varchar(1000)	NULL	Lý do khám bệnh
rating	int4	NULL	Kết quả đánh giá lịch khám của người dùng.
status	varchar(255)	CHECK	Trạng thái: PENDING, WAITING, CANCELLED, FINISHED, NO_SHOW, ADMIN_CANCELLED.
created_at / updated_at	timestamp(6)	NULL	Thời điểm lịch hẹn được tạo và cập nhật lần cuối trên hệ thống.

Bảng: public.appointment_result (Kết quả khám)

Bảng 3.14. Mô tả public.appointment_result

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa / Mô tả
result_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bản ghi kết quả khám.
appointment_id	int8	NOT NULL, FOREIGN KEY	Khóa ngoại tham chiếu trực tiếp đến public.appointment(appointment_id). Đảm bảo mỗi kết quả khám bắt buộc phải gắn liền với một cuộc hẹn cụ thể.
title_result	varchar(255)	NOT NULL	Tiêu đề kết quả hoặc kết luận chẩn đoán sơ bộ ban đầu (Ví dụ: *Khám tổng quát, Chẩn đoán Viêm phế quản*...).
description_result	text	NULL	Nội dung mô tả chi tiết kết quả khám bệnh, chỉ định cận lâm sàng, đơn thuốc hoặc lời dặn chi tiết từ bác sĩ.
img_result_url	varchar(255)	NULL	Đường dẫn lưu trữ hình ảnh đơn thuốc, phiếu kết quả xét nghiệm máu, X-quang hoặc siêu âm (Liên kết đám mây).
result_status	varchar(255)	NULL, CHECK	Trạng thái xử lý kết quả khám, giới hạn nghiêm ngặt trong tập giá trị: PENDING (Chờ khám), IN_PROGRESS (Đang tiến hành), COMPLETED (Đã hoàn thành chẩn đoán).

created_at / updated_at	timestamp(6)	NULL	Thời điểm khởi tạo và cập nhật lần cuối bản ghi kết quả trên hệ thống.

Bảng: public.payment (Quản lý thanh toán lich hẹn)

Bảng 3.15. Mô tả public.payment

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa / Mô tả
payment_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của hóa đơn giao dịch.
appointment_id	int8	NOT NULL, UNIQUE, FOREIGN KEY	Khóa ngoại tham chiếu trực tiếp đến public.appointment(appointment_id). Ràng buộc UNIQUE đảm bảo một lịch hẹn chỉ gắn liền với một hóa đơn thanh toán duy nhất.
payment_cost	int4	NOT NULL	Số tiền thực tế bệnh nhân phải trả cho ca hẹn khám (tính bằng VNĐ).
payment_method	varchar(255)	NOT NULL, CHECK	Phương thức giao dịch, giới hạn nghiêm ngặt trong tập giá trị: BANK_TRANSFER (Chuyển khoản ngân hàng).
payment_status	varchar(255)	NULL, CHECK	Trạng thái của hóa đơn giao dịch: PENDING (Đang chờ thanh toán) hoặc COMPLETED (Đã thanh toán thành công).
session_id	varchar(255)	NULL	Mã định danh phiên giao dịch được cấp bởi cổng thanh toán

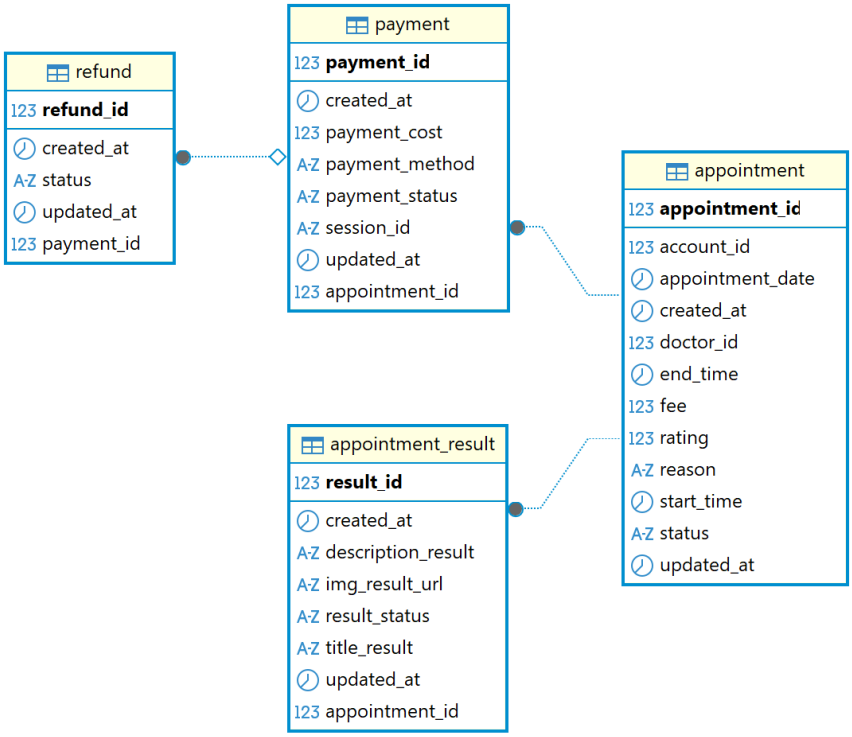
			đối tác, phục vụ việc kiểm tra trạng thái (Webhook/IPN) và đối soát.
created_at	timestamp(6)	NULL	Thời điểm phát hành hóa đơn hệ thống.
updated_at	timestamp(6)	NULL	Thời điểm cập nhật trạng thái hóa đơn gần nhất (khi nhận được phản hồi thanh toán thành công).

Bảng: public.refund (Quản lý hoàn trả chi phí lich hen)

Bảng 3.16. Mô tả public.refund

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa / Mô tả**
refund_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bản ghi yêu cầu hoàn tiền.
payment_id	int8	UNIQUE, FOREIGN KEY, NULL	Khóa ngoại tham chiếu trực tiếp đến public.payment(payment_id). Ràng buộc UNIQUE đảm bảo một hóa đơn chỉ được phép tạo ra tối đa một yêu cầu hoàn tiền.
status	varchar(255)	NULL, CHECK	Trạng thái xử lý tiến trình hoàn trả, giới hạn nghiêm ngặt trong tập giá trị: PENDING (Đang chờ xử lý), REFUNDED (Đã hoàn tiền thành công), REJECTED (Yêu cầu bị từ chối).
created_at	timestamp(6)	NULL	Thời điểm khởi tạo đơn yêu cầu hoàn tiền trên hệ thống.

updated_at	timestamp(6)	NULL	Thời điểm cập nhật trạng thái đơn hoàn tiền gần nhất.



Hình 3.23. Mối quan hệ của các bảng trong appointment_serivce_db

- **Dịch vụ nhắn tin (chat_service_db)**

Bảng: public.chat (Quản lý các đoạn hội thoại)

Bảng 3.17. Mô tả public.chat

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa / Mô tả
chat_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của phòng hội thoại.
account_id	int8	NOT NULL	Mã tài khoản của Bệnh nhân / Bác sĩ (User/Doctor) chủ thể khởi tạo cuộc hội thoại.
assigned_doctor_account_id	int8	NULL	Mã tài khoản của Bác sĩ tiếp nhận

			phòng chat này (nếu có).
assigned_admin_account_id	int8	NULL	Mã tài khoản của Admin (thuộc nhóm ADMIN_SUPPORT) chịu trách nhiệm hỗ trợ ca chat (nếu có).
chat_type	varchar(255)	NOT NULL, CHECK	Phân loại luồng tương tác, giới hạn nghiêm ngặt trong tập giá trị: AI (Hội thoại với Chatbot), DOCTOR (Tư vấn với Bác sĩ), ADMIN_SUPPORT (Hỗ trợ kỹ thuật/thủ tục).
is_human_doctor	bool	NOT NULL	Cờ logic xác định phòng chat có sự tham gia điều phối trực tiếp từ bác sĩ con người (true) hay không (false).
status	varchar(255)	NOT NULL, CHECK	Trạng thái xử lý của phòng chat, giới hạn trong tập: WAITING (Chờ tiếp nhận), IN_PROGRESS (Đang tương tác), RESOLVED (Đã xử lý), CLOSED (Đã đóng phòng).

created_at	timestamp(6)	NULL	Thời điểm phòng hội thoại được tạo trên hệ thống.
updated_at	timestamp(6)	NULL	Thời điểm cập nhật lần cuối đoạn hội thoại.

Bảng: public.chat_member (Quản lý thành viên trong đoạn hội thoại)

Bảng 3.18. Mô tả public.chat_member

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa / Mô tả**
chat_member_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của bản ghi thành viên phòng chat.
chat_id	int8	FOREIGN KEY, NULL	Khóa ngoại tham chiếu trực tiếp đến public.chat(chat_id) nhằm xác định thành viên thuộc phòng chat nào.
account_id	int8	NOT NULL	Mã tham chiếu logic sang tài khoản người dùng (user-service), xác định danh tính chủ thể tham gia phòng hội thoại.
chat_name	varchar(255)	NULL	Tên hiển thị dành riêng cho đoạn hội thoại của người dùng này.
is_hidden_permanently	bool	NULL	Cờ logic xác định đoạn hội thoại có bị xóa bởi toàn bộ thành viên trong đoạn hội thoại hay không.

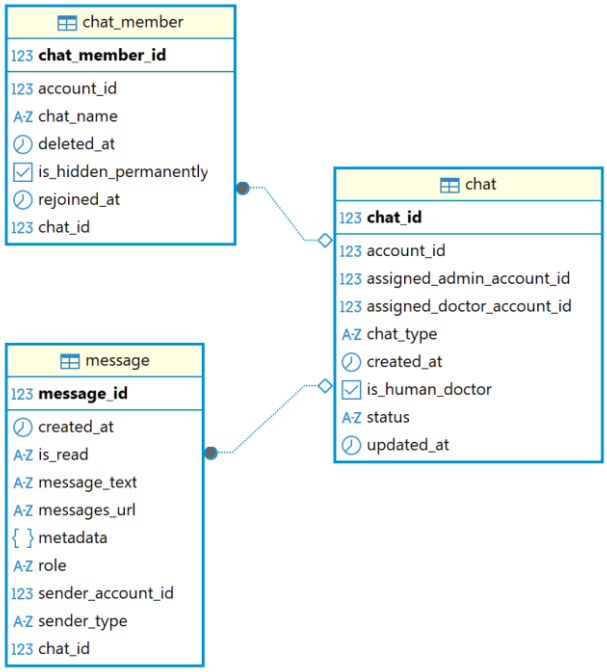
deleted_at	timestamp(6)	NULL	Thời điểm thành viên rời khỏi phòng chat hoặc xóa phòng chat.
rejoined_at	timestamp(6)	NULL	Thời điểm thành viên quay trở lại tái tham gia phòng chat cũ sau khi đã rời đi trước đó.

Bảng: public.message (Quản lý tin nhắn trong cuộc hội thoại)

Bảng 3.19. Mô tả public.message

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa / Mô tả**
message_id	int8	PRIMARY KEY, IDENTITY	Mã định danh duy nhất của từng tin nhắn.
chat_id	int8	FOREIGN KEY, NULL	Khóa ngoại liên kết đến public.chat(chat_id), xác định tin nhắn này thuộc phòng hội thoại nào.
sender_account_id	int8	NULL	Mã tham chiếu logic đến tài khoản người gửi (Account ID), để trống nếu tin nhắn do AI gửi.
sender_type	varchar(255)	NULL, CHECK	Phân loại đối tượng gửi tin: AI (Trợ lý ảo), DOCTOR (Bác sĩ), USER (Bệnh nhân), ADMIN_SUPPORT (Admin), hoặc SYSTEM (Thông báo từ hệ thống).
role	varchar(255)	NOT NULL	Phân vai gửi tin theo chuẩn giao thức hội thoại.

message_text	text	NOT NULL	Nội dung văn bản chi tiết của tin nhắn hoặc câu thoại tư vấn.
messages_url	varchar(255)	NULL	Đường dẫn lưu trữ tệp tin đính kèm như hình ảnh đơn thuốc hoặc tài liệu y tế đi kèm tin nhắn.
metadata	jsonb	NULL	Dữ liệu bán cấu trúc JSONB. Lưu trữ các thuộc tính động mở rộng của tin nhắn (cấu hình định dạng tin nhắn phức tạp).
is_read	varchar(255)	NOT NULL, CHECK	Trạng thái đọc của tin nhắn, giới hạn trong hai giá trị: UNREAD (Chưa đọc) hoặc READ (Đã đọc).
created_at	timestamp(6)	NULL	Thời điểm tin nhắn được gửi



Hình 3.24. Mối quan hệ của các bảng trong chat_serivce_db

3.5. Thiết kế workflow AI và cơ chế xử lý

HealthAssistant áp dụng hướng tiếp cận tích hợp các công nghệ AI hiện đại thay vì huấn luyện mô hình từ đầu. Thiết kế hệ thống tập trung vào tính khả thi, khả năng mở rộng và tối ưu chi phí triển khai nhằm phù hợp với điều kiện thực tế của các phòng khám tư nhân quy mô nhỏ.

3.5.1. Kiến trúc workflow AI tổng thể

Hệ thống sử dụng kiến trúc workflow AI được xây dựng trên nền tảng n8n [7], kết hợp giữa mô hình LLM, cơ chế truy xuất tri thức (RAG), bộ nhớ hội thoại và các công cụ hỗ trợ nghiệp vụ. Kiến trúc hệ thống bao gồm hai AI Agent chính:

- General AI Agent: sử dụng kiến thức tổng quát của mô hình Gemini 3.5 Flash kết hợp với Memory và Tool Calling để xử lý các yêu cầu chung như tư vấn sức khỏe cơ bản, hỗ trợ tìm bác sĩ và hỗ trợ quy trình đặt lịch khám.
- Personal AI Agent: tích hợp thêm cơ chế RAG nhằm truy xuất tri thức chuyên biệt do bác sĩ cung cấp cho từng vai trò AI.
- Mỗi AI Agent đều được tích hợp:
- Mô hình Gemini 3.5 Flash để xử lý ngôn ngữ tự nhiên.
- Redis Chat Memory để duy trì ngữ cảnh hội thoại.
- Tool Calling nhằm tương tác với các API backend.

Riêng Personal AI Agent được bổ sung cơ chế RAG nhằm hỗ trợ truy xuất tri thức từ KB trước khi sinh phản hồi.

3.5.2. Lựa chọn mô hình ngôn ngữ lớn

Hệ thống sử dụng mô hình Gemini 3.5 Flash làm nền tảng xử lý ngôn ngữ tự nhiên cho các workflow AI chính. Mô hình được tích hợp trong n8n nhằm hỗ trợ sinh phản hồi hội thoại, xử lý yêu cầu từ người dùng và hỗ trợ các tác vụ tư vấn sức khỏe. Việc lựa chọn Gemini 3.5 Flash mang lại các ưu điểm:

- Tốc độ phản hồi nhanh, phù hợp với các hệ thống chatbot thời gian thực.
- Khả năng xử lý ngôn ngữ tự nhiên tốt trong các tác vụ hội thoại và tư vấn.
- Dễ dàng tích hợp với workflow AI trên nền tảng n8n.

Ngoài workflow hội thoại chính, hệ thống còn sử dụng mô hình Gemini 2.5 Flash cho quy trình xử lý hình ảnh triệu chứng. Mô hình này được sử dụng để phân tích

hình ảnh do người dùng cung cấp như các chỉ số y tế, báo cáo xét nghiệm... Kiến trúc hệ thống được thiết kế theo hướng linh hoạt, cho phép thay đổi hoặc nâng cấp các mô hình AI trong tương lai khi cần thiết.

3.5.3. Thiết kế RAG và Knowledge Base

Hệ thống sử dụng cơ chế RAG nhằm hỗ trợ truy xuất tri thức từ dữ liệu do bác sĩ cung cấp thay vì chỉ phụ thuộc vào kiến thức tổng quát của mô hình ngôn ngữ. Trong hệ thống hiện tại, RAG chỉ được áp dụng cho Personal AI Agent nhằm hỗ trợ cá nhân hóa nội dung tư vấn theo từng vai trò AI.

Embedding Model

Hệ thống sử dụng mô hình embedding của Cohere để chuyển đổi tài liệu y khoa thành vector phục vụ tìm kiếm theo ngữ nghĩa (semantic search). Việc sử dụng embedding giúp hệ thống có khả năng truy xuất các đoạn tri thức liên quan dựa trên ngữ cảnh và ý nghĩa của câu hỏi thay vì chỉ dựa trên từ khóa.

Vector Store

Hệ thống sử dụng Pinecone làm vector database để lưu trữ và truy xuất dữ liệu embedding. Các tài liệu tri thức sau khi được chuyển đổi thành vector sẽ được lưu trữ trong Pinecone Index. Dữ liệu của từng vai trò AI được phân tách thông qua metadata nhằm hỗ trợ truy xuất tri thức phù hợp cho từng workflow AI.

Quy trình RAG

Indexing

Bác sĩ tải tài liệu lên hệ thống. Sau đó, tài liệu được chia thành nhiều đoạn nhỏ (chunking), chuyển đổi thành vector embedding thông qua mô hình embedding của Cohere và lưu trữ trong Pinecone Vector Store tương ứng.

Retrieval

Khi người dùng gửi câu hỏi, hệ thống chuyển nội dung câu hỏi thành vector embedding và thực hiện semantic search trên Pinecone để truy xuất các đoạn tri thức có mức độ liên quan cao nhất từ KB tương ứng với vai trò AI đang hoạt động.

Generation

Các nội dung được truy xuất sẽ được đưa vào prompt dưới dạng context trước khi gửi đến mô hình Gemini 3.5 Flash để sinh phản hồi. Nhờ cơ chế RAG, phản hồi

của Personal AI Agent được định hướng dựa trên tri thức chuyên môn do bác sĩ cung cấp thông qua KB, giúp tăng độ chính xác và tính phù hợp của nội dung tư vấn.

3.5.4. Thiết kế AI Agent và Tool Calling

AI Agent được xây dựng theo kiến trúc ReAct (Reasoning + Acting) trong n8n:

- System Prompt: Thiết kế riêng cho từng vai trò, định nghĩa rõ vai trò và giới hạn của trợ lý ảo.
- Memory: Sử dụng Redis Chat Memory để lưu trữ lịch sử hội thoại, giúp Agent duy trì ngữ cảnh trong suốt cuộc trò chuyện.
- Tools: Agent có khả năng gọi các công cụ bên ngoài như kiểm tra lịch làm việc của bác sĩ, lấy các khung giờ trống, xác định chuyên khoa phù hợp, tìm bác sĩ và xử lý hình ảnh (nếu bệnh nhân gửi kèm ảnh).

3.5.5. Cơ chế an toàn y tế (Safety Mechanism)

Do hệ thống HealthAssistant hoạt động trong lĩnh vực hỗ trợ tư vấn sức khỏe, cơ chế an toàn y tế được xem là thành phần quan trọng nhằm giảm thiểu nguy cơ cung cấp thông tin sai lệch hoặc gây hiểu nhầm cho người dùng. Hệ thống triển khai một số cơ chế an toàn nhằm giới hạn phạm vi tư vấn và hỗ trợ kiểm soát nội dung phản hồi của AI.

- **Cảnh báo tư vấn sơ bộ:** Giao diện trò chuyện luôn hiển thị thông báo rằng AI có thể mắc lỗi và hãy tham khảo ý kiến của bác sĩ khi cần thông tin y tế quan trọng.
- **Phát hiện triệu chứng nghiêm trọng:** Các tài liệu tri thức trong KB được gắn kèm thông tin về mức độ nghiêm trọng của triệu chứng. Trong quá trình truy xuất tri thức bằng cơ chế RAG, AI sử dụng các thông tin này để điều chỉnh phản hồi và đưa ra khuyến nghị phù hợp đối với các trường hợp có nguy cơ cao.
- **Giới hạn phạm vi tư vấn:** AI được định hướng hoạt động ở vai trò hỗ trợ tư vấn sức khỏe sơ bộ, giải thích thông tin cơ bản và hỗ trợ quy trình đặt lịch khám. Hệ thống không hướng đến việc thay thế bác sĩ trong các hoạt động chẩn đoán hoặc điều trị chuyên sâu.
- **Human-in-the-loop:** Bác sĩ có thể xem lại lịch sử hội thoại và can thiệp thủ công khi cần thiết.

- **Kiểm soát tri thức:** Chỉ cho phép bác sĩ cập nhật dữ liệu vào KB nhằm đảm bảo độ tin cậy của nguồn tri thức.

3.6. Thiết kế giao diện và Trải nghiệm người dùng (UI/UX)

Thiết kế giao diện người dùng (User Interface - UI) và trải nghiệm người dùng (User Experience - UX) đóng vai trò quan trọng trong quá trình xây dựng các hệ thống công nghệ thông tin nói chung và hệ thống HealthAssistant nói riêng. Một giao diện trực quan và thân thiện không chỉ giúp người dùng thao tác thuận tiện hơn mà còn nâng cao khả năng tiếp cận hệ thống đối với nhiều nhóm người dùng khác nhau như bệnh nhân, bác sĩ và quản trị viên.

Trong phạm vi đề tài, giao diện hệ thống được tập trung thiết kế chủ yếu cho nền tảng desktop nhằm đảm bảo khả năng hiển thị đầy đủ các chức năng và tối ưu trải nghiệm sử dụng. Đối với nền tảng mobile, giao diện sẽ được điều chỉnh theo hướng responsive và có thể linh hoạt thay đổi tùy theo cách triển khai của người phát triển, miễn vẫn đảm bảo tính trực quan và thuận tiện trong quá trình sử dụng hệ thống.

3.6.1. Mục tiêu

Hệ thống được xây dựng với mục tiêu tạo ra một môi trường tương tác thuận tiện giữa người dùng với AI, người dùng với người dùng (bác sĩ, quản trị viên hỗ trợ) thông qua nền tảng này. Vì vậy, giao diện của hệ thống được thiết kế theo hướng hiện đại, trực quan và dễ sử dụng nhằm hỗ trợ người dùng thao tác nhanh chóng trong quá trình sử dụng. Các mục tiêu chính trong quá trình thiết kế giao diện bao gồm:

- Xây dựng giao diện trực quan, thân thiện và dễ tiếp cận đối với nhiều nhóm người dùng khác nhau như bệnh nhân, bác sĩ và nhóm quản trị viên (quản trị viên, quản trị viên quản lý bác sĩ, quản trị viên kiểm duyệt bác sĩ, quản trị viên hỗ trợ).
- Đảm bảo tính đồng bộ giữa các màn hình và các nhóm chức năng trong toàn hệ thống.
- Hỗ trợ khả năng mở rộng giao diện trong tương lai khi bổ sung thêm các chức năng mới.
- Thiết kế giao diện theo xu hướng hiện đại và tương đồng với các nền tảng chăm sóc sức khỏe phổ biến nhằm giúp người dùng dễ làm quen và nâng cao trải nghiệm sử dụng hệ thống.

Bên cạnh đó, do hệ thống thuộc lĩnh vực chăm sóc sức khỏe nên giao diện được thiết kế với các gam màu nhẹ, trong đó màu xanh dương được sử dụng làm màu chủ đạo, kết hợp với nền trắng và xám nhạt nhằm tạo cảm giác thân thiện, dễ quan sát và tăng độ tin cậy cho người dùng trong quá trình sử dụng. Bố cục giao diện được sắp xếp rõ ràng, hạn chế các thành phần gây rối mắt nhằm giúp người dùng tập trung hơn vào các chức năng chính của hệ thống như tư vấn sức khỏe, trò chuyện và đặt lịch khám.

3.6.2. Nguyên tắc

Quá trình thiết kế giao diện của hệ thống được thực hiện dựa trên một số nguyên tắc thiết kế UI/UX phổ biến nhằm nâng cao trải nghiệm sử dụng, tăng tính trực quan và hỗ trợ người dùng thao tác hiệu quả hơn trong quá trình sử dụng hệ thống.

- Hiển thị trạng thái hệ thống (Visibility of System Status): Hệ thống luôn cung cấp phản hồi trực quan cho người dùng trong quá trình thao tác thông qua thông báo, cập nhật dữ liệu của các chức năng sau khi hoàn thành thao tác. Điều này giúp người dùng dễ dàng theo dõi quá trình tương tác và hạn chế nhầm lẫn khi sử dụng hệ thống.
- Tính nhất quán (Consistency and Standards): Các thành phần giao diện như màu sắc, biểu tượng, button, popup và bố cục được thiết kế đồng nhất giữa các màn hình và phân hệ chức năng. Việc duy trì tính nhất quán giúp người dùng dễ làm quen với hệ thống và giảm thời gian thao tác khi chuyển đổi giữa các chức năng khác nhau.
- Thiết kế tối giản và thẩm mỹ (Aesthetic and Minimalist Design): Giao diện được thiết kế theo hướng tối giản, chỉ hiển thị các thông tin và chức năng cần thiết nhằm giảm sự rối mắt và giúp người dùng tập trung hơn vào các chức năng chính như tư vấn sức khỏe, trò chuyện và đặt lịch khám.
- Tăng khả năng nhận diện thay vì ghi nhớ (Recognition Rather Than Recall): Các chức năng được tổ chức theo nhóm rõ ràng và kết hợp với biểu tượng trực quan nhằm giúp người dùng dễ dàng nhận biết chức năng tương ứng mà không cần ghi nhớ quá nhiều thao tác hoặc vị trí chức năng trong hệ thống.

3.6.3. Nền tảng và công cụ thiết kế

Trong giai đoạn hiện thực hóa ý tưởng và xây dựng giao diện người dùng (UI/UX), việc lựa chọn một công cụ thiết kế phù hợp đóng vai trò quan trọng nhằm đảm bảo tính trực quan, hiệu quả làm việc và khả năng cộng tác trong quá trình phát triển hệ thống. Nhóm quyết định lựa chọn Figma làm nền tảng chủ đạo để phác thảo giao diện, thiết kế chi tiết các chức năng cho hệ thống HealthAssistant. Việc lựa chọn Figma xuất phát từ nhiều ưu điểm phù hợp với định hướng phát triển của đề tài.

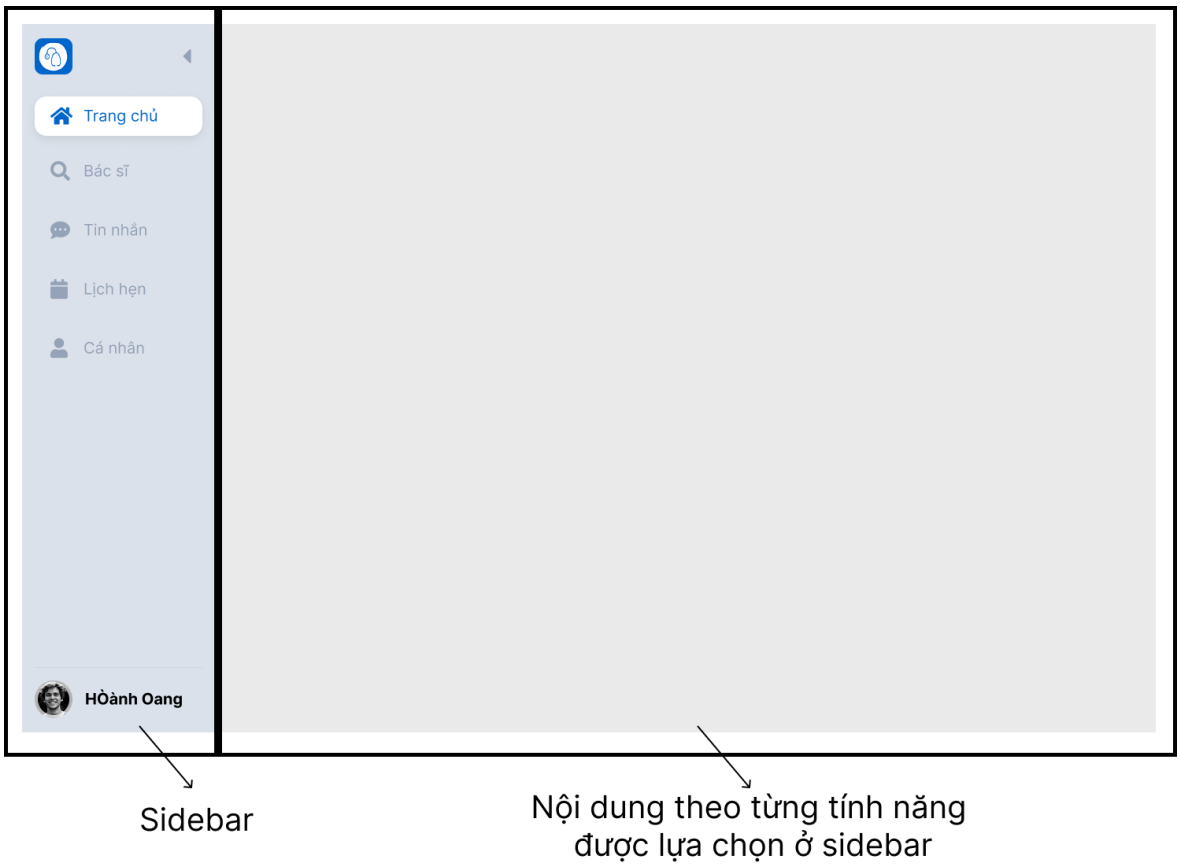
- Figma hỗ trợ thiết kế giao diện trực quan thông qua cơ chế kéo thả linh hoạt.
- Figma hoạt động trên nền tảng web và hỗ trợ làm việc theo thời gian thực, cho phép nhiều thành viên trong nhóm cùng tham gia chỉnh sửa, trao đổi và đóng góp ý tưởng trên cùng một bản thiết kế

Trong phạm vi đề tài, Figma được sử dụng chủ yếu để thiết kế giao diện cho nền tảng desktop nhằm đảm bảo khả năng hiển thị đầy đủ các chức năng và tối ưu trải nghiệm sử dụng. Đối với nền tảng mobile, giao diện được triển khai theo hướng responsive và có thể được điều chỉnh linh hoạt tùy theo cách triển khai thực tế của người phát triển chức năng.

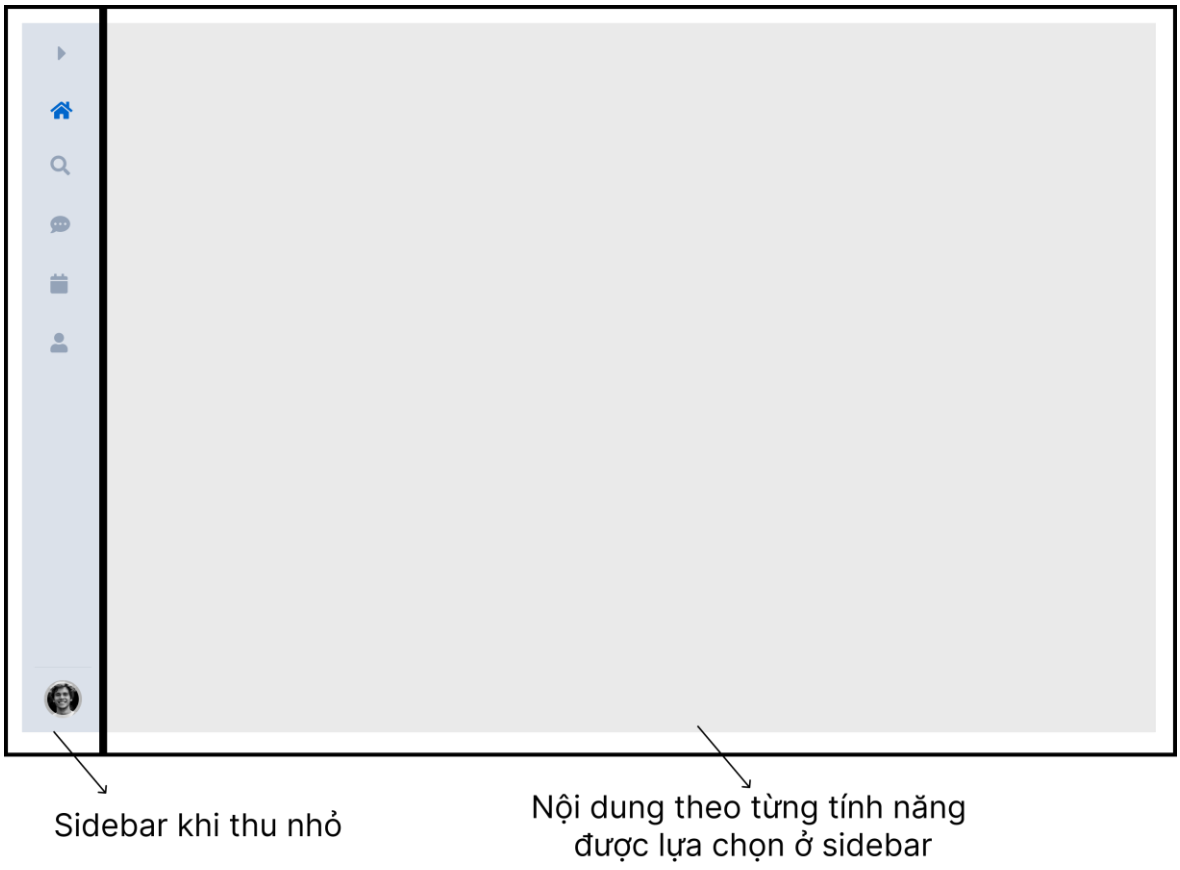
3.6.4. Thiết kế tổng thể

Giao diện trên nền tảng desktop được thiết kế theo mô hình hai cột (two-column layout) với thanh điều hướng (sidebar) ở bên trái và khu vực nội dung chính ở trung tâm. Thiết kế này giúp người dùng dễ dàng chuyển đổi giữa các chức năng và thao tác thuận tiện hơn trong quá trình sử dụng hệ thống.

Sidebar được sử dụng để hiển thị các chức năng theo quyền tài khoản như quản lý tin nhắn, lịch hẹn,… Thiết kế này giúp người dùng dễ dàng chuyển đổi giữa các chức năng mà không cần tải lại toàn bộ giao diện.

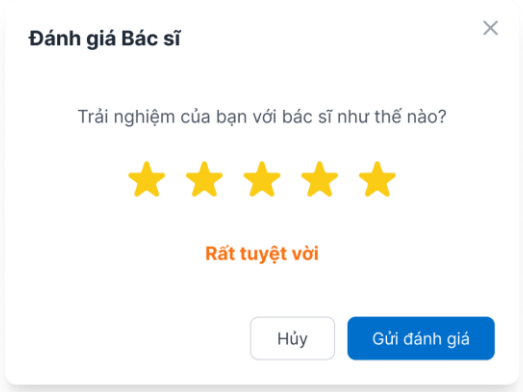


Hình 3.25. Bố cục tổng quan của hệ thống



Hình 3.26. Bố cục tổng quan của hệ thống khi thu nhỏ Sidebar

Ngoài ra, các hộp thoại (Modal) được sử dụng trong nhiều thao tác như cập nhật trạng thái lịch khám, đổi tên cuộc trò chuyện hoặc xác nhận thao tác xóa dữ liệu…. Việc sử dụng popup giúp giảm số lần chuyển trang và tối ưu trải nghiệm người dùng trong quá trình thao tác.



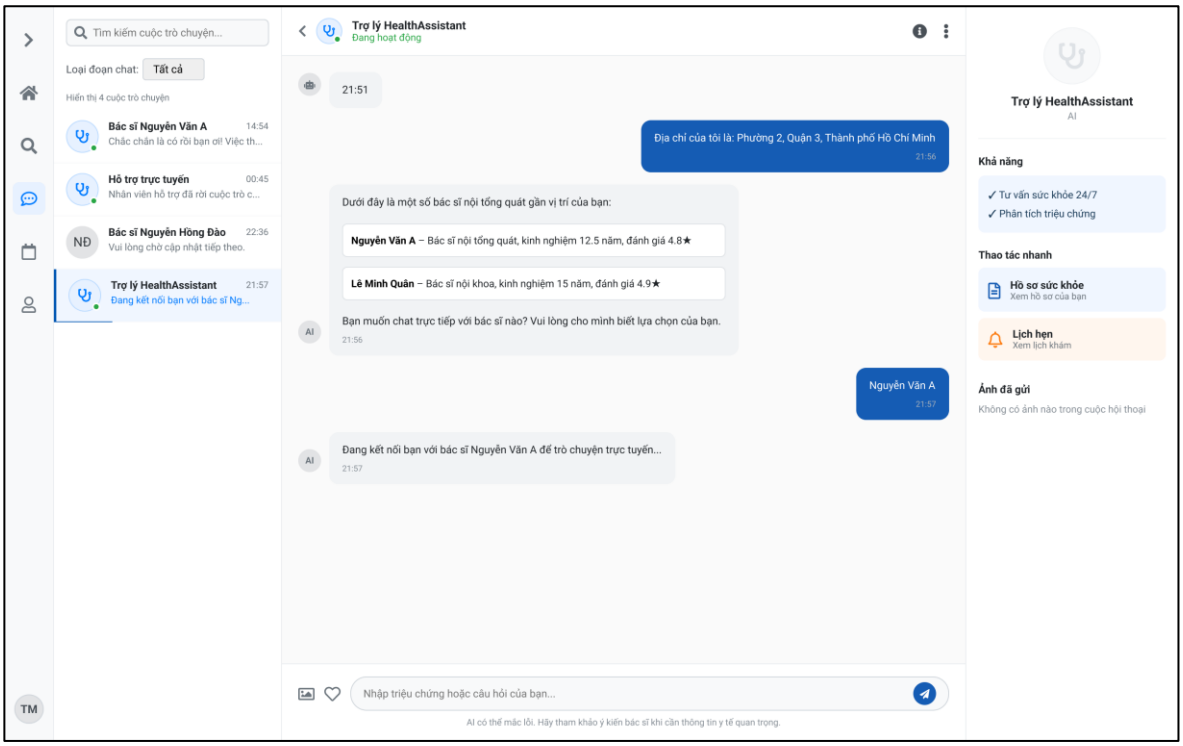
Hình 3.27. Minh họa về Modal: Đánh giá bác sĩ

3.6.5. Phân tích thiết kế các chức năng chính

Trong nội dung này, nhóm tập trung phân tích thiết kế của hai chức năng trọng tâm trong hệ thống bao gồm: trò chuyện tư vấn sức khỏe và đặt lịch khám. Đây là các chức năng cốt lõi được xây dựng nhằm hỗ trợ người dùng thao tác thuận tiện, trực quan và đảm bảo tính liên tục trong quá trình sử dụng hệ thống.

Chức năng nhắn tin

Chức năng trò chuyện được xem là trung tâm của hệ thống, đóng vai trò hỗ trợ tương tác giữa người dùng với AI Chatbot hoặc bác sĩ hoặc quản trị viên hỗ trợ thông qua cơ chế nhắn tin thời gian thực. Giao diện chức năng này được thiết kế theo mô hình nhiều cột nhằm tối ưu khả năng theo dõi hội thoại và hỗ trợ người dùng thao tác thuận tiện hơn trong quá trình sử dụng.



Hình 3.28. Bố cục chức năng nhắn tin

Bố cục giao diện bao gồm bốn khu vực chính:

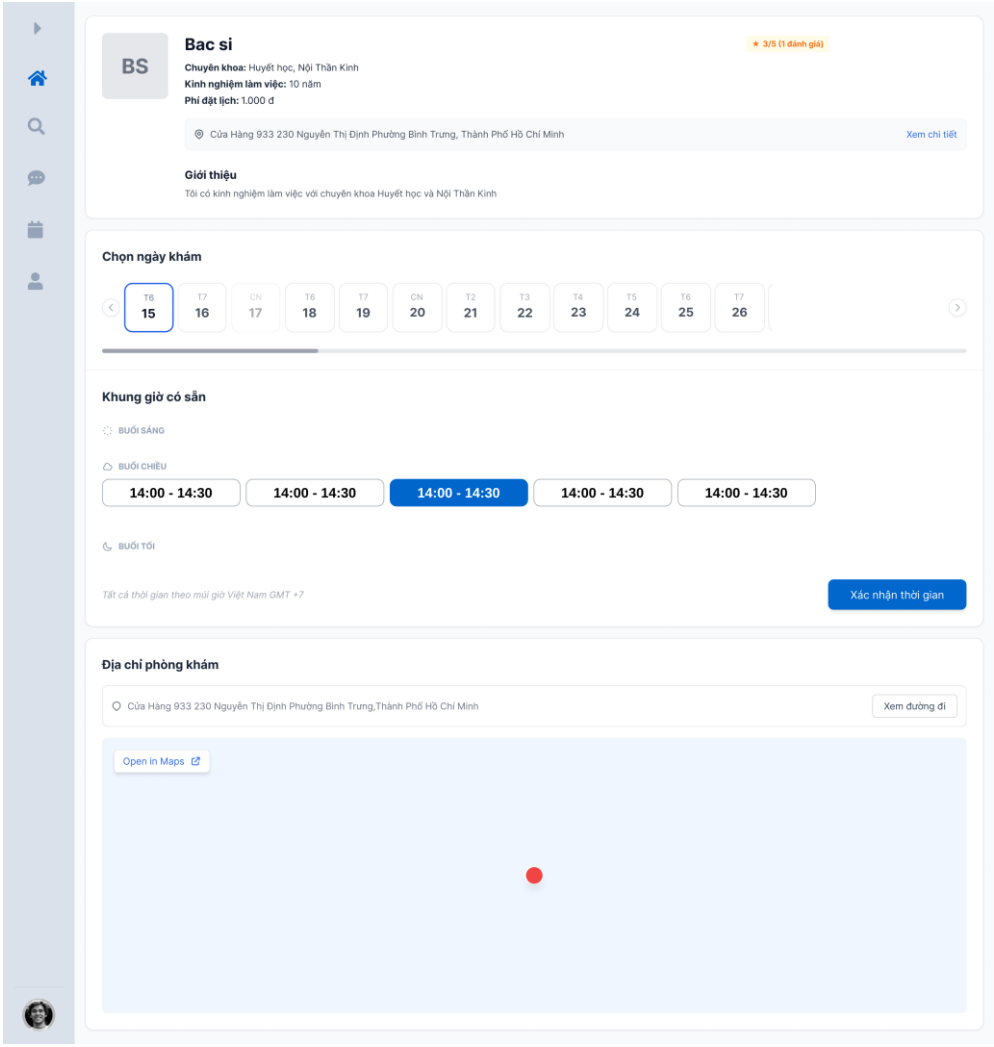
- Thanh điều hướng (sidebar) ở phía bên trái dùng để truy cập nhanh đến các chức năng của hệ thống như trang chủ, bác sĩ, tin nhắn, lịch hẹn và cá nhân.
- Danh sách cuộc trò chuyện được bố trí kế bên sidebar nhằm hiển thị các đoạn hội thoại gần đây giữa người dùng với AI, bác sĩ hoặc quản trị viên hỗ trợ.
- Khu vực hội thoại chính nằm ở trung tâm giao diện, hỗ trợ hiển thị nội dung trò chuyện theo thời gian thực.
- Khu vực các hình ảnh trong đoạn hội thoại đặt ở phía bên phải nhằm hỗ trợ xem nhanh các hình ảnh liên quan.

Đối với trải nghiệm người dùng, giao diện được thiết kế theo hướng tối ưu hóa không gian hiển thị cho nội dung hội thoại. Sidebar được thu gọn nhằm giúp người dùng tập trung hơn vào nội dung trò chuyện. Ngoài ra, hệ thống còn hỗ trợ cập nhật trạng thái hoạt động theo thời gian thực như trạng thái trực tuyến, gửi tin nhắn hoặc chuyển cuộc trò chuyện sang bác sĩ phù hợp.

Đặt lịch khám

Chức năng đặt lịch khám được thiết kế nhằm hỗ trợ người dùng dễ dàng tìm kiếm và lựa chọn thời gian khám phù hợp với bác sĩ mong muốn. Giao diện chức năng này

tập trung vào tính trực quan, khả năng thao tác nhanh và tối ưu quá trình đặt lịch trên nền tảng trực tuyến.



Hình 3.29. Bố cục chức năng đặt lịch khám

Bố cục giao diện được chia thành 3 sections thông tin:

- Thông tin bác sĩ: Hiển thị ở phía trên giao diện bao gồm tên bác sĩ, chuyên khoa, số năm kinh nghiệm, chi phí khám và thông tin giới thiệu cơ bản. Ngoài ra, hệ thống còn hiển thị đánh giá của người dùng nhằm hỗ trợ người dùng có thêm cơ sở tham khảo trước khi lựa chọn lịch khám.
- Ngày khám và khoảng giờ trống: Thời gian khám được thiết kế theo dạng timeline kết hợp danh sách khung giờ trực quan. Người dùng có thể lựa chọn ngày khám thông qua thanh thời gian nằm ngang, sau đó tiếp tục chọn các khung giờ còn trống tương ứng với lịch làm việc của bác sĩ. Các khung giờ đã

được lựa chọn sẽ được làm nổi bật bằng màu sắc nhằm giúp người dùng dễ dàng nhận biết trạng thái hiện tại.

- Địa chỉ phòng khám: Hỗ trợ người dùng dễ dàng xác định vị trí khám bệnh. Người dùng có thể xem nhanh vị trí phòng khám hoặc mở trực tiếp trên nền tảng bản đồ để tra cứu đường đi.

Về mặt trải nghiệm người dùng, giao diện được thiết kế theo hướng giảm thiểu số bước thao tác trong quá trình đặt lịch khám. Người dùng chỉ cần thực hiện các bước chính bao gồm lựa chọn ngày khám, khung giờ phù hợp, xác nhận lịch hẹn và tiến hành thanh toán. Các thành phần giao diện được bố trí rõ ràng và đồng nhất nhằm giúp người dùng dễ làm quen và thao tác thuận tiện hơn trong quá trình sử dụng hệ thống.

3.6.6. Trải nghiệm người dùng (UX)

Bên cạnh yếu tố giao diện, trải nghiệm người dùng (UX) cũng là một trong những nội dung được chú trọng trong quá trình phát triển hệ thống. Các chức năng được thiết kế theo hướng giảm bớt các thao tác không cần thiết, đồng thời tăng khả năng tương tác giữa người dùng và hệ thống.

Đối với các chức năng được sử dụng thường xuyên như nhắn tin hoặc tìm kiếm bác sĩ, hệ thống áp dụng cơ chế thu gọn thanh điều hướng (collapsible sidebar) nhằm mở rộng không gian hiển thị nội dung chính, giúp người dùng dễ tập trung hơn trong quá trình sử dụng. Bên cạnh đó, tính năng nhắn tin còn tích hợp cơ chế thông báo tức thời và phản hồi thời gian thực nhằm đảm bảo quá trình tương tác diễn ra liên tục.

Ngoài ra, giao diện của hệ thống cũng được phân chia theo từng vai trò người dùng khác nhau như bệnh nhân, bác sĩ, quản trị viên, quản trị viên quản lý bác sĩ, quản trị viên kiểm duyệt bác sĩ, quản trị viên hỗ trợ. Mỗi nhóm đối tượng sẽ được hiển thị các chức năng phù hợp với nhu cầu sử dụng thực tế, từ đó giúp giao diện trở nên trực quan hơn và hỗ trợ người dùng thao tác thuận tiện hơn trong quá trình sử dụng hệ thống.

CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ

4.1. Môi trường triển khai

4.1.1. Cấu hình phần cứng và nền tảng triển khai

Hệ thống được triển khai trên môi trường máy chủ ảo (Virtual Private Server - VPS) nhằm mô phỏng điều kiện vận hành thực tế của một hệ thống tư vấn sức khỏe tích hợp AI. Việc triển khai trên VPS giúp hệ thống có thể hoạt động độc lập, hỗ trợ truy cập từ xa và đánh giá khả năng giao tiếp giữa các thành phần trong kiến trúc Microservices. Máy chủ triển khai được cấu hình nhằm đáp ứng nhu cầu xử lý đồng thời giữa backend, workflow AI và cơ sở dữ liệu. Ngoài ra, môi trường triển khai còn cần đảm bảo khả năng duy trì hội thoại và xử lý các yêu cầu gọi workflow AI theo thời gian thực.

Error! Reference source not found. trình bày cấu hình kỹ thuật của máy chủ VPS được sử dụng trong quá trình triển khai hệ thống.

Bảng 4.1. Thông số kỹ thuật của VPS

Thuộc tính	Thông số
Loại máy chủ	AWS EC2 VPS
Instance Type	m7i-flex.large
CPU	2 vCPU
RAM	8 GB
Ổ lưu trữ	SSD GP3
Hệ điều hành	Ubuntu Server 24.04 LTS (Ubuntu Noble)
Công nghệ triển khai	Docker Engine và Docker Compose

Cấu hình trên được lựa chọn nhằm đáp ứng nhu cầu triển khai và kiểm thử hệ thống trong môi trường prototype. Với 2 vCPU và 8 GB RAM, máy chủ có khả năng xử lý đồng thời các service backend, workflow AI và cơ sở dữ liệu trong quá trình vận hành.

Hệ điều hành Ubuntu Server 24.04 LTS (Ubuntu Noble) được sử dụng do có tính ổn định cao, hỗ trợ lâu dài và tương thích tốt với môi trường Docker. Ngoài ra, Docker Engine và Docker Compose được sử dụng để chuẩn hóa môi trường triển khai, giúp các service có thể được vận hành độc lập và dễ dàng quản lý trong quá trình phát triển cũng như kiểm thử hệ thống.

4.1.2. Công nghệ và công cụ sử dụng

Hệ thống được xây dựng theo kiến trúc Microservices kết hợp workflow AI, do đó việc lựa chọn công nghệ cần đảm bảo khả năng mở rộng, tính ổn định và thuận tiện cho quá trình phát triển. Ở lớp backend, Java Spring Boot được sử dụng để xây dựng các dịch vụ xử lý nghiệp vụ và REST API [20]. Spring Boot hỗ trợ tốt cho kiến trúc Microservices, đồng thời cung cấp khả năng quản lý dependency và cấu hình hệ thống hiệu quả.

Đối với workflow AI, nền tảng n8n được sử dụng để xây dựng luồng xử lý hội thoại và giao tiếp với mô hình Gemini AI. Việc sử dụng n8n giúp quá trình tổ chức workflow trực quan hơn và thuận tiện cho việc mở rộng trong tương lai. Redis được sử dụng để lưu trữ bộ nhớ hội thoại nhằm hỗ trợ duy trì ngữ cảnh giữa các lần trao đổi của người dùng với chatbot. Ngoài ra, hệ thống sử dụng PostgreSQL để lưu trữ dữ liệu người dùng và lịch sử hội thoại.

Bảng 4.2. Công nghệ sử dụng trong hệ thống

Công nghệ	**Vai trò**
Java Spring Boot	Xây dựng backend Microservices
n8n	Xây dựng workflow AI
Gemini 3.5 Flash	Sinh phản hồi AI
PostgreSQL	Lưu trữ dữ liệu hệ thống
Redis Cloud	Lưu bộ nhớ hội thoại
Docker	Container hóa hệ thống
Docker Compose	Quản lý triển khai các service

Postman	Kiểm thử API
GitHub	Quản lý mã nguồn

4.1.3. Tổng quan môi trường triển khai

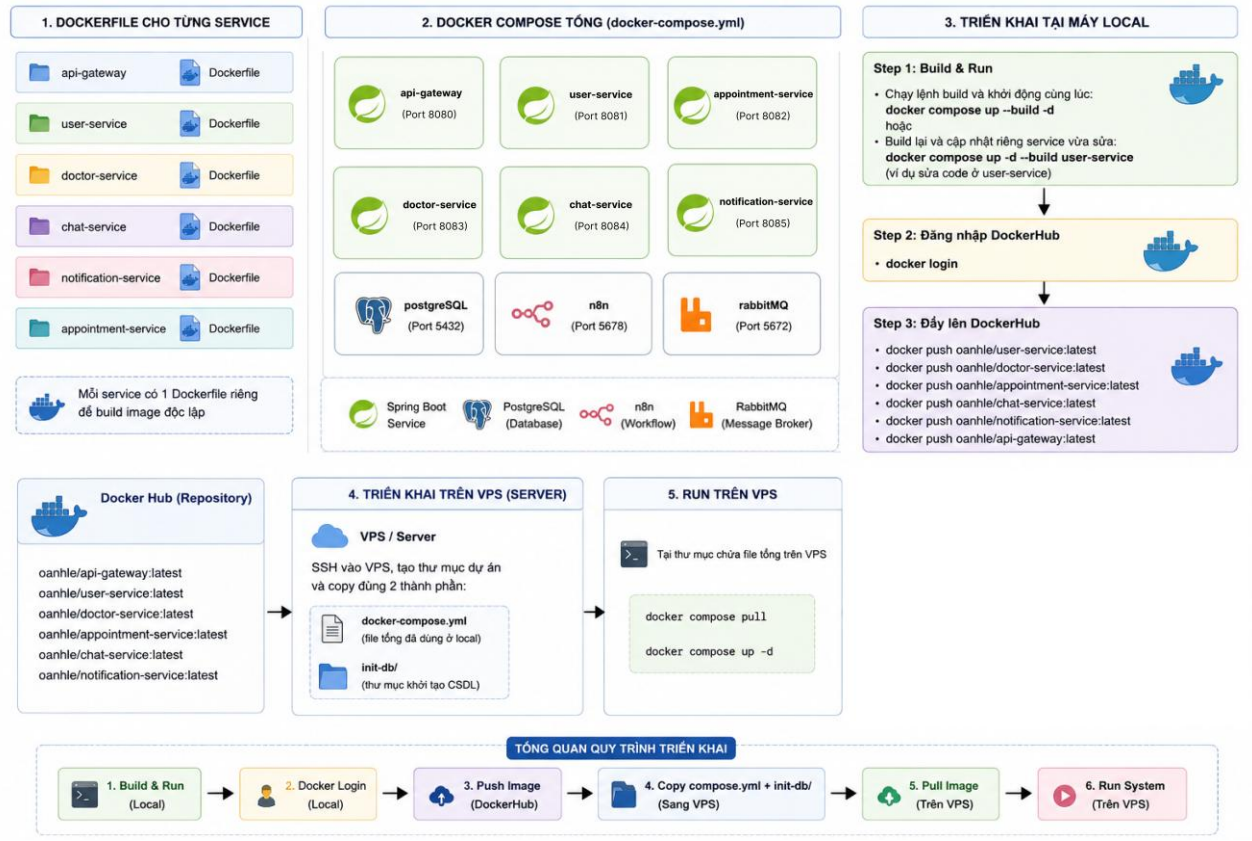
Môi trường triển khai của hệ thống được xây dựng theo mô hình phân tán cơ bản nhằm mô phỏng kiến trúc vận hành của các hệ thống AI hiện đại. Trong kiến trúc này, giao diện web đóng vai trò là nơi tương tác giữa người dùng và hệ thống. Backend Microservices chịu trách nhiệm xử lý các chức năng nghiệp vụ và giao tiếp với workflow AI thông qua REST API.

Workflow AI được triển khai độc lập trên nền tảng n8n nhằm xử lý luồng hội thoại, duy trì ngữ cảnh và giao tiếp với mô hình Gemini AI. Redis được sử dụng để lưu trữ trạng thái hội thoại giúp chatbot có khả năng phản hồi theo ngữ cảnh. Các thành phần trong hệ thống giao tiếp với nhau thông qua REST API nhằm đảm bảo tính độc lập giữa các service và thuận tiện cho việc mở rộng trong tương lai.

Môi trường triển khai này không chỉ đáp ứng yêu cầu vận hành của hệ thống mà còn hỗ trợ quá trình kiểm thử và đánh giá khả năng hoạt động của kiến trúc Microservices kết hợp workflow AI trong điều kiện gần với thực tế sử dụng.

4.2. Triển khai hệ thống Microservices và workflow AI

4.2.1. Triển khai hệ thống Microservices



Hình 4.1. Quy trình triển khai thủ công được áp dụng

Hệ thống được xây dựng theo kiến trúc Microservices, trong đó các chức năng nghiệp vụ được phân tách thành các service độc lập nhằm tăng khả năng mở rộng và thuận tiện cho việc bảo trì hệ thống. Các service chính của hệ thống bao gồm:

- user-service: Quản lý thông tin người dùng và xác thực.
- doctor-service: Quản lý thông tin bác sĩ, lịch làm việc, trợ lý AI, bảng lương và các yêu cầu kiểm duyệt bác sĩ.
- chat-service: Xử lý giao tiếp giữa backend và workflow AI, lưu trữ lịch sử hội thoại.
- appointment-service: Quản lý các thao tác với lịch hẹn, thanh toán, thống kê hệ thống.
- notification-service: Thực hiện gửi mail khi cần.
- api-gateway : Đóng vai trò cổng giao tiếp trung gian giữa frontend và các backend service, thực hiện định tuyến request đến service phù hợp.

Toàn bộ các service backend được phát triển bằng Java và Spring Boot, đồng thời được đóng gói dưới dạng container thông qua công nghệ Docker. Mỗi service sở hữu một Dockerfile riêng nhằm cấu hình môi trường runtime, dependency và cổng giao tiếp tương ứng. Trong báo cáo này, nhóm chỉ minh họa Dockerfile của api-gateway do các service còn lại có cấu trúc triển khai tương tự. Việc chuẩn hóa Dockerfile giúp đảm bảo tính nhất quán giữa các môi trường triển khai và hỗ trợ khả năng vận hành độc lập của từng service.

```
FROM maven:3.9.6-eclipse-temurin-17-alpine AS build_stage
WORKDIR /build

RUN mvn dependency:go-offline -B

COPY src ./src

RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app

COPY --from=build_stage /build/target/*.jar app.jar

# Expose the corresponding port based on the service
EXPOSE 8080

# Run the application and force inject the RAM limit variable $JAVA_OPTS from docker-compose
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

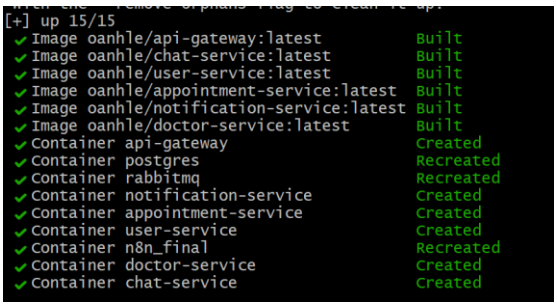
Hình 4.2. Dockerfile của api-gateway

Sau khi hoàn thiện, các Docker image được build và khởi chạy trên VPS AWS thông qua Docker Compose. Việc sử dụng Docker Compose cho phép quản lý đồng thời nhiều service trong cùng một môi trường triển khai, đồng thời đơn giản hóa quá trình khởi động và cập nhật hệ thống.

```
services:
  postgres:
    image: postgis/postgis:16-3.4
    container_name: postgres
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: 123
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./init-db:/docker-entrypoint-initdb.d # Đảm bảo thư mục này chứa file SQL tạo sẵn các DB: user_service_db, doctor_service_db, n8n_db, ...
    restart: unless-stopped
    # Giới hạn Postgres ăn tối đa 500MB RAM
    deploy:
      resources:
        limits:
          memory: 500M

  n8n:
    image: n8nio/n8n:latest
    container_name: n8n_final
    ports:
      - "5678:5678"
    environment:
      DB_TYPE: postgresdb
      DB_POSTGRESDB_HOST: postgres
      DB_POSTGRESDB_PORT: 5432
      DB_POSTGRESDB_DATABASE: n8n_db #: Tách riêng DB cho n8n
      DB_POSTGRESDB_USER: user
      DB_POSTGRESDB_PASSWORD: 123
```

Hình 4.3. Trích đoạn docker-compose.yml dùng trong triển khai.



Hình 4.4. Kết quả build image và khởi tạo các container ở local



Hình 4.5. Trạng thái hoạt động của các container sau khi triển khai trên VPS

Thông qua cơ chế container hóa, toàn bộ hệ thống có thể được tái tạo nhanh chóng trên môi trường mới mà không phụ thuộc vào cấu hình vật lý của máy chủ. Đồng thời, việc triển khai theo kiến trúc Microservices giúp các service có thể được cập nhật hoặc mở rộng độc lập mà không ảnh hưởng đến toàn bộ hệ thống.

4.2.2. Triển khai workflow AI trên nền tảng n8n

Workflow AI được triển khai trên nền tảng n8n nhằm xử lý luồng hội thoại tư vấn sức khỏe và điều phối giao tiếp giữa backend với workflow AI. Workflow được xây dựng theo hướng trực quan thông qua các node xử lý độc lập, cho phép dễ dàng chỉnh sửa và mở rộng trong quá trình phát triển hệ thống.

Trong quá trình triển khai thực tế, hệ thống AI được tổ chức thành hai AI Agent chính bao gồm General AI Agent và Personal AI Agent nhằm xử lý các loại yêu cầu tư vấn khác nhau. General AI Agent được sử dụng cho các cuộc tư vấn sức khỏe tổng quát giữa người dùng và hệ thống AI. Trong khi đó, Personal AI Agent được sử dụng cho các cuộc tư vấn riêng giữa người dùng và bác sĩ trên hệ thống.

Khi người dùng gửi yêu cầu từ giao diện web, dữ liệu sẽ được chuyển đến hệ thống AI thông qua Webhook. Sau khi tiếp nhận dữ liệu đầu vào, hệ thống sẽ kiểm tra key status trên Redis Cloud để xác định nên chuyển đến AI Agent nào. Việc phân tách thành nhiều AI Agent giúp hệ thống dễ dàng mở rộng và quản lý các luồng xử lý AI khác nhau trong tương lai.

Ngoài dữ liệu văn bản, hệ thống còn hỗ trợ xử lý hình ảnh nếu người dùng gửi kèm ảnh liên quan đến chỉ số sức khỏe hoặc tình trạng bệnh lý. Hình ảnh sẽ được phân tích trước khi gửi đến AI Agent và kết quả phân tích sẽ được bổ sung vào đầu vào ban đầu nhằm hỗ trợ quá trình sinh phản hồi chính xác hơn.

Trong một số trường hợp, hệ thống có thể kích hoạt thêm quy trình tóm tắt triệu chứng nhằm hỗ trợ quá trình đặt lịch khám hoặc phục vụ cho bác sĩ trong quá trình theo dõi thông tin tư vấn. Cuối cùng, phản hồi từ AI sẽ được lưu vào hệ thống và gửi trả về backend thông qua Webhook để hiển thị trên giao diện web.

Việc sử dụng n8n giúp quá trình xây dựng và quản lý workflow AI trở nên trực quan hơn so với việc xử lý toàn bộ logic AI trực tiếp trong backend. Đồng thời, Redis Cloud hỗ trợ duy trì bộ nhớ hội thoại hiệu quả và giúp hệ thống có khả năng phản hồi theo ngữ cảnh giữa nhiều phiên trao đổi khác nhau.

4.2.3. Triển khai cơ sở dữ liệu và bộ nhớ hội thoại

Hệ thống sử dụng PostgreSQL làm cơ sở dữ liệu chính để lưu trữ thông tin người dùng và dữ liệu hội thoại. PostgreSQL được triển khai dưới dạng container Docker nhằm đảm bảo tính nhất quán giữa môi trường phát triển và môi trường triển khai thực tế.

Ngoài ra, Redis Cloud được sử dụng để lưu trữ bộ nhớ hội thoại phục vụ cho workflow AI. Redis Cloud cho phép truy xuất dữ liệu nhanh và hỗ trợ duy trì ngữ cảnh hội thoại trong quá trình chatbot xử lý yêu cầu từ người dùng. Việc sử dụng Redis Memory giúp hệ thống có khả năng phản hồi theo ngữ cảnh thay vì xử lý từng câu hỏi một cách độc lập.

4.2.4. Triển khai hệ thống trên VPS AWS

Sau khi hoàn thiện quá trình phát triển, toàn bộ hệ thống được triển khai thử nghiệm trên VPS AWS EC2 sử dụng hệ điều hành Ubuntu Server 24.04 LTS. Các service được đóng gói dưới dạng container Docker và quản lý thông qua Docker Compose. Hệ thống được vận hành trực tiếp trên VPS thông qua địa chỉ IP public của máy chủ.

Trong phạm vi đề tài, hệ thống chưa triển khai tên miền riêng hoặc reverse proxy như Nginx mà chủ yếu tập trung vào mục tiêu nghiên cứu, triển khai thử nghiệm và đánh giá khả năng hoạt động của kiến trúc Microservices kết hợp workflow AI.

Mặc dù chưa được tối ưu hoàn toàn cho môi trường production thực tế, môi trường triển khai hiện tại vẫn đáp ứng tốt yêu cầu vận hành thử nghiệm và hỗ trợ kiểm tra khả năng hoạt động của hệ thống sau khi triển khai.

4.3. Kết quả triển khai giao diện

4.3.1. Triển khai giao diện dự án lên nền tảng Netlify

Sau khi hoàn thiện các chức năng ở môi trường phát triển, hệ thống frontend được triển khai lên nền tảng **Netlify** để phục vụ mục đích kiểm thử, thử nghiệm tốc độ và đánh giá thực tế.

Việc triển khai frontend lên **Netlify** giúp đơn giản hóa quá trình deploy, giảm phụ thuộc vào cấu hình server truyền thống và tăng tốc độ đưa sản phẩm vào thử nghiệm thực tế. Tuy nhiên, Netlify chỉ phù hợp cho frontend tĩnh, do đó backend của hệ thống vẫn được triển khai riêng biệt trên môi trường VPS riêng để đảm bảo khả năng xử lý nghiệp vụ phức tạp.

Giới thiệu về nền tảng Netlify

Netlify là nền tảng cloud hosting chuyên dùng để triển khai các ứng dụng web tĩnh và frontend hiện đại như ReactJS, VueJS, Angular, đồng thời hỗ trợ các ứng dụng có Server-Side Rendering thông qua serverless functions. Nền tảng này hỗ trợ triển khai tự động thông qua Git repository (GitHub, GitLab, Bitbucket) và cung cấp các tính năng:

- CI/CD tự động
- HTTPS miễn phí cho mọi dự án
- CDN toàn cầu giúp tăng tốc độ truy cập (thế nhưng hạ tầng chủ yếu đặt tại Mỹ, độ trễ với người dùng Việt Nam có thể cao hơn so với các nền tảng tại Đông Nam Á)
- Hỗ trợ routing cho Single Page Application (SPA)
- Dễ dàng rollback phiên bản khi có lỗi nhờ mỗi lần deploy được lưu trữ độc lập

Nhờ các ưu điểm này, Netlify được lựa chọn để triển khai frontend của hệ thống nhằm thực nghiệm và kiểm thử.

Quy trình triển khai frontend lên Netlify

Bước 1: Build project ReactJS (tại local để kiểm tra lỗi và sửa lỗi, trước khi tiến hành triển khai lên Netlify)

Dùng lệnh: npm run build

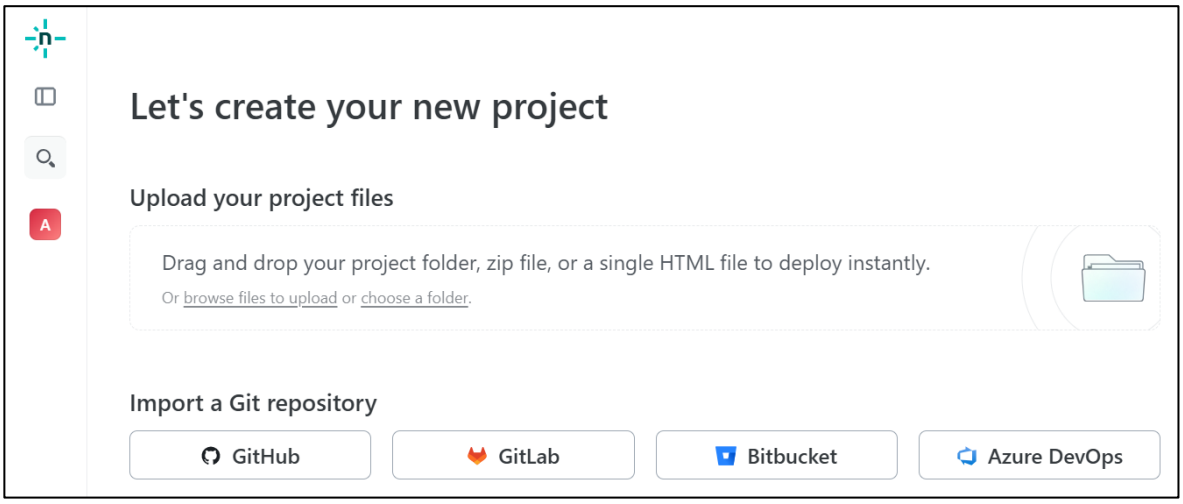
Bước 2: Kết nối dự án với Netlify

- Chọn “Add new project”

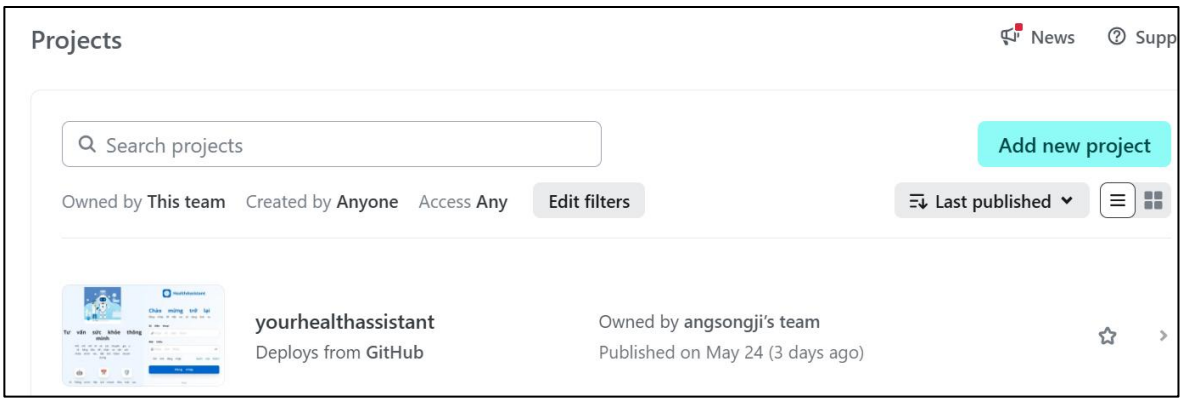


Hình 4.6. Giao diện trang Projects trên Netlify

- Chọn cách thêm dự án vào Netlify



Hình 4.7. Giao diện thêm dự án lên Netlify

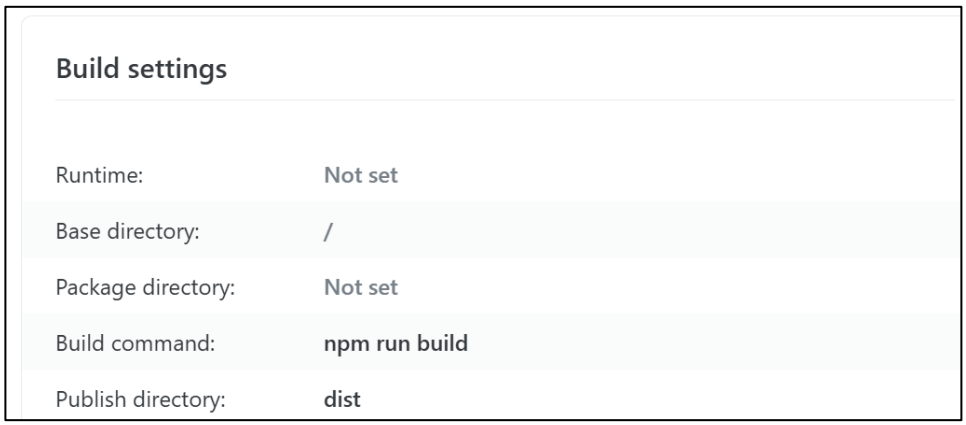


Hình 4.8. Kết quả sau khi thêm dự án lên Netlity

Bước 3: Cấu hình build settings

Thiết lập thông số triển khai:

- Build command: npm run build (Lệnh đóng gói: biên dịch để tự động thu gom, tối ưu hóa và nén toàn bộ mã nguồn lập trình của bạn (React, Next.js, Vue, hay mã JavaScript/CSS gốc) thành các tệp tin nhẹ nhất, chuẩn nhất)
- Publish directory: dist (Thư mục xuất bản: tên của thư mục chứa toàn bộ sản phẩm hoàn chỉnh sau khi lệnh npm run build ở trên chạy xong, mặc định của React là dist)

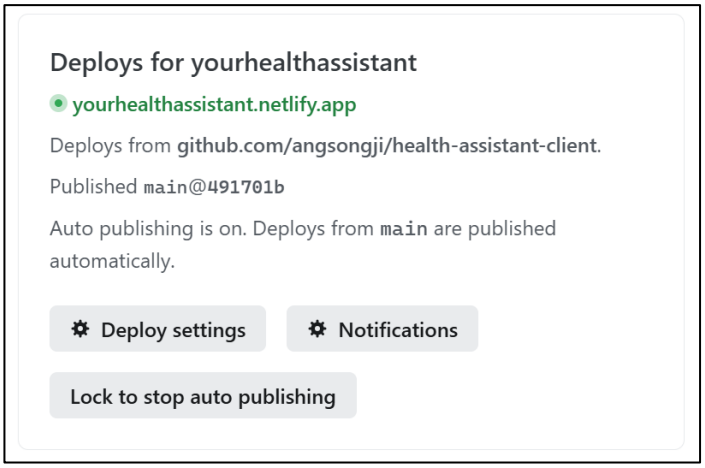


Hình 4.9. Cấu hình xuất bản ứng dụng

Bước 4: Triển khai tự động

Sau khi cấu hình, Netlify tự động:

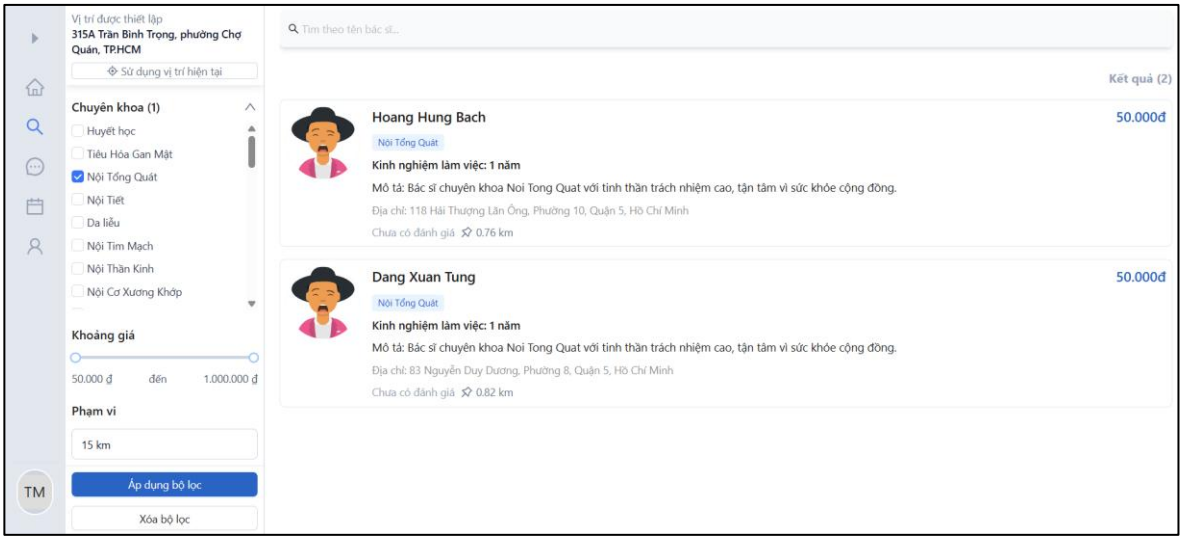
- Build project mỗi khi project được đẩy phiên bản mới
- Deploy lên CDN toàn cầu
- Cấp domain truy cập



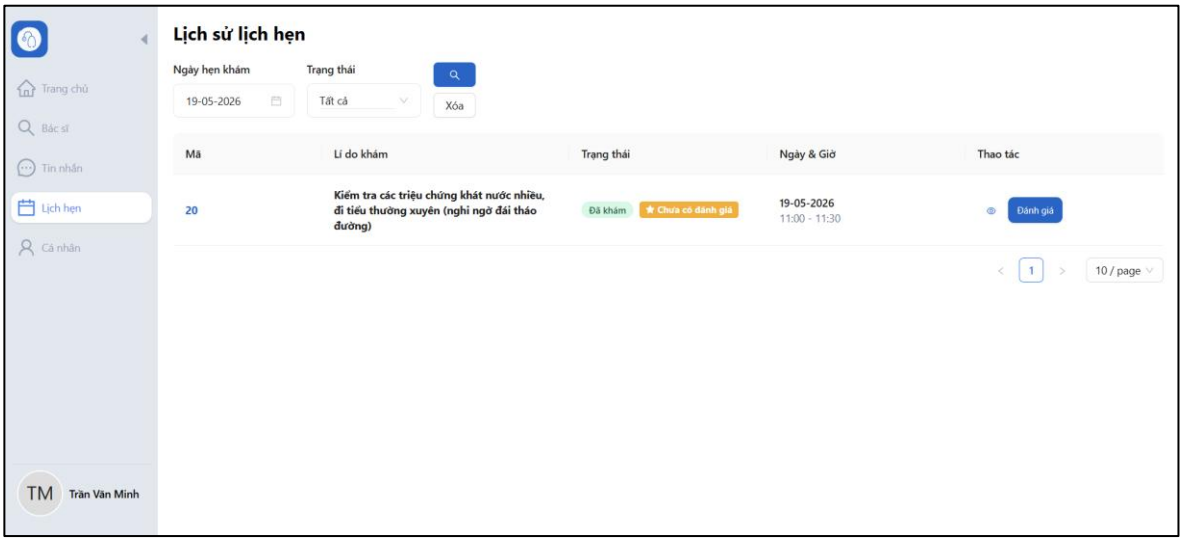
Hình 4.10. Thông tin về phiên bản được xuất bản gần nhất

4.3.2. Kết quả giao diện các chức năng cốt lõi

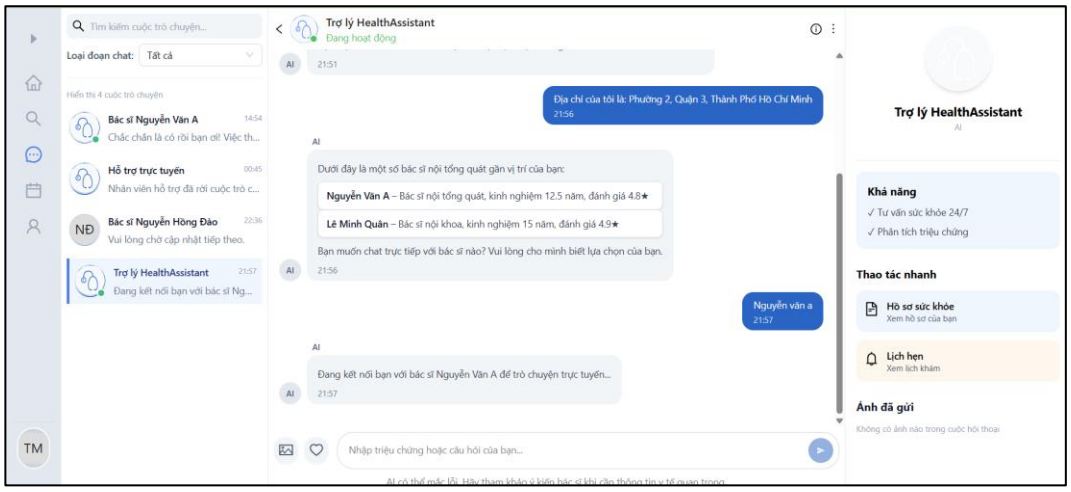
Giao diện chế độ người dùng



Hình 4.11. Giao diện chức năng tìm kiếm bác sĩ

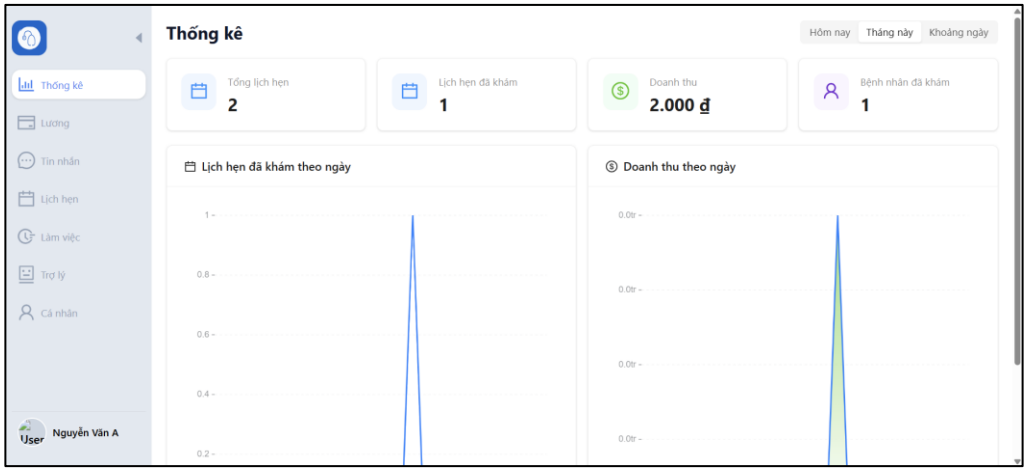


Hình 4.12. Giao diện chức năng quản lý lịch hẹn

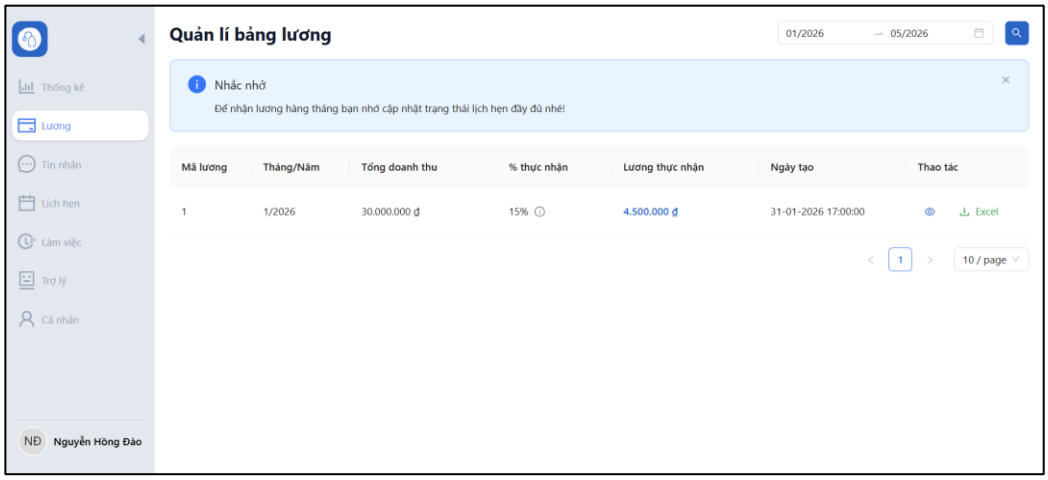


Hình 4.13. Giao diện chức năng nhắn tin với AI ChatBot

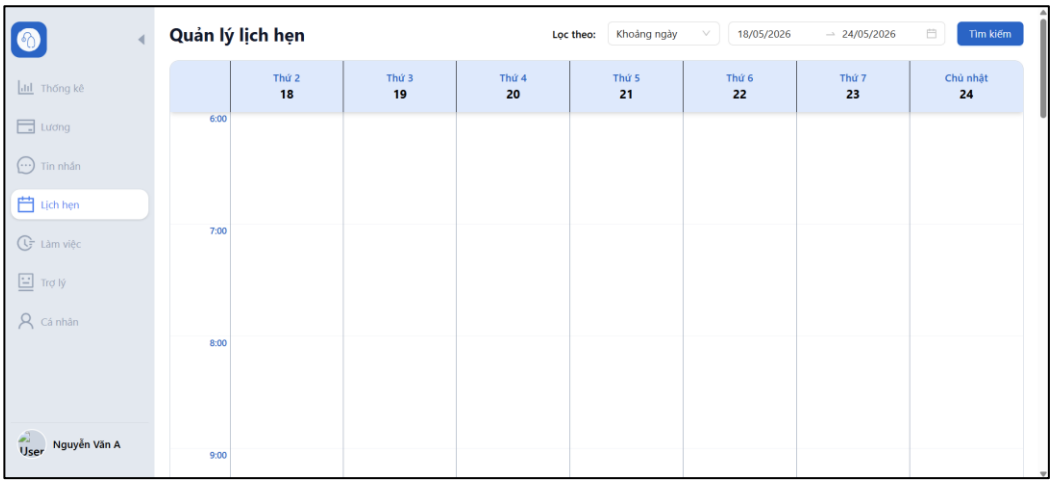
Giao diện chế độ bác sĩ



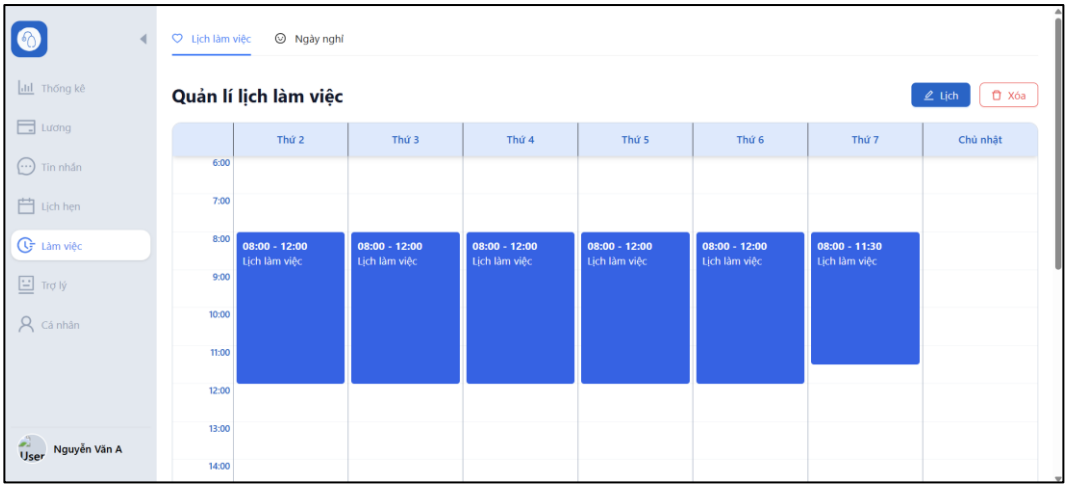
Hình 4.14. Giao diện chức năng thống kê



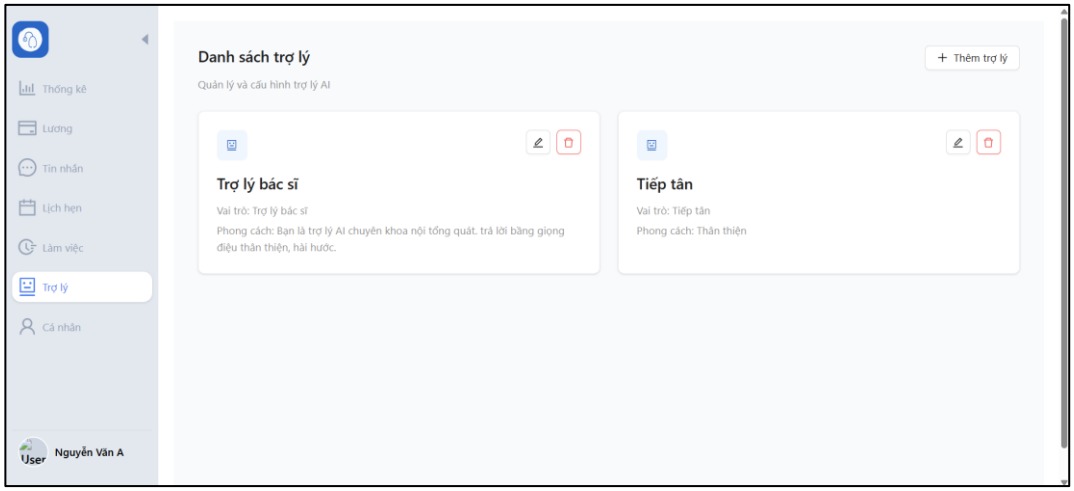
Hình 4.15. Giao diện chức năng quản lý lương



Hình 4.16. Giao diện chức năng quản lý lịch hẹn

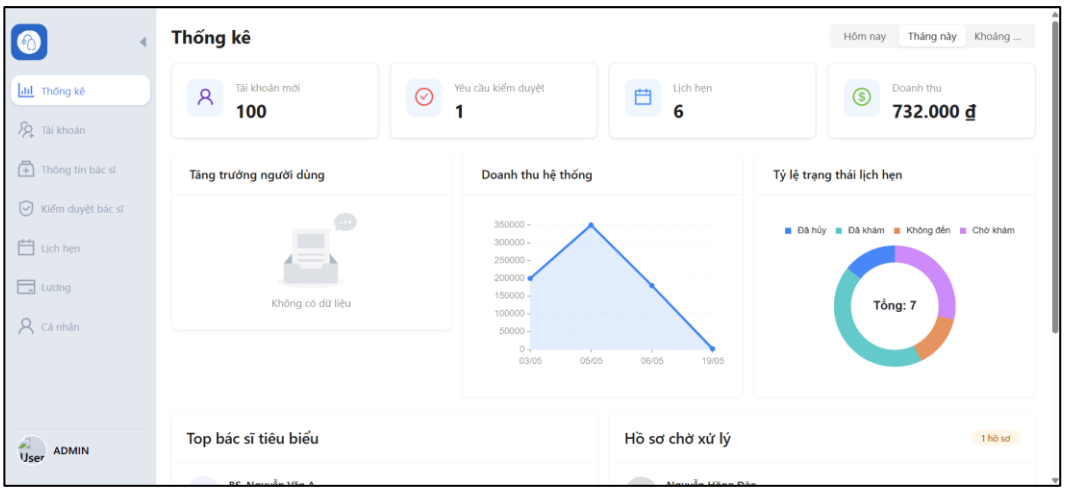


Hình 4.17. Giao diện chức năng quản lý lịch làm việc

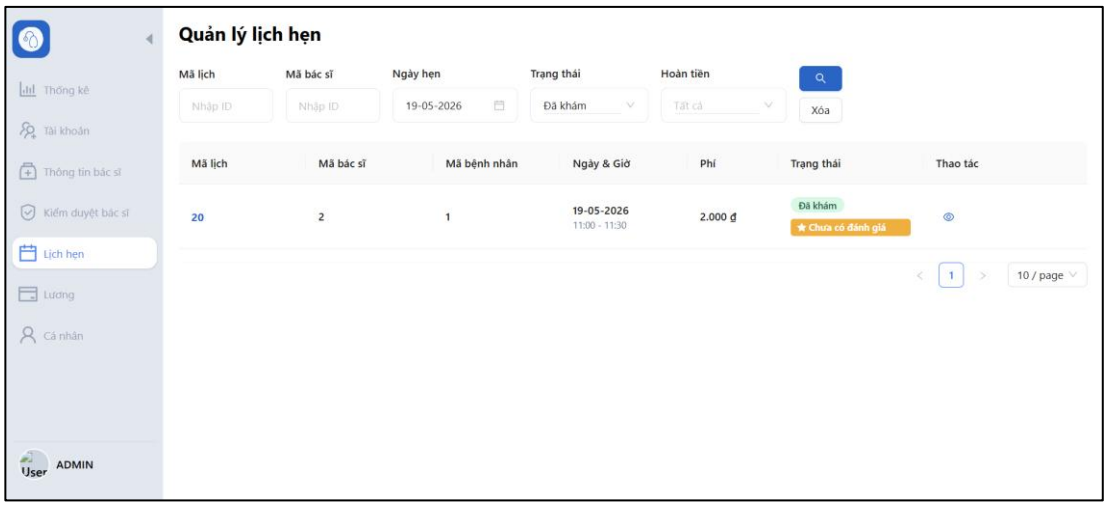


Hình 4.18. Giao diện quản lý trợ lý

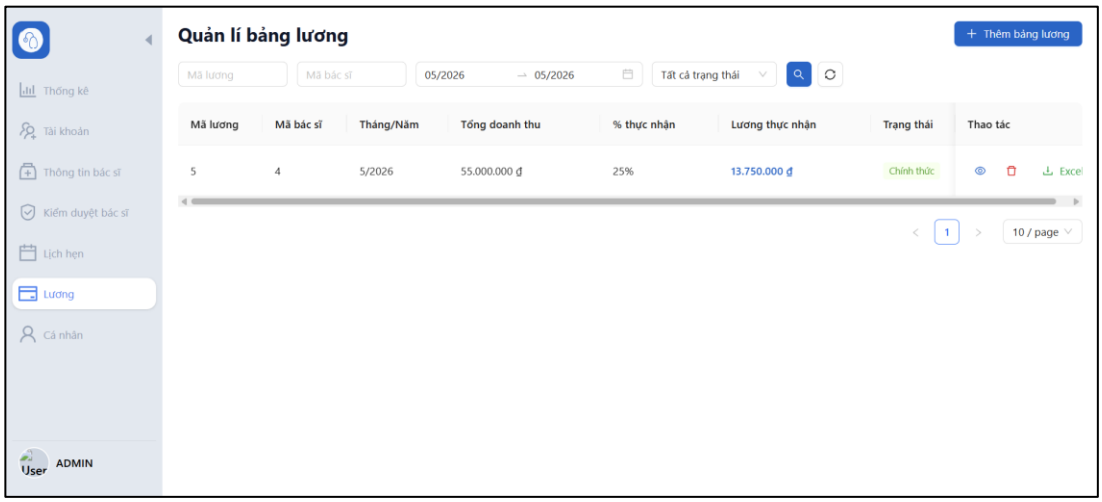
Giao diện chế độ quản trị viên



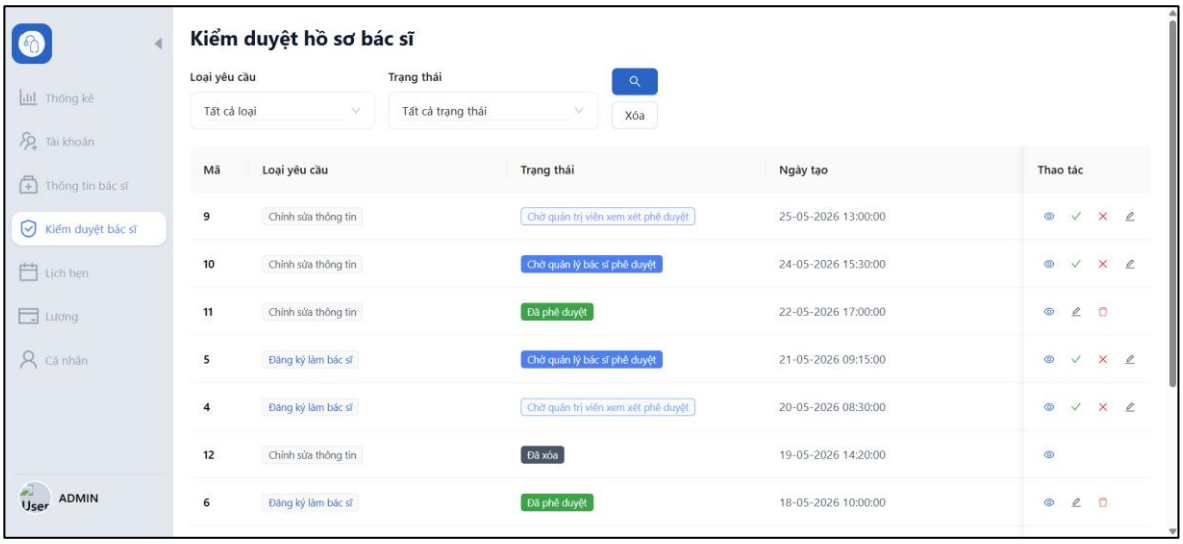
Hình 4.19. Giao diện chức năng thống kê



Hình 4.20. Giao diện chức năng quản lý lịch hẹn



Hình 4.21. Giao diện chức năng quản lý bảng lương



Hình 4.22. Giao diện chức năng kiểm duyệt bác sĩ

4.4. Tổ chức thực nghiệm và kịch bản kiểm thử

4.4.1. Mục tiêu kiểm thử

Quá trình kiểm thử được thực hiện nhằm đánh giá khả năng hoạt động của hệ thống sau khi triển khai, bao gồm các chức năng chính của giao diện web, backend service và workflow AI. Các nội dung kiểm thử tập trung vào:

- Khả năng hoạt động của giao diện web đối với từng vai trò người dùng.
- Tính ổn định của các backend service.
- Khả năng xử lý request của hệ thống.
- Chất lượng phản hồi của workflow AI.
- Khả năng lưu trữ và duy trì lịch sử hội thoại.

Quá trình kiểm thử được thực hiện trên môi trường phát triển cục bộ (local environment), trong đó frontend React kết nối với backend Spring Boot API và workflow AI đang hoạt động trong môi trường Docker.

4.4.2. Kiểm thử giao diện người dùng

Kiểm thử giao diện người dùng được thực hiện bằng framework Playwright nhằm đánh giá khả năng hoạt động của các chức năng chính trên hệ thống web. Bộ kiểm thử được xây dựng theo hướng End-to-End (E2E Testing), mô phỏng các thao tác thực tế của người dùng trên trình duyệt bao gồm nhập liệu, điều hướng trang và tương tác với các thành phần giao diện. Các module chức năng được kiểm thử bao gồm:

- Trang đăng nhập.

- Trang quản lý bảng lương.
- Trang hồ sơ sức khỏe người dùng.

Phạm vi và dữ liệu kiểm thử

Phạm vi: Tổng cộng 15 ca kiểm thử được chia đều cho 3 module chức năng đã đề cập bên trên.

Dữ liệu kiểm thử: Để đảm bảo tính thực tế, bộ kiểm thử sử dụng các tài khoản hệ thống sau:

- USER: Số điện thoại: 0123456789 / Mật khẩu: 123456
- DOCTOR: Số điện thoại: 0123456788 / Mật khẩu: 123456
- ADMIN: Số điện thoại: 0123456790 / Mật khẩu: 123456

Chi tiết kịch bản các ca kiểm thử

1. Module Đăng nhập

Bảng 4.3. Bảng test case module Đăng nhập

Mã test case	Mô tả	Kịch bản	Kết quả mong đợi
TC_AUTH_01	Kiểm tra lỗi validation khi để trống thông tin.	1. Click nút "Đăng nhập" khi chưa nhập số điện thoại và mật khẩu.	Hiển thị thông báo lỗi dưới các trường: "Vui lòng nhập số điện thoại" và "Vui lòng nhập mật khẩu".
TC_AUTH_02	Kiểm tra lỗi định dạng số điện thoại.	1. Nhập số điện thoại sai định dạng (ví dụ: 123) 2. Click Đăng nhập.	Hiển thị lỗi "Số điện thoại không hợp lệ (10 chữ số)".
TC_AUTH_03	Đăng nhập thất bại với tài khoản sai/không tồn tại.	1. Nhập đúng định dạng số điện thoại và mật khẩu nhưng sai mật khẩu hoặc tài khoản chưa đăng ký.	Hiển thị thông báo lỗi toast từ backend: "Thông tin đăng nhập không hợp lệ!".

		2. Click Đăng nhập.	
TC_AUTH_04	Kiểm tra Modal Quên mật khẩu.	1. Click link "Quên mật khẩu?", kiểm tra Modal xuất hiện. 2. Nhập SĐT không hợp lệ (099) bấm "Gửi mật khẩu mới" để kiểm tra lỗi validation. 3. Nhập tiếp SĐT hợp lệ nhưng không đăng ký (0999999999) bấm gửi. 4. Cuối cùng bấm nút "Hủy".	Modal đóng mở chính xác. Hiển thị lỗi validation "Số điện thoại không hợp lệ" và lỗi từ backend "Không tìm thấy tài khoản với số điện thoại này".
TC_AUTH_05	Đăng nhập thành công.	1. Nhập thông tin đăng nhập hợp lệ (USER SĐT 0123456789, Mật khẩu 123456). 2. Click Đăng nhập.	Thông báo "Đăng nhập thành công", tự động điều hướng sang trang chủ User (/user).

2. Module Quản lý bảng lương

Bảng 4.4. Bảng test case module Quản lý bảng lương

Mã test case	Mô tả	Kịch bản	Kết quả mong đợi
TC_SALARY_01	Hiển thị giao diện và danh sách bảng lương.	1. Đăng nhập tài khoản ADMIN 2. Truy cập trang /admin/salary.	Trang tải thành công, tiêu đề "Quản lý bảng lương" hiển thị, bảng lương xuất hiện đầy đủ các cột và hiển thị dòng dữ liệu.
TC_SALARY_02	Bộ lọc tìm kiếm hoạt động chính xác.	1. Nhập Mã bác sĩ = 6 2. Chọn trạng thái "Chính thức" từ dropdown 3. Bấm nút tìm kiếm.	Bảng dữ liệu tự động cập nhật hiển thị các kết quả lọc tương ứng (gọi đúng API với các tham số query và cột Mã bác sĩ hiển thị đúng giá trị 6).
TC_SALARY_03	Reset bộ lọc dữ liệu.	1. Nhập các giá trị lọc bất kỳ 2. Bấm nút Reset/Làm mới.	Các ô input và select bộ lọc quay về giá trị mặc định, bảng lương hiển thị dữ liệu gốc ban đầu.
TC_SALARY_04	Xem chi tiết bảng lương (Modal Detail).	1. Click nút biểu tượng "Con mắt" (xem chi tiết) ở một hàng bảng lương.	Modal chi tiết bảng lương mở ra, hiển thị các thông tin chi tiết: Doanh thu, phần trăm hoa hồng và danh sách các lịch hẹn liên quan.

TC_SALARY_05	Thêm bảng lương mới cho bác sĩ.	1. Bấm "Thêm bảng lương" 2. Chọn tháng 05/2026 3. Click "Tải bác sĩ" 4. Chọn bác sĩ có mã là 6 5. Bấm "Xem trước" để hiển thị thông tin tính lương tạm tính 6. Sau đó click nút "Thêm".	Modal preview hiển thị thông tin chính xác. Sau khi bấm tạo thành công thì hiển thị thông báo "Thêm bảng lương thành công" và danh sách bảng lương được cập nhật.

3. Module Hồ sơ sức khỏe

Bảng 4.5. Bảng test case module Hồ sơ sức khỏe

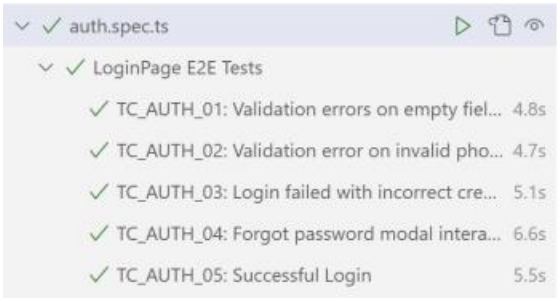
Mã test case	Mô tả	Kịch bản	Kết quả mong đợi
TC_HEALTH_01	Hiển thị thông tin hồ sơ sức khỏe hiện tại.	1. Đăng nhập tài khoản USER, 2. Truy cập trang cá nhân /user/profile 3. Click vào tab "Hồ sơ sức khỏe".	Hiển thị thông tin "Mô tả bản thân" / "Cá nhân hóa" và danh sách các chỉ số y tế hiện tại (nếu có). Nút "Chỉnh sửa" hiển thị.

TC_HEALTH_02	Thêm mới chỉ số sức khỏe trong chế độ chỉnh sửa.	1. Bấm nút “Chỉnh sửa” 2. Cuộn xuống form thêm ở dưới 3. Nhập nhãn (ví dụ: “Nhóm máu”) và giá trị (ví dụ: “O+”) 4. Bấm nút “Thêm”.	Chỉ số mới được đưa vào danh sách hiển thị tạm thời trên form và xuất hiện trong danh sách.
TC_HEALTH_03	Chỉnh sửa và xóa chỉ số sức khỏe.	1. Trong chế độ chỉnh sửa 2. Bấm biểu tượng thùng rác để xóa 3. Bấm xác nhận trong hộp thoại confirm của Antd.	Chỉ số bị xóa hoàn toàn khỏi giao diện tạm thời.
TC_HEALTH_04	Hủy bỏ các thay đổi (Cancel).	1. Thực hiện thay đổi mô tả bản thân hoặc chỉ số y tế 2. Sau đó bấm nút “Quay về” (Hủy).	Hệ thống thoát khỏi chế độ chỉnh sửa, giao diện khôi phục lại toàn bộ các giá trị gốc cũ trước khi sửa.

TC_HEALTH_05	Tải lên và phân tích hình ảnh sức khỏe bằng AI.	1. Từ giao diện xem (không ở chế độ chỉnh sửa) 2. Chọn/tải ảnh sức khỏe 3. Chọn ảnh từ đường dẫn local 4. Click nút "Phân tích" 5. Chờ kết quả.	Hiển thị thông báo "Phân tích hoàn tất!" (trong thời gian chờ tối đa 60s do dịch vụ AI xử lý ảnh). Giao diện tự động chuyển sang chế độ chỉnh sửa (isEditing=true) với các thông tin sức khỏe được điền sẵn.

Kết quả kiểm thử được trình bày trong bảng dưới đây.

Bảng 4.6. Kết quả kiểm thử giao diện người dùng

Module	Kết quả kiểm thử	Đánh giá
Đăng nhập		- Số ca kiểm thử: 5 - Passed: 5/5 - Tỷ lệ thành công: 100%
Quản lý lương		- Số ca kiểm thử: 5 - Passed: 5/5 - Tỷ lệ thành công: 100%

Hồ sơ sức khỏe		- Số ca kiểm thử: 5 - Passed: 5/5 - Tỷ lệ thành công: 100%

Kết quả cho thấy toàn bộ các test case đều thực thi thành công và các chức năng giao diện hoạt động ổn định trong quá trình kiểm thử.

4.4.3. Kiểm thử đơn vị

Kiểm thử đơn vị (Unit Test) được thực hiện nhằm đánh giá tính chính xác của các hàm xử lý nghiệp vụ trong backend service.

Quá trình kiểm thử sử dụng framework JUnit 5 kết hợp Mockito để cô lập các dependency và kiểm tra logic xử lý của hệ thống mà không cần kết nối trực tiếp đến cơ sở dữ liệu hoặc các service bên ngoài. Ngoài việc xây dựng test thủ công, hệ thống còn sử dụng Diffblue Cover để hỗ trợ sinh tự động các test case cho mã nguồn Java. Công cụ này giúp tăng độ bao phủ kiểm thử, giảm thời gian xây dựng kiểm thử đơn vị và hỗ trợ phát hiện sớm các lỗi tiềm ẩn trong quá trình phát triển hệ thống.

Kết quả kiểm thử được kết hợp với JaCoCo để đánh giá độ bao phủ mã nguồn thông qua các chỉ số như line coverage và branch coverage, từ đó hỗ trợ đánh giá mức độ đầy đủ của bộ kiểm thử backend. Module được tập trung kiểm thử là "Tìm kiếm và lọc bác sĩ".

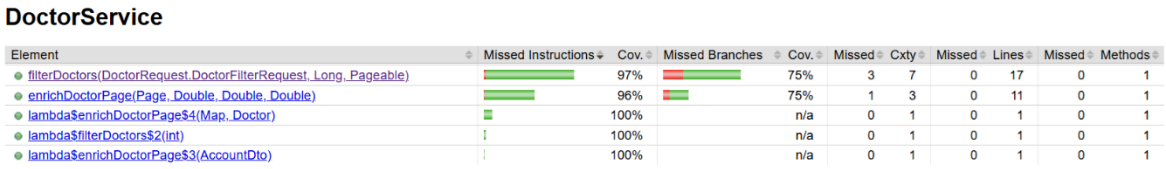
Bảng 4.7. Bảng test case module Tìm kiếm và lọc bác sĩ

Mã TC	**Kịch bản kiểm thử**	**Điều kiện / Input**	**Kết quả mong đợi**	**Hình thức**
TC01	Lọc bác sĩ thành công	Tham số lọc hợp lệ	Trả về danh sách bác sĩ phù hợp	Diffblue AI
TC02	Không tìm thấy bác sĩ phù hợp	findDoctorIds() trả về danh sách rỗng	Trả về page rỗng	Diffblue AI

TC03	Phân trang không hợp lệ	page = -1	Trả về thông báo lỗi “Page index must not be less than zero”	Diffblue AI
TC04	Không truyền vị trí người dùng	lat = null, lng = null	Hệ thống sử dụng vị trí từ tài khoản người dùng	Diffblue AI
TC05	Bác sĩ có tài khoản hợp lệ	Doctor chứa accountId hợp lệ	Trả về thông tin bác sĩ đã mapping	Diffblue AI
TC06	Có nhiều thông tin tài khoản bác sĩ	findDoctorsByDistance() trả về nhiều account	Hệ thống xử lý mapping thành công	Diffblue AI
TC07	Không có dữ liệu khoảng cách	findDoctorsByDistance() trả về rỗng	Hệ thống trả về kết quả an toàn	Diffblue AI
TC08	Lỗi khi lấy thông tin tài khoản	accountClient.getAccountById() phát sinh exception	Ném ra ResponseStatusException	Diffblue AI
TC09	Có truyền đầy đủ latitude và longitude	lat = 10.0, lng = 20.0	Hệ thống sử dụng tọa độ được truyền vào	Thủ công
TC10	Lấy danh sách bác sĩ thành công	Danh sách bác sĩ và account hợp lệ	Trả về danh sách DoctorDto đầy đủ	Diffblue AI
TC11	Danh sách bác sĩ rỗng	doctorsPage.isEmpty() = true	Trả về page rỗng	Diffblue AI

TC12	Fallback khi tìm bác sĩ bằng khoảng cách lỗi	findDoctorsByDistance() phát sinh exception	Hệ thống trả về DTO cơ bản	Diffblue AI

Kết quả độ bao phủ kiểm thử:



DoctorService

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
filterDoctors(DoctorRequest,DoctorFilterRequest, Long, Pageable)		97%		75%	3	7	0	17	0	1
enrichDoctorPage(Page, Double, Double, Double)		96%		75%	1	3	0	11	0	1
lambda$enrichDoctorPage$4(Map, Doctor)		100%		n/a	0	1	0	1	0	1
lambda$filterDoctors$2(int)		100%		n/a	0	1	0	1	0	1
lambda$enrichDoctorPage$3(AccountDto)		100%		n/a	0	1	0	1	0	1

Hình 4.23. Màn hình report của JaCoCo

Kết quả kiểm thử cho thấy các phương thức trọng tâm của module "Tìm kiếm và lọc bác sĩ" đạt độ bao phủ cao:

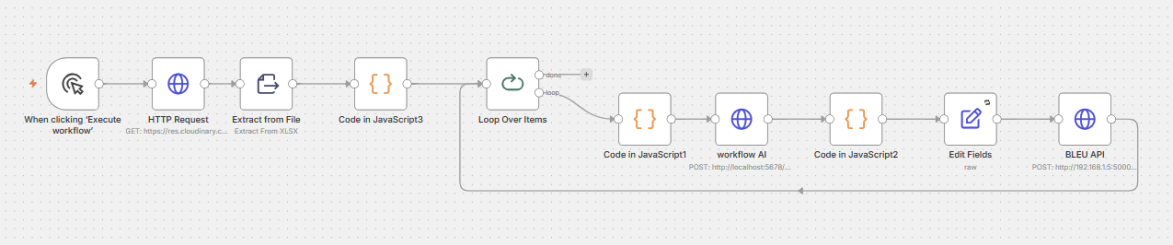
- filterDoctors()
    - Instruction Coverage: 97%
    - Branch Coverage: 75%
- enrichDoctorPage()
    - Instruction Coverage: 96%
    - Branch Coverage: 75%

Kết quả kiểm thử cho thấy toàn bộ các test case đều được thực thi thành công và các hàm xử lý nghiệp vụ của module "Tìm kiếm và lọc bác sĩ" hoạt động đúng theo yêu cầu đặt ra. Độ bao phủ kiểm thử đạt mức cao đối với các phương thức trọng tâm, giúp đảm bảo tính ổn định và độ tin cậy của hệ thống trong quá trình xử lý dữ liệu và lọc thông tin bác sĩ.

4.4.4. Kiểm thử chất lượng phản hồi của workflow AI

Chất lượng hoạt động của workflow AI được đánh giá thông qua các tiêu chí liên quan đến chất lượng phản hồi. Quá trình đánh giá được thực hiện trên tập dữ liệu kiểm thử gồm 20 yêu cầu tư vấn sức khỏe mô phỏng các tình huống hội thoại thực tế giữa người dùng và hệ thống. Các chỉ số đánh giá được sử dụng bao gồm:

- **Latency:** thời gian phản hồi trung bình của hệ thống.
- **BLEU Score:** đánh giá mức độ tương đồng giữa phản hồi của AI và câu trả lời tham chiếu.
- **ROUGE-1 và ROUGE-L:** đánh giá mức độ liên quan và khả năng bao phủ nội dung của phản hồi.



Hình 4.24. Workflow kiểm thử chất lượng phản hồi của workflow AI

Kết quả đánh giá được trình bày trong bảng bên dưới.

Bảng 4.8. Kết quả đánh giá workflow AI

Chỉ số	Giá trị
Tổng số yêu cầu kiểm thử	20
Thời gian phản hồi trung bình	11910.35 ms
BLEU trung bình	0.0571
ROUGE-1 trung bình	0.3111
ROUGE-L trung bình	0.2810

Kết quả cho thấy hệ thống có khả năng tạo phản hồi tương đối phù hợp với nội dung tham chiếu thông qua các chỉ số ROUGE. Tuy nhiên, chỉ số BLEU ở mức thấp do đặc thù của bài toán hội thoại tư vấn sức khỏe, trong đó mô hình có thể diễn đạt cùng một nội dung theo nhiều cách khác nhau thay vì khớp chính xác với câu trả lời mẫu.

Bên cạnh đó, thời gian phản hồi trung bình của hệ thống còn ở mức tương đối cao do workflow AI phải thực hiện nhiều bước xử lý như truy xuất tri thức bằng RAG, gọi API backend, duy trì bộ nhớ hội thoại và xử lý dữ liệu từ mô hình LLM. Tuy

nhiên, kết quả này vẫn phù hợp với môi trường prototype và có thể tiếp tục được tối ưu trong các giai đoạn phát triển tiếp theo.

Nhìn chung, việc kết hợp giữa n8n, cơ chế RAG, Redis Memory và mô hình Gemini 3.5 Flash giúp hệ thống đạt được khả năng hỗ trợ tư vấn sức khỏe và xử lý hội thoại theo ngữ cảnh với mức độ ổn định phù hợp cho mục tiêu nghiên cứu và thử nghiệm ban đầu.

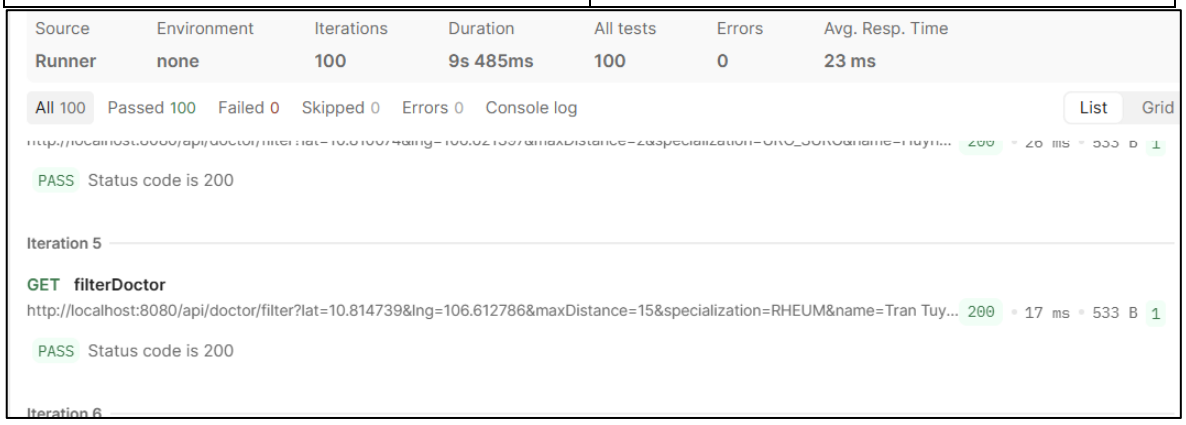
4.4.5. Kiểm thử API và khả năng xử lý request

Quá trình kiểm thử API được thực hiện bằng Postman Collection Runner nhằm đánh giá khả năng xử lý request của backend service trong quá trình vận hành hệ thống.

Kịch bản kiểm thử được thực hiện bằng cách gửi liên tiếp nhiều request đến API tìm kiếm bác sĩ nhằm kiểm tra tính ổn định và thời gian phản hồi của hệ thống. Kết quả kiểm thử cho thấy hệ thống xử lý thành công toàn bộ 100 request liên tiếp mà không phát sinh lỗi trong quá trình thực thi.

Bảng 4.9. Kết quả kiểm thử API bằng Postman Collection Runner

Thông số	Giá trị
Tổng số request	100
Số lượng Passed	100
Số lượng Error	0
Thời gian phản hồi trung bình	23 ms



Hình 4.25. Kết quả kiểm thử API bằng Postman Collection Runner

Kết quả kiểm thử cho thấy backend service hoạt động ổn định và có khả năng xử lý request liên tục với thời gian phản hồi thấp trong môi trường kiểm thử cục bộ.

4.5. Đánh giá và phân tích kết quả

4.5.1. Đánh giá khả năng hoạt động của hệ thống

Qua quá trình triển khai và kiểm thử, hệ thống đã đáp ứng được các mục tiêu chính của đề tài về xây dựng nền tảng tư vấn sức khỏe thông minh theo kiến trúc Microservices kết hợp workflow AI.

Kiến trúc Microservices giúp hệ thống phân tách rõ ràng các chức năng nghiệp vụ giữa các service backend như quản lý người dùng, quản lý bác sĩ, lịch hẹn, hội thoại AI và thông báo hệ thống. Việc phân tách này giúp giảm sự phụ thuộc giữa các thành phần, đồng thời thuận tiện cho việc mở rộng và bảo trì hệ thống trong tương lai.

Bên cạnh đó, việc sử dụng Docker và Docker Compose giúp quá trình triển khai hệ thống trở nên đồng nhất giữa các môi trường phát triển và triển khai thực tế. Các service có thể được khởi chạy và quản lý độc lập, hỗ trợ thuận tiện cho quá trình cập nhật và vận hành hệ thống. Ngoài ra, frontend và backend được xây dựng theo hướng tách biệt thông qua REST API, giúp hệ thống dễ dàng mở rộng và hỗ trợ phát triển độc lập giữa các thành phần.

Nhìn chung, hệ thống hoạt động ổn định trong môi trường thử nghiệm và đáp ứng được các chức năng chính theo yêu cầu của đề tài.

4.5.2. Đánh giá chất lượng phản hồi của workflow AI

Kết quả thực nghiệm cho thấy workflow AI có khả năng xử lý các yêu cầu tư vấn sức khỏe và duy trì ngữ cảnh hội thoại giữa nhiều lần trao đổi của người dùng. Việc phân tách workflow thành General AI Agent và Personal AI Agent giúp hệ thống xử lý linh hoạt các loại yêu cầu tư vấn khác nhau. Trong đó, General AI Agent phục vụ tư vấn tổng quát, còn Personal AI Agent hỗ trợ các cuộc tư vấn riêng giữa người dùng và bác sĩ.

Redis Cloud được sử dụng để lưu trữ bộ nhớ hội thoại giúp chatbot có khả năng phản hồi theo ngữ cảnh thay vì xử lý từng câu hỏi độc lập. Điều này giúp cải thiện tính liên tục của hội thoại và nâng cao trải nghiệm người dùng trong quá trình tương tác với hệ thống AI.

Ngoài dữ liệu văn bản, workflow còn hỗ trợ xử lý hình ảnh sức khỏe do người dùng cung cấp. Kết quả phân tích hình ảnh được bổ sung vào ngữ cảnh trước khi gửi đến mô hình Gemini 3.5 Flash để sinh phản hồi.

Kết quả đánh giá thông qua các chỉ số BLEU, ROUGE-1 và ROUGE-L cho thấy phản hồi của hệ thống có mức độ phù hợp tương đối tốt với các câu trả lời mẫu được xây dựng dựa trên nguồn tri thức của hệ thống. Bên cạnh đó, thời gian phản hồi của workflow AI còn ở mức tương đối cao do workflow phải thực hiện nhiều bước xử lý như duy trì bộ nhớ hội thoại, xử lý dữ liệu hình ảnh, gọi API backend và sinh phản hồi từ mô hình AI. Tuy nhiên, với phạm vi của một hệ thống prototype phục vụ mục tiêu nghiên cứu và thử nghiệm, kết quả đạt được vẫn phù hợp với mục tiêu đề tài.

4.5.3. Đánh giá kết quả kiểm thử hệ thống

Kết quả kiểm thử giao diện, Unit Test và kiểm thử API cho thấy hệ thống hoạt động ổn định trong môi trường thử nghiệm. Đối với kiểm thử giao diện, các chức năng chính của hệ thống đều hoạt động đúng theo yêu cầu thiết kế. Việc áp dụng Playwright giúp tự động hóa quá trình kiểm thử giao diện và hỗ trợ phát hiện sớm các lỗi phát sinh trong quá trình phát triển hệ thống.

Đối với Unit Test, các hàm xử lý nghiệp vụ quan trọng trong backend service hoạt động đúng với các kịch bản kiểm thử được xây dựng. Việc sử dụng JUnit 5 kết hợp Mockito và Diffblue Cover giúp kiểm tra chính xác business logic, tự động sinh test case và nâng cao độ bao phủ kiểm thử của hệ thống.

Ngoài ra, kết quả kiểm thử API bằng Postman Collection Runner cho thấy backend service có khả năng xử lý ổn định nhiều request liên tiếp mà không phát sinh lỗi trong quá trình thực thi. Nhìn chung, các kết quả kiểm thử cho thấy hệ thống đáp ứng tương đối tốt các yêu cầu chức năng trong môi trường phát triển và thử nghiệm hiện tại.

4.5.4. Phân tích ưu điểm và hạn chế của hệ thống

Ưu điểm của hệ thống

Hệ thống được xây dựng theo kiến trúc Microservices giúp phân tách rõ ràng các chức năng nghiệp vụ và hỗ trợ thuận tiện cho việc mở rộng trong tương lai. Việc sử dụng Docker và Docker Compose giúp chuẩn hóa môi trường triển khai, hỗ trợ quản lý và vận hành các service một cách linh hoạt hơn.

Ngoài ra, workflow AI được triển khai trên nền tảng n8n giúp tổ chức luồng xử lý trực quan và dễ dàng mở rộng thêm các chức năng AI khi cần thiết. Redis Cloud hỗ trợ duy trì bộ nhớ hội thoại hiệu quả, giúp chatbot có khả năng phản hồi theo ngữ cảnh và cải thiện trải nghiệm tương tác của người dùng.

Bên cạnh đó, hệ thống đã tích hợp được nhiều chức năng như quản lý lịch hẹn, tư vấn AI, quản lý bác sĩ, kiểm duyệt và quản lý dữ liệu sức khỏe trong cùng một nền tảng.

Hạn chế của hệ thống

Mặc dù đạt được các kết quả khả quan, hệ thống hiện vẫn còn một số hạn chế nhất định. Hệ thống hiện mới được triển khai ở mức prototype và chưa tối ưu hoàn toàn cho môi trường production thực tế. Một số thành phần như HTTPS, tên miền riêng và reverse proxy chưa được triển khai. Ngoài ra, thời gian phản hồi của workflow AI còn phụ thuộc vào tốc độ xử lý của mô hình Gemini API và các bước xử lý trong workflow. Bộ dữ liệu đánh giá AI hiện còn tương đối nhỏ và chưa bao phủ đầy đủ các tình huống tư vấn sức khỏe thực tế.

Bên cạnh đó, hệ thống chưa được kiểm thử với số lượng lớn người dùng đồng thời và chưa triển khai đầy đủ các cơ chế monitoring, logging hoặc bảo mật chuyên sâu cho kiến trúc Microservices. Các cơ chế bảo mật hiện tại chủ yếu dừng ở mức xác thực JWT và phân quyền cơ bản giữa các vai trò người dùng.

Hệ thống mới chỉ bảo vệ các trường thông tin nền tảng như mật khẩu, số tài khoản ngân hàng, nhưng lại chưa có đảm bảo an toàn cho các thông tin định danh cá nhân (số điện thoại, email), hồ sơ sức khỏe, lịch sử tin nhắn, kết quả chẩn đoán khám bệnh đều đang được lưu giữ dưới dạng văn bản thô. Việc thiếu các cơ chế bảo mật đồng bộ có thể dẫn đến nguy cơ rò rỉ thông tin trên diện rộng và ảnh hưởng nghiêm trọng đến quyền riêng tư cũng như an toàn dữ liệu y tế của người bệnh khi hệ thống bị tấn công hoặc truy cập trái phép từ bên ngoài.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Tóm tắt kết quả nghiên cứu

Trong quá trình thực hiện đề tài, nhóm đã xây dựng thành công hệ thống tư vấn sức khỏe thông minh theo kiến trúc Microservices kết hợp workflow AI. Hệ thống hỗ trợ nhiều chức năng như quản lý người dùng, quản lý bác sĩ, đặt lịch khám, quản lý lịch sử hội thoại, hỗ trợ tư vấn sức khỏe bằng AI thông qua giao diện web...

Đối với mục tiêu xây dựng kiến trúc hệ thống, đề tài đã triển khai backend theo kiến trúc Microservices bằng Spring Boot, trong đó các chức năng nghiệp vụ được phân tách thành các service độc lập nhằm tăng khả năng mở rộng và thuận tiện cho việc bảo trì hệ thống. Các service giao tiếp với nhau thông qua REST API và được đóng gói bằng Docker để có thể thực hiện triển khai thử nghiệm trên môi trường VPS AWS EC2.

Đối với mục tiêu xây dựng workflow AI, hệ thống đã triển khai workflow trên nền tảng n8n nhằm xử lý luồng hội thoại tư vấn sức khỏe. Workflow hỗ trợ tiếp nhận dữ liệu từ backend, duy trì bộ nhớ hội thoại thông qua Redis Cloud, xử lý hình ảnh sức khỏe và giao tiếp với mô hình Gemini 3.5 Flash để sinh phản hồi cho người dùng. Ngoài ra, hệ thống còn hỗ trợ phân tách giữa General AI Agent và Personal AI Agent nhằm xử lý linh hoạt các loại yêu cầu tư vấn khác nhau giữa người dùng và bác sĩ.

Về mặt triển khai và kiểm thử, hệ thống đã được triển khai thử nghiệm trên VPS AWS EC2 thông qua Docker và Docker Compose. Tuy nhiên, các bài kiểm thử chính của hệ thống được thực hiện trên môi trường phát triển cục bộ. Các chức năng chính của hệ thống đã được kiểm thử thông qua Playwright, JUnit 5, Mockito và Postman Collection Runner. Kết quả kiểm thử cho thấy các chức năng hoạt động ổn định trong môi trường phát triển và thử nghiệm hiện tại.

Bên cạnh đó, workflow AI cũng được đánh giá thông qua các chỉ số BLEU, ROUGE-1, ROUGE-L và thời gian phản hồi trung bình nhằm kiểm tra khả năng xử lý hội thoại của hệ thống AI trong phạm vi dữ liệu kiểm thử của đề tài.

5.2. Đóng góp của đề tài

5.2.1. Đóng góp trên phương diện kỹ thuật

Về mặt kỹ thuật, đề tài đã xây dựng được một hệ thống tư vấn sức khỏe thông minh dựa trên kiến trúc Microservices kết hợp workflow AI tương đối hoàn chỉnh trong phạm vi nghiên cứu. Đề tài đã áp dụng kiến trúc Microservices để phân tách các chức năng nghiệp vụ thành các service độc lập như quản lý người dùng, quản lý bác sĩ, lịch hẹn, hội thoại AI và thông báo hệ thống. Cách tiếp cận này giúp hệ thống có khả năng mở rộng và thuận tiện cho việc bảo trì trong tương lai.

Ngoài ra, đề tài cũng triển khai workflow AI trên nền tảng n8n nhằm tổ chức luồng xử lý AI theo hướng trực quan và linh hoạt hơn so với việc xử lý toàn bộ logic AI trực tiếp trong backend. Bên cạnh đó, Redis Cloud được tích hợp để duy trì bộ nhớ hội thoại giúp chatbot có khả năng phản hồi theo ngữ cảnh giữa nhiều lần trao đổi của người dùng. Hệ thống cũng hỗ trợ xử lý hình ảnh sức khỏe và phân loại các luồng tư vấn phù hợp giữa General AI Agent và Personal AI Agent.

Đối với triển khai, hệ thống đã được triển khai container hóa bằng Docker và Docker Compose, giúp chuẩn hóa môi trường triển khai và hỗ trợ vận hành hệ thống thuận tiện hơn.

5.2.2. Đóng góp trên phương diện thực tiễn

Về mặt thực tiễn, đề tài đã xây dựng được một nền tảng hỗ trợ tư vấn sức khỏe thông minh có khả năng ứng dụng trong các hệ thống hỗ trợ y tế và chăm sóc sức khỏe trực tuyến. Thông qua hệ thống, người dùng có thể thực hiện tư vấn sức khỏe bằng AI, quản lý lịch sử hội thoại và đặt lịch khám trực tuyến với bác sĩ trên cùng một nền tảng. Ngoài ra, hệ thống còn hỗ trợ bác sĩ quản lý lịch làm việc, lịch hẹn khám bệnh và dữ liệu tri thức phục vụ trợ lý AI. Các chức năng quản trị cũng được xây dựng nhằm hỗ trợ quản lý người dùng, bác sĩ và dữ liệu vận hành của hệ thống.

Việc triển khai hệ thống trên VPS AWS EC2 cho thấy khả năng vận hành thực tế của kiến trúc Microservices kết hợp workflow AI trong môi trường triển khai thử nghiệm với chi phí hạ tầng tương đối thấp.

5.3. Hạn chế của nghiên cứu

Mặc dù đề tài đã hoàn thành các mục tiêu nghiên cứu đề ra, hệ thống hiện vẫn còn một số hạn chế nhất định. Hệ thống hiện mới được triển khai ở mức prototype

phục vụ mục đích nghiên cứu và thử nghiệm, chưa được tối ưu hoàn toàn cho môi trường production thực tế. Quá trình kiểm thử chủ yếu được thực hiện trên môi trường phát triển cục bộ (local environment) và chưa được đánh giá trên quy mô lớn với nhiều người dùng đồng thời trong môi trường thực tế.

Ngoài ra, thời gian phản hồi của workflow AI còn phụ thuộc vào tốc độ xử lý của mô hình Gemini API cũng như các bước xử lý trong workflow như duy trì bộ nhớ hội thoại và xử lý hình ảnh. Bộ dữ liệu đánh giá AI còn tương đối nhỏ và chưa bao phủ đầy đủ các tình huống tư vấn sức khỏe thực tế. Do đó, chất lượng phản hồi của chatbot vẫn có thể thay đổi tùy theo ngữ cảnh và dữ liệu đầu vào.

Một số thành phần như HTTPS, tên miền riêng, reverse proxy, monitoring và logging chuyên sâu cho kiến trúc Microservices hiện vẫn chưa được triển khai đầy đủ trong phạm vi đề tài. Các cơ chế bảo mật hiện tại chủ yếu dừng ở mức xác thực JWT và phân quyền cơ bản giữa các vai trò người dùng.

Ngoài ra, hệ thống hiện chưa triển khai đầy đủ cơ chế mã hóa dữ liệu nhạy cảm của người dùng. Các thông tin cá nhân như số điện thoại, địa chỉ email và nội dung tin nhắn hội thoại hiện được lưu trữ ở dạng văn bản thô trong cơ sở dữ liệu, chưa được mã hóa theo các tiêu chuẩn bảo vệ dữ liệu y tế. Đây là một hạn chế đáng kể cần được khắc phục trước khi triển khai hệ thống trong môi trường thực tế.

5.4. Hướng phát triển trong tương lai

Dựa trên các hạn chế hiện tại cũng như tiềm năng mở rộng của hệ thống, nghiên cứu này đề xuất một số hướng phát triển trong tương lai nhằm nâng cao hiệu quả xử lý của workflow AI, khả năng mở rộng và tính ứng dụng thực tiễn của hệ thống tư vấn sức khỏe thông minh.

Tối ưu hiệu năng và thời gian phản hồi của hệ thống AI

Trong phiên bản hiện tại, workflow AI vẫn cần thực hiện nhiều bước xử lý như duy trì bộ nhớ hội thoại, xử lý dữ liệu hình ảnh, giao tiếp giữa các service và gọi mô hình Gemini API. Điều này làm cho thời gian phản hồi của hệ thống trong một số trường hợp còn tương đối cao.

Trong tương lai, hệ thống có thể được tối ưu bằng cách cải thiện luồng xử lý workflow, giảm các bước xử lý trung gian và tối ưu cơ chế lưu trữ bộ nhớ hội thoại. Ngoài ra, việc nghiên cứu các cơ chế xử lý bất đồng bộ hoặc cache dữ liệu cũng là

hướng tiếp cận phù hợp nhằm giảm độ trễ trong quá trình xử lý hội thoại thời gian thực. Bên cạnh đó, việc lựa chọn các mô hình AI phù hợp hơn với bài toán tư vấn sức khỏe hoặc tối ưu prompt engineering cũng có thể góp phần cải thiện tốc độ và chất lượng phản hồi của hệ thống.

Mở rộng dữ liệu và nâng cao chất lượng đánh giá AI

Hiện tại, bộ dữ liệu đánh giá AI của hệ thống còn tương đối nhỏ và chưa bao phủ đầy đủ các tình huống tư vấn sức khỏe thực tế.

Trong tương lai, hệ thống cần được mở rộng tập dữ liệu kiểm thử nhằm nâng cao độ chính xác và tính ổn định của chatbot trong nhiều ngữ cảnh hội thoại khác nhau. Việc xây dựng dữ liệu đa dạng hơn về triệu chứng, tình huống sức khỏe và hành vi người dùng sẽ giúp cải thiện khả năng phản hồi của hệ thống AI.

Ngoài ra, các nghiên cứu tiếp theo có thể mở rộng đánh giá chatbot dựa trên nhiều tiêu chí khác nhau như mức độ hài lòng của người dùng, khả năng duy trì ngữ cảnh dài hạn hoặc độ phù hợp của phản hồi trong từng nhóm tình huống sức khỏe cụ thể.

Hoàn thiện hạ tầng triển khai và bảo mật hệ thống

Hiện tại, hệ thống mới được triển khai thử nghiệm trên VPS AWS EC2 và chưa triển khai đầy đủ các thành phần dành cho môi trường production như HTTPS, tên miền riêng hoặc reverse proxy.

Trong tương lai, hệ thống cần được hoàn thiện về mặt hạ tầng nhằm nâng cao tính ổn định và bảo mật trong quá trình vận hành thực tế. Các thành phần như Nginx, HTTPS và cơ chế quản lý cấu hình môi trường sẽ đóng vai trò quan trọng trong quá trình mở rộng hệ thống.

Bên cạnh đó, việc triển khai các cơ chế monitoring, logging, HTTPS, quản lý truy cập và bảo mật dữ liệu cho kiến trúc Microservices cũng là hướng phát triển cần thiết nhằm nâng cao độ an toàn và tính ổn định của hệ thống trong môi trường thực tế.

Bảo mật dữ liệu

Hệ thống cần chú trọng phát triển cơ chế bảo mật dữ liệu toàn diện cho người dùng. Toàn bộ thông tin cá nhân (số điện thoại, email, …) khi lưu trữ và nội dung tin nhắn trong quá trình gửi đi đều được mã hóa tự động để chống truy cập trái phép. Giải pháp này không chỉ bảo vệ quyền riêng tư tuyệt đối cho người dùng, mà còn

giúp hệ thống chuẩn hóa theo các khung pháp lý và tiêu chuẩn quốc tế về bảo mật dữ liệu y tế của WHO và ISO/IEC 27001.

Mở rộng khả năng xử lý đồng thời và triển khai thực tế

Trong phạm vi đề tài, hệ thống chủ yếu được kiểm thử trên môi trường phát triển cục bộ và chưa được đánh giá trên quy mô lớn với nhiều người dùng đồng thời. Do đó, trong tương lai cần tiếp tục nghiên cứu và tối ưu khả năng xử lý đồng thời của hệ thống nhằm đáp ứng nhu cầu triển khai thực tế. Các hướng phát triển có thể bao gồm mở rộng hạ tầng triển khai theo hướng phân tán, triển khai container orchestration hoặc áp dụng cơ chế cân bằng tải cho các backend service.

Ngoài ra, việc triển khai thử nghiệm hệ thống trong môi trường thực tế với người dùng thật cũng là bước quan trọng nhằm đánh giá chính xác hiệu quả hoạt động và trải nghiệm sử dụng của hệ thống.

5.5. Kết luận

Khóa luận đã hoàn thành mục tiêu xây dựng hệ thống tư vấn sức khỏe thông minh dựa trên kiến trúc Microservices kết hợp workflow AI. Hệ thống hỗ trợ các chức năng chính như quản lý người dùng, quản lý bác sĩ, đặt lịch khám, quản lý lịch sử hội thoại và tư vấn sức khỏe bằng AI thông qua giao diện web.

Về mặt kỹ thuật, đề tài đã triển khai thành công backend theo kiến trúc Microservices bằng Spring Boot, đồng thời xây dựng workflow AI trên nền tảng n8n nhằm xử lý luồng hội thoại và điều phối giao tiếp với mô hình Gemini 3.5 Flash. Redis Cloud được sử dụng để duy trì bộ nhớ hội thoại, giúp chatbot có khả năng phản hồi theo ngữ cảnh giữa nhiều lần trao đổi của người dùng.

Ngoài ra, hệ thống đã được triển khai thử nghiệm trên môi trường VPS AWS EC2 thông qua Docker và Docker Compose nhằm phục vụ quá trình triển khai và kiểm tra hoạt động cơ bản của hệ thống sau khi hoàn thiện.

Kết quả kiểm thử giao diện, Unit Test và kiểm thử API cho thấy các chức năng chính của hệ thống hoạt động ổn định trong môi trường phát triển hiện tại. Bên cạnh đó, workflow AI cũng cho thấy khả năng hỗ trợ xử lý hội thoại tư vấn sức khỏe với mức độ phù hợp tương đối thông qua các chỉ số đánh giá như BLEU và ROUGE.

Mặc dù hệ thống vẫn còn một số hạn chế như thời gian phản hồi AI còn tương đối cao, dữ liệu đánh giá chưa lớn và chưa được tối ưu đầy đủ cho môi trường

production, đặc biệt là các cơ chế bảo mật và bảo vệ dữ liệu người dùng vẫn chưa được triển khai chuyên sâu, kết quả đạt được vẫn cho thấy tính khả thi của việc kết hợp Microservices, workflow AI và mô hình LLM trong bài toán hỗ trợ tư vấn sức khỏe thông minh.

Nhìn chung, đề tài đã xây dựng được một nền tảng hệ thống tương đối hoàn chỉnh phục vụ cho mục tiêu nghiên cứu và là cơ sở để tiếp tục mở rộng, tối ưu và phát triển các ứng dụng AI hỗ trợ y tế trong tương lai.

TÀI LIỆU THAM KHẢO

[1] Singhal K. và cộng sự. (2023), "Large Language Models Encode Clinical Knowledge", *Nature*, 620(7972), 172-180.

[2] Brown T.B. và cộng sự. (2020), "Language Models are Few-Shot Learners", *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 1877-1901.

[3] Lewis P. và cộng sự. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459-9474.

[4] Microsoft. (2026), *AutoGen Framework Documentation*. https://microsoft.github.io/autogen/. [Truy cập ngày 26/05/2026].

[5] CrewAI. (2026), *CrewAI Documentation*. https://docs.crewai.com/. [Truy cập ngày 26/05/2026].

[6] LangChain AI. (2026), *LangGraph Documentation*. https://langchain-ai.github.io/langgraph/. [Truy cập ngày 26/05/2026].

[7] n8n. (2026), *n8n Documentation*. https://docs.n8n.io/. [Truy cập ngày 26/05/2026].

[8] Russell S. và Norvig P. (2021), *Artificial Intelligence: A Modern Approach*, Pearson, Harlow.

[9] Goodfellow I., Bengio Y. và Courville A. (2016), *Deep Learning*, MIT Press, Cambridge.

[10] Topol E. (2019), *Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again*, Basic Books, New York.

[11] Weng L. (2023), *LLM Powered Autonomous Agents*. https://lilianweng.github.io/posts/2023-06-23-agent/. [Truy cập ngày 26/05/2026].

[12] Zhang H. và cộng sự. (2025), "Reducing Hallucinations and Trade-Offs in Responses in Generative AI Chatbots for Cancer Information", *Journal of Medical Internet Research*. https://pmc.ncbi.nlm.nih.gov/articles/PMC12425422/. [Truy cập ngày 26/05/2026].

[13] World Health Organization. (2021), *Ethics and Governance of Artificial Intelligence for Health: WHO Guidance*, World Health Organization, Geneva. https://www.biodiritto.org/AI-Legal-Atlas/AHEAD-OBSERVATORY/AHEAD-Legislative-and-regulatory-framework/WHO-Ethics-ad-Governance-of-Artificial-Intelligence-for-Health-WHO-Guidance. [Truy cập ngày 26/05/2026].

[14] Alotaibi M.A. và cộng sự. (2025), "Transforming Healthcare with Chatbots: Uses and Applications-A Scoping Review", *Healthcare*. https://pmc.ncbi.nlm.nih.gov/articles/PMC11915287/. [Truy cập ngày 26/05/2026].

[15] Kim J. và cộng sự. (2025), "Development and Evaluation of a Retrieval-Augmented Generation-Based Electronic Medical Record Chatbot System", *npj Digital Medicine*. https://pmc.ncbi.nlm.nih.gov/articles/PMC12370418/. [Truy cập ngày 26/05/2026].

[16] Docker Inc. (2026), *Docker Documentation*. https://docs.docker.com/. [Truy cập ngày 26/05/2026].

[17] Meta Platforms. (2026), *React Documentation*. https://react.dev/. [Truy cập ngày 26/05/2026].

[18] Ant Design. (2026), *Ant Design Documentation*. https://ant.design/. [Truy cập ngày 26/05/2026].

[19] Zustand. (2026), *Zustand Documentation*. https://zustand.docs.pmnd.rs/learn/getting-started/introduction. [Truy cập ngày 26/05/2026].

[20] VMware. (2026), *Spring Boot Documentation*. https://spring.io/projects/spring-boot. [Truy cập ngày 26/05/2026].

[21] Redis Ltd. (2026), *Redis Documentation*. https://redis.io/docs/latest/. [Truy cập ngày 26/05/2026].

[22] Cloudinary. (2026), *Cloudinary Documentation*. https://cloudinary.com/documentation. [Truy cập ngày 26/05/2026].

[23] VMware. (2026), *RabbitMQ Documentation*. https://www.rabbitmq.com/documentation.html. [Truy cập ngày 26/05/2026].

[24] Carnell J. và Huaylupo I. (2021), *Spring Microservices in Action*, Manning Publications, Shelter Island.

[25] JUnit Team. (2026), *JUnit 5 User Guide*. https://junit.org/junit5/docs/current/user-guide/. [Truy cập ngày 26/05/2026].

[26] Spring Framework. (2026), *Spring Testing Documentation*. https://docs.spring.io/spring-framework/reference/testing/. [Truy cập ngày 26/05/2026].

[27] International Software Testing Qualifications Board. (2026), *Software Testing Foundations*. https://www.istqb.org/. [Truy cập ngày 26/05/2026].

[28] Mockito. (2026), *Mockito Official Documentation*. https://site.mockito.org/. [Truy cập ngày 26/05/2026].

[29] Diffblue. (2026), *Diffblue Cover Documentation*. https://docs.diffblue.com/. [Truy cập ngày 26/05/2026].

[30] Diffblue. (2026), *Diffblue Official Website*. https://www.diffblue.com/. [Truy cập ngày 26/05/2026].

[31] JaCoCo. (2026), *JaCoCo Java Code Coverage Library*. https://www.jacoco.org/jacoco/. [Truy cập ngày 26/05/2026].

[32] JaCoCo. (2026), *JaCoCo Maven Plugin Documentation*. https://www.jacoco.org/jacoco/trunk/doc/maven.html. [Truy cập ngày 26/05/2026].

[33] SonarSource. (2026), *Code Coverage and Quality Analysis*. https://www.sonarsource.com/. [Truy cập ngày 26/05/2026].

[34] Microsoft. (2026), *Playwright Documentation*. https://playwright.dev/. [Truy cập ngày 26/05/2026].

[35] Microsoft. (2026), *Playwright for Java*. https://playwright.dev/java/. [Truy cập ngày 26/05/2026].

[36] BrowserStack. (2026), *What Is End-to-End Testing*. https://www.browserstack.com/guide/end-to-end-testing. [Truy cập ngày 26/05/2026].

[37] Postman. (2026), *Postman API Platform*. https://www.postman.com/. [Truy cập ngày 26/05/2026].

[38] Postman. (2026), *Postman Learning Center*. https://learning.postman.com/. [Truy cập ngày 26/05/2026].

[39] Postman. (2026), *Collection Runner*. https://learning.postman.com/docs/collections/running-collections/intro-to-collection-runs/. [Truy cập ngày 26/05/2026].

[40] Shukla, A. K. (2017), Building Web Apps with Spring 5 and Angular. O'Reilly Media. https://www.oreilly.com/library/view/building-web-apps/9781787284661/c06d01b0-529c-42b7-8659-7142a2abcb76.xhtml. [Truy cập ngày 26/05/2026].

[41] JRebel. (2026), Mock Unit Testing with Mockito. https://www.jrebel.com/blog/mock-unit-testing-with-mockito. [Truy cập ngày 26/05/2026].

[42] Diffblue. (2026), What is Diffblue Cover - Diffblue Cover Documentation. https://cover-docs.diffblue.com/get-started/what-is-diffblue-cover. [Truy cập ngày 26/05/2026].

[43] Baeldung. (2026), Intro to JaCoCo. https://www.baeldung.com/jacoco. [Truy cập ngày 26/05/2026].

[44] MM Software. (2026), Playwright - Modern End-to-End Testing for Modern Web Apps. https://www.mm-software.com/en/insights/detail/playwright/. [Truy cập ngày 26/05/2026].

[45] Postman. (2026), Introduction to Collection Runs. https://learning.postman.com/docs/tests-and-scripts/running-collections/intro-to-collection-runs/. [Truy cập ngày 26/05/2026].